

# Six attempts at quasi-polynomial unknot recognition: synthesis, cross-validation, and a faster exact recognizer

Synthesis report prepared with Claude Code

17–18 September 2026; updated 8 October 2026

## Abstract

Six independently produced archives each tried to implement the quasi-polynomial unknot recognition algorithm announced in Marc Lackenby’s February 2021 talk. This report extracts them into numbered directories, reads their reports, reviews their code, re-runs their test suites, and cross-validates them against each other on shared inputs. All six reach the same honest conclusion: the quasi-polynomial algorithm was not implemented, because the accelerated hierarchy operations named on the last slide have no published algorithmic specification with cost bounds. What they deliver instead is consistent and correct as far as we can test: five reduced-Khovanov recognizers agree on 44 shared inputs, six ball-pattern testers agree on 2000 random spherical patterns, and the grid-diagram search of archive 01 agrees with Khovanov homology on every rectangular diagram we checked. Building on their ideas we add a seventh recognizer, **fastunknot**, whose exact part is a scanning computation of Khovanov homology (Bar-Natan’s local algorithm with delooping and Gaussian elimination) preceded by an Alexander-polynomial filter. It is only justified by an exponential general bound; its practical cost depends on scan width and intermediate complex size. For example, the 80-crossing unknot family on which every cube implementation needs  $2^{80}$  states is decided in eleven milliseconds. Nine further runs then proposed accelerations of that recognizer. We extracted and measured all nine on one machine, integrated every idea with a measurable effect into version 0.2 (the input that timed out at 600 seconds now takes under a second, and recognition of the Conway knot went from 59 ms to 1.3 ms), recorded a per-idea ablation, and wrote a Rust implementation that is faster again by an order of magnitude. The  $n^{O(\log n)}$  target remains open; Section 5 states precisely what is missing. The October continuation integrates report 07: a linear structural certificate and exact component sharing, including a conditional finite-type complexity bound. Section 12 separates this bound from the unresolved general target.

## Contents

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>1</b> | <b>Task, sources, and outcome</b>             | <b>8</b>  |
| 1.1      | What was asked . . . . .                      | 8         |
| 1.2      | The talk . . . . .                            | 8         |
| 1.3      | The 2026 preprint . . . . .                   | 9         |
| 1.4      | Outcome in one paragraph . . . . .            | 10        |
| <b>2</b> | <b>The six archives</b>                       | <b>10</b> |
| 2.1      | Extraction . . . . .                          | 10        |
| 2.2      | What each archive delivers . . . . .          | 10        |
| 2.3      | Common structure of the six reports . . . . . | 11        |

|           |                                                                           |           |
|-----------|---------------------------------------------------------------------------|-----------|
| <b>3</b>  | <b>Code review</b>                                                        | <b>12</b> |
| 3.1       | The Khovanov recognizers (02–06)                                          | 12        |
| 3.2       | The pattern testers (all six)                                             | 12        |
| 3.3       | Normal coordinates and potentials                                         | 12        |
| 3.4       | The grid search of archive 01                                             | 13        |
| <b>4</b>  | <b>Cross-validation</b>                                                   | <b>13</b> |
| 4.1       | Five Khovanov implementations on shared inputs                            | 13        |
| 4.2       | Six pattern testers on random spherical patterns                          | 14        |
| 4.3       | Grid search versus Khovanov homology                                      | 14        |
| 4.4       | What the cross-validation does and does not show                          | 14        |
| <b>5</b>  | <b>What is missing for <math>n^{O(\log n)}</math></b>                     | <b>15</b> |
| <b>6</b>  | <b>A faster exact recognizer: fastunknot 0.1</b>                          | <b>15</b> |
| 6.1       | Design                                                                    | 15        |
| 6.2       | The scanning computation                                                  | 16        |
| 6.3       | Validation                                                                | 17        |
| 6.4       | Measurements                                                              | 17        |
| 6.5       | Limits of the new package                                                 | 18        |
| <b>7</b>  | <b>Nine acceleration proposals</b>                                        | <b>18</b> |
| 7.1       | What was delivered                                                        | 18        |
| 7.2       | The ideas                                                                 | 18        |
| 7.3       | Same-machine comparison                                                   | 19        |
| <b>8</b>  | <b>fastunknot 0.2</b>                                                     | <b>20</b> |
| 8.1       | What was adopted                                                          | 20        |
| 8.2       | Ablation                                                                  | 21        |
| 8.3       | Complexity: what changed and what did not                                 | 22        |
| 8.4       | Verification                                                              | 22        |
| <b>9</b>  | <b>A Rust implementation</b>                                              | <b>23</b> |
| 9.1       | Design                                                                    | 23        |
| 9.2       | Verification                                                              | 23        |
| 9.3       | Profile                                                                   | 23        |
| 9.4       | Where the three versions stand                                            | 24        |
| <b>10</b> | <b>A third round: constant factors in Python, and how to measure them</b> | <b>24</b> |
| 10.1      | What changed                                                              | 24        |
| 10.2      | Results                                                                   | 25        |
| 10.3      | Measured and not adopted                                                  | 29        |
| 10.4      | An episode about measurement                                              | 29        |

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| <b>11 A port to Bend 2, and what its parallelism delivered</b>                   | <b>30</b> |
| 11.1 Setting . . . . .                                                           | 30        |
| 11.2 What was ported, and how it was checked . . . . .                           | 30        |
| 11.3 What the fork delivers . . . . .                                            | 31        |
| 11.4 Where the time goes . . . . .                                               | 31        |
| 11.5 Verdict . . . . .                                                           | 32        |
| <b>12 Structural certificates and compressed complexes: October continuation</b> | <b>32</b> |
| 12.1 Intake, implementation, and scope . . . . .                                 | 32        |
| 12.2 A linear structural decision . . . . .                                      | 33        |
| 12.3 What exact sharing represents . . . . .                                     | 33        |
| 12.4 Safe decision compression and early obstructions . . . . .                  | 34        |
| 12.5 The conditional complexity statement . . . . .                              | 34        |
| 12.6 Local measurements and reproducibility . . . . .                            | 35        |
| 12.7 Rechecking structure after crossing reduction . . . . .                     | 36        |
| 12.8 Preparing suffix geometry within the inference budget . . . . .             | 36        |
| 12.9 Replacing Euler state enumeration by link connectivity . . . . .            | 37        |
| 12.10 Source screening: the OpenAI mathematics collection . . . . .              | 38        |
| <b>13 Further incoming reports and checked integrations</b>                      | <b>39</b> |
| 13.1 Intake status and provenance . . . . .                                      | 39        |
| 13.2 Complete recognition for checked short-strand braids . . . . .              | 39        |
| 13.3 Canonical visible factors from interlacement . . . . .                      | 41        |
| 13.4 Sparse modular Alexander elimination . . . . .                              | 42        |
| 13.5 Resource handling and the corrected frontier bound . . . . .                | 42        |
| 13.6 Later algebra and twist reports: production cross-checks . . . . .          | 42        |
| 13.7 Typed scalar shortcuts in the default planar algebra . . . . .              | 43        |
| 13.8 Opt-in component contraction in the production scanner . . . . .            | 44        |
| <b>14 Twist compression and a restricted quasi-polynomial family</b>             | <b>45</b> |
| 14.1 Local replacement and exact assembly . . . . .                              | 46        |
| 14.2 Counting the selected complex . . . . .                                     | 46        |
| 14.3 A sharper degree-sensitive bound . . . . .                                  | 47        |
| 14.4 Integration, provenance, and failure semantics . . . . .                    | 48        |
| 14.5 Production checks and observed scaling . . . . .                            | 48        |
| <b>15 Survivor prediction and adaptive cancellation</b>                          | <b>49</b> |
| 15.1 What binary residue homology determines . . . . .                           | 50        |
| 15.2 An exact shortcut, and an essential non-shortcut . . . . .                  | 50        |
| 15.3 Switching implementations in response to progress . . . . .                 | 51        |
| 15.4 The stronger report and its boundary assumptions . . . . .                  | 52        |
| 15.5 Validation and measured limits . . . . .                                    | 52        |

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>16 Recovered grading, homogeneous algebra, and sharper type bounds</b>      | <b>53</b> |
| 16.1 The grading that survives after its labels are erased . . . . .           | 53        |
| 16.2 Cheaper multiplication with an explicit applicability check . . . . .     | 54        |
| 16.3 A sharper sufficient regime for component sharing . . . . .               | 55        |
| 16.4 Adaptive controls that satisfy the grading constraint . . . . .           | 56        |
| 16.5 A limit of fixed finite-quotient portfolios . . . . .                     | 57        |
| <b>17 Exact homological windows and adaptive widening</b>                      | <b>57</b> |
| 17.1 Why deleting chain groups can be exact . . . . .                          | 57        |
| 17.2 Mirror choice and a parameterized complexity bound . . . . .              | 58        |
| 17.3 An introspective recognition policy . . . . .                             | 59        |
| 17.4 Validation and measured tradeoffs . . . . .                               | 59        |
| 17.5 Review boundaries for the adjacent reports . . . . .                      | 60        |
| <b>18 Interval normalization, continuation quotients, and graded splitting</b> | <b>60</b> |
| 18.1 Exact decomposition of a pure component . . . . .                         | 61        |
| 18.2 Why length two is enough for the decision observation . . . . .           | 61        |
| 18.3 Certified scalar splitting with recovered grading . . . . .               | 62        |
| 18.4 A sharper checkpoint bound . . . . .                                      | 63        |
| 18.5 Validation and practical scope . . . . .                                  | 64        |
| <b>19 Verified rank-two braid preprocessing</b>                                | <b>65</b> |
| 19.1 Faithful local states without copying prefixes . . . . .                  | 65        |
| 19.2 An optimal pass over bounded-overlap intervals . . . . .                  | 65        |
| 19.3 Independent replay and evidence association . . . . .                     | 66        |
| 19.4 A constructive quasi-polynomial input class and its barrier . . . . .     | 67        |
| 19.5 Production validation and end-to-end measurements . . . . .               | 67        |
| <b>20 Minimal extremal windows with certified binomial bounds</b>              | <b>68</b> |
| 20.1 Cancellation and truncation do not commute . . . . .                      | 68        |
| 20.2 Minimality controls all exhaustive pivot choices . . . . .                | 68        |
| 20.3 The geometric certificate and inherited object count . . . . .            | 69        |
| 20.4 A uniform bound without the all-state-circle factor . . . . .             | 70        |
| 20.5 Raw endpoint depth has an exact padding obstruction . . . . .             | 70        |
| 20.6 Validation and comparison with the current window engine . . . . .        | 71        |
| <b>21 Exact Potts filters and independent component tensors</b>                | <b>71</b> |
| 21.1 Normalization without approximate algebraic numbers . . . . .             | 72        |
| 21.2 A half-frontier bound for every supplied crossing order . . . . .         | 73        |
| 21.3 Independent processed components and exact orbit division . . . . .       | 73        |
| 21.4 A specialization collision that changing primes cannot repair . . . . .   | 74        |
| 21.5 Integration, budgets, and validation . . . . .                            | 74        |

|           |                                                                                |           |
|-----------|--------------------------------------------------------------------------------|-----------|
| 21.6      | Measurements and the quasi-polynomial objective . . . . .                      | 74        |
| <b>22</b> | <b>Disk-certified transfer after an adaptive pause</b>                         | <b>75</b> |
| 22.1      | Certifying the boundary from the processed rotation system . . . . .           | 75        |
| 22.2      | Retaining the maps that the binary profile cannot determine . . . . .          | 76        |
| 22.3      | The adaptive implementation and its transactional boundary . . . . .           | 77        |
| 22.4      | Cost separation and the missing global parameter bound . . . . .               | 77        |
| 22.5      | Why changing crossing order alone cannot suffice . . . . .                     | 78        |
| 22.6      | Validation and measured scope . . . . .                                        | 78        |
| <b>23</b> | <b>Exact long-run recurrences and streamed macro homology</b>                  | <b>79</b> |
| 23.1      | The repeated differential, not just repeated dimensions . . . . .              | 79        |
| 23.2      | Why the slope is positive . . . . .                                            | 80        |
| 23.3      | Compressed output and a fixed-context bound . . . . .                          | 80        |
| 23.4      | Direct structural evaluation on run-encoded braids . . . . .                   | 81        |
| 23.5      | Streaming changes storage, with an explicit time tradeoff . . . . .            | 81        |
| 23.6      | Validation and measurements . . . . .                                          | 82        |
| <b>24</b> | <b>Arithmetic continuations and checked Montesinos sources</b>                 | <b>82</b> |
| 24.1      | What the source describes . . . . .                                            | 83        |
| 24.2      | Provenance, budgets, and integration . . . . .                                 | 84        |
| 24.3      | Literal local obstructions with a checked disk . . . . .                       | 85        |
| 24.4      | Why arithmetic helps when a small linear summary cannot . . . . .              | 85        |
| 24.5      | Production validation and measured scope . . . . .                             | 86        |
| <b>25</b> | <b>Full transfer ordered by quantum grading</b>                                | <b>87</b> |
| 25.1      | Retaining the grading rather than reconstructing it afterwards . . . . .       | 87        |
| 25.2      | A contraction with representatives and homotopies . . . . .                    | 88        |
| 25.3      | Computing a path sum without enumerating its paths . . . . .                   | 89        |
| 25.4      | Adaptive scheduling and installation . . . . .                                 | 89        |
| 25.5      | Validation and measurement scope . . . . .                                     | 90        |
| <b>26</b> | <b>Succinct transport, the earlier rank law, and repair dependencies</b>       | <b>91</b> |
| 26.1      | An earlier total-rank threshold does not give a smaller cap for free . . . . . | 91        |
| 26.2      | JSON is part of the succinct representation contract . . . . .                 | 92        |
| 26.3      | What an acyclic repair certificate would need to establish . . . . .           | 92        |
| <b>27</b> | <b>Survivor corridors and adaptive transfer direction</b>                      | <b>93</b> |
| 27.1      | From perturbation transfer to a directed graph . . . . .                       | 93        |
| 27.2      | What the direction heuristic bounds . . . . .                                  | 94        |
| 27.3      | Sparse scalar components and complete costs . . . . .                          | 94        |
| 27.4      | A strict separation, and its scope . . . . .                                   | 95        |

|           |                                                                        |            |
|-----------|------------------------------------------------------------------------|------------|
| 27.5      | Integration, validation, and measured limits . . . . .                 | 95         |
| <b>28</b> | <b>Certified cyclic Garside preprocessing</b>                          | <b>97</b>  |
| 28.1      | Exact states and a separately checked equality witness . . . . .       | 97         |
| 28.2      | Sharing the algebra across cuts and targets . . . . .                  | 98         |
| 28.3      | Height-sensitive costs and the small-core theorem . . . . .            | 99         |
| 28.4      | Two limitations of short-word approaches . . . . .                     | 100        |
| 28.5      | Production contract and empirical results . . . . .                    | 101        |
| <b>29</b> | <b>Compressed surface covers and ordered attachment points</b>         | <b>103</b> |
| 29.1      | A checked presentation and three unmarked families . . . . .           | 103        |
| 29.2      | Orientation and peripheral information are retained . . . . .          | 104        |
| 29.3      | An exact extension for ordered marked points . . . . .                 | 105        |
| 29.4      | Implementation bounds, verification and measurements . . . . .         | 106        |
| <b>30</b> | <b>Causal RIII search: review and unambiguous certificates</b>         | <b>108</b> |
| 30.1      | What the causal argument can give . . . . .                            | 108        |
| 30.2      | Why immediate inverse pruning needs care . . . . .                     | 109        |
| 30.3      | Integrated prerequisite: identify the actual triangular face . . . . . | 109        |
| 30.4      | Integrated search and adaptive dispatch . . . . .                      | 110        |
| 30.5      | Actual-diagram evidence and limits . . . . .                           | 110        |
| <b>31</b> | <b>Marked determinant continuations and an adaptive exact fallback</b> | <b>111</b> |
| 31.1      | Why the marked completion is necessary . . . . .                       | 112        |
| 31.2      | A four-coordinate lower bound . . . . .                                | 112        |
| 31.3      | Computing the vector without enumerating the cube . . . . .            | 113        |
| 31.4      | A budgeted observer on the same exact scan . . . . .                   | 113        |
| 31.5      | A reusable terminal kernel, including singular interiors . . . . .     | 114        |
| 31.6      | Independent checks and actual performance . . . . .                    | 115        |
| 31.7      | What is still missing for a general complexity theorem . . . . .       | 116        |
| <b>32</b> | <b>Classical closures, first jets, and bounded diagram resets</b>      | <b>116</b> |
| 32.1      | Source audit and the exact scope of the rerun . . . . .                | 116        |
| 32.2      | Why a first jet controls a knot completion . . . . .                   | 117        |
| 32.3      | Pure blocks and the indispensable link distinction . . . . .           | 118        |
| 32.4      | Recognition control, splicing, and fallback . . . . .                  | 118        |
| 32.5      | Actual reset gaps and what they do not prove . . . . .                 | 119        |
| 32.6      | Expanded discovery, validation, and paired measurements . . . . .      | 119        |
| <b>33</b> | <b>Adaptive boundary summaries for repeated completion queries</b>     | <b>120</b> |
| 33.1      | Two path summaries determine the raw Euler value . . . . .             | 121        |
| 33.2      | When the maintained backend switches . . . . .                         | 121        |

|           |                                                                           |            |
|-----------|---------------------------------------------------------------------------|------------|
| 33.3      | A common Tait graph made from cut-face fragments . . . . .                | 122        |
| 33.4      | Independent geometry audit . . . . .                                      | 123        |
| 33.5      | Measured reuse and its limits . . . . .                                   | 123        |
| <b>34</b> | <b>Signed Tait block products for actual observer workloads</b>           | <b>124</b> |
| 34.1      | Why the signed product identity is exact . . . . .                        | 124        |
| 34.2      | Implementation, resources, and complexity . . . . .                       | 125        |
| 34.3      | Validation and workload evidence . . . . .                                | 126        |
| <b>35</b> | <b>Preserving a cheap observer after marked-work exhaustion</b>           | <b>128</b> |
| 35.1      | Two limits with separate meanings . . . . .                               | 128        |
| 35.2      | Why partially completed observations are safe to abandon . . . . .        | 129        |
| 35.3      | Transition tests and benchmark design . . . . .                           | 129        |
| <b>36</b> | <b>Certified square-root frontiers and a subexponential scalar filter</b> | <b>130</b> |
| 36.1      | A bounded-radius middle band . . . . .                                    | 131        |
| 36.2      | Three short paths from a dual centroid . . . . .                          | 131        |
| 36.3      | From the hierarchy to a certified order . . . . .                         | 132        |
| 36.4      | Exactly which subexponential computation follows . . . . .                | 132        |
| 36.5      | Validation and measurements . . . . .                                     | 133        |
| <b>37</b> | <b>Deferring separator work until the scalar query needs it</b>           | <b>134</b> |
| 37.1      | The policy and its persistent state . . . . .                             | 134        |
| 37.2      | A single transition budget, including discarded work . . . . .            | 134        |
| 37.3      | Why the deferral preserves the asymptotic scalar bound . . . . .          | 135        |
| 37.4      | Independent checks and measured scope . . . . .                           | 135        |
| 37.5      | A relevant geometric result, with a different scope . . . . .             | 136        |
| <b>38</b> | <b>A collision-free Jones query and full polynomial recovery</b>          | <b>137</b> |
| 38.1      | Uniform coefficient and degree bounds . . . . .                           | 137        |
| 38.2      | Choosing a specialization that cannot hide nontriviality . . . . .        | 137        |
| 38.3      | Recovering all coefficients from two integers . . . . .                   | 138        |
| 38.4      | Why exponentially many colors do not cause exponential work . . . . .     | 138        |
| 38.5      | Interface, evidence and resource semantics . . . . .                      | 139        |
| 38.6      | Validation and measurement . . . . .                                      | 139        |
| 38.7      | Using proved identity to avoid redundant reconstruction . . . . .         | 140        |
| <b>39</b> | <b>Auditing a claimed route from Jones identity to recognition</b>        | <b>141</b> |
| 39.1      | An explicit host and a twist with fixed exterior . . . . .                | 141        |
| 39.2      | An independent polynomial calculation . . . . .                           | 142        |
| 39.3      | Reproducibility and the scope of the finding . . . . .                    | 142        |
| <b>40</b> | <b>A complete polynomial test on certified treewidth-two projections</b>  | <b>143</b> |

|           |                                                                            |            |
|-----------|----------------------------------------------------------------------------|------------|
| 40.1      | The structural restriction that makes determinant one sufficient . . . . . | 143        |
| 40.2      | A graph certificate before any positive invariant verdict . . . . .        | 143        |
| 40.3      | Exact integer arithmetic and independent verification . . . . .            | 144        |
| 40.4      | Adaptive integration and the exact scope of the guarantee . . . . .        | 145        |
| 40.5      | Validation and whole-query measurements . . . . .                          | 145        |
| <b>41</b> | <b>An adaptive normal-surface fallback in an isolated worker</b>           | <b>145</b> |
| 41.1      | A complete geometric decision with a different bottleneck . . . . .        | 145        |
| 41.2      | Convention checks and the trust boundary . . . . .                         | 146        |
| 41.3      | Switching after cheap filters, with bounded discarded work . . . . .       | 146        |
| 41.4      | Measurements with explicit censoring . . . . .                             | 147        |
| <b>42</b> | <b>Cyclic knot-group certificates and adaptive Whitehead descent</b>       | <b>148</b> |
| 42.1      | A positive certificate from the full fundamental group . . . . .           | 148        |
| 42.2      | A deterministic presentation and reversible eliminations . . . . .         | 148        |
| 42.3      | Cut-based changes of free basis . . . . .                                  | 149        |
| 42.4      | A reproducible convention audit of a new preprint . . . . .                | 149        |
| 42.5      | Independent replay and local resource accounting . . . . .                 | 150        |
| 42.6      | What can and cannot be bounded . . . . .                                   | 150        |
| 42.7      | Whole-query measurements and regression tests . . . . .                    | 151        |
| <b>43</b> | <b>Assessment and recommendations</b>                                      | <b>151</b> |
| <b>A</b>  | <b>Files produced</b>                                                      | <b>154</b> |

## 1 Task, sources, and outcome

### 1.1 What was asked

The original request to each of the six runs was an implementation of the main theorem of the supplied talk [16]: an algorithm that decides whether an  $n$ -crossing knot diagram represents the unknot in time  $n^{O(\log n)}$ . The present task was to unpack the six resulting archives, read their reports, review their code, and synthesize what was achieved and what obstacles remain. A second task was added mid-way: write the asymptotically fastest algorithm possible, trying to reach  $n^{O(\log n)}$ . A third task followed: unpack nine proposals for accelerating that algorithm, compare their ideas, incorporate those that give a measurable speedup, and write an optimized Rust implementation.

### 1.2 The talk

The supplied file `docs/quasipolynomial-talk.pdf` has 109 pages including progressive-reveal duplicates. Its logical content is short:

- pp. 11–14: the theorem,  $n^{O(\log n)}$  for an  $n$ -crossing diagram;
- pp. 15–30: hierarchies, boundary patterns, essential patterns (a proper disc meeting the pattern at most three times must cut off an empty disc, an arc, or a tripod), and the Waldhausen–Johannson characterization: the knot is nontrivial iff the exterior admits an essential hierarchy

ending in balls;

- pp. 38–53: simplification of an inessential hierarchy by a violating disc (compress or pattern-compress the last surface the disc runs over, discard later surfaces), and the basic algorithm as a flowchart;
- pp. 54–65: termination by a lexicographic complexity  $(g(S_1), \dots, g(S_\ell))$ , with the warning that “genus” may mean  $\chi_-$  or a pattern-sensitive variant, and the estimate “running time at most  $Lg^L$ ” if every surface has complexity at most  $g$  and hierarchies have length at most  $L$ ;
- pp. 66–74: the four speed-ups needed for  $n^{O(\log n)}$ : (1) surfaces with  $g \leq O(n^2)$ , (2) compressed hierarchy encodings (Agol–Hass–Thurston), (3) multi-surfaces, (4) generalised Heegaard splittings and Cheeger regions, “the effect of this is to ensure that  $L \leq O(\log n)$ ”;
- pp. 75–108: sketches of each speed-up, including the Cheeger-region condition  $g(\partial M_i) \leq \frac{1}{3} \min\{g(H_i), g(H) - g(H_i)\}$ ;
- p. 109: the final flowchart with the boxes BUILD HIERARCHICAL MULTI-SURFACE, CUT ALONG SURFACE, USE CHEEGER REGION, HAKEN’S LEMMA, WEAKLY REDUCE, SIMPLIFY MULTI-SURFACE, and the terminal test “does the graph dual to the boundary pattern have a violating cycle?”.

The slides describe the strategy; they contain no data structures, no algorithms for the boxes, and no proof of the depth bound.

### 1.3 The 2026 preprint

All six archives located Lackenby’s July 2026 preprint [17] (“Incompressible surfaces, hierarchies and unknot recognition”, 54 pages). We downloaded it and read Section 9 (“The number of steps”) ourselves. It opens: “There are various steps in the algorithm from Theorem 1.3 that are not described with enough precision to be able to estimate their running time. So, we will initially content ourselves with an analysis of the number of times that the algorithm goes through any of the steps”, in terms of the maximal hierarchy length  $L$  and the maximal pattern-complexity  $g$ , and “At present, we will not attempt to bound these two quantities.” Proposition 9.1 then gives the iteration bound  $L(g+1)^L$  using exactly the base- $(g+1)$  digit argument that the talk sketches in base  $g$ . Section 10 refines the algorithm to obtain a bound on  $L$  that is *linear* in the initial  $q$ -complexity of the handle structure, not logarithmic in  $n$ . The preprint also supplies two facts the archives use: Proposition 2.6 (a pattern on a 3-ball is essential iff it is empty, a circle, or a connected graph whose planar dual is simple and 4-connected or  $K_3$  or  $K_4$ ) and Definition 3.4 (pattern-complexity  $\chi_P(S) = -4\chi(S) + |S \cap P|$ ).

So the accelerated, logarithmic-depth algorithm of the talk remains announced but not specified in any source available to us on rechecking the author’s page and the preprint on 8 October 2026. This is the single obstacle behind every “partial” verdict below, and it is mathematical, not an engineering shortfall.

**Plausibility and the precise target.** Later seminar abstracts, including the November 2024 talk [9], state  $2^{O((\log n)^3)} = n^{O((\log n)^2)}$ . The earlier supplied slides state the stronger  $n^{O(\log n)}$  target used elsewhere in this report; these are distinct formulations. Both are quasi-polynomial and hence  $2^{o(n)}$ . The hierarchy strategy is a credible route to such a bound: it replaces exhaustive global witness search with decomposition, compression and controlled corrections. The preprint supplies substantial topological machinery for that route. This is our assessment of plausibility, not a verification of its announced asymptotics. Even given the iteration estimate  $L(g+1)^L$ , an implementation must bound the bit cost  $C(n)$  of each operation on its largest intermediate encoding. A sufficient accounting condition is

$$\log C(n) + \log(L+1) + L \log(g+1) = O((\log n)^3).$$

Efficiently handling one compressed surface, or proving termination, does not establish this condition. No general sub-exponential bound for our recognizer follows from the current report or from adaptive switching among its present backends.

For intuition,  $L = O(\log n)$  and  $g = n^{O(1)}$  would make the iteration count  $2^{O((\log n)^2)}$ ; allowing suitably bounded quasi-polynomial operation cost could still fit the later cubic-logarithm exponent. The difficulty is proving compatible bounds throughout all corrections, including the size of intermediate compressed descriptions. A short certificate or a terminating hierarchy does not itself give a fast way to find one. Conversely, the growth of our explicit Khovanov complexes is not evidence against Lackenby’s different hierarchy algorithm. Our assessment is that the claim is mathematically plausible and worth pursuing, while the available specification is insufficient to verify or implement its announced asymptotics. Section 36 proves a general subexponential scalar computation, and Section 38 extends this to full Jones polynomial recovery. Neither computation supplies the missing complete recognition bound.

## 1.4 Outcome in one paragraph

None of the six archives, and not the new package either, implements a recognizer with a proved  $n^{O(\log n)}$  bound. Each archive contains a complete exact recognizer with an exponential bound, plus genuinely implemented polynomial pieces of the hierarchy strategy (the terminal ball-pattern test, cocycle-dual normal coordinates, the potential arithmetic), and each report says so without hedging. We verified every test suite, cross-validated the archives against each other and against independent computations, found no incorrect verdict, and identified one gap in a correctness argument (Section 3.4). The new package **fastunknot** improves the exact part substantially in practice (Section 6); its worst case is still  $2^{O(n)}$ .

# 2 The six archives

## 2.1 Extraction

The six ZIP files in **reports/** were extracted into **reports/01** through **reports/06**, each with the single wrapper directory removed, in the order of the archive modification times:

| Dir | archive                                           | package              | Python lines | tests |
|-----|---------------------------------------------------|----------------------|--------------|-------|
| 01  | unknot_recognition_implementation.zip             | unknot (grid search) | 1753         | 40    |
| 02  | unknot_recognition_partial_implementation.zip     | unknotlab            | 1939         | 64    |
| 03  | unknot_workbench.zip                              | unknot_lab           | 1431         | 49    |
| 04  | unknot_recognition_partial.zip                    | unknot               | 1422         | 62    |
| 05  | unknot_recognition_partial_implementation (1).zip | unknot_recognition   | 1882         | 58    |
| 06  | unknot-recognition-implementation.zip             | unknot               | 1278         | 45    |

Every archive contains a README, a LaTeX report with PDF, examples, recorded results, and a standard-library-only Python package. All six test suites were re-run here on CPython 3.14.4 under Windows (the archives recorded CPython 3.13.5 on Linux) and pass with exactly the claimed number of tests.

## 2.2 What each archive delivers

**Archive 01: rectangular diagrams and Dynnikov moves.** The only archive that does not use Khovanov homology. It represents knots as grid diagrams, searches the finite space of grids reachable by non-increasing moves (exchanges and destabilizations, modulo cyclic shifts) and relies on Dynnikov’s monotonic simplification theorem [23]: an unknot reaches the  $2 \times 2$  grid, a knot is

certified by a state set closed under all moves. Certificates (positive path, non-unit determinant, closed set) are replayed by a verifier that enumerates moves independently. Bound  $2^{O(g \log g)}$  in the grid size  $g$ ; the report is careful that  $g$  is not the crossing number. It validated all 44 719 connected grids of size  $\leq 6$  (1285 shift orbits). It also contains the pattern test, exact rational  $H^1$  of a simplicial complex, cocycle-dual normal coordinates, and the potential arithmetic.

**Archive 02: Khovanov cube with fast certificates.** Descending-diagram test (a diagram traversed meeting every crossing first on its over-strand is trivial), Fox determinant, then the full reduced Khovanov complex over  $\mathbb{F}_2$ . Its normal-surface module accepts genuine face-pairing triangulations, checks orientability and sphere/disc vertex links, and computes cocycle-dual normal coordinates without expanding discs. Six braid/determinant pairs were checked against Wolfram KnotData.

**Archive 03: workbench with an explicit exponential lower bound.** Khovanov cube with default resource ceilings (65 536 states), a bond-enumeration pattern test with checkable witnesses, and a `bound` command for the conditional  $L(g+1)^L$  arithmetic. Its report proves that its own implementation is not quasi-polynomial: on the unknot  $\sigma_1 \widehat{\sigma_2 \cdots \sigma_n}$  the cube has  $2^n$  states and  $3^n$  generators, and the determinant filter cannot help since the determinant is 1.

**Archive 04: Khovanov cube with Knot Atlas fixtures.** Khovanov cube, dual-4-connectivity pattern test, potential arithmetic. It checks  $10_{124} = T(3, 5)$  from an Atlas PD code and contains a clean chain-level proof that the unreduced  $\mathbb{F}_2$  complex is two copies of the reduced one, which justifies reading reduced ranks off published unreduced tables.

**Archive 05: Khovanov cube with Reidemeister preprocessing.** Crossing-decreasing Reidemeister I/II moves with replayable traces, determinant filter, Khovanov cube; bond-enumeration pattern test; simplicial normal coordinates with manifold validation; the pattern-complexity  $-4\chi + |S \cap P|$  of the preprint. It exhausts all 682 two-strand words of odd length  $\leq 9$  and gives an independent braid-theoretic proof that its eight-crossing “hard unknot” is trivial.

**Archive 06: Khovanov cube with 11-crossing tests.** Khovanov cube, bond-enumeration pattern test, a `verify` command that recomputes a report with  $d^2 = 0$  checks. Its fixtures include the Kinoshita–Terasaka and Conway knots (Alexander polynomial 1, reduced rank 33). Its source audit notes that a 34-page Oxford version of the slides uses the factor 1/10 instead of 1/3 in the Cheeger condition and displays  $O((\log n)^2)$  for the hierarchy length; substituting that depth into the digit count would give  $n^{O((\log n)^2)}$ .

## 2.3 Common structure of the six reports

Read side by side, the reports agree on every substantive point, which is itself evidence since they were produced independently:

1. The requested bound is not established; the delivered recognizer is a different, exponential algorithm; resource limits return UNKNOWN and never a verdict.
2. The Khovanov route is justified by Kronheimer–Mrowka [21] plus a universal-coefficient argument:  $\dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(K; \mathbb{F}_2) \geq \text{rank } \widetilde{\text{Kh}}(K; \mathbb{Z})$ , and the rational reduced homology is nonzero because its Euler characteristic is the reduced Jones polynomial, so  $\mathbb{F}_2$ -rank one forces rational rank one.
3. Determinant one is never a positive certificate ( $T(3, 5)$  and the Kinoshita–Terasaka knot are the standard witnesses).

4. The talk’s “base  $g$ ” digit argument needs base  $g + 1$  when the digit  $g$  is allowed, since  $(1, 0)$  and  $(0, g)$  have the same base- $g$  value; all six use base  $g + 1$ , as does Proposition 9.1 of the preprint.
5. The terminal pattern test is implemented only under an explicit “already a 3-ball” hypothesis, and  $b_1 = 0$  is not ball recognition ( $S^2 \times I$  is the standard counterexample).
6. The missing operations are listed by name: initial layered handle structure of the exterior, compressed cutting with provenance, hierarchical multi-surfaces with the pattern-sensitive quadratic bound, lifting a violating dual cycle to a disc in an earlier stage, Cheeger-region detection and Heegaard modification, Haken’s lemma and weak reduction, and an end-to-end bit-cost proof including restarts.

### 3 Code review

#### 3.1 The Khovanov recognizers (02–06)

All five have the same architecture, evidently converged upon rather than copied (variable names, data layouts and gradings differ): enumerate the  $2^n$  resolutions with a disjoint-set structure, encode a reduced generator as a bit mask over the unmarked circles with the marked circle fixed to  $x$ , build each differential column as a Python integer (XOR arithmetic over  $\mathbb{F}_2$ ), and compute ranks by pivot elimination. Points checked in the review:

- Input validation is uniformly careful: one component, spherical rotation system via  $V - E + F = 2$  on the ribbon graph (so virtual diagrams are rejected, not misclassified), labels normalized, `{"pd": []}` defined as one circle.
- The Frobenius algebra tables ( $m(x \otimes x) = 0$ ,  $\Delta(1) = 1 \otimes x + x \otimes 1$ ,  $\Delta(x) = x \otimes x$ ) are identical everywhere, and every implementation checks that the marked circle keeps its  $x$  label.
- $d^2 = 0$  is available as an exhaustive optional check in all five and is exercised by their tests.
- Gradings are deliberately unnormalized; only the total rank is used, so writhe shifts are irrelevant. This is correct for recognition.
- Resource semantics are consistent (UNKNOWN, exit code 2 or 3; the codes differ between archives, which is a minor interoperability wart).
- Archive 03 imposes default ceilings, so its default invocation is incomplete above 16 crossings; the README says so.

We found no correctness problem. The determinant filters (Fox colouring matrix, Bareiss elimination with exact-division checks) are also equivalent.

#### 3.2 The pattern testers (all six)

Three different criteria are implemented for “is this cubic spherical pattern essential on a ball”: dual simple and 4-connected with the  $K_3$ ,  $K_4$  exceptions (01, 04), no dual loop, parallel pair or non-facial dual triangle (02), and no bond of size 1 or 2 and no three-edge bond with more than one vertex on each side (03, 05, 06). Each report proves its own criterion from the definition. The proofs are correct; the bond argument in archive 06 (the cycle rank of a side with  $k$  vertices and  $r$  cut edges is  $(k - r + 2)/2$ ) is the cleanest. Input conventions differ (darts  $2e, 2e + 1$ ; signed edge ends; positional darts; edge identifiers occurring twice), which is why a converter was needed for cross-validation.

#### 3.3 Normal coordinates and potentials

Archives 01, 02 and 05 compute integral cocycles spanning  $H^1(\cdot; \mathbb{Q})$  in a spanning-tree gauge and convert a cocycle to normal coordinates by half-integral level sets of local vertex heights. The constructions agree (seven coordinates per tetrahedron, matching equations checked, Euler

characteristic without disc expansion). All are honest that this is not a saturated  $\mathbb{Z}$ -basis, not a connected surface, and not a cut. The four potential modules implement the same base- $(g + 1)$  lemma and differ only in interface.

### 3.4 The grid search of archive 01

This is the one place where the review found a gap in an argument. The recognizer’s negative certificate is a set of grids closed under all implemented moves; soundness needs the implemented move set to be exactly a set for which monotonic simplification holds. The report states that “rows sharing an endpoint are not accepted by this implementation’s exchange rule” and cites Dynnikov’s Proposition 6, but does not argue that Dynnikov’s theorem still holds when exchanges of two adjacent rows (or columns) with a common corner column are excluded. Depending on the reading of Dynnikov’s definition of interleaving, that exclusion may remove legal moves, in which case a closed set could exist for an unknot. We could not settle this from the sources at hand, so we tested it: Section 4.3 compares the search verdict with Khovanov homology on every grid of size  $\leq 6$  and on random larger grids, and found no disagreement. The gap is in the proof, not (as far as tested) in the answers.

Everything else in archive 01 checked out: the move implementation, the Booth canonicalization, the independent verifier, and the exhaustive counts ( $g!(g - 1)!/2$  connected grids per size).

## 4 Cross-validation

None of the archives compared itself with another; each ran its own tests. We wrote three scripts (in `synthesis/data/`) that load the packages side by side.

### 4.1 Five Khovanov implementations on shared inputs

The same braid words and Knot Atlas PD codes were passed to all five packages (after converting label conventions). Reduced  $\mathbb{F}_2$  ranks:

| Input                                                | 02         | 03 | 04 | 05 | 06 | agree |
|------------------------------------------------------|------------|----|----|----|----|-------|
| unknot circle                                        | 1          | 1  | 1  | 1  | 1  | yes   |
| trefoil                                              | 3          | 3  | 3  | 3  | 3  | yes   |
| figure-eight                                         | 5          | 5  | 5  | 5  | 5  | yes   |
| T(2,5)                                               | 5          | 5  | 5  | 5  | 5  | yes   |
| T(2,7)                                               | 7          | 7  | 7  | 7  | 7  | yes   |
| T(3,4)                                               | 5          | 5  | 5  | 5  | 5  | yes   |
| T(3,5) det 1                                         | 7          | 7  | 7  | 7  | 7  | yes   |
| unknot s1 s2                                         | 1          | 1  | 1  | 1  | 1  | yes   |
| unknot s1s2s3s4                                      | 1          | 1  | 1  | 1  | 1  | yes   |
| conjugated unknot                                    | 1          | 1  | 1  | 1  | 1  | yes   |
| hard unknot 05                                       | 1          | 1  | 1  | 1  | 1  | yes   |
| KT 11n42 (Atlas PD)                                  | 33         | 33 | 33 | 33 | 33 | yes   |
| Conway 11n34 (Atlas PD)                              | 33         | 33 | 33 | 33 | 33 | yes   |
| 10_124 (Atlas PD)                                    | 7          | 7  | 7  | 7  | 7  | yes   |
| 30 random braid closures (3–4 strands, 6–10 letters) | ranks 1–13 |    |    |    |    | 30/30 |

All 44 inputs agree, including the two Alexander-polynomial-one 11-crossing knots and 30 random closures whose ranks range from 1 to 13.

## 4.2 Six pattern testers on random spherical patterns

Random perfect matchings of the darts of  $V$  trivalent vertices were kept when every component's rotation system is spherical, converted to the six input formats, and classified by all six testers (400 distinct systems per vertex count) together with six named patterns:

| Vertices | spherical systems tested | random matchings drawn | essential | disagreements |
|----------|--------------------------|------------------------|-----------|---------------|
| 2        | 400                      | 883                    | 102       | 0             |
| 4        | 400                      | 737                    | 12        | 0             |
| 6        | 400                      | 1179                   | 0         | 0             |
| 8        | 400                      | 2603                   | 0         | 0             |
| 10       | 400                      | 6144                   | 0         | 0             |

  

| Pattern      | verdict     | all six agree |
|--------------|-------------|---------------|
| theta        | essential   | yes           |
| tetrahedron  | essential   | yes           |
| cube         | essential   | yes           |
| prism        | inessential | yes           |
| dodecahedron | essential   | yes           |
| dumbbell     | inessential | yes           |

No disagreement among 2006 patterns and three different criteria. (Random matchings on many vertices are almost always inessential because short bonds are common; the named cube and dodecahedron are the essential large examples.)

## 4.3 Grid search versus Khovanov homology

We wrote an independent grid-to-PD converter (traversal of the grid, crossing signs from the directions of the vertical and horizontal strands) and compared archive 01's search (determinant filter disabled) with archive 04's Khovanov rank:

| Grid size | sample                        | diagrams | unknots | knots | disagreements |
|-----------|-------------------------------|----------|---------|-------|---------------|
| 2         | exhaustive (all shift orbits) | 1        | 1       | 0     | 0             |
| 3         | exhaustive (all shift orbits) | 2        | 2       | 0     | 0             |
| 4         | exhaustive (all shift orbits) | 6        | 6       | 0     | 0             |
| 5         | exhaustive (all shift orbits) | 64       | 62      | 2     | 0             |
| 6         | exhaustive (all shift orbits) | 1212     | 1185    | 27    | 0             |
| 7         | random, $\leq 14$ crossings   | 300      | 286     | 14    | 0             |
| 8         | random, $\leq 14$ crossings   | 200      | 188     | 12    | 0             |
| 9         | random, $\leq 14$ crossings   | 100      | 82      | 18    | 0             |

The knotted orbits of size 5 are the two trefoils (grid number 5) and those of size 6 are 26 trefoil presentations and one figure-eight, exactly as the grid-number theory predicts. No disagreement was found, which supports but does not prove the completeness of the restricted move set. The sample stops at size 9 because the search itself becomes expensive there (one size-9 grid needed 900 863 states, and the 100 grids took 1.9 hours), while the Khovanov side stayed below 14 crossings by construction.

## 4.4 What the cross-validation does and does not show

Agreement of five independently written cubes is strong evidence that all five compute the same invariant, and agreement with the published ranks (trefoil 3, figure-eight 5,  $T(3, 5)$  7,  $K11n34$

and  $K11n42$  33) ties that invariant to reduced Khovanov homology. It says nothing about the quasi-polynomial question, and all runs are on diagrams of at most 14 crossings because the cubes cannot go further.

## 5 What is missing for $n^{O(\log n)}$

Consolidating the six audits with our own reading of the sources, an implementation with a proved  $n^{O(\log n)}$  bound would require the following, none of which exists in any archive or, to our knowledge, in print.

1. **A representation.** A layered handle structure of the knot exterior with a Morse function, boundary-pattern labels, the ambient embedding in  $S^3$  (used by the generalised Seifert construction of pp. 75–78), compressed normal-surface data, and regluing maps, with a proved bound on the bit size of all of this after arbitrarily many operations. The archives’ normal-coordinate modules are local ingredients only.
2. **Bounded multi-surfaces.** An algorithm producing independent relative homology classes with embedded representatives whose *pattern-sensitive* complexity is  $O(n^2)$ , and a proof that compression and pattern-compression preserve the invariants that allow later surfaces to be kept. A rational  $H^1$  basis is not this.
3. **Compressed cutting.** Cutting, component extraction, Euler characteristic, orientation and pattern transport on compressed data without expanding parallel discs, with provenance sufficient to lift a violating dual cycle in a terminal ball to a disc in the correct earlier stage. Agol–Hass–Thurston orbit counting is a tool, not this procedure.
4. **Cheeger regions and weak reduction.** Recognition of the Heegaard–Morse hypotheses, construction of the modified generalised splitting, Haken’s lemma and weak reduction, and a global progress measure bounding restarts. The inequality  $3g(\partial M_i) \leq \min\{g(H_i), g(H) - g(H_i)\}$  is a test on three integers, not the operation.
5. **The depth bound.** A proof that the executable process, after every rebuild and restart, has hierarchy length  $O(\log n)$  (the talk) or  $O((\log n)^2)$  (the Oxford version cited by archive 06), together with polynomial digit bounds. The preprint’s Section 10 gives a linear bound in the initial  $q$ -complexity, and its Section 9 explicitly declines to bound  $g$  and  $L$ .
6. **The cost accounting.** Only after all of the above does the conditional lemma proved in every archive,  $L(g+1)^L$  iterations times a polynomial cost per iteration times polynomially many restarts, become a running-time theorem:  $\log((g+1)^L) = O((\log n)^2)$ .

The archives also list, and we agree with, the substitutions that do not work: an exponential search with a time cap is not a total algorithm; a polynomial iteration count does not bound work per iteration; expanding a compressed normal surface loses the compression;  $b_1 = 0$  is not ball recognition; a Jones or Alexander test does not detect the unknot.

## 6 A faster exact recognizer: **fastunknot** 0.1

### 6.1 Design

Since the geometric route cannot be completed from the available sources, the second task was addressed by making the *exact* route as fast as we could while keeping every verdict provable. The package **fast/fastunknot** (standard library only, Python  $\geq 3.10$ ) runs:

1. validation (PD codes, braid closures, or grid diagrams in the archive-01 convention, converted by the traversal of Section 4.3);
2. crossing-decreasing Reidemeister I/II moves, as in archive 05;
3. the descending-diagram test of archive 02;

4. the *Alexander polynomial*, computed exactly by fraction-free Bareiss elimination over  $\mathbb{Z}[t]$  on a first minor of the Fox matrix of the Wirtinger presentation (rows  $(1 - t)x_o + tx_u - x_v$  at positive and  $(1 - t)x_o - x_u + tx_v$  at negative crossings, with signs from the traversal);  $\Delta \neq 1$  certifies knottedness and strictly dominates the determinant filter of the archives ( $T(3, 5)$  and all torus knots are now rejected in polynomial time; the Kinoshita–Terasaka and Conway knots are not);
5. reduced Khovanov rank over  $\mathbb{F}_2$  by *scanning*.

## 6.2 The scanning computation

This is Bar-Natan’s local algorithm [22] specialised to  $\mathbb{F}_2$  and implemented directly in terms of the arc-algebra description of morphisms. The crossings are processed one at a time in a greedy order chosen (from up to twelve starting crossings) to keep the boundary of the processed region small. After  $k$  crossings the partial complex has:

- objects: perfect matchings of the boundary edges, each with a homological degree; closed circles are delooped, one object with a circle becoming two objects (labels 1 and  $x$ );
- morphisms  $a \rightarrow b$ :  $\mathbb{F}_2$ -combinations of dot-subsets of the circles of  $a \cup \bar{b}$ , i.e. elements of  $A^{\otimes \text{circles}}$  with  $A = \mathbb{F}_2[x]/(x^2)$ .

Composition is the two-dimensional TQFT of the glued surface: a component of genus  $\geq 1$  or with two dots is zero (because the handle operator  $m \circ \Delta = 2x$  and  $x^2$  vanish), a closed component is 1 exactly when it carries one dot, and a planar component with  $\beta$  boundary circles and  $d \in \{0, 1\}$  dots is  $\Delta^{(\beta-1)}(x^d)$ , which over  $\mathbb{F}_2$  is  $x \otimes \cdots \otimes x$  for  $d = 1$  and the sum over one undotted position for  $d = 0$ . Delooping is the same computation with a cup on the source circle (dotted for label  $x$ ) and a cap on the target circle (dotted to extract the coefficient of 1, since the dual basis of  $\{1, x\}$  under the counit is  $\{\varepsilon(x \cdot), \varepsilon\}$ ). After every crossing all invertible entries are cancelled: an entry  $b \rightarrow c$  between equal matchings is a unit of the local ring  $\text{End}(a) \cong A^{\otimes k}$  iff its expansion contains the undotted term, its inverse is the finite geometric series of the nilpotent part, and Bar-Natan’s cancellation lemma replaces  $\varepsilon$  by  $\varepsilon - \gamma\varphi^{-1}\delta$ . When the last crossing is added the boundary is empty, every morphism is a scalar, so a minimal complex has zero differential and its number of objects is the unreduced total rank; the reduced rank is half of it (archive 04’s two-copies lemma) and equals 1 exactly for the unknot.

**Proposition 6.1** (Correctness). *On a validated one-component diagram, **fastunknot** returns **UNKNOT** iff the knot is trivial, unless a user-selected ceiling stops it, in which case it returns **UNKNOWN**.*

*Proof.* Steps 2–3 are isotopies and a sufficient triviality criterion; step 4 uses  $\Delta_{\text{unknot}} = 1$ . Step 5 computes the homology of the Khovanov complex: the tensor product of crossing complexes in Bar-Natan’s category, delooping, and Gaussian elimination are chain homotopy equivalences, and the closed complex is the standard one. The decision rule is the detection theorem with the universal-coefficient argument of the archives.  $\square$

**Proposition 6.2** (Cost). *Steps 1–4 are polynomial in  $n$ . Step 5 is  $2^{O(n)}$  in the worst case.*

*Proof.* The Alexander determinant is  $O(n^3)$  polynomial operations of degree  $\leq n$  with polynomially bounded coefficients. In step 5 the objects before elimination after crossing  $k$  number at most  $2 \cdot 2^c$  times the objects after crossing  $k - 1$ , where  $c \leq 2$  closed circles can appear, so the whole computation is dominated by the cube; compositions cost polynomial time in the boundary size.  $\square$

The interesting quantity is the size after elimination. Empirically it stays close to (number of crossingless matchings of the boundary)  $\times$  (a multiplicity that grows with the tangle’s homology),

i.e. roughly  $2^{O(w)} \cdot \text{poly}$  for scan width  $w$ . **We have no proof of this**, and there cannot be one in general for the multiplicity: the reduced rank of  $\#_k K11n34$  is  $33^k$  with Alexander polynomial 1, so any method that outputs the full homology is exponential on such inputs, whatever the scan width. A subexponential unknot recognizer along these lines would have to detect “rank  $> 1$ ” without finishing the computation; we do not know how.

### 6.3 Validation

The package has 19 unit tests (polynomial arithmetic; Alexander polynomials of the trefoil, figure-eight,  $5_2$ ,  $T(3,5)$ ,  $\Delta = 1$  for the two 11-crossing knots and two hard unknots; symmetry  $\Delta(t) = \Delta(t^{-1})$ ,  $|\Delta(1)| = 1$ , mirror invariance and odd determinant on 40 random braids; Reidemeister moves; the local endomorphism ring;  $d^2 = 0$  after every crossing; independence of the processing order; resource-limit semantics; known ranks; an independent dense reduced-cube computation on 25 random braids; the command line). Beyond the tests, the scanner agrees with archive 04’s cube on 120 random braid closures (3–5 strands, 6–13 letters, with random processing orders), and reproduces the Atlas ranks 33, 33 and 7.

### 6.4 Measurements

Timings are on the same Windows machine as everything else in this report; they illustrate the dependence on scan width and are not complexity evidence.

| Family                  | $n$ | reduced rank | max. boundary | objects before | objects after | seconds |
|-------------------------|-----|--------------|---------------|----------------|---------------|---------|
| unknot sigma_1..sigma_n | 8   | 1            | 2             | 6              | 2             | 0.001   |
| unknot sigma_1..sigma_n | 16  | 1            | 2             | 6              | 2             | 0.004   |
| unknot sigma_1..sigma_n | 32  | 1            | 2             | 6              | 2             | 0.013   |
| unknot sigma_1..sigma_n | 64  | 1            | 2             | 6              | 2             | 0.036   |
| unknot sigma_1..sigma_n | 128 | 1            | 2             | 6              | 2             | 0.133   |
| unknot sigma_1..sigma_n | 256 | 1            | 2             | 6              | 2             | 0.475   |
| T(2,n)                  | 5   | 5            | 4             | 30             | 10            | 0.003   |
| T(2,n)                  | 11  | 11           | 4             | 66             | 22            | 0.011   |
| T(2,n)                  | 21  | 21           | 4             | 126            | 42            | 0.043   |
| T(2,n)                  | 41  | 41           | 4             | 246            | 82            | 0.153   |
| T(3,n)                  | 8   | 5            | 6             | 42             | 10            | 0.010   |
| T(3,n)                  | 10  | 7            | 6             | 54             | 14            | 0.022   |
| T(3,n)                  | 14  | 9            | 6             | 78             | 18            | 0.032   |
| T(3,n)                  | 16  | 11           | 6             | 90             | 22            | 0.045   |
| T(3,n)                  | 20  | 13           | 6             | 114            | 26            | 0.092   |
| T(3,n)                  | 22  | 15           | 6             | 126            | 30            | 0.089   |
| random 3-braid          | 20  | 75           | 6             | 762            | 150           | 0.217   |
| random 3-braid          | 40  | 321          | 6             | 3354           | 642           | 1.035   |
| random 4-braid          | 15  | 5            | 6             | 30             | 10            | 0.015   |
| random 4-braid          | 21  | 25           | 8             | 438            | 146           | 0.576   |
| random 4-braid          | 31  | 45           | 8             | 604            | 216           | 0.947   |
| random 4-braid          | 41  | 109          | 8             | 654            | 218           | 2.683   |
| random 5-braid          | 24  | 13           | 6             | 109            | 42            | 0.099   |
| random 5-braid          | 36  | timeout      | timeout       | timeout        | timeout       | 600.147 |
| random 6-braid          | 31  | 85           | 8             | 548            | 216           | 1.194   |

| Example                        | $n$ | verdict | deciding step            | seconds |
|--------------------------------|-----|---------|--------------------------|---------|
| conway.json                    | 11  | KNOTTED | reduced-khovanov-F2-scan | 0.0528  |
| figure_eight.json              | 4   | KNOTTED | alexander-polynomial     | 0.0001  |
| grid_determinant_one_knot.json | 10  | KNOTTED | alexander-polynomial     | 0.0010  |
| grid_scrambled_unknot.json     | 6   | UNKNOT  | reidemeister-reduction   | 0.0004  |
| hard_unknot_8.json             | 8   | UNKNOT  | reduced-khovanov-F2-scan | 0.0022  |
| kinoshita_terasaka.json        | 11  | KNOTTED | reduced-khovanov-F2-scan | 0.0593  |
| torus_3_5.json                 | 10  | KNOTTED | alexander-polynomial     | 0.0009  |
| trefoil.json                   | 3   | KNOTTED | alexander-polynomial     | 0.0001  |
| unknot.json                    | 0   | UNKNOT  | reidemeister-reduction   | 0.0000  |
| unknot_braid40.json            | 40  | UNKNOT  | reidemeister-reduction   | 0.0090  |

One case timed out: a random 36-letter closure on five strands exceeded the 600-second budget, while a 31-letter closure on six strands took about a second. When this section was first written we attributed the difference to the size of the minimal complexes of the intermediate tangles. **That explanation was wrong.** Section 7 shows that the minimal complexes of this input never exceed 5898 objects; the time went into fill-in caused by cancelling pivots in last-in-first-out order, and with min-fill pivots the same scan takes under a second.

Two comparisons with the archives' cubes on identical inputs: the Kinoshita–Terasaka knot (11 crossings, 2048 states, 11 559 generators) takes 0.24–0.31 s in the cubes and 0.07 s in the scan with a boundary of six edges and at most 66 objects after elimination; the unknot  $\widehat{\sigma_1 \cdots \sigma_n}$ , archive 03's explicit exponential family, needs  $2^n$  states in every cube and is decided by the scan in 0.011 s for  $n = 80$  with two objects after every elimination.

## 6.5 Limits of the new package

It is a reference implementation in pure Python; a compiled implementation of the same algorithm (as in Bar-Natan's or the KnotJob programs) would be far faster. Its default scan-order heuristic is greedy; the optional certified separator order added in Section 36 guarantees  $O(\sqrt{n})$  width. Grid-like projections can still require  $\Omega(\sqrt{n})$  raw width, and minimal complexes can be large. Homology verdicts do not produce a spanning disc or a Reidemeister sequence. No general quasi-polynomial recognition bound is proved.

# 7 Nine acceleration proposals

## 7.1 What was delivered

After version 0.1 of `fastunknot` (Section 6) was written, nine further runs were each asked to accelerate it. Their archives were extracted into `proposals/01–09` (wrapper directories removed, ZIP files deleted, archive order recorded in `proposals/ORDER.txt`). Every proposal is a complete replacement package with its own report, tests, an unchanged copy of the 0.1 baseline and same-machine benchmarks, and every one of them states that the general worst case remains exponential. As with the six original archives, the independent runs converged on the same ideas, which is evidence that the ideas are the natural ones.

## 7.2 The ideas

1. **Units are involutions.** In  $\text{End}(m) \cong \mathbb{F}_2[x_1, \dots, x_k]/(x_i^2)$  a unit is  $1 + \nu$  with  $\nu$  in the maximal ideal, and  $\nu^2 = 0$  in characteristic two (squares of monomials vanish, cross terms occur twice). So  $(1 + \nu)^{-1} = 1 + \nu$  and the geometric series of 0.1 is unnecessary. All nine.

2. **Min-fill pivots.** Version 0.1 cancelled invertible entries in last-in-first-out order. Choosing the pivot  $b \rightarrow c$  that minimizes the Markowitz count  $(\text{out}(b) - 1)(\text{in}(c) - 1)$  through a lazy heap limits fill-in [24]. Seven of nine (not 05, 06).
3. **Cheaper cobordism algebra.** The topology of a gluing (which discs merge, Euler characteristics, boundary circles) does not depend on the dots, so it can be computed once per triple of matchings and reused; identity and square shortcuts; memoized results. All nine; proposal 03 goes furthest and packs a morphism into one integer whose bit number is a dot mask.
4.  $O(n \log n)$  **scan order** and  $O(n)$  **descending test.** The greedy score of a crossing changes only when a neighbour is processed, so a lazy heap reproduces the 0.1 order exactly; the descending test becomes a sweep over a difference array of bad cyclic intervals. All nine.
5. **Modular Alexander test.** Evaluate the first Alexander minor in a prime field; for the unknot it is  $\pm t^k$ ,  $0 \leq k < n$ , so any other value certifies knottedness in  $O(n^3)$  field operations. Six of nine, three of them only at  $t = -1$  (the determinant).
6. **Modular Jones test by a frontier scan.** Evaluate the Kauffman bracket at a fixed  $A \in \mathbb{F}_p$  while scanning the diagram, adding the weights of partial states as soon as they induce the same boundary matching. A value different from  $\delta(-A^3)^w$  certifies knottedness. All nine. This is the most valuable single idea for recognition: it rejects the Kinoshita–Terasaka and Conway knots, which have trivial Alexander polynomial, in about a millisecond, and its cost is *provably*  $(w - 1)!! \cdot \text{poly}(n)$  for scan width  $w$ , because states with equal matchings are merged; there is no multiplicity factor.
7. **Visible connected sums.** Two edges bordering the same two faces are a two-edge cut of the projection, so the diagram is a connected sum of the two sides. Reduced Khovanov homology over a field is multiplicative, so ranks multiply and degree counts convolve; for recognition the summands are examined separately. Seven of nine.
8. **Crossing signs from both strands.** Version 0.1 assumed that a PD row starts at the incoming under-port (true for its own converters); seven proposals compute the sign from both incoming ports, which the Jones normalization needs.
9. **Incremental Reidemeister simplification** (02 only), **no cancellation on the last crossings followed by linear algebra** (06 only), **an exact integer bracket after the modular one** (05 only), and **witness replay commands** (six proposals).

### 7.3 Same-machine comparison

The harness `synthesis/data/proposals_bench.py` runs every engine on the same inputs, each measurement in a fresh process with a 120-second limit; timers exclude start-up, imports and input construction; medians of three runs (one run above five seconds). “scan” is the raw Khovanov scan with factorization disabled, “rank” the rank of a connected sum of eight Conway knots through each engine’s factored API when it has one, “recognize” the default pipeline. Milliseconds; 0.1 is the baseline, 0.2 the integrated version of Section 8, and Rust the port of Section 9.

| Task                          | 0.1    | 01   | 02     | 03   | 04   | 05     | 06   | 07   | 08   | 09   | 0.2  | Rust  |
|-------------------------------|--------|------|--------|------|------|--------|------|------|------|------|------|-------|
| scan: conway                  | 34.4   | 11.6 | 10.4   | 9.86 | 11.3 | 10.2   | 10.4 | 9.80 | 10.0 | 12.5 | 8.44 | 0.59  |
| scan: kinoshita_terasaka      | 41.5   | 12.4 | 11.2   | 10.5 | 12.8 | 12.3   | 13.3 | 10.8 | 11.3 | 13.0 | 9.05 | 0.62  |
| scan: T(3,11)                 | 51.8   | 29.9 | 29.4   | 27.9 | 31.2 | 28.8   | 31.7 | 26.9 | 30.3 | 30.8 | 26.8 | 1.22  |
| scan: braid3_40               | 611    | 103  | 115    | 94.0 | 108  | 133    | 140  | 143  | 164  | 201  | 84.1 | 6.88  |
| scan: braid4_41               | 984    | 263  | 274    | 212  | 350  | 520    | 321  | 273  | 268  | 284  | 160  | 10.2  |
| scan: braid6_31               | 495    | 96.7 | 108    | 88.2 | 115  | 95.1   | 93.6 | 94.6 | 104  | 114  | 67.7 | 5.25  |
| scan: braid5_36               | >120 s | 1064 | 1208   | 887  | 1153 | 44995  | 7708 | 1080 | 1163 | 1491 | 691  | 68.1  |
| scan: chain256                | 312    | 28.8 | 24.9   | 29.7 | 26.9 | 24.8   | 335  | 24.1 | 24.2 | 25.3 | 19.3 | 0.97  |
| scan: conway_sum_2            | 1481   | 239  | 241    | 173  | 204  | 220    | 205  | 213  | 207  | 261  | 136  | 11.9  |
| rank: conway_sum_8            | >120 s | 31.5 | >120 s | 20.2 | 31.4 | >120 s | 16.0 | 29.3 | 51.7 | 18.6 | 20.7 | 1.90  |
| recognize: trefoil            | 0.13   | 0.19 | 0.19   | 0.13 | 1.67 | 0.15   | 0.15 | 0.28 | 0.16 | 0.15 | 0.27 | 0.008 |
| recognize: torus_3_5          | 0.98   | 0.34 | 1.03   | 0.32 | 2.24 | 1.06   | 0.80 | 1.04 | 0.32 | 0.66 | 0.56 | 0.027 |
| recognize: hard_unknot_8      | 2.84   | 5.05 | 2.40   | 4.02 | 3.86 | 2.45   | 3.07 | 2.18 | 2.16 | 3.03 | 2.78 | 0.056 |
| recognize: unknot_braid40     | 11.2   | 8.00 | 0.41   | 7.82 | 8.31 | 7.64   | 7.50 | 7.93 | 8.64 | 8.42 | 0.17 | 0.005 |
| recognize: conway             | 58.8   | 1.53 | 0.89   | 1.79 | 4.01 | 3.50   | 4.05 | 1.57 | 3.95 | 1.20 | 1.27 | 0.055 |
| recognize: kinoshita_terasaka | 65.6   | 2.12 | 1.02   | 1.49 | 3.43 | 1.55   | 3.31 | 1.08 | 1.81 | 0.72 | 1.25 | 0.030 |
| recognize: conway_sum_2       | 1817   | 3.54 | 2.05   | 3.47 | 2.75 | 2.32   | 12.6 | 2.07 | 2.77 | 2.17 | 1.56 | 0.059 |
| recognize: conway_sum_3       | 58090  | 6.77 | 2.96   | 5.15 | 3.60 | 3.43   | 35.3 | 2.10 | 2.78 | 2.51 | 2.47 | 0.099 |
| recognize: T(3,61)            | 4747   | 3.71 | 12.5   | 5780 | 13.8 | 19.0   | 5811 | 22.3 | 5.91 | 8.80 | 8.84 | 0.19  |
| recognize: braid5_36          | 9.89   | 2.53 | 0.69   | 2.38 | 8.63 | 2.63   | 11.2 | 6.20 | 2.40 | 2.37 | 0.90 | 0.023 |

All eleven Python engines and the Rust binary return identical ranks and verdicts on every input. Observations:

- The stress case **braid5\_36**, which 0.1 abandoned after 600 seconds, takes about a second wherever min-fill pivots are used, 45 seconds in proposal 05 (LIFO kept) and 7.7 seconds in proposal 06 (LIFO plus the tail trick). Its reduced rank is 2949. **This corrects our own 0.1 analysis:** we had attributed the timeout to large minimal complexes of the intermediate tangles. The minimal complexes are small (at most 5898 objects); what exploded was fill-in caused by the pivot order.
- Proposal 03 has the fastest raw scanner, which is why its representation was adopted.
- On  $T(3,61)$  proposals 03 and 06 take 5.8 seconds. Proposal 06 runs the symbolic Alexander polynomial before anything modular. Proposal 03 is more instructive: it evaluates the minor at  $t = 2$  modulo  $2^{31} - 1$ , where 2 has multiplicative order 31, so the set of “units”  $\{\pm 2^k\}$  has only 62 elements and is closed under the cyclotomic arithmetic of torus knots;  $\Delta_{T(3,61)}(2) \equiv -2^{29}$  and the filter misses. Our own first implementation had the same structural weakness (2 has order 61 modulo  $2^{61} - 1$ ) even though it happened to fire. Version 0.2 and the Rust port evaluate at fixed generic field elements instead, for both the Alexander and the Jones test. The filters are one-sided, so this was never a soundness issue, only a lost speedup.
- Tiny inputs get slower: the trefoil goes from 0.1 ms to 0.2–0.3 ms in most engines, because more stages run before one fires.

## 8 fastunknot 0.2

### 8.1 What was adopted

Version 0.2 (**fast/**) integrates every proposed idea that produced a measurable speedup, re-implemented in the structure of the existing package; the 0.1 scanner is kept unchanged as **scan\_reference** so that each idea can be switched off and so that the new code has an independent oracle.

The pipeline is now: validation; incremental R1/R2 reduction; linear descending test; visible connected-sum factorization (summands examined smallest first, the first knotted one decides, and a sum is trivial exactly when all summands are); then per summand the Alexander minor at  $t = -1$  and at a generic field element modulo  $p = 2^{61} - 1$ , the Kauffman bracket at a generic  $A$  modulo  $p$  (skipped when it exceeds 4096 frontier states), the exact Alexander polynomial over

$\mathbb{Z}[t]$ , and finally the Khovanov scan with bit-packed morphisms, compiled plans and min-fill pivots. Every filter is one-sided: only a mismatch with the unknot’s value gives a verdict, and that verdict is KNOTTED.

Not adopted: the exact integer bracket (equality with the unknot’s value is inconclusive whether it is computed modulo  $p$  or over  $\mathbb{Z}$ , so it cannot change a verdict, and it showed no measurable effect); replay commands (useful, but not a speedup); tails longer than one crossing (slower, see below).

## 8.2 Ablation

`fast/ablation.py` measures each idea in isolation, again in fresh processes (limit 300 s). First the scanner, in milliseconds:

| Scanner configuration      | conway | braid3_40 | braid4_41 | braid6_31 | braid5_36 |
|----------------------------|--------|-----------|-----------|-----------|-----------|
| 0.1 reference scanner      | 42.9   | 1115      | 1264      | 612       | >300 s    |
| sets, LIFO, series inverse | 44.1   | 1044      | 1206      | 618       | >300 s    |
| sets, LIFO, self-inverse   | 43.0   | 703       | 1113      | 618       | >300 s    |
| sets, min-fill             | 43.0   | 594       | 845       | 553       | 10376     |
| bits, LIFO                 | 11.4   | 106       | 203       | 73.4      | 35622     |
| bits, min-fill (default)   | 11.1   | 103       | 224       | 78.6      | 670       |
| bits, min-fill, tail 1     | 10.3   | 97.0      | 232       | 81.8      | 566       |
| bits, min-fill, tail 2     | 13.7   | 105       | 225       | 84.6      | 815       |

Then the polynomial-time stages and the filters:

| Task                                        | new (ms)                  | old (ms)                         | ratio |
|---------------------------------------------|---------------------------|----------------------------------|-------|
| greedy order, 2048 crossings                | heap $O(n \log n)$ : 32.6 | 0.1 rescoring $O(n^2)$ : 26685   | 818×  |
| descending test, 2048 crossings             | linear sweep: 3.72        | 0.1 quadratic: 314               | 84×   |
| R1/R2 reduction, 2048 crossings             | incremental: 3.77         | 0.1 rebuild per move: 28089      | 7458× |
| $T(3,61)$ rejection, 122 crossings          | modular minor: 8.07       | 0.1 exact $\mathbb{Z}[t]$ : 5829 | 722×  |
| recognize Conway                            | with Jones filter: 0.81   | without Jones filter: 10.0       | 12×   |
| recognize Conway#Conway#Conway,<br>no Jones | with factorization: 12.6  | without factorization: 6781      | 539×  |
| rank of Conway#Conway#Conway                | factored: 20.7            | one scan: 6972                   | 337×  |

What the ablation shows:

- **The bit-packed algebra** is the across-the-board gain: 4 to 10 times on every scan, with either pivot rule.
- **Min-fill pivots** matter on exactly one kind of input, and there they are decisive. On the stress case the bit-packed scanner needs 35.6 s with LIFO pivots and 0.67 s with min-fill; the set-based scanner goes from more than 300 s to 10.4 s. On the other four inputs the pivot rule is neutral to slightly negative (the heap costs a few percent). The final complex is the same in all configurations, so the difference is fill-in during cancellation, not the size of the homology.
- **Self-inverse units** save between nothing and 30%, depending on how many cancellations involve a non-identity unit.
- **The tail trick** is mixed: one crossing without cancellation is 16% faster on the stress case and 6% faster on one other input, a few percent slower on the rest; two crossings are slower almost everywhere because the uncanceled complex quadruples. It is kept as an option (`tail`) with default zero.
- **Heap ordering, incremental reduction, modular Alexander test**: three to four orders of magnitude at 2048 crossings and at  $T(3, 61)$ . These are asymptotic improvements of polynomial

stages: the old code spent 27 s on ordering and 28 s on R1 moves for a diagram that reduces to nothing.

- **The linear descending test** needs an adversarial input to show (a 2048-crossing chain with one crossing flipped: 3.7 ms against 314 ms); on typical inputs the old test exits early and the two are equal.
- **The Jones filter** turns recognition of the Conway knot from a 10 ms Khovanov scan into a 0.8 ms bracket evaluation, and **factorization** turns a 6.8 s scan of three Conway summands into 12.6 ms even with the Jones filter disabled.

### 8.3 Complexity: what changed and what did not

The polynomial stages improved asymptotically: scan ordering from  $O(n^2)$  to  $O(n \log n)$  per start, the descending test from  $O(n^2)$  to  $O(n)$ , R1/R2 reduction from one  $O(n)$  revalidation per move to  $O(1)$  per move, and the Alexander rejection from fraction-free elimination over  $\mathbb{Z}[t]$  to  $O(n^3)$  field operations.

Two statements with an exponential flavour are now *proved* rather than observed:

**Proposition 8.1.** *The Jones filter costs  $O(n \cdot (w - 1)!! \cdot \text{poly}(w))$  field operations for a scan of width  $w$ .*

*Proof.* After each crossing the state is a map from perfect matchings of at most  $w$  boundary points to  $\mathbb{F}_p$ ; each entry spawns two successors. The earlier Catalan claim required an unproved common-disk hypothesis. Report 09 supplies explicit counterexamples; the October integration corrects this statement without changing the exact bracket evaluation.  $\square$

**Proposition 8.2.** *If the visible factors of the reduced diagram have  $n_1, \dots, n_k$  crossings, recognition and the factored rank cost  $\text{poly}(n) + \sum_i 2^{O(n_i)}$ . In particular the cost is polynomial when every  $n_i = O(\log n)$  and quasi-polynomial when every  $n_i = O(\log^2 n)$ .*

This is a statement about a restricted class of diagrams (several proposals make it too), not about unknot recognition: a composite knot can be drawn without any two-edge cut, and prime knots gain nothing. The family  $\#_k K11n34$ , which Section 6 used to show that a scan cannot be subexponential if it must output the whole homology, is exactly the family that factorization now handles in polynomial time; the obstruction moves to diagrams in which the summands are hidden.

The general worst case of the exact fallback is unchanged,  $2^{O(n)}$ , and the  $n^{O(\log n)}$  target of Section 5 is no closer: nothing here touches the hierarchy machinery.

### 8.4 Verification

The package now has 33 unit tests. The 14 new ones compare bit-packed with set-based composition on random planar matchings, check every unit of a three-circle endomorphism ring, run all scanner configurations against the 0.1 scanner, check that heap orders equal the 0.1 orders from every start, check sign and filter invariance under half-turns of PD rows, check that no filter fires on any unknot diagram and that every filter hit has reduced Khovanov rank above one, check  $33^k$  for  $k = 2, 3, 8$  Conway summands and equality of factored and unfactored degree counts, replay simplifier traces, and recompute the stress case (rank 2949, with and without the tail).

## 9 A Rust implementation

### 9.1 Design

`rust/` is a dependency-free crate (about 2500 lines) implementing the 0.2 pipeline except the exact Alexander polynomial over  $\mathbb{Z}[t]$ , which would need big integers and on which exactness never depended. It was designed for Rust rather than transliterated:

- matchings are interned, so objects, plans and caches hold 32-bit ids;
- a morphism is a bitset over dot masks, a single inline `u64` when the Hom space has at most six circles (every scan of width at most twelve) and boxed words otherwise;
- composition and transfer plans are a few integer masks per surface component, evaluated with popcounts;
- memo tables use an Fx-style multiplicative hasher; measured against the standard SipHash it is worth 22% on the stress case (65 ms against 86 ms) and 18% on two Conway summands;
- objects live in flat vectors, the min-fill queue is a binary heap, and field arithmetic modulo  $2^{61} - 1$  uses 128-bit products;
- factored ranks such as  $33^{16}$  use a small base- $10^9$  integer.

### 9.2 Verification

Four unit tests (known ranks under both pivot rules, tails 0–2 and mirrors; input validation; one-sidedness of the filters; the pipeline), and `rust/cross_check.py`, which compares ranks by cube degree in three configurations and verdicts in two with the Python package on random braid closures: 200 closures, no disagreement.

### 9.3 Profile

`rust/profile.py` records the binary’s own timers (medians of five runs inside one process, after parsing). The comparison columns of the table in Section 7 show the Rust binary against the Python engines on identical inputs. Larger random braid closures, raw scan, one run each with a 300-second budget:

| Input                         | reduced rank              | boundary | objects | total s | transfer s | eliminate s |
|-------------------------------|---------------------------|----------|---------|---------|------------|-------------|
| random 3-braid, 80 crossings  | 5189                      | 6        | 12240   | 0.235   | 0.101      | 0.122       |
| random 3-braid, 160 crossings | budget of 300 s exhausted |          |         |         |            |             |
| random 4-braid, 61 crossings  | 643                       | 8        | 3310    | 0.113   | 0.055      | 0.055       |
| random 4-braid, 81 crossings  | budget of 300 s exhausted |          |         |         |            |             |
| random 5-braid, 48 crossings  | 39                        | 10       | 450     | 0.015   | 0.011      | 0.004       |
| random 5-braid, 60 crossings  | 481                       | 10       | 7398    | 4.663   | 0.525      | 4.108       |
| random 6-braid, 43 crossings  | 177                       | 10       | 530     | 0.023   | 0.014      | 0.008       |
| random 6-braid, 55 crossings  | 1171                      | 10       | 13294   | 1.484   | 0.434      | 1.021       |
| random 7-braid, 48 crossings  | 29                        | 10       | 1838    | 0.070   | 0.032      | 0.036       |
| random 8-braid, 49 crossings  | 1709                      | 12       | 3418    | 0.309   | 0.153      | 0.149       |

What the profile shows:

- On identical inputs the Rust scanner is 10 to 17 times faster than Python 0.2 (Conway 0.59 ms against 8.4 ms, the 40-crossing 3-braid 6.9 ms against 84 ms, the stress case 68 ms against 691 ms) and recognition takes between 5 and 200 microseconds. Against version 0.1 the stress case went from more than 600 s to 0.068 s.
- Time splits roughly evenly between adding crossings (gluing, delooping, transfer plans) and cancellation, except when fill-in dominates: with LIFO pivots the stress case takes 2.1 s in

Rust, 2.03 s of it in cancellation, against 0.068 s with min-fill. The pivot rule matters as much in Rust as in Python; on the other inputs it is again neutral.

- One uncanceled tail crossing saves 5 to 20% in Rust as well.
- **The exponential wall is still there.** A random 3-strand closure with 80 crossings has reduced rank 5189 and takes 0.23 s; with 160 crossings the scan exhausted 300 s. A 4-strand closure with 61 crossings (rank 643) takes 0.11 s; with 81 crossings it exhausted 300 s. The scan width is 6 or 8 throughout, so this is the multiplicity factor of Section 6 and not the boundary: the rank of the homology itself grows exponentially with the length of a random braid word, and any algorithm that writes the homology down inherits that. (Both runs overshoot their cooperative deadline by 40 to 70 seconds, because the deadline is checked between cancellations.) For *recognition* these inputs are easy, since the Alexander test rejects them in microseconds; the hard inputs for recognition are diagrams of the unknot and of knots with trivial Alexander and Jones values at our evaluation points, where the scan must run to completion.

## 9.4 Where the three versions stand

| Task                                           | Python 0.1 | Python 0.2 | Rust     |
|------------------------------------------------|------------|------------|----------|
| Scan, stress closure (36 crossings, rank 2949) | > 600 s    | 0.69 s     | 0.068 s  |
| Scan, Conway knot                              | 34 ms      | 8.4 ms     | 0.59 ms  |
| Recognize Conway knot                          | 59 ms      | 1.3 ms     | 0.055 ms |
| Recognize Conway#Conway#Conway                 | 58 s       | 2.5 ms     | 0.099 ms |
| Recognize $T(3, 61)$                           | 4.7 s      | 8.8 ms     | 0.19 ms  |
| Rank of eight Conway summands ( $33^8$ )       | > 120 s    | 21 ms      | 1.9 ms   |
| Recognize the 8-crossing hard unknot           | 2.8 ms     | 2.8 ms     | 0.056 ms |

The last row is the honest one for the original question: on an unknot diagram that no filter can decide, the accelerations of the Python version change nothing, because the Khovanov scan still has to run; only the constant factor of the Rust port helps. (This describes 0.1 against 0.2. The later work of Section 10 does speed up the scan itself, and this case with it, by a factor of 1.4.)

## 10 A third round: constant factors in Python, and how to measure them

After version 0.2 the Rust work was paused and the Python package was optimized further, with no change to the mathematics, the verdicts or the asymptotic behaviour. This section records what was gained, what was tried and rejected, and an episode about measurement that changed how every number below was obtained. All verdicts and ranks are exact and identical to those of 0.2 (the deciding step can differ: the Reidemeister III help below settles some unknots before the scan): 41 unit tests pass, and `crosscheck.py` compares the ranks by homological degree of the new default scanner with the generic 0.2 scanner and the 0.1 set algebra on random braid closures (400 closures with 2 to 5 strands, no mismatch).

### 10.1 What changed

A profile of 0.2 on the stress closure showed that the cobordism algebra was no longer the bottleneck: of 1.6 s under the profiler, the scanner’s own bookkeeping (method calls per differential entry, tuple-keyed dictionaries, hashing of nested frozensets, a heap of pivot candidates) took most of it.

- **A scanner written for CPython** (`scan_fast.py`). Objects of a stage are numbered  $0, \dots, N - 1$  and live in lists; the objects created from one old object form a contiguous block,

so a delooping label is an offset; matchings are interned as integers, so every cache key is a tuple of small integers; transfers of an edge are cached per (source matching, target matching, value) for both smoothings at once; cancellation is one inlined loop. The generic scanner of 0.2 is kept for the ablation configurations.

- **A bucket queue for pivots.** On the stress closure 114 197 unit entries are candidates but only 16 884 are used; the rest die when a neighbour is cancelled. Markowitz costs are tiny, so one list per small cost plus a heap for the rare large ones replaces the heap.
- **Integer planar geometry** (`planar.py`). Circles of  $a \cup \bar{b}$  are found by walking partners alternately; a crossing is glued by following the at most four strands through it; a transfer plan is compiled only over the crossing and the circles it touches, the other circles being renumbered; and since monomials of different components live on disjoint circles, multiplying a bit-packed sum by a monomial  $c$  is the shift  $\ll c$ .
- **Outside the scanner.** The modular Alexander filter builds the matrix by sparse rows (at most three nonzero entries per row) and evaluates each distinct entry once; the boundary profile of a greedy scan order is tracked inside the greedy in  $O(1)$  per step and a losing start is abandoned early; the Jones frontier scan uses the interned matchings and the strand-following gluing. Each was checked to be output-preserving against the previous implementation on 500 to 600 random diagrams, including diagrams with kinks.

## 10.2 Results

Same harness on both sides (`perf.py`: in one process, best of  $N$ , timers around the call only), so the columns are comparable with each other but *not* with the fresh-process medians of the previous section.

| Task (milliseconds)                            | 0.2    | after the scanner work | ratio |
|------------------------------------------------|--------|------------------------|-------|
| Scan, stress closure (36 crossings, rank 2949) | 575.4  | 216.8                  | 2.65  |
| Scan, 41-crossing 4-strand closure             | 135.3  | 51.1                   | 2.65  |
| Scan, 40-crossing 3-strand closure             | 71.0   | 26.2                   | 2.71  |
| Scan, 31-crossing 6-strand closure             | 59.3   | 21.4                   | 2.77  |
| Scan, Conway#Conway (unfactored)               | 114.9  | 41.7                   | 2.75  |
| Scan, $T(3, 11)$                               | 22.8   | 8.7                    | 2.61  |
| Scan, Conway knot                              | 7.7    | 3.0                    | 2.54  |
| Scan, chain of 256 kinks                       | 15.9   | 10.9                   | 1.47  |
| Recognize the 8-crossing hard unknot           | 1.26   | 0.90                   | 1.41  |
| Recognize Conway knot, exact steps only        | 7.8    | 3.2                    | 2.43  |
| Total of the 14 cases of <code>perf.py</code>  | 1022.2 | 394.6                  | 2.59  |

The right-hand column is a run whose control loop (see below) spread by 4.8%. The changes outside the scanner came later and were measured as paired ratios new/old with 95% confidence intervals, which do not need a quiet machine: recognition of  $T(3, 61)$  0.60 [0.53, 0.67] and then 0.69 [0.62, 0.72], together about 0.41; the chain of 256 kinks 0.78 [0.73, 0.82]; the Jones scan of the stress closure 0.67 [0.46, 0.72]; recognition of the Conway knot 0.92 [0.89, 0.93]. Four later sequential runs were all rejected by the control rule and are not used; they are flagged in `perf_log.json`.

**A cache that began as a trade-off.** The last change was at first not a free win; how it became nearly one is told in order. Every crossing brings new edge labels, so plans cached per stage are never reused across stages, although a long braid compiles the same few pictures at every crossing: at boundary size 6 there are five matchings, and about 35 transfer plans were compiled per crossing of  $T(3, n)$ . Transfer plans are now also cached under a label-independent key, the matchings and the crossing relabelled by rank. This is sound because the relabelling is monotone and circles are numbered by their minimal label whichever matching is walked first (tested on

random matchings). The reuse is 80% of the plans for  $T(3, 11)$ , 94% for  $T(3, 31)$ , 99% for the chain of kinks, 38% and 19% for the random 3- and 4-strand closures, but 1% for the stress closure and none for the Conway knot. With the cache always on, three agreeing A/B runs: scan of  $T(3, 11)$  0.61 [0.59, 0.66], chain of kinks 0.78, 40-crossing 3-strand closure 0.97; *scan of the Conway knot* 1.06 [1.04, 1.07] *and recognition of the hard unknot* 1.07, *slower*; no claim on the large scans. Timing partial variants against the old code (A/A ratio 0.999) splits the cost into ranking the boundary at each stage (+1.5%), building and looking up the key (+3.0%) and storing (+1.3%). Two explanations offered first were wrong: interning the shapes as integers, against the hashing of nested tuples, changed nothing, and the ratio was the same with the cyclic garbage collector disabled; `shape_cache=False` restores the previous speed exactly (1.003 [0.999, 1.007]).

A first attempt at an automatic switch failed, and I concluded too early that none was possible: an early hit count predicts the wrong way, since the inputs that profit get their first hit only after 35 to 150 lookups, while the stress closure gets 12 hits in its first 64 lookups and then none. A different predictor works. A plan can be reused only in a stage whose picture, the position of the four crossing slots among the boundary points up to monotone relabelling, has occurred before, and that is known from the scan order alone, before the scan. The number of such stages is 0 for the Conway knot, the Kinoshita–Terasaka knot and Conway#Conway, 1 of 36 for the stress closure, 4 of 31 and 11 of 41 for the 6- and 4-strand closures, 16 of 40 for the 3-strand closure and 16 of 22 for  $T(3, 11)$ . The default now turns the cache on when the diagram has at least 16 crossings and an eighth of its stages repeat. Transfer and composition *results* are shared across stages too, but only after the first reused plan of a scan, because before that no shared result can exist; without that gate they cost a further 2.6% on the small irregular scans. Against the code before any of this, with 301 interleaved pairs: scan of  $T(3, 11)$  0.505 [0.486, 0.508], Conway knot 1.008 [1.005, 1.010], hard unknot 1.007; on the corpus,  $T(3, 11)$  0.51, chain of kinks 0.81, and nothing slower.

**A stage that was not needed.** The benchmark corpus contained one input that no filter decides, the hard unknot with 8 crossings, and that hid the cost of a whole stage. A generator of scrambled unknot braids (`hard_unknots.py`: the closure of  $v\sigma_1\cdots\sigma_{n-1}v^{-1}$  rewritten by random braid relations) gives diagrams of which about half survive the Reidemeister I/II reduction with 15 to 27 crossings and pass every filter. On 26 of them the exact Alexander polynomial over  $\mathbb{Z}[t]$  took a median 24%, and up to 63%, of the recognition. That stage can only answer KNOTTED, and only for a polynomial that is not a unit, which the modular test has just failed to see at  $t = -1$  and at a generic point of  $\mathbb{F}_p$ ; and whatever it might still catch, the exact scan decides anyway, so verdicts cannot change. It now runs only when the modular test did not. Over 720 diagrams the two modes never disagree, and the exact stage decided nothing once the modular test had passed. Recognition of the 55 scan-decided scrambled unknots is faster on all 55: median ratio 0.81, range 0.53 to 0.93.

**Reidemeister III, the one algorithmic gain of this round.** The scan is exponential in what the reduction leaves, and the reduction used to stop wherever a Reidemeister III move was needed. On the dart structure of the reduction a triangular face is a 3-cycle of the face walk through three distinct crossings; it can be inverted if and only if some side passes over at both ends; and inverting it only reverses the order in which each of the three strands meets the other two, a relinking of at most twelve darts that keeps every crossing's slots and the direction of both strands through it, hence its sign. When no I/II move is left, each such triangle is inverted on trial, kept if that creates a I/II move and undone otherwise (the move is an involution). The move was tested on its own before it was used: 150 random moves give valid spherical diagrams with the same Alexander polynomial, writhe and Khovanov ranks by degree, and a second move restores the structure exactly (300 of 300). On 1500 random closures the helped reduction never leaves more crossings than I/II alone, leaves no I/II move, replays from its trace, and preserves the

Alexander polynomial and the Khovanov rank; 26% of the diagrams shrink further and 0.80 of the crossings remain. The 8-crossing “hard unknot” used throughout this report goes from 6 crossings to none, and so do all 120 scrambled unknots above, which is no surprise, their scrambling being made of such moves. Honest hard inputs had to be found by search: 117 closures among 40 000 random words that survive I, II, the III search and the Alexander filter, and are unknots by reduced Khovanov rank 1, with up to 21 crossings. Verdicts with and without the search agree on 900 diagrams.

Two lessons came with it. *Placement*: run together with the first reduction, the search cost 58% on  $T(3, 61)$ , 20% on a random 5-strand closure and 4 to 6% on the Conway knot, inputs that a filter decides a moment later. It now runs only after every filter has failed, immediately before the scan; then no slowdown can be claimed on those inputs, while recognition of the hard unknot takes 0.48 and of a scrambled unknot of 27 crossings 0.18 of the time. *Fewer crossings is not always less work*: one survivor was flagged 1.43 times slower, correctly, since the help takes it from 22 to 20 crossings and its scan work from 2081 to 3277. Over the 54 survivors where crossings are removed (3.4 on average) the scan work is 0.43 of what it was, cheaper on 44 and dearer on 10, with per-diagram ratios from 0.03 to 7.3; over all 117 it is 0.69.

*Depth, and a budget*. One move of lookahead left those 117 unknots untouched. Sequences of two moves, the second next to the first, reduce 97 of them to nothing, three moves 109 and four moves 110, without a violation at any depth. What costs is a search that fails: on  $T(3, 61)$ , full of triangles that lead nowhere, depths 1 to 4 take 2, 10, 44 and 189 ms. Useful sequences are found early, so the search watches its own cost and deepens only while fewer than a fixed number of trial moves per crossing have been made; budgets of 5, 15, 40 and none give the same 97, 109 and 110, and a budget of 5 caps the  $T(3, 61)$  case at 10 ms. With the defaults (depth 4, budget 10) only 15 of 60 000 random closures are unknots that survive the reduction, the largest with 16 crossings.

**Introspective selection and racing (two suggestions of the user).** The first suggestion was to choose algorithms from a runtime profile, as introsort does. It works where a cheap signal separates the populations and switching is free, and three parts of the package are now of that kind: the shape cache chosen from the scan order, result sharing enabled by the first reused plan, and the budgeted deepening just described. For scan *orders* it fails with the rules at hand, in both forms tried. Restarting with another tie-break rule when a growing work budget is overrun was simulated exactly on the saved work of three rules on 81 diagrams: all 27 policies cost 1.4 to 4.8 times the default. Introsort can switch because heapsort carries a guarantee and recursion depth signals degeneracy; here no rule has a guarantee, expensive is not degenerate, and an abandoned scan is lost. Without restarts (the previous stage is never mutated, so tied candidates can each be tried from the same state and the smallest real complex kept) the cost with all trial work counted is 0.41 on the scrambled unknots but 1.44 to 1.52 on random closures and about 1.4 overall: the size of the complex now does not predict the cost of the choice later.

The second suggestion, racing two algorithms on separate cores, is where the same data does promise something, because by wall clock a race costs only the winner’s time: the better of two rules needs 0.75 of the default on the expensive inputs, and 0.07, 0.24 and 0.30 on single ones. In Python a competitor has to be a process, and starting one costs 30 ms bare, 76 ms with the package imported and 111 ms for a complete scan of the trefoil from the command line, against scans that mostly finish within 300 ms. Following the user’s instruction for that case, the race was built in the Rust version, where a thread costs microseconds, together with a port of the Reidemeister III search: up to three greedy orders (ties by crossing index, by longest wait, by most recent touch) are scanned on separate threads and the first to finish stops the others through a flag; the rank does not depend on the order, so the result is the same whoever wins. On 40 random closures of 34 to 55 crossings whose single scan takes 30 ms to 8 s, with the settings interleaved over five rounds, racing two orders needs 0.650 of the total time and racing three 0.689. The gain is insurance against the heavy tail and not uniform: per input the ratio runs from 0.05

to 1.68; 6.8 s become 0.34 s and 890 ms become 140 ms, but on the half of the inputs where the default order was the best anyway the race costs 10 to 60%. I first blamed the shared heap for that premium and was wrong: two or three separate processes running the same single scan at once slow each other down exactly as much as the racing threads (1.15 against 1.18, 2.11 against 2.06), so it is contention in the hardware of this machine and will differ elsewhere. The race is therefore an option (`-race 2`), not the default.

Dismissing a race in Python was, on reflection, a wrong comparison: the cost of starting a process was set against the typical scan, while a race only matters on the heavy tail, where Python scans take seconds to minutes. The Python package therefore has it too, in the form the two suggestions combine to: the default order runs alone for a head start of one second, and only then are other greedy orders started as separate processes, which the scanner polls where it looks at the clock; the first to finish wins. On 26 random closures with single scans of 0.15 to 3.6 s the total is 0.910 of single with two orders and 0.925 with three. That total consists of two inputs on which a competitor wins in every round (3.22 s to 1.74 s, 3.55 s to 1.56 s) and of a cost of 10 to 20% on scans of 1 to 3 s that the default order wins. The other per-input ratios of that experiment are noise, and the data says so itself: 18 inputs finish inside the head start, where nothing is started and the ratio must be 1, yet they read 0.80 to 1.43 with three rounds each. Measured with 61 interleaved pairs, an armed race that starts nothing costs 1.010 [0.984, 1.041] against an A/A control of 1.042, that is, nothing.

**Start-up, the largest gain for a command-line user.** Everything above saves well under a millisecond on a small diagram, while a whole `python -m fastunknot recognize` process took 115 ms, 29.5 of them for the bare interpreter. Importing the package cost 56 ms: 25 for `dataclasses`, which loads `inspect`, `re`, `dis`, `ast` and `tokenize`, used for five simple records (now written out by hand with the same constructors, equality, hashing, immutability and `repr`); 9 for `subprocess` and `json`, needed only when a race starts; 5 for `typing`, used only in annotations that are never evaluated. Then `argparse`: 55 to 59 ms in Python 3.14, where it loads `_colorize`, `dataclasses` again and `shutil`. The options are now one table feeding a small direct parser for the well-formed common case and, on demand, the real `argparse` parser for everything else, so help and error messages are unchanged; tests check that the two agree wherever the fast one accepts. With 41 interleaved pairs of whole processes a run takes 0.52 of the time (0.510 to 0.541; 115.5 to 60.9 ms for the Conway knot) on every short invocation and 0.86 on the half-second stress scan. My first two such comparisons showed no difference and were invalid: `python -m` puts the current directory first on the module path and both arms ran from the working tree, which was thus compared with itself.

Profiling `recognize` by cumulative time, rather than the scanner by own time, then found four more things of the same unglamorous kind. The Jones filter paid for the bookkeeping of a shape cache it never uses (9% of a recognition of the Conway knot); the reduction rebuilt and revalidated the diagram even when no move applied (12%); the dart involution of an immutable diagram was recomputed seven times per recognition; and on the commonest input of all, a reducible closure that the Alexander filter decides, a quarter of the time went into validating the reduced diagram from scratch, although the moves can only break its topology, which is now checked directly on the darts (one traversal through every crossing twice,  $n + 2$  faces; of 282 deliberately corrupted structures 273 are rejected and the other 9 also pass the full validation). Against the code before: recognition of the Conway knot 0.765 [0.733, 0.794], of  $T(3, 61)$  0.759, of `Conway#Conway#Conway` 0.731, and of the random 5-strand closure 0.865 and then 0.796 [0.772, 0.831], with status, deciding step, reduced size and rank identical on 700 diagrams and reduced diagrams and traces identical on 1500. The `tail` option was re-tested on the present scanner and stays at 0: one crossing of tail is a wash (0.94 to 1.01), two or three are worse, up to three times.

Two remarks. First, the closing sentence of the previous section, that on an unknot diagram no filter can decide the Python accelerations change nothing, was true of 0.1 against 0.2 and is no

longer true: that case is now 1.4 times faster, because the scan itself is. Second, the gain is a constant factor: between 2.5 and 2.8 on every scan in the table except the chain of kinks (1.5), where fixed work per crossing dominates. Nothing here touches the exponential growth, and the Rust port remains several times faster still.

### 10.3 Measured and not adopted

All of these were decided with deterministic counters (compositions performed during cancellation, differential entries written while adding crossings), and the ranks were identical in every variant.

- *One queued pivot candidate per source object.* Fewer queue operations, but 42 197 instead of 29 347 compositions on the stress closure: the queued candidate goes stale. Rejected.
- *Other pivot rules.* First-in-first-out inside a cost bucket needs 64% more compositions on the stress closure; the cost  $(\text{out} - 1) + (\text{in} - 1)$  is worse on all four inputs tried. The product cost with last-in-first-out stays.
- *Another score for scan orders.* Scoring greedy orders by  $\sum 2^{|\partial|/2}$  selects the same start as the current rule (maximal, then total boundary size) on all four inputs.
- *Scoring all greedy starts instead of twelve.* On 40 random closures the order changed in 18: equal work in 8, less in 6, more in 4, with ratios from 0.29 to 6.69. The total, 0.952, is carried by three inputs and changes sign without one of them. Not adopted.
- *Cancelling saddle isomorphisms before writing their entries.* Between 74 and 83% of the entries written are incident to a cancelled pair when it is cancelled, but only 30 to 53% of the cancellations are between the two smoothings of one old object, the predictable kind, and they account for 24 to 36% of the entries. That bounds the gain near 15% on the stress closure before the cost of the extra code. Not pursued.
- *A fast path for single-entry transfers.* It would apply to 13% of the edges of the stress closure: too little to measure. Not written.

The experiments on scan orders have a positive content. The score is the weak point, not the number of candidates: on the stress closure the greedy starts 14 to 16 need 23 710 compositions and have boundary profile (8, 226), while start 0, with profile (8, 230), needs 895 182, that is 38 times more, with 2.6 times the peak size; 20 of the 36 starts do not finish in 4 s. A matchings-only simulation cannot replace the score, because the gap occurs at equal boundary size, where there are at most 14 matchings: it is all multiplicity, i.e. homology of the partial tangle, which is what the computation computes. A cheap predictor of that multiplicity would be worth more than everything in this section.

A larger experiment, on 81 diagrams (36 scrambled unknots of the kind described above and 45 random closures with 20 to 35 crossings), confirms this and closes one more door. The greedy rule breaks ties by crossing index, which is arbitrary. Breaking them towards the most recently touched crossing costs 1.09 and 1.05 of the present total work on the two families, and towards the longest-waiting crossing 1.52 and 1.57; per diagram the ratios run from 0.01 to 28, better on about as many inputs as worse, so the rules are incomparable rather than ranked. A portfolio does not rescue this. The best of the three rules on each diagram would need 0.31 of the work on the unknots and 0.74 overall, but running them interleaved until the first one finishes costs three times the winner: 0.94 on the unknots and 2.2 overall, because 8 of the 81 inputs carry 86% of the work and there the rules lie within a factor two of each other.

### 10.4 An episode about measurement

Midway, a benchmark run showed everything 40% slower, including code that had not been touched. No process of ours was running; the desktop's own load had changed. A fixed pure-Python control loop, added to the harness, went from 5.3 ms to 12 ms *inside one run*. Sequential before/after timing was therefore abandoned for decisions: `perf.py` now times the control next to every case

and rejects a run whose control spreads by more than 10% (eight of the fifteen logged runs are rejected), and `ab.py` times the old and the new code alternately in one process.

The first criterion of `ab.py`, “the interquartile range of the paired ratios lies below 1”, was then caught making false claims in both directions. A change confined to order selection, which is 0.2 to 1.8% of the scans concerned and was measured on its own as 32 to 58% faster with identical output, was reported as 14% slower on one input and 4% faster on another. The bound came from asking what share of the run time the change could touch at all. The tool now times the old code twice per round; on mid-sized scans this A/A control shows identical code differing by 20 to 30% between adjacent runs. A difference is claimed only if the order-statistic 95% confidence interval of the median ratio excludes 1 *and* the median lies outside the interquartile range of the A/A ratios. The one claim that had been published under the lenient criterion (7 to 13% from the refinements of the geometry on small scans, no claim on large ones) was re-tested between the two revisions under the strict criterion and holds: 0.87 to 0.92 with intervals inside  $[0.87, 0.95]$ , no claim on the large scans.

A smaller finding of the same kind: the new scanner at first overshot a time budget of 1 s by up to 0.24 s (the generic scanner: 0.01 s), because a single crossing was added without looking at the clock. With a check every 512 objects the overshoot is at most 0.03 s on the same runs.

## 11 A port to Bend 2, and what its parallelism delivered

Bend 2 (version 2.0.10) is a pure, total, affine functional language that compiles through C to native code and promises parallelism for free: a parallel let `a b = f(x) g(y)` forks, the type checker guarantees that the two calls share nothing, and the result cannot depend on the number of threads. The algorithm of `fastunknot` was ported to it (directory `bend/` of the standalone Knots repository; the port was not carried into ProveIt when that repository was merged on 4 October 2026) to find out what that promise is worth for this problem. Nothing in this section changes the complexity discussion: the scan is exponential in the boundary width, in every language.

### 11.1 Setting

Bend runs on Linux, macOS and WSL only, so all Bend work ran over SSH on an Ubuntu virtual machine with 6 virtual CPUs of an i7-3770. That CPU has no AVX2 and the distributed binary died with an illegal instruction; Bend was run from source under the baseline build of its JavaScript runtime instead. All times below are from that host, and the Python package was copied there for every comparison, so the two sides of a ratio always share the hardware. Timings are interleaved with an A/A control as in Section 10; the control’s ratio stayed within 0.96 to 1.05.

### 11.2 What was ported, and how it was checked

- the modular Alexander filter (Wirtinger matrix, determinant modulo 65521, because U32 is the only machine integer);
- the planar geometry, the dotted cobordism algebra over  $\mathbb{F}_2$  and Bar-Natan’s scan with delooping and Gaussian elimination, that is `scan_fast.py` with, of its caches, only the transfer plans;
- a greedy scan order from eight starting crossings;
- a standalone recognizer that chains the three.

Not ported: diagram simplification, the Jones filter, factorization, and the cache of composition plans. Each layer was compared with its Python counterpart before the next was written: 200 Alexander minors, 1920 gluings and circle numberings, 2736 compositions and transfers, the Khovanov ranks of 160 random closures and of the timing inputs (up to rank 5898), and the verdicts of the recognizer on 60 diagrams with up to 40 crossings, 40 of them unknots, among

the knots Conway’s and the Kinoshita–Terasaka knot: no mismatch anywhere. Eighteen laws about the control logic (for instance: of two scan orders with equal profile the left one wins, so the work does not depend on scheduling) are stated in `LAWS.bend` and proved in `PROOF.bend`; they were drafted by the AI as proposals, and they do not say that a rank is right, which Bend cannot express about U32 arithmetic.

### 11.3 What the fork delivers

Measured on kernels first: a deep, balanced, uniform recursion reaches 5.0 on 6 threads; 32 coarse leaves reach 2.1; every switch from sequential to parallel execution costs about 80 microseconds; forks inside the sparse elimination of the Alexander determinant made it 8 to 15 times *slower*; one fork over four independent determinants gained nothing. Tasks are dealt once and never moved, and there is no cancellation, so the race of scan orders, the best parallel gain of the Rust port (0.65 of the time in total), cannot be written.

In the scan, the transfer of all objects of a stage through a crossing is a map that shares nothing, and it was written as a fork over the tree of the array. Its first effect was negative: with six threads the scan took 1.28 to 1.85 s instead of 1.04 s. The cause was found in the runtime source and the emitted C, and then confirmed by measurement: generated code makes a non-tail call either by pushing a stack frame or, in “parallel mode”, by allocating the continuation as a task on the heap; a program runs in parallel mode wherever a fork is reachable, which for a program with one fork is everywhere. With the forking transfer *switched off* and six threads the scan took 1.85 s: a factor 1.8 for the mere presence of a fork. The remedy is to make every long sequential phase a task of its own, a fork next to a trivial second child, since a task whose function reaches no fork runs in the fast mode. Three natural ways of writing that child were silently compiled away (inlined, constant-folded, dropped as unused), leaving no fork; it has to be recursive, depend on run-time data, and its result has to be used. After that: 1.02 s on one thread, 0.81 s on six; the transfer alone became 2.7 times faster.

### 11.4 Where the time goes

| version                                    | braid closure, rank 218 |             | stress closure, rank 5898 |           |
|--------------------------------------------|-------------------------|-------------|---------------------------|-----------|
|                                            | 1 thread                | 6 threads   | 1 thread                  | 6 threads |
| first working scan, first-found pivots     | 1.04 s                  | 1.28–1.85 s | 63 s                      | 58 s      |
| sequential phases as tasks                 | 1.05 s                  | 0.89 s      | 63 s                      | 58 s      |
| least-fill pivots in batches, work lists   | 0.65 s                  | 0.50 s      | 7.0 s                     | 4.9 s     |
| one transfer plan per pair of matchings    | 0.36 s                  | 0.39 s      | 4.07 s                    | 3.9 s     |
| composition plans once per source matching | 0.32 s                  |             | 3.59 s                    |           |
| one alternative evaluated instead of all   | 0.24 s                  |             | 2.65 s                    |           |
| number of sources kept, not counted        | 0.23 s                  | 0.25 s      | 2.20 s                    | 1.99 s    |
| Python package, same host                  | 0.134 s                 |             | 0.542 s                   |           |

The largest gain was algorithmic and sequential: cancelling only pivots whose fill-in is within a limit, and raising the limit to the least fill-in that was skipped, brought the stress closure from 58 to 5 s. A sampling profile of the emitted C then showed 43% of the time in reference counting: a value used more than once is shared and counted, and every step through a shared list bumps and drops counts. Matchings are shared by design, and the linear lookups in them were most of those steps; a differential profile put the transfers at 48 to 61% of the single-thread time.

Python avoids that work by interning matchings and caching every plan (2817 plans serve 251 406 transferred entries on the stress closure). In Bend a cache is a value every call must thread, the

only map has string keys, and an array cannot be shared between the branches of a fork. The port therefore splits a transfer into its plan, which depends on the two matchings only, and its application to a morphism, and before the forked map of a stage a sequential pass builds a table with one entry per pair of matchings that occurs: a binary trie over 20 bits of a hash of the pair, which is plain data and may be shared, read-only, by all branches. A leaf remembers its pair, and a lookup that finds another pair or nothing computes the plan on the spot, so a collision costs time and never correctness (two of the laws). That halved the single-thread time (0.53 and 0.56, A/A control within 0.98 to 1.01).

Two further findings were about the language rather than the algorithm. Bend is strict and has no `if: Bool.pick` evaluates both alternatives and `&&` both sides. The port had used them as conditionals, and so ran twelve rounds of list work per surface component whether needed or not, compared matchings for every entry of every visited row, and iterated a connectivity closure sixteen times after it had converged; evaluating one alternative takes a separate function that matches on a Boolean parameter. Five such repairs gave 0.75 of the time. And a third of all reference-count drops then came from counting a list of sources at every pivot candidate; the list now carries its length.

The table of plans also ended the parallel gain. It took the work out of the one part that forks: the map fell to 22 to 25% of the time, and with six threads the two inputs took 0.39 and 3.9 s against 0.36 and 4.07 s with one (at the end: 0.25 and 1.99 s against 0.23 and 2.20 s). What had been running in parallel was the redundant work.

As a process the native recognizer (one thread) beats the Python command line on every input tried up to 30 crossings, because Python needs 28 ms to start (2 ms against 27 ms on an 8-crossing unknot, 34 against 44 ms at 30 crossings); in-process, Python is on a par up to 25 crossings on the same steps, and 1.7 to 4.1 times faster on the two large scans of the table.

## 11.5 Verdict

The promise holds in the narrow sense and not in the useful one. Writing a fork is trivial and safe. But the scan alternates a parallel map with a Gaussian elimination in which each cancellation changes the rows the next one reads. While the map recomputed every plan for every entry it was 20 to 60% of the time and six threads gave 1.2 to 1.4 overall; once the plans were computed once per pair of matchings, which is the first thing a sequential programmer does, one thread was as fast as six. The cost model (a fork anywhere taxes all sequential code by 1.8 unless it is wrapped as a task) is invisible in the language, and the compiler removes the wrapper unless it is written just so. For this algorithm the measured ranking stays: Rust, then Python, then Bend, the last 1.7 to 4.1 times behind the Python package on large scans, after becoming 4.5 to 29 times faster than its own first version by sequential changes alone.

## 12 Structural certificates and compressed complexes: October continuation

### 12.1 Intake, implementation, and scope

Report 07 [15] extends the Python recognizer at a pinned repository snapshot. Its patch was reviewed and integrated on 7 October 2026 as version 0.3.0. The delivered archive stays unchanged; this section is the maintained account of the implementation. The Rust implementation still uses the earlier pipeline. The complete integrated Python suite passes all 69 tests on CPython 3.13.14, including randomized comparisons with the inherited scanner, generic set algebra, independent cube calculations in the inherited suite, certificate replay, and stagewise checks that  $d^2 = 0$ . These checks support the implementation; they are not formal proofs of its algebra or of the topological

theorems it imports.

The new default first tries a signed Seifert-graph certificate. If it is inconclusive, the previous pipeline runs. The ordinary scanner remains the default backend, because compression adds overhead on complexes that do not split. Three optional backends provide exact component sharing, saturated decision weights, and early Euler obstructions. They preserve the distinction between a proved decision and resource exhaustion. No universal quasi-polynomial running-time guarantee follows from this integration.

## 12.2 A linear structural decision

Let  $D$  be a validated, oriented, one-component classical diagram, with  $n$  crossings,  $r$ , and  $s$  Seifert circles. The Seifert graph has one vertex per circle and one signed edge per crossing. Let  $c_+$  and  $c_-$  count the connected components of its positive and negative spanning subgraphs, including isolated vertices in both counts. Define

$$g_D = (n - s + 1)/2, \quad \delta = s + 1 - c_+ - c_-.$$

The Seifert surface consists of  $s$  discs and  $n$  bands and has one boundary component; its Euler characteristic is  $s - n = 1 - 2g_D$ . Abe’s criterion identifies  $\delta = 0$  with homogeneity. For a homogeneous diagram, the canonical Seifert surface realizes knot genus [13, Theorems 2.3 and 3.4]. The Kawamura–Lobb bounds give

$$r - s + 2c_+ - 1 \leq s(K) \leq r + s - 2c_- + 1.$$

Here  $s(K)$  is Rasmussen’s invariant, distinct from the circle count  $s$ ; the upper bound follows by applying the lower bound to the mirror [14, 13]. The interval width is  $2\delta$ .

Consequently the implementation certifies the unknot when  $g_D = 0$ , a nontrivial knot when  $\delta = 0$  and  $g_D > 0$ , and a nontrivial knot when the interval excludes zero. The first condition gives a spanning disc. The second gives positive knot genus. The third uses  $s(U) = 0$ . In every other case the stage is inconclusive. In particular, the interval  $[0, 0]$  alone does not certify triviality: the figure-eight knot is a homogeneous positive-genus example with that interval.

The implementation follows the oriented smoothing and edge-pairing involutions to label circles, then traverses each signed subgraph. This costs  $O(n)$  word operations after input validation and normalization. Each word holds an  $O(\log n)$ -bit index; this statement does not include reading arbitrarily long external edge labels. The separate certificate verifier rebuilds orientation from the PD code and uses union by size with path compression, costing  $O(n\alpha(n))$  word operations. It checks all counts and the decision criterion rather than trusting cached production data. Both implementations share the classical one-component input promise and the cited topology.

## 12.3 What exact sharing represents

At each scan frontier the inherited Bar–Natan complex has matching objects and differential entries in the finite dot-mask algebra over  $\mathbb{F}_2$ . Connect two objects whenever a differential entry between them is nonzero. The connected components of this graph are literal direct summands. The shared scanner stores

$$C \simeq \bigoplus_a \bigoplus_h T_a[h]^{\oplus p_{a,h}}, \quad p_a(z) = \sum_h p_{a,h} z^h \in \mathbb{N}[z].$$

A template  $T_a$  records a complete component with its minimum homological degree subtracted. Its key includes the ordered matching identifiers, relative degrees, and every differential target and coefficient. Equality of complete keys exhibits a chain isomorphism. A hash collision cannot identify unequal keys; a missed isomorphism merely loses a saving. Multiplicity coefficients are

nonnegative integers, never scalars in  $\mathbb{F}_2$ . Two identical isolated generators have rank two, not rank zero.

Crossing extension is additive and commutes with shifts. Unit elimination preserves direct sums: the Schur-complement correction  $\gamma\phi^{-1}\eta$  remains in the pivot's parent block. Thus it suffices to extend and contract one representative, split its resulting differential graph, and attach each child's normalization shift to its parent's weight. Merging equal child keys adds their shifted weight polynomials. Induction proves preservation of the represented chain-homotopy type. At closure all nonzero differential entries are units and have been eliminated; summing weights gives the exact unreduced rank and its raw homological-degree counts. The inherited characteristic-two splitting yields the reduced rank by dividing by two. The method does not output a homology basis, quantum grading, or integral torsion.

## 12.4 Safe decision compression and early obstructions

The map  $q(m) = \min(3, m)$  is a semiring homomorphism from nonnegative integer bookkeeping to saturated addition and multiplication. Evaluating each  $p_a$  at one and applying  $q$  therefore preserves the final total rank capped at three. It discards absolute grading but does not modify the differential algebra. For a validated knot, unreduced  $\mathbb{F}_2$  rank two identifies the unknot, while a capped rank of three identifies a nontrivial knot, using the inherited rank criterion and [21]. No decision is made from large multiplicities at an incomplete scan stage.

An early decision needs information about the actual remaining crossings. Let  $F_i$  extend through that suffix and close the diagram. The completed component bound is

$$B_i = \sum_a p_a(1) |\chi(F_i(T_a))| \leq \dim_{\mathbb{F}_2} H(C).$$

This follows by additivity of homology and  $|\chi(X)| \leq \dim H(X)$  for each completed summand. When a component splits, the triangle inequality implies  $B_i \leq B_{i+1}$ ; at complete closure the bound equals the rank. The multiplicity uses  $p_a(1)$ , not  $p_a(-1)$ , since shifts do not change the absolute Euler characteristic of an individual copy.

The memoized suffix functional on a matching is

$$E_i(m) = 2^{\ell_0} E_{i+1}(m_0) - 2^{\ell_1} E_{i+1}(m_1), \quad E_n(\emptyset) = 1,$$

where smoothing  $j$  creates  $\ell_j$  closed circles and matching  $m_j$ . The terminal empty object here is the monoidal unit; the separate empty input-PD convention returns one unknot of rank two. A component's completed Euler value is  $\sum_v (-1)^{h_v} E_i(m_v)$ . Signed intermediate values must remain exact: saturating them before subtraction would invalidate the bound. Only the final nonnegative bound and multiplicities may be capped. The implementation uses an explicit dependency stack, avoiding recursion depth failures on long narrow diagrams.

The default Euler budget registers at most 4096 suffix states. Exhaustion disables this optional inference and continues the saturated scanner. Object or time exhaustion instead propagates to an UNKNOWN result. An early result exposes `rank_lower_bound_capped`; a completed decision exposes `rank_capped`. Neither is reported as an exact rank when it equals three.

## 12.5 The conditional complexity statement

Fix the complete scan order. Let  $w$  bound every frontier, including those in an unvisited suffix after an early decision, and let  $b$  bound the size of every differential component after elimination and before sharing. These are execution parameters, not knot invariants or conjecturally optimal decompositions. Write  $M(0) = 1$  and  $M(w) = (w - 1)!!$  for positive even  $w$ . The number of

complete template keys of size at most  $b$  is at most

$$K(b, w) = \sum_{t=1}^b t^t M(w)^t 2^{t^2 2^{w/2}}.$$

For a numbered connected component with  $t$  vertices, relative degrees lie in  $\{0, \dots, t-1\}$ , giving  $t^t$  assignments. There are at most  $M(w)$  boundary matchings per vertex. A morphism has at most  $2^{w/2}$  coefficient bits, so it has up to  $2^{2^{w/2}}$  possible values. Allowing all  $t^2$  entries gives the last factor. Confusing coefficient bits with the number of morphisms would incorrectly remove an exponential here.

At a compressed stage there are at most  $bK(b, w)$  physical objects. A crossing creates at most eight objects per parent before elimination: each of its two smoothings can create at most two circles. Matrix elimination is polynomial in physical size; coefficient operations cost  $2^{O(w)}$  times polynomial factors. After  $j$  crossings the virtual expanded state has at most  $8^j$  objects, so each multiplicity coefficient has  $O(j)$  bits and each exact weight has at most  $j+1$  degree positions. The conservative bit bound, including exact multiplicity arithmetic, is

$$T(n; b, w) \leq n^{O(1)} (bK(b, w))^{O(1)} 2^{O(w)}.$$

The suffix Euler table has at most  $(n+1)M(w)$  states with  $O(n)$ -bit values and fits within the same conservative bound.

In particular  $b^2 2^{w/2} = O(\log^2(n+2))$  suffices for  $n^{O(\log n)}$  time. This includes fixed  $w$  with  $b = O(\log n)$ , or fixed  $b$  with  $w \leq 4 \log_2 \log(n+2) + O(1)$ . The order must be supplied or efficiently constructed. Neither the existence of such orders for every knot nor the required component-size bound has been established. Small experimental values do not prove either statement. Moreover, report 07 supplies RI/RII-reduced unknot diagrams of homogeneity defect one with arbitrarily large canonical genus; defect alone cannot close the missing bound. The hierarchy obligations in Section 5 remain a separate unresolved route.

## 12.6 Local measurements and reproducibility

The integrated benchmark `fast/benchmark_structural.py` records seven randomized paired rounds, with five fresh Diagram records per arm and an identical-baseline A/A control. Parsing, validation, imports, and serialization are excluded from every arm. The baseline disables the Seifert stage; the treatment enables it. These are recognition-function timings, not end-to-end command-line times. The raw record is `fast/results/structural_integration_20261007.json`.

On this host the median paired baseline/new ratios were 27.10 for a 1000-crossing weaving braid, 26.34 for a 1000-crossing positive three-braid, and 50.56 for a 1004-crossing positive five-braid. Their median A/A ratios were 0.993, 1.002, and 1.008. Inconclusive controls show the added cost: Conway 0.97, hard unknot 0.95, and the tiny scrambled-grid unknot 0.60. Ratios below one mean the added stage slowed the recognition call. These are descriptive measurements with host-dependent variability, not fitted complexity exponents or hardware-independent speed guarantees.

Separate raw-scanner checks bypass all geometric factorization and recognition filters. Exact sharing agrees with the standard rank and degree counts on one and two Conway summands; eight summands give  $2 \cdot 33^8 = 2,812,817,236,482$  without expanding that many generators. This demonstrates representation compression. It does not claim a corresponding recognition speedup: the existing recognition pipeline already factors visible connected sums. The archive's original timing observations remain separate from these local checks.

## 12.7 Rechecking structure after crossing reduction

The first integration checks the original diagram only. This misses an inexpensive opportunity: cancelling opposite-sign crossing pairs may turn an inconclusive signed graph into a homogeneous one. The maintained pipeline therefore repeats the Seifert check after the initial RI/RII pass, provided that the crossing count strictly decreased and is nonzero. An unchanged diagram incurs no second graph traversal. Disabling Seifert certificates or reduction preserves the corresponding opt-out behavior.

Correctness is a composition of two certificates: the recorded legal Reidemeister moves preserve the knot, and the signed-graph certificate decides the reduced diagram. The evidence key `seifert_after_reduction` explicitly identifies this target. To verify it, replay `reidemeister_trace` on the original diagram, then call the independent Seifert verifier on the resulting diagram. The original-input key `seifert_certificate` is not populated with a certificate for a different diagram. This distinction prevents a numerically correct verdict from carrying misleading replay evidence.

For example, append the cancelling braid word  $\sigma_2\sigma_2^{-1}\sigma_1^{-1}\sigma_1$  to  $(\sigma_1\sigma_2^{-1})^m$ , with  $3 \nmid m$  and  $m \geq 2$ . The resulting knot diagram has both signs on both braid generators and a Rasmussen interval containing zero, so the original Seifert test is inconclusive. Reduction exposes the homogeneous weaving diagram of positive genus. The recheck certifies nontriviality before factorization or matrix filters. The added work is one  $O(n)$  traversal; combined with the incremental RI/RII reducer it gives linear word-operation recognition on this family after validation. This is a family result, not a global complexity bound.

The reproduction script `fast/benchmark_reduction.py` loads only the baseline recognition module from commit `1323898c3`, sharing the other implementation modules to isolate this change. Nine paired rounds with fresh diagrams, five calls per arm, and A/A controls measured speedups 1.99, 2.80, and 8.24 for reduced weaving sizes 10, 100, and 1000 (input sizes 14, 104, and 1004). The corresponding A/A ratios were 1.011, 0.993, and 1.023. Control ratios for Conway, the hard unknot, and the scrambled grid were 0.99, 0.95, and 0.95; no improvement is claimed for them. Raw samples are in `fast/results/reduction_structural_20261007.json`. All 71 integrated tests pass, including mirrors, independent certificate replay on the reduced diagram, and a regression asserting that the matrix filter is not entered on this family.

## 12.8 Preparing suffix geometry within the inference budget

The delivered Euler engine eagerly constructed a separate geometry object for every crossing, retaining all intermediate frontier sets. This cost was incurred even with a zero-state budget, or if scanning finished without attempting an Euler inference. The maintained implementation instead constructs a stage's geometry only after registering a state within the budget. The complete matching of that state supplies the frontier. All matchings at a fixed suffix index have the same boundary labels, so a single cached geometry object serves every visit to that stage. This changes preparation, not the Euler recurrence or its values.

Construction still checks closure in  $O(n)$  word operations by toggling each of the four label occurrences at every crossing. Occurrences must be toggled individually: a loop label appearing twice in one crossing does not remain on the frontier. The pass keeps only the current boundary set and checks the time deadline throughout. The crossing list occupies  $O(n)$  references. For a budget of  $B$  registered states, at most  $\min(n, B)$  stage geometries are prepared; their frontier storage is  $O(\min(n, B)w)$ . Thus the geometry preparation storage is  $O(n + \min(n, B)w)$  rather than  $O(nw)$ , before accounting for the separately budgeted memo and gluing entries. This is especially useful when the optional inference is disabled or rejected immediately. It does not change the worst-case complexity theorem.

The additional statistic `prepared_stages` reports actual geometry construction. Tests check

zero- and one-state budgets on a 1201-crossing diagram, deadline exhaustion during construction, the existing exact suffix comparisons against independent generic-algebra continuation, and the long-order case. All 73 tests pass on CPython 3.13.14.

`fast/benchmark_euler_setup.py` isolates construction and the first budget-exhausted evaluation against commit 9765612b8, with seven paired rounds and A/A controls. On 1201 crossings, budgets zero and one reduced traced allocation peaks from about 2.87 MB to 12.0 and 16.5 kB; on 4801 crossings they reduced approximately 11.02 MB to 43.7 and 48.4 kB. Setup time improved by 33–43 times, with A/A ratios 0.979–0.994. Allocation peaks were measured separately from timing, using Python’s `tracemalloc`; they are not process resident memory. The raw data are in `fast/results/lazy_euler_setup_20261007.json`. These measurements concern optional inference setup only, and make no claim of an equivalent speedup for full recognition.

## 12.9 Replacing Euler state enumeration by link connectivity

The suffix dynamic program computes more intermediate information than the ungraded Euler functional needs. A surviving matching comes from a partial resolution of the original classical diagram. Closing it through the unprocessed crossings therefore gives a classical link  $L_m$ , possibly with several components. Gaussian elimination removes objects and changes differentials but does not introduce unrealizable matching objects. Let  $c(L_m)$  be the component count and let  $n_-(L_m)$  count negative crossings after choosing an orientation on each completed component. Then, in the scanner’s raw homological convention,

$$E_i(m) = (-1)^{n_-(L_m)} 2^{c(L_m)}.$$

This formula concerns each completed matching separately; using the original knot’s writhe or component count would be incorrect.

To justify it, the normalized graded Euler characteristic of Khovanov homology is the Jones polynomial in the normalization with unknot value  $q + q^{-1}$  [25]. At  $q = 1$ , a classical  $c$ -component link has value  $2^c$ ; equivalently the specialized skein relation makes the normalized value unchanged by crossing changes, which reduce the link to the unlink. The homological normalization shifts raw cube degree by  $-n_-$ , accounting for the sign. Reversing a component changes signs only at its intersections with other components. The total number of such crossings is even for classical plane curves, so the parity of  $n_-$  is independent of this choice. The empty monoidal closure has  $c = 0$  and value one. Removed crossingless circles are already represented by the scanner’s delooping multiplicities and are not counted twice.

**ClosureEuler** constructs the remaining crossings’ dart pairing: internal labels pair their two occurrences, and the supplied matching pairs the remaining boundary occurrences. Alternating straight-through crossing passages with edge pairings orients and counts all components. The incoming under/over slots determine each sign using the same PD convention as the rest of the package. Each evaluation takes  $O(n - i + w)$  word operations, plus linear-bit integer construction, instead of enumerating up to  $M(w)$  smoothing states per suffix stage. At most one stage’s base graph is retained; complete query values are memoized. Thus a budget of  $B$  queries gives polynomial work  $O(n + B(n + w))$  at this level, and removes the exponential dependence on frontier size from the Euler inference itself. The shared complex can still be exponentially large. This does not settle the general target.

The default `euler` backend now uses this evaluator; the exact **SuffixEuler** recurrence remains available as an independent oracle. The state budget now counts requested completed matchings, not recursive smoothing states. The zero-budget fallback is unchanged. This formula is used only for matchings arising from validated classical diagrams; the low-level method is not an evaluator for arbitrary virtual closures.

A separate exploratory comparison checked 34,447 live matching queries on 500 seeded ran-

dom classical knot diagrams with shuffled orders, with no differences. The maintained tests include 100 random diagrams across all live suffix matchings, exact component-bound equality, mirrors, the monoidal empty closure, malformed frontier rejection, and a 1201-crossing suffix evaluated using a single query. All 76 tests pass. Seven paired raw Euler-assisted scanner runs against commit `1fe98cb7e` reduced the eight-Conway-summand example’s Euler states from 286 to one and measured a 1.326 baseline/new time ratio (A/A 1.013), with the same early decision at stage 11. Conway and the hard-unknot controls performed no Euler inference and provide no evidence of speedup. The script `fast/benchmark_euler_connectivity.py` and `fast/results/euler_connectivity_20261007.json` preserve the measurement boundaries and raw samples.

## 12.10 Source screening: the OpenAI mathematics collection

At the user’s suggestion, we screened the manuscript catalogue at <https://github.com/openai/math>, pinned to commit `adc7f1241b42e322a6451854ab7e4b4c146bf78a`. The catalogue contained no directly named unknot-recognition or Khovanov algorithm. This is a catalogue screening result, not an assertion that every manuscript was audited. The collection itself distinguishes different verification stages. No theorem from it is imported as a correctness dependency of the recognizer.

The September 25 manuscript *Deterministic quasipolynomial-time mean-payoff games*, family 104, asks for forced boundary labels rather than the entire numerical fixed point. Its complexity section separately counts recursion nodes and the bit cost of local operations. The useful methodological lesson here is to compute the required observable directly when possible: the preceding connectivity evaluator calculates Euler values without constructing the suffix smoothing state space. Its proof uses the classical Jones specialization, independently of the game’s claimed result.

One recursion-counting lemma from that manuscript is directly elementary. If every path has length at most  $H$ , has at most  $B$  exceptional edges, and each node has at most one ordinary child and at most  $s$  exceptional children, the node count is bounded by

$$\sum_{\ell=0}^H \sum_{j=0}^{\min(\ell, B)} \binom{\ell}{j} s^j \leq (H+1)(B+1)(1+Hs)^B.$$

Choose the exceptional positions and their child indices; every other position is forced. Thus polynomial  $H$ ,  $s$  and logarithmic  $B$  imply a quasi-polynomial call count, provided local bit cost is also polynomial. For an unknot hierarchy, constructing such a budget and proving that all but one child spend it would be an alternative precise research target. The current scanner has no such proved budget; relabelling its branches does not supply one.

Family 089’s planar  $L_1$  embedding manuscript controls metric distortion and charging across scales. Even its asserted constant distortion would not bound the number of edges in our frontier or the size of connected differential components. Family 306’s cosmetic-surgery manuscript has a topological conclusion about filled manifolds and uses instanton and spinor counts; its introductory construction supplies no bounded-bit algorithm for these counts on a PD input. Neither is presently an algorithmic replacement for the missing width or hierarchy operations. Their introductions and relevant source sections were inspected for applicability; this is not a full verification or rejection of those manuscripts’ main mathematical claims.

## 13 Further incoming reports and checked integrations

### 13.1 Intake status and provenance

Reports 08–12 arrived during the October continuation and were merged from the shared main branch before further integration. Their delivered contents remain unchanged. Local reruns passed 59 test methods for report 08, 69 for report 09, 15 for report 10’s numerical kernels, and 62 for report 11. These are separate suites and their counts must not be added as though they were independent mathematical cases. Logs and local cross-checks are retained under `synthesis/data/`. The current production suite passes 209 tests. Report 12’s 29 archive test methods also pass locally. The braid specialization of report 08, selected preprocessing of report 09, component contraction from report 12, and the twist backend from report 11 are integrated. Report 10’s parallel kernel was cross-checked; report 09’s pointed backend and report 12’s succinct block layer remain under integration review. Section 14 gives the restricted-family complexity proof and degree-sensitive refinement.

### 13.2 Complete recognition for checked short-strand braids

Report 08 contributes a complete decision procedure for the closure of an explicit braid on at most three strands. This is a restricted input class, not a method for reducing every diagram to three strands. The public raw-word entry point is `braid_certificate(m, word)`. It checks integer generators, strand bounds, and that the strand permutation is one cycle before issuing a knot verdict. If the word has  $n$  letters, one-component closure requires  $m \leq n + 1$ , so even permutation validation uses linear space and word operations.

Write  $e$  for the exponent sum. Bennequin’s inequality applied to a braid and its mirror gives  $|e| \leq m - 1$  for an unknot closure. A violation therefore proves knottedness on any number of strands. For one or two strands, this test leaves only the trivial closure classes, so it already gives a complete answer. Its converse is not asserted for arbitrary  $m$ .

For three strands put  $a = \sigma_1$ ,  $b = \sigma_2$ ,  $x = aba$ ,  $y = ab$ , and  $z = x^2 = y^3$ . The central quotient is

$$B_3/\langle z \rangle \cong C_2 * C_3 = \langle x, y \mid x^2 = y^3 = 1 \rangle.$$

The Birman–Menasco classification says that the three conjugacy classes with unknot closure are represented by  $ab$ ,  $(ab)^{-1}$ , and  $ab^{-1}$ ; see Theorem 4.1(i) of Datta’s account of the classification. Only this specific classification result is needed here. The implementation maps letters to at most two quotient tokens:

$$a \mapsto y^2x, \quad a^{-1} \mapsto xy, \quad b \mapsto xy^2, \quad b^{-1} \mapsto yx.$$

A stack multiplies adjacent tokens in the same cyclic factor. A deque then removes or combines equal-factor end tokens until the word is cyclically reduced. Each push, pop, and end operation can be charged to an input token, so the total is  $O(n)$  symbol operations. Cyclic words are compared up to rotation, not by an expensive conjugator search.

The exact test accepts the following exponent/quotient pairs:

$$(e, \text{cyclic class}) = (2, [y]), \quad (-2, [y^2]), \quad (0, [xyxy^2]).$$

For the last case the implementation counts two  $x$  tokens, one  $y$ , and one  $y^2$  in a reduced word of length four. Alternation forces precisely the desired cyclic class. To justify lifting the quotient test, lift a quotient conjugator to  $B_3$ . The remaining difference from the target is  $z^k$ ; since  $e(z) = 6$ , equality of exponent sums forces  $k = 0$ . Thus forgetting the center has not lost information needed by the classifier. A conservative bit bound, including counters and indices, is  $O((n + m) \log(n + m + 2))$ .

An optional independent backend multiplies the integral matrices

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix}.$$

The map  $B_3 \rightarrow \mathrm{SL}_2(\mathbb{Z})$  has kernel  $\langle z^2 \rangle$ , whose exponent sum is a multiple of twelve. Trace one has two conjugacy classes, lifted from the order-three elements of  $C_2 * C_3$ ; exponent residue distinguishes them. Trace three has the single class of  $AB^{-1}$ . One way to see the latter claim is to express a hyperbolic projective conjugacy class as a positive word in  $L = A$  and  $R = B^{-1}$ . A mixed word can be rotated to start with  $L^r R^s$ ,  $r, s \geq 1$ , of trace  $2 + rs$ . Each additional letter increases the trace of this strictly positive matrix. Trace three forces  $r = s = 1$  and no further letters. Lifting and matching exponent sums again eliminates the central ambiguity.

Consequently the exact trace/exponent pairs for the unknot are  $(2, 1), (-2, 1), (0, 3)$ . After the writhe bound and knot-permutation validation, this is equivalently  $|2 - \mathrm{tr}(M)| = 1$ . Without the writhe condition the determinant test is unsound as a recognizer:  $(ab)^5$  has determinant one but is the nontrivial torus knot  $T(3, 5)$ . Matrix entries can have  $\Theta(n)$  bits and the addition-only implementation uses  $O(n^2)$  bit time. The symbolic backend is therefore the default; matrix witnesses use hexadecimal strings to avoid decimal serialization limits.

For more strands, an optional sufficient simplification freely and cyclically cancels inverse letters, then looks for an endpoint generator  $\sigma_1^{\pm 1}$  or  $\sigma_{m-1}^{\pm 1}$  occurring exactly once. Cyclic rotation exposes an elementary Markov destabilization. Deleting that letter and relabeling the remaining strands preserves the closure. There are at most  $m - 3$  steps, each using  $O(n)$  work. The accompanying verifier replays the supplied choices and validates every singleton, rather than trusting the asserted final word. This  $O(nm)$  procedure does not find every destabilization: Morton’s four-strand unknot remains inconclusive and exercises the existing exact diagram fallback.

**Integration and provenance.** Only `Diagram.from_braid` attaches checked braid provenance. PD and grid input acquire none; a reduced factor also does not inherit the original word. Mirrors negate the word’s letters, while serialized provenance is reconstructed and compared to the stored PD, allowing a half-turn of an individual crossing row. Equality and hashing continue to depend only on the PD. Standard JSON output remains PD; the explicit `preserve_braid=True` option retains the source word.

Recognition now tries the linear braid test before the Seifert stage. For  $m > 3$ , the potentially quadratic endpoint descent waits until the Seifert, Reidemeister I/II, repeated Seifert, and descending tests have failed. Resource handling surrounds both braid stages. The evidence records which representation made the decision, and any Markov trace. `-no-braid`, `-braid-backend matrix`, and `-no-braid-reduction` expose ablations. Existing structural and scanner tests explicitly disable earlier braid dispatch where needed to keep testing their intended code paths.

**Validation and measurement scope.** The integrated tests exhaust all 46,376 one-component three-braid words of lengths two, four, six, and eight against both representations. For the 2,856 words through length six they also compare the verdict to an independent dense Khovanov cube. Separate checks cover Alexander determinants, Artin relations, conjugation, mirror and pickle behavior, invalid inputs, endpoint replay, resource limits, and the four-strand fallback. The raw symbolic counter records exactly two token operations per input letter; timings do not substitute for its amortized proof. `fast/benchmark_braid.py` compares fresh diagram construction plus recognition to the current `use_braid=False` ablation, including the already-integrated Seifert shortcut. Python startup and JSON parsing are excluded. Its raw-word scaling experiment separately avoids PD construction. The raw samples are `fast/results/braid_integration_20261007.json`.

**Local braid results.** Nine paired rounds, rerun after correctness tests completed, measured baseline/new ratios 3.30–4.89 on the five scrambled three-braid unknots with default Reidemeister III enabled, and 9.73–41.12 when that optional search was disabled in both arms. Positive three-braid cases improved 1.23–1.35 times, and the hard eight-crossing braid fixture 6.14 times. Conway, Kinoshita–Terasaka, grid input, and the four-strand fallback support no timing claim under the recorded paired/A–A rule. The 256-strand stabilized unknot was about 2.8% slower: the additional linear provenance check has a real constant cost when a different cheap certificate would already finish. These are finite local results, not a uniform speedup claim. The symbolic raw-word test took 0.112 seconds at 128,000 letters in the committed run, with exactly 256,000 token operations.

### 13.3 Canonical visible factors from interlacement

Traverse the validated spherical knot diagram and record the crossing number at each visit, obtaining a double-occurrence word  $W$  of length  $2n$ . Join two crossing labels if their occurrences alternate cyclically. Report 09 identifies the connected components of this interlacement graph with the atomic crossing sets of visible connected-sum factorization. This is a statement about the given diagram; it does not find hidden prime factors of an arbitrary knot presentation.

For completeness, the central argument is short. A chord not adjacent to any chord of a connected component has both endpoints in one gap between successive endpoints of that component. Connectedness forces every chord of another component into the same gap. Consequently the component intervals are nested or disjoint. An innermost component is a contiguous closed interval: every crossing it visits is visited twice. Only the two edges at the ends of that interval connect its projection subgraph to the rest. In the projection sphere these form a visible splitting curve. Removing the interval preserves interlacements among the remaining labels, so induction splits all graph components. Conversely, a visible two-edge split separates whole intervals and cannot divide a connected interlacement component. The induced traversal, with original crossing rotations and over/under data, determines each factor diagram.

The implementation never builds the potentially quadratic graph. A stack tracks active components, union–find merges them when a closing chord crosses later-open components, and circular linked lists maintain an open representative of each component. Every successful merge decreases the number of components. Each chord is opened and closed once, giving  $O(n\alpha(n))$  combinatorial work and  $O(n)$  storage. Checked PD output construction sorts labels, for  $O(n \log(n+1))$  total interface work.

The certificate contains a partition of crossing labels and a forest of actual interlaced pairs. A separate verifier checks every forest edge, connectivity within each proposed group, and proper component nesting. It does not rerun the discovery algorithm. Production recognition now uses this factorizer and exposes `connected_sum_factorization` evidence. The historical API and cut-trace schema remain available through `factor_backend="legacy"` or `-legacy-factor`.

On a cyclic sum of  $k$  standard trefoil diagrams, the old recursive factorizer peels off one factor per internal call. Its internal crossing visits total

$$3 \sum_{j=2}^k j = 3(k(k+1)/2 - 1),$$

which is quadratic in input size. The new factorizer reads the word once and constructs constant-size factors directly. Seven local paired kernel runs measured baseline/new ratios 14.20, 56.59, and 117.20 for  $k = 32, 128, 256$ ; A/A ratios were 1.007, 1.005, and 0.985. These are factorizer timings, not default recognition speedups: the Seifert stage already decides this homogeneous family before factorization.

### 13.4 Sparse modular Alexander elimination

The Alexander minor starts with at most three nonzero entries per row. Report 09’s sparse determinant keeps row dictionaries and column incidence sets. It selects a shortest active row, then a least-populated column in that row, and records row and column permutation signs. Removing a pivot updates only its incident Schur-complement positions. The determinant equals the signed product of pivots, by the same block-elimination identity as dense Gaussian elimination; a zero active row proves singularity. All arithmetic is exact in the fixed prime field.

If  $m$  is the minor dimension,  $E$  its initial nonzero count, and

$$F = \sum_{\text{pivots}} (\text{row degree} - 1)(\text{column degree} - 1),$$

the arithmetic work is  $O(m + E + F)$  plus at most  $m$  inversions. Heap maintenance and stable neighbor order add logarithmic bookkeeping. The bound is conditional on actual fill;  $F$  may still be cubic. The logical number of live entries also differs from Python container capacity and resident memory. The implementation reports these work counters rather than claiming linear complexity for every Alexander matrix.

The sparse path is selected automatically at 256 crossings; dense and sparse modes remain explicit low-level options. On positive three-braid examples with 62, 262, and 514 crossings, local paired sparse-kernel speedups were 0.67, 1.10, and 1.98 (A/A ratios 0.999, 0.999, 1.002). The small-case regression motivates retaining the dense path below the threshold. These kernel measurements also bypass the Seifert stage. The exact witness dictionaries agree, including determinant signs. The reproduction script is `fast/benchmark_factor_sparse.py`; raw samples are in `fast/results/factor_sparse_20261007.json`.

### 13.5 Resource handling and the corrected frontier bound

Factorization and preprocessing now execute inside the same resource handler as the exact backend. Cooperative deadline checks occur before and after coarse operations and between factors; sparse elimination receives the callback at each pivot. An exhausted deadline or a memory allocation failure yields UNKNOWN. Tests inject these failures and advance a simulated clock during preprocessing, ensuring an expired operation does not leak an exception or produce a late verdict.

Report 09 also corrects an older theoretical statement. The scan stores endpoint pairings but supplies no certificate that all frontier points lie on one disk boundary in a common cyclic order. On arbitrary frontiers the safe scalar-state bound is

$$s_i \leq \min\{2^i, (w_i - 1)!!\}, \quad T \leq \text{poly}(n, w, \log p)(w - 1)!!.$$

The Catalan count is valid only with that additional disk condition. We reproduced three counterexamples by independent prefix-resolution enumeration: an explicit six-crossing input, a twelve-crossing greedy order, and the existing double-Conway fixture under its default best order. Each has a six-endpoint frontier with six nonzero Laurent and modular matching states, exceeding the third Catalan number five. The data are in `synthesis/data/frontier-counterexamples-integration.json`. The earlier proposition and user documentation have been corrected; the exact bracket calculation and all verdicts are unchanged.

### 13.6 Later algebra and twist reports: production cross-checks

Report 10’s component-contraction kernel completed 453,611 matching comparisons against the current production algebra, with all completed comparisons equal. Both inspected source hashes matched at the preintegration checkpoint; Conway and Kinoshita–Terasaka scans agree in every

raw homological degree. Their adaptive runs selected no compressed calls, so those checks establish compatibility but no ordinary-diagram speedup. Report 11’s twist backend agreed on 200 production reduced-rank comparisons. Report 12’s preintegration source hash checks also passed; baseline, adaptive, and forced contraction agreed on each degree for six production fixtures, with  $d^2 = 0$  checked after every crossing. The 29 standalone tests pass. These results are saved as the corresponding `report10`, `report11`, and `report12-production-crosscheck.json` files under `synthesis/data/`; the delivered reports themselves are unchanged.

The scope of the later claims matters. A faster dense cobordism product reduces one algebra operation, while the number of objects and calls can still be exponential. Twist compression requires a supplied braid-block presentation and measures size in expanded crossings. Report 12’s succinct Schur cancellation assumes that tensor factors for a same-matching block and its interfaces have already been supplied. Its trillion-generator example verifies an algebraic identity and is not a knot decision. General extraction of these descriptions and control of their size under all crossing and cancellation operations remain unresolved. In particular, a scalar unit in a square-free characteristic-two algebra is its own inverse, but a matrix unit need not be: radical corrections require a finite matrix geometric series. That distinction must survive any future block integration.

### 13.7 Typed scalar shortcuts in the default planar algebra

The report 12 adapter drew attention to a small mismatch between the two production algebra engines: the older bit engine shortcuts identities and scalar squares, whereas the default integer-matching engine compiled them as general cobordisms. The default `Planar.compose` now returns zero immediately for a zero factor, and recognizes the identity only when its source and target matching agree. A polynomial encoded by the integer one in a different morphism space is not an identity.

For an endomorphism of one matching, the coefficient ring is  $R_b = \mathbb{F}_2[x_1, \dots, x_b]/(x_i^2)$ . If  $F = c + \sum_{S \neq \emptyset} f_S x_S$ , then

$$F^2 = c^2 + \sum_{S \neq \emptyset} f_S^2 x_S^2 = c,$$

because cross terms cancel in characteristic two and every nonconstant square-free monomial has square zero. Thus equal scalar endomorphisms compose to their constant coefficient. This avoids the old Cartesian loop over  $|\text{supp } F|^2$  monomial pairs. Comparing packed integers may still read the coefficient vector; it is not constant bit time for arbitrary dense inputs. Equal coefficients in differently typed morphism spaces do not satisfy this identity, and matrix-valued endomorphisms are explicitly outside its scope.

The new tests compare every three-arc scalar and typed identity against the original compiled evaluator, enumerate equal coefficients across all small matching triples to catch a missing type guard, and check 1,000 random four-arc products. The suite at this scalar-only checkpoint passed 116 tests. Nine paired raw-scan runs are recorded in `fast/results/planar_shortcuts_20261007.json`, with a pinned pre-change implementation, fresh algebra objects, and separate untimed call profiling. Ordinary scans support no general timing gain: the scanner already shortcuts most identities before calling this layer. For example, Conway calls the general algebra only 32 times, three of which are scalar squares; its exact square-zero debug checks call the algebra much more frequently. The separate dense scalar microcases measure the algebra operation alone and must not be advertised as full recognition speedups. The reproduction command is `python benchmark_planar_shortcuts.py -output RESULTS.json`.

### 13.8 Opt-in component contraction in the production scanner

Reports 10 and 12 share an exact algebraic improvement. We have integrated report 12's contraction kernel and adapter, preserving its attribution to the established fast subset-convolution algorithm. The independent matrix-block and tensor prototypes remain in the research archive. The production option is `composition="component"`; the explicit `component-dense` mode selects the dense asymptotic algorithm on every nontrivial composition. The standard scalar engine remains default.

Let the middle matching have  $b$  arcs and hence  $w = 2b$  boundary points. A compiled gluing plan decomposes into connected surfaces. For component  $j$ , write  $L_j, Q_j$  for its input-circle sets,  $B_j$  for its output-circle set, and  $e_j$  for its fixed dots. Sending every left input variable in  $L_j$  to  $z_j$ , and every right input variable in  $Q_j$  to the same  $z_j$ , defines algebra maps into  $R_c = \mathbb{F}_2[z_1, \dots, z_c]/(z_j^2)$ , where  $c$  counts components. Two dots entering one component vanish. Distinct input monomials with the same projected support cancel by XOR; keeping only the number of dots would lose this information.

Define a linear output map  $\Lambda$  componentwise. Total dot occupancy at least two vanishes. An open component with one dot gives the monomial on all its output circles; with zero dots it gives the sum of monomials omitting exactly one output circle. A closed component gives one exactly when its total dot occupancy is one, and otherwise zero. Multiplication across disjoint output blocks defines the complete map. The composition identity is

$$\mathcal{B}(F, G) = \Lambda(\pi_L(F)\pi_R(G)).$$

Checking input monomials proves it: the projections and multiplication kill precisely the repeated-dot cases, while the output rule is the original Frobenius evaluation. Bilinearity extends the equality to arbitrary coefficient vectors. A positive-genus component vanishes because a handle acts by  $2x = 0$  over  $\mathbb{F}_2$ .

The ring product is disjoint subset convolution,

$$(FG)_S = \sum_{T \subseteq S} F_T G_{S \setminus T}.$$

We use the ranked transform of Björklund, Husfeldt, Kaski, and Koivisto, *Fourier meets Möbius*. Group coefficients by subset cardinality, take the subset zeta transform at each rank, convolve the rank polynomials, and invert the transform. At output subset  $S$ , selecting rank  $|S|$  retains exactly disjoint pairs whose union is  $S$ . In characteristic two the inverse transform uses the same XOR butterfly. The cost is  $O((c+1)^2 2^c)$  field operations and  $O((c+1)2^c)$  coefficient storage; this algorithm is not a new invention.

If the left, right, and output spaces have  $a, d, h$  circles, the complete factorized operation has cost

$$O((a+1)2^a + (d+1)2^d + (c+1)^2 2^c + (h+1)2^h)$$

in field and subset-mask operations. For the output step, component  $j$  with  $r_j$  output circles contributes at most  $r_j + 1$  alternatives, and  $\prod_j (r_j + 1) \leq 2^{\sum_j r_j} = 2^h$ . Each surviving component contains a middle arc, so  $a, d, h, c \leq b$  in ordinary composition. Consequently the conservative bit cost is  $\text{poly}(w)2^{w/2}$ , improving the dense Cartesian-pair baseline, which can visit  $4^b = 2^w$  pairs. Python coefficient integers contain up to  $2^b$  bits and are not treated as unit-cost words. The implementation unpacks their bytes once when traversing support and uses rank polynomials with only  $O(b)$  bits in the dense transform.

Sparse input requires a different choice. The adaptive mode delegates small support products to the existing engine. After projection, it also uses the characteristic-two identities

$$FG = F(F + G) + F(0) = G(F + G) + G(0)$$

to choose the cheapest of three sparse pair enumerations. A ranked transform is selected only when its estimated work is lower. The adaptive threshold is a performance heuristic; the explicit dense mode is the implementation to which the displayed asymptotic bound applies.

**Implementation boundaries.** The adapter retains the current `Planar` matching IDs and circle numbering. It strips only the redundant fifth field of each compiled component. Its added plan cache is cleared at each stage, and installation is permitted only on a fresh scanner before any crossing. It does not replace the specialized crossing-transfer operation. The options are available through both `recognize` and `khovanov_rank`, and through `-composition standard|component|component-dense` in the CLI. The current integration requires the standard scanner, minimum-fill pivots, bit coefficients, and no racing; incompatible combinations are rejected before an early certificate can hide them. Earlier recognition filters still run normally, and explicit rank computation bypasses them.

`composition_max_variables`, or the CLI flag of the same name with hyphens, defaults to eighteen. It bounds the component and output variables on selected factored allocations. This is an allocation guard, not a process-memory bound or a theorem for arbitrarily large uncapped inputs. Exceeding it produces `UNKNOWN` in recognition and a resource exception in the raw Python rank API; the CLI reports resource exhaustion with exit code three. Kernel deadline callbacks share the scanner’s deadline. Counts of delegated, factored, and scalar-shortcut calls are reported under `composition_stats`. Neither a declined dense call nor an allocation estimate is evidence of knottedness.

The dense improvement leaves the number and size of live chain objects uncontrolled. The finite-type sharing theorem and the need for a uniform topological progress bound therefore remain unchanged. Faster local algebra alone does not prove quasi-polynomial unknot recognition.

**Integrated validation and measurements.** At the component-contraction checkpoint, all 134 production tests passed. The contraction-specific tests exhaust 65,536 three-variable ring products, compare 2,245 small matching/monomial compositions to both the production evaluator and an independent set-based engine, and check all 2,744 four-arc matching triples with seeded arbitrary polynomials. Complete scans, per-stage  $d^2 = 0$ , and exact homological-degree counts agree on supplied fixtures and forty additional random braid closures. Pipeline tests exercise allocation failure, option rejection, evidence fields, and both CLI resource paths. They also exposed an existing unclosed input file in the CLI loader, which now uses a context manager.

The seven-round production benchmark includes separate standard/standard controls. Adaptive isolated dense products improved 2.87, 10.25, and 37.90 times at six, eight, and ten variables; nearly equal dense products improved 6.51, 35.10, and 182.84 times through polarization. The ten-variable sparse control was about 1.13 times slower adaptively and 43.86 times slower under forced dense contraction. No adaptive factored calls were selected on the six ordinary scanner fixtures. Their observed adaptive baseline/new ratios were 0.94–1.04, with A/A variation, and establish no broad scan speedup. The forced path was slower on several ordinary scans. Raw samples, exact inputs, counters, and timing boundaries are retained in `fast/results/component_algebra_20261007.json`; reproduce with `fast/benchmark_component_algebra.py`. The dense-kernel improvement is real for the supplied synthetic morphisms, but their density is not claimed to arise on every difficult knot input.

## 14 Twist compression and a restricted quasi-polynomial family

Report 11 supplies a different way to reduce the exponential dependence: simplify an entire two-strand twist before constructing the closed chain complex. We have integrated this backend as

an explicit option, after checking its local model, degree convention, and production comparisons. Unlike component contraction, which changes one morphism product, this replacement reduces the number of terms in the selected complex. The parameter is the number of signed runs in a supplied braid word. It is neither the minimal braid index nor an invariant of an arbitrary PD presentation.

### 14.1 Local replacement and exact assembly

Write the braid on  $b$  strands as

$$\beta = \sigma_{i_1}^{\epsilon_1 m_1} \cdots \sigma_{i_t}^{\epsilon_t m_t}, \quad m_j > 0, \quad \epsilon_j \in \{-1, 1\}, \quad n = \sum_j m_j.$$

The production adapter groups maximal equal signed letters. It does not perform a hidden search for a shorter braid presentation. The standalone run API also accepts binary-encoded exponents, but every complexity bound below is in the expanded crossing count  $n$ , not their encoding length.

Let  $I$  be the identity smoothing and  $E$  the turnback. A positive  $m$ -crossing twist has the local representative

$$I \xrightarrow{s} E \xrightarrow{D} E \xrightarrow{D} \cdots \xrightarrow{D} E, \quad D = X_{\text{top}} + X_{\text{bottom}},$$

with terms in degrees  $0, \dots, m$ . The negative twist is its dual, in degrees  $-m, \dots, 0$ , with the saddle entering the final  $I$ . This is the characteristic-two, quantum-ungraded specialization of Thompson's Proposition 4.1 (version 2, December 2018), using Bar-Natan's local delooping and cancellation. It is a homotopy equivalence relative to the boundary, so inserting it into the surrounding braid preserves homology. The integer dot sums and differences become the same map over  $\mathbb{F}_2$ . This locality theorem, not merely a  $d^2 = 0$  check, justifies replacing the crossing cube.

A macro state chooses  $k_j \in \{0, \dots, m_j\}$  in each block. Its homological degree is  $\sum_j \epsilon_j k_j$ , and its geometry depends only on the support  $S = \{j : k_j > 0\}$ . The implementation closes the corresponding  $I/E$  picture on a  $(t+1)$ -by- $b$  grid and caches its circle partition by  $S$ . A marked closing arc fixes one circle's label to  $X$  in  $\mathbb{F}_2[X]/(X^2)$ ; the other circles carry binary labels. A state with  $c(S)$  circles therefore contributes  $2^{c(S)-1}$  reduced basis elements.

Transitions at the end of a run use the multiplication or comultiplication saddle. Internal transitions apply the two dot maps. If the top and bottom arcs lie on one circle their dot contributions cancel; a dot on the marked circle vanishes. Different blocks commute, so cross terms cancel in characteristic two. Exact bit-column elimination computes each boundary rank and hence each homology degree. The existing rank-one reduced Khovanov detector then decides the knot.

### 14.2 Counting the selected complex

For each support  $S$ , precisely  $\prod_{j \in S} m_j$  states have that support. Thus the full reduced chain dimension is exactly

$$N = \sum_{S \subseteq [t]} 2^{c(S)-1} \prod_{j \in S} m_j. \quad (1)$$

This is a chain dimension, not a Betti number. The same formula with  $m_j$  replaced by  $z^{\epsilon_j} + \cdots + z^{m_j \epsilon_j}$  gives the dimension in every degree. The Temperley–Lieb trace of  $\prod_j (I + m_j E_{i_j})$  at loop value two, divided by two for the mark, computes  $N$  without expanding the run lengths. This provides the adapter's exact size preflight. A separate polynomial transfer computes the degree profile using sliding windows for these consecutive powers.

Here is the useful structural bound from report 11. If  $d(S)$  counts distinct generator indices active in  $S$ , then

$$c(S) \leq b + |S| - 2d(S).$$

Begin with  $b$  vertical circles and activate turnbacks in word order. At the first use of index  $i$ , no prior connection crosses the cut between positions  $i$  and  $i + 1$ . Its two arcs are therefore in different circles, and that saddle must merge them. Every other activation can increase the circle count by at most one. Counting the  $d(S)$  forced merges proves the inequality; its slack is an even nonnegative integer.

Set  $Q_i = \prod_{j:i_j=i} (1 + 2m_j)$ . Inserting the bound into (1) and grouping subsets by their generator gives

$$N \leq 2^{b-1} \prod_{i=1}^{b-1} \left(1 + \frac{Q_i - 1}{4}\right) = U := \prod_{i=1}^{b-1} \frac{Q_i + 3}{2}.$$

A knot closure uses every generator index: otherwise its strand permutation preserves the two nonempty sides of an unused cut. Consequently  $Q_i \geq 3$ ,  $b \leq t + 1$ , and, for  $t > 0$ ,

$$N \leq U \leq \prod_j (1 + 2m_j) \leq \left(1 + \frac{2n}{t}\right)^t. \quad (2)$$

The last step is arithmetic–geometric mean. The bound is sharp for two-strand presentations and when each generator appears in just one block. It is not a bound on the smallest possible presentation of a given knot.

There are at most  $N$  macro states, each with polynomial bookkeeping in  $n + b$ . Differential matrices have at most  $N$  rows and columns, and their bit elimination costs at most a fixed polynomial times  $N^3$ , with storage at most a fixed polynomial times  $N^2$ . Therefore the uncapped algorithm satisfies

$$T \leq \text{poly}(n + b)N^3 = \text{poly}(n)2^{O(t \log(2+n/t))}.$$

In particular, for every fixed  $c$ , supplied knot braids with  $t \leq c \log(n + 2)$  admit a uniform  $2^{O(\log^2(n+2))}$  bound. Fixed  $t$  gives a polynomial whose exponent may depend on  $t$ ; this is not an FPT claim. Finite allocation and time limits can still cause an invocation to return UNKNOWN. No algorithm here constructs such a short-run braid for every knot diagram.

### 14.3 A sharper degree-sensitive bound

The actual implementation eliminates each homological degree separately. Writing  $d_h = \dim C^h$ , define

$$M = \sum_h d_h d_{h+1}, \quad W = \sum_h d_h d_{h+1} \min(d_h, d_{h+1}).$$

There are at most  $d_h \min(d_h, d_{h+1})$  column reductions for the  $h$ th differential, each touching at most  $d_{h+1}$  bits. Thus  $W$  bounds their total XOR bit work; reading columns and bookkeeping add at least  $O(N)$  work and polynomial factors in index bit lengths.  $M$  is the dense coefficient-bit storage bound. These more informative quantities are reported by both the assembly and degree-profile preflight as `matrix_bit_upper_bound` and `rank_xor_bit_upper_bound`. They are analytical bounds, not sampled CPU times or process-memory measurements.

This gives an additional consequence for a single lengthening block. Hold the strand count, block positions, signs, and all  $m_j$  except  $m_r$  fixed. For any assignment of the other local indices and any degree  $h$ , the equation  $\sum_j \epsilon_j k_j = h$  determines at most one value of  $k_r$ . Every state has dimension at most  $2^{b+t-1}$ . Hence

$$d_h \leq 2^{b+t-1} \prod_{j \neq r} (m_j + 1),$$

independent of  $m_r$ . The number of degrees and macro states is  $O(m_r + 1)$ , while the finite list of support geometries and local maps is unchanged. It follows that  $N, M, W = O(m_r + 1)$ , with constants depending on the fixed surrounding word. The code therefore has linear word-level work

in this family, with logarithmic factors for index arithmetic in a conservative bit model. This conclusion is substantially sharper than substituting a linear  $N$  into the generic cubic bound.

For a single positive two-strand twist, explicitly  $d_0 = 2$  and  $d_1 = \dots = d_m = 1$ . Thus  $N = m + 2$ , the peak degree dimension is two, and  $M = W = m + 1$ . Tests compare these closed formulas to independently computed profiles and assembled complexes at lengths 3, 31, and 301. This is a homology-computation improvement; recognition already has the faster elementary two-strand certificate.

#### 14.4 Integration, provenance, and failure semantics

`backend="twist"` or `-backend twist` selects this backend after the existing reduction, factorization, and invariant stages. The direct rank CLI uses `khovanov -twist`. The adapter accepts only retained, checked source-braid provenance. If the entire diagram was simplified, its original braid still represents the same knot. If factorization produced several diagrams, that braid is never passed to an individual factor. Cheap tests may still decide a factor and thus the sum. Remaining factors are recorded as inconclusive with `twist_deferred`; if necessary the backend then decides the original whole braid once, with its rank recorded only at the top level. This avoids losing the restricted run-count bound by sending factors without braid provenance to an unrestricted scanner. Ordinary PD or grid input has no source word at all and uses the standard scanner, recording `twist_skipped`. The raw twist-rank API instead rejects input without provenance.

The macro degree convention differs from the scanner's cube degree:

$$h_{\text{macro}} = n_+ - h_{\text{cube}}, \quad \dim H_{\text{unred}}^{h_{\text{cube}}} = 2 \dim H_{\text{red}}^{h_{\text{macro}}}.$$

Here  $n_+$  counts positive letters of the *original source braid*. The adapter converts its output to that original cube grading and labels the evidence explicitly, even when the current PD has fewer crossings. A guessed shift or a factor's crossing count would be wrong. Normalized macro degrees are also retained for inspection.

The adapter first rejects a macro-state count exceeding its budget, then computes the exact reduced basis count by Temperley–Lieb transfer. Only a successful count within the basis ceiling permits matrix assembly. Preflight and assembly consume one common time budget. The preflight has separate limits of 4096 matchings and 256 strands; exhausting a guard raises a resource limit, not a verdict. The all- $I$  state's dimension also provides an immediate lower bound before any large geometry is allocated. The state-count guard uses bounded multiplication, avoiding construction and decimal formatting of enormous products for succinct run inputs.

The production recognition option `twist_max_basis` defaults to one million, independently of the standard scanner's object ceiling. The raw `Budget` also controls macro states, matrix bits, XORs, and time. Matrix-bit limits concern coefficient capacity rather than Python resident memory. Allocation/time exhaustion yields `UNKNOWN` in recognition and the CLI, and `ScanLimit` or `MemoryError` in the raw production adapter. The separate `fastunknot.twist` module retains the advanced signed-run, homology, and preflight interfaces from the report.

For context, Kelomäki and Schütz (2026) give a polynomial homology algorithm for all three-braids and show exponential behavior of ordinary scanning on simple three-braid families. The run-count bound above is a different restriction and does not subsume their result. Our existing three-braid recognition shortcut already decides that knot class without computing its homology; it remains ahead of this optional backend.

#### 14.5 Production checks and observed scaling

The full production suite passes 209 tests, including the report's 62 local algebra/topology tests and new production integration regressions. A seeded corpus of 100 knot braid closures on two

through five strands agrees with the existing scanner in every raw cube degree, with  $d^2 = 0$  checked in both engines. The earlier 200-case production cross-check remains recorded separately. These are comparisons of presentations, not counts of distinct knots. New tests also exercise mirrors, whole-diagram simplification, proper-factor deferral, cheap factor obstructions, missing provenance, huge encoded counts, exact preflight rejection, shared time accounting, and CLI resource handling.

Seven randomized paired rounds compare fresh raw homology computations, with a second standard-scanner arm as an A/A control. Diagram creation and input loading are outside the timed interval; scanner ordering and twist size preflight are inside. Every timed computation must agree in rank and every original cube degree. The results are:

| Input family                                | Size parameter | Scanner/twist | A/A   |
|---------------------------------------------|----------------|---------------|-------|
| Single two-strand twist                     | 51             | 31.61         | 1.013 |
|                                             | 201            | 134.74        | 0.973 |
|                                             | 801            | 574.45        | 1.004 |
| Fixed four-strand context                   | 3              | 0.19          | 1.012 |
|                                             | 15             | 0.30          | 1.002 |
|                                             | 51             | 0.48          | 0.995 |
|                                             | 201            | 1.26          | 1.016 |
|                                             | 801            | 4.29          | 1.024 |
| Alternating three-braid, no run compression | 8 crossings    | 0.22          | 0.995 |
| Morton's four-strand unknot                 | 11 crossings   | 0.06          | 0.972 |

The four-strand word is  $\sigma_1^m \sigma_2 \sigma_3^{-1} \sigma_2 \sigma_3 \sigma_1 \sigma_2^{-1}$  with odd  $m$ . Its measured chain dimension is  $N = 177m + 234$ , verified by exact assembly and the independent preflight in each sample. Its peak degree dimension is 177 for the listed  $m \geq 15$ , while the small  $m = 3$  example peaks at 160. Thus the code exhibits the fixed-context bound above; this finite sample is not a proof of the general bound. The larger context measurements were run separately after the first corpus, with the same paired protocol.

The raw scanner took approximately 2.302 seconds on the 801-crossing two-strand case, compared with 0.004 seconds for the macro adapter. These are *homology* timings: the default recognition pipeline already decides all the long-twist examples by its braid obstruction. The alternating control is about 4.5 times slower under the macro backend, and the Morton example about 15.8 times slower. These regressions preclude an unconditional default switch. In particular, a small run count gives a useful asymptotic theorem but need not beat the scanner's cancellations at a particular finite size.

Reproduce with `fast/benchmark_twist.py`; add `-context-only` for the larger four-strand cases. Raw timings, inputs, degree ranks, work bounds, and the method chosen by default recognition are saved in `fast/results/twist_integration_20261007.json` and `fast/results/twist_context_scaling_20261007.json`. No archive files were changed to obtain these results.

## 15 Survivor prediction and adaptive cancellation

The incoming package `unknot_radical_transfer_20261007.zip` is now preserved as report 15. It analyzes minimal complexes before performing cobordism-valued elimination. We reviewed an isolated extraction of the delivered archive, reran its checks, and integrated a narrower operation into the production scanner: exact prediction of the surviving matching multiplicities, followed by a certified shortcut when no differential can remain. An adaptive variant uses this operation only after ordinary cancellation becomes expensive. The delivered report and the production measurements are separate artifacts.

## 15.1 What binary residue homology determines

At a fixed frontier with  $2k$  endpoints, write  $c(a, b)$  for the number of circles in the overlay of two matching objects. A dotted basis morphism  $x_S : a \rightarrow b$  has nonnegative intrinsic weight

$$\omega(x_S) = k - c(a, b) + 2|S|.$$

Weight zero means exactly the undotted identity of one matching. The Euler-characteristic calculation in the composition engine makes weights additive on every nonzero term of a valid cobordism composition. More explicitly, for input dot counts  $u, v$  and output dot count  $q$ , the nonzero genus-zero terms obey

$$2q = 2u + 2v + k - c(a, b) - c(b, c) + c(a, c),$$

which gives the stated additivity after substitution. Consequently the operation  $\rho$  that retains only the scalar identity coefficient between equal matchings is multiplicative. An undotted coefficient between *different* matchings is discarded. Forgetting this typing would give an incorrect binary complex.

Let  $m_{a,h}$  count copies of matching  $a$  in degree  $h$  of a valid partial complex  $C$ . The residue differential  $d_0 = \rho(d)$  is a direct sum, over  $a$ , of ordinary binary complexes. Put

$$r_{a,h} = \text{rank}_{\mathbb{F}_2}(d_0 : C_{a,h} \rightarrow C_{a,h+1}), \quad \mu_{a,h} = m_{a,h} - r_{a,h} - r_{a,h-1}.$$

**Proposition 15.1** (Exact survivor profile). *Complete scalar-unit cancellation leaves exactly  $\mu_{a,h}$  copies of each matching in each degree. This profile is independent of pivot order at the fixed processed tangle.*

*Proof.* An allowed pivot is an endomorphism  $\phi$  with scalar coefficient one, so  $\rho(\phi) = \rho(\phi^{-1}) = 1$ . Applying  $\rho$  to its cobordism Schur update gives the ordinary binary pivot update. Hence each cancellation preserves residue homology. Cancellation terminates, removing two objects at each step. Its final residue differential is zero: a remaining nonzero scalar coefficient would be another allowed pivot. The final multiplicities are therefore the dimensions of the original residue homology, namely  $\mu_{a,h}$ .  $\square$

This argument uses only the valid compositions already required by the scanner. It does not require a universal Catalan count, the full classification of minimal complexes, or the incoming report's sharp nilpotence exponent. In particular, pivot heuristics can improve fill, composition work, and intermediate memory, but cannot decrease the final matching multiplicities of a fixed prefix. Changing the scan order changes the prefix and is a different operation.

## 15.2 An exact shortcut, and an essential non-shortcut

The profile is not itself the surviving differential. For example,  $a \xrightarrow{x} a$  in degrees zero and one has zero residue differential, yet its dotted map is nonzero. Cancelling other scalar pivots can also create such maps between surviving objects.

**Proposition 15.2** (Degree-gap shortcut). *Suppose the occupied degrees of  $\mu$  contain no consecutive integers. Then  $C$  is homotopy equivalent, relative to its frontier, to the direct sum specified by  $\mu$ , with zero differential. The same conclusion holds at a closed frontier without the degree-gap hypothesis.*

*Proof.* Perform unit cancellation conceptually and apply the survivor proposition. A differential increases homological degree by one, so the degree-gap hypothesis forbids every remaining entry, even between unequal matchings. At a closed frontier  $k = 0$ , all coefficients are scalars and a fully cancelled differential is zero. The usual local cancellation equivalences therefore identify  $C$  with exactly the stated zero-differential complex.  $\square$

The implementation need not carry out that conceptual sequence: it computes binary ranks and constructs the resulting objects directly. There are no cobordism compositions in the shortcut. Because the equivalence is relative to the frontier, subsequent crossing gluing preserves it. Zero survivors are allowed. Nonconsecutive integer degrees and previously cancelled object slots are handled explicitly. The raw helper assumes a valid complex, including  $d^2 = 0$ ; a nonnegative profile alone is not a validity certificate.

### 15.3 Switching implementations in response to progress

Eager residue prediction can be slower than the existing sparse cancellation loop. The adaptive mode follows an introsort-style policy, with a mathematical distinction: the alternative operation is a conditional shortcut, not a universal asymptotic fallback.

At the beginning of each crossing stage let  $N$  be the live object count. Start the existing minimum-fill cancellation with an allowance

$$B = \max(256, 4N)$$

for Schur update pairs. Before cancelling a pivot from  $b$  to  $c$ , its candidate work is

$$(|\text{out}(b)| - 1)(|\text{in}(c)| - 1).$$

These are the Markowitz costs already available to the queue. They bound touched pairs, not actual expensive compositions: scalar identities, zero products, and caches may make a pair cheaper. When the next pivot would exceed  $B$ , pause before that pivot, finish the counters, and discard the candidate queues. All completed pivots, live maps, and composition caches are retained.

Compute the residue profile of this partially reduced complex. If the degree-gap or closed-boundary certificate applies, finish the stage using the shortcut. Otherwise resume ordinary cancellation on the same partial result. There is at most one prediction attempt per crossing stage, so a failed attempt cannot cause oscillation between implementations. A later crossing starts a fresh allowance. The constants are a transparent cost heuristic, not a proved optimal threshold. Deadline and object limits keep their existing meaning; neither slow progress nor a failed shortcut decides whether a knot is trivial.

The shared state is what makes this switch inexpensive and sound. Every pause lies between complete homotopy equivalences; a partially applied Schur update is never exposed to another engine. Rebuilding the queue on fallback costs a fresh pass over its remaining entries, while the completed algebra is reused. The additional binary rank work is not bounded by elapsed sparse work, so this policy does not claim the constant-factor competitive guarantee of a time-sliced portfolio. In particular, it does not acquire introsort's global worst-case theorem merely by adopting its switching principle.

**Interfaces and evidence.** Both recognition and raw homology accept the Python option `reduction="adaptive"` or `reduction="residue"`; the latter predicts at every reduced stage and is an ablation/control. The CLI uses `-reduction adaptive|residue|standard`. This controls chain cancellation and is distinct from the existing `-no-reduction` switch for Reidemeister simplification. The default remains **standard**. The new modes require the standard backend, bit coefficients, minimum-fill pivots, and no race; they compose with component contraction and the existing tail option. Unsupported combinations are rejected before early recognition filters. Counters expose prediction attempts, successful shortcuts, switches, fallbacks, and Schur update pairs. Earlier recognition certificates remain enabled, so raw homology measurements are not pipeline speedups.

## 15.4 The stronger report and its boundary assumptions

For noncrossing matchings in a *prescribed common cyclic order*, the incoming report proves the sharp arc-algebra radical exponent  $J^{2k} = 0 \neq J^{2k-1}$  for  $k \geq 1$ . Its dot derivative and central marked dot also yield a two-pass transfer formula. Given a scalar contraction  $(i, p, h)$  and  $\delta = d - d_0$ , the induced differential is

$$D = \sum_{r=0}^{2k-2} p\delta(h\delta)^r i.$$

This is a finite specialization of the homological perturbation lemma, whose inverse formula and special contraction conditions are stated in Crainic’s Section 2. The report supplies sharp-depth examples, so truncating this general expression after the first correction would be unsound.

Those stronger formulas are not installed into the arbitrary PD scanner here. Its frontier pairing representation has no global common-disk certificate. As a simple warning, the three pairings  $(01)(23), (02)(13), (03)(12)$  are not simultaneously noncrossing in one cyclic order; blindly composing the associated abstract triple in the planar gluing engine yields an impossible surface component. The narrow production proof above needs only its existing valid geometric compositions. A full transfer adapter must establish its category and geometry assumptions separately. The report’s provided profile adapter targets the older dictionary-based scanner, not the current interned-list implementation; our adapter is new.

The survivor profile also exposes the main asymptotic obstruction. A small number of matching *types* does not bound their *multiplicities*. Even fixed-width scans can have exponentially many surviving copies. The report’s parameterized transfer estimate still depends on both boundary half-width and the largest minimal survivor count. No bound forcing these parameters into a quasi-polynomial regime for every input has been established.

## 15.5 Validation and measured limits

The isolated report rerun passed 18 unit tests, 2,244 exhaustive composition/derivation comparisons, 500 full transfer certificates, 500 minimal-profile comparisons, 308 marked-transfer certificates, 306 braid closures against an independent crossing cube, and 1,365 intermediate-stage contraction certificates. Sharp radical witnesses through  $k = 12$  and transfer-depth witnesses through  $k = 8$  also passed. These checks cover the report’s model; they do not prove a global runtime bound or establish compatibility with arbitrary scanner geometry.

Production tests compare each predicted profile to actual complete cancellation at every stage of 120 seeded knot closures on two to five strands, using both greedy and shuffled crossing orders. Every final homological degree agrees for eager, adaptive, and ordinary scans; twenty cases also use the independent older set-based engine. Additional tests check a nontrivial unit, nonzero maps after a declined shortcut, pause/resume validity, both adaptive outcomes, budget exhaustion, CLI and pipeline evidence, and incompatible options. All 222 integrated tests passed. The integrated run and the report validation summaries are retained under `synthesis/data/`.

| Input                  | Eager ratio | Adaptive ratio | A/A   |
|------------------------|-------------|----------------|-------|
| two_strand_201         | 0.699       | 0.983          | 0.998 |
| torus_3_10             | 0.897       | 1.004          | 0.997 |
| alternating_3_4        | 0.977       | 0.995          | 1.021 |
| morton                 | 0.897       | 0.986          | 1.020 |
| conway                 | 0.922       | 0.998          | 1.006 |
| hard_unknot_8          | 0.934       | 0.986          | 1.006 |
| Synthetic, 33 objects  | 87.29       | 4.91           | 1.001 |
| Synthetic, 65 objects  | 272.23      | 132.33         | 0.995 |
| Synthetic, 129 objects | 409.70      | 203.12         | 1.002 |

Ratios are median paired standard/new times; values above one are faster.

The benchmark includes seven shuffled paired trials with a second ordinary-scanner control. Diagram construction is excluded and scan order selection is included. Synthetic two-term complexes are timed only during elimination, after deterministic construction: they have a dense dotted differential, a full-rank binary residue, and a single survivor. Their density is not asserted to occur on a measured knot scan. The adaptive mode is useful on this kernel family, but the ordinary diagram results do not establish a general recognition gain. Raw inputs, all timings, counters, and controls are stored in `fast/results/residue_integration_20261007.json`; reproduce with `fast/benchmark_residue.py`. These measurements keep both new modes opt-in while giving subsequent adaptive policies a tested, interruptible cancellation interface.

## 16 Recovered grading, homogeneous algebra, and sharper type bounds

The incoming package `unknot_recognition_dense_algebra_20261008.zip` contains another dense coefficient engine, a recovered-grading argument, and a bounded finite-quotient certificate search. It is now preserved as report 16. Its component contraction overlaps the implementation already integrated from reports 10 and 12. We therefore retain the current adapter and incorporate the additional homogeneous multiplication case, rather than replacing the production tree with the archive’s older baseline. We also derive a packed shortcut for products in the top degree and audit the recovered grading against the current scanner.

### 16.1 The grading that survives after its labels are erased

Start the empty object with an auxiliary quantum shift  $q = 0$ . When smoothing  $i \in \{0, 1\}$  of one crossing creates  $c$  closed circles, its delooping label  $\ell$  gives the child shift

$$q' = q + i + c - 2 \text{popcount}(\ell).$$

For matching objects  $a_u, a_v$  at a frontier of  $2m$  endpoints, every monomial with  $d$  dots in a nonzero differential entry  $u \rightarrow v$  obeys

$$c(a_u, a_v) - m - 2d + q_v - q_u = 0. \quad (3)$$

Here  $c(a_u, a_v)$  counts the circles of their overlay, and the normalization is that of the raw scanner, before oriented link shifts. This is the dotted grading of Bar–Natan’s tangle formalism, specialized to the implementation’s smoothing and delooping order.

**Proposition 16.1.** *Every actual differential entry in the ordinary scanner is homogeneous in dot degree. Every cancellable entry between equal matchings is the scalar identity 1.*

*Proof.* A  $k$ -disc cobordism carrying  $d$  dots has normalized degree  $k - m - 2d$ . A crossing saddle contributes  $-1$ , compensated by the smoothing shift. Delooping a circle contributes shifts  $+1$  and  $-1$ , giving the displayed recurrence and establishing (3) after crossing extension.

The intrinsic weights from Section 15 add under nonzero composition; their negatives are these normalized degrees. If a map between equal matchings has scalar coefficient one, homogeneity forces its dot degree to be zero. There is only one zero-dot monomial, so the map is exactly 1 and the two shifts agree. The source and target shifts telescope in a Schur correction through this identity. That correction has the same degree as the existing entry, and their sum remains homogeneous. Induction over the cancellation sequence proves the claim.  $\square$

The auxiliary labels are a proof and diagnostic device; the normal scanner still does not store quantum grading. Component sharing preserves the conclusion: identical connected keys have identical relative shift differences along their nonzero entries, and an overall shift of a disconnected summand is irrelevant to homogeneity. The adaptive degree-gap shortcut produces a zero differential, which also admits such a grading before subsequent crossing extension. This does not turn its output into a computation of quantum-graded homology; those discarded absolute labels are not reconstructed.

The production audit explicitly propagates  $q$  and checks every term before and after elimination, for ordinary and forced component composition. It completed 520 scans of 87 diagrams, using input, greedy, and seeded shuffled orders where specified: 196,928 entries and 220,736 supported terms satisfy the equation. The largest dot degree observed was two. This finite observation is not a universal bound on dot degree. Inputs, orders, exact ranks, and counts are in `fast/results/grading_audit_20261008.json`; reproduce with `fast/audit_grading.py`.

## 16.2 Cheaper multiplication with an explicit applicability check

In the component ring  $R_r = \mathbb{F}_2[x_1, \dots, x_r]/(x_1^2, \dots, x_r^2)$ , suppose  $F, G$  have homogeneous dot degrees  $a, b$ . Apply the ordinary subset zeta transform to their binary coefficient arrays, multiply pointwise, and apply the inverse transform, also XOR over  $\mathbb{F}_2$ . The resulting coefficient at  $S$  is

$$U_S = \sum_{A \cup B = S} F_A G_B.$$

Retain only subsets of size  $a + b$ . Each supported pair has  $|A| + |B| = a + b$ , so this restriction is equivalent to  $A \cap B = \emptyset$ . The surviving array is exactly the square-free product  $FG$ . If  $a + b > r$ , the product is zero.

This branch uses  $O((r + 1)2^r)$  binary field operations. The generic ranked transform, retained for mixed-degree inputs, has the usual  $O((r + 1)^2 2^r)$  ring-operation bound of Bjoerklund–Husfeldt–Kaski–Koivisto. These are arithmetic bounds; array indices, monomial masks, and coefficient conversions still carry their bit costs. Both remain within the earlier  $\text{poly}(r)2^r$  coefficient envelope. The ranked implementation itself is cheaper on many special inputs, so the two displayed bounds are not a prediction of a factor- $r$  measured speedup on every homogeneous product.

The final size restriction is essential. For example,  $x_1(x_1 + x_2) = x_1x_2$ , whereas unfiltered union convolution would incorrectly keep  $x_1$ . The software verifies the dot degree of each actual operand and falls back to ranked convolution for mixed degrees. It never infers this property merely from a caller selecting the scanner API. Projection to component variables preserves degree for every surviving monomial; a repeated component annihilates a term and collisions may cancel terms, but cannot create another degree. Thus the optimization applies after the existing component projection.

**Complementary subsets in the top degree.** We add a further specialization when  $a + b = r$ . Only the full monomial  $x_{[r]}$  can survive, and its coefficient is

$$[x_{[r]}]FG = \sum_{A \subseteq [r]} F_A G_{[r] \setminus A}.$$

In the packed coefficient convention, complementing the subset index is exactly reversing its  $2^r$  coefficient bits. Reverse the bytes and the bits within each byte, remove padding when  $2^r < 8$ , intersect with the other polynomial, and take the parity of the population count. This computes the coefficient using linear work in the packed bit length after the degrees have been checked, with no zeta transform. The homogeneity check still incurs its own support and index-processing cost. Generic mixed-degree inputs cannot use this shortcut to compute their other output degrees.

The new cases are inside the existing opt-in component-contraction backend. No additional recognition flag is required; the ordinary sparse engine remains the default. The old ranked function remains available for independent comparisons. Existing allocation limits, deadline callbacks, scalar identities, and polarization are retained.

### 16.3 A sharper sufficient regime for component sharing

Let  $B_m = \binom{m}{\lfloor m/2 \rfloor}$  and let  $M(2m) = (2m - 1)!!$ , with  $M(0) = 1$ . The latter is the safe unrestricted pairing count; this argument does not restore a Catalan bound without a common-disk certificate.

A homogeneous coefficient with at most  $m$  circle variables lies in one of at most  $m + 1$  degree layers, each of dimension at most  $B_m$ . There are therefore at most

$$H_m = (m + 1)2^{B_m}$$

possibilities, intentionally counting zero more than once. A connected component on  $s$  objects has normalized homological degrees between zero and  $s - 1$ : each edge changes degree by one, and a simple path from a minimum-degree object has at most  $s - 1$  edges. Consequently the number of ordered serialized keys with at most  $b$  objects is bounded by

$$K(m, b) = \sum_{s=1}^b s^s M(2m)^s H_m^{s^2}.$$

We have overcounted zero entries, invalid degree differences, disconnected graphs, and failures of  $d^2 = 0$ , so no unproved realizability assertion enters this upper bound. Taking logarithms,

$$\log K = O(b^2 B_m + bm \log(m + 1) + b \log(b + 1)) = O(b^2 B_m).$$

The existing exact-sharing argument now gives the stronger sufficient condition

$$b^2 \binom{m}{\lfloor m/2 \rfloor} = O(\log^2(n + 2))$$

for quasi-polynomial cost, provided it bounds every reduced component and frontier actually encountered. Extending a representative through one crossing only multiplies its object count by a constant; different parent components stay independent. Shift weights have polynomial degree and polynomial bit length, as in Section 12. For bounded  $b$ , writing  $L = \log_2(n + 2)$  and  $\ell = \log_2 L$ , the sufficient frontier scale improves from  $2m \leq 4\ell + O(1)$  to

$$2m \leq 4\ell + \log_2 \ell + O(1).$$

This is a refinement of a conditional parameter regime, not a proof that arbitrary diagrams admit such scans. Actual component size is still uncontrolled, and large multiplicities still defeat an explicit-object implementation.

## 16.4 Adaptive controls that satisfy the grading constraint

The mixed-dot two-term matrices used in the previous adaptive benchmark are valid ungraded complexes, but generally cannot be genuine scan differentials: their entries mix dot degrees. Their large speedups therefore do not establish the behavior of a graded scan. We retain those samples and add a separate graded family.

For size  $s$ , begin with an invertible binary  $s$ -by- $s$  matrix, constructed by reversible row additions, and add one source column. Interpret each nonzero entry as the identity of a fixed matching. The resulting two-term complex has  $s + 1$  sources,  $s$  targets, quantum shifts all zero, and exactly one surviving source object. It satisfies both  $d^2 = 0$  and the recovered grading equation. Dense scalar Schur updates can still be expensive even though they require no cobordism products. The existing adaptive allowance detects that update work and switches to binary residue homology.

The new graded controls are still synthetic: we do not claim that our crossing order naturally generates their density. This makes them a stronger applicability check for the switch, not evidence of a broad recognition improvement.

| Kernel             | Product ratio | Full component ratio |
|--------------------|---------------|----------------------|
| homogeneous_8_2_3  | 1.14          | 1.08                 |
| homogeneous_10_3_4 | 1.08          | 1.07                 |
| homogeneous_12_4_5 | 1.19          | 1.13                 |
| homogeneous_14_5_6 | 1.20          | 1.13                 |
| homogeneous_8_4_4  | 16.25         | 4.69                 |
| homogeneous_10_5_5 | 22.62         | 5.21                 |
| homogeneous_12_6_6 | 28.86         | 5.97                 |
| homogeneous_14_7_7 | 34.21         | 6.54                 |
| mixed_12           | 1.01          | 1.00                 |
| sparse_12          | 1.10          | 1.11                 |

| Graded two-term objects | Adaptive ratio | A/A   |
|-------------------------|----------------|-------|
| 65                      | 2.72           | 1.026 |
| 129                     | 8.91           | 1.005 |
| 257                     | 20.74          | 1.001 |
| 513                     | 58.52          | 0.987 |

| Forced component scan | Ranked/new ratio | A/A   |
|-----------------------|------------------|-------|
| trefoil               | 0.803            | 1.058 |
| conway                | 0.991            | 1.015 |
| kinoshita_terasaka    | 1.092            | 1.060 |
| torus_3_5             | 0.987            | 0.990 |
| hard_unknot_8         | 1.006            | 1.002 |

All speedups are median paired old/new times; values above one are faster.

All measurements use seven shuffled paired trials and an identical old/old control. Product timings compare the prior ranked transform to the checked dispatch; full-component timings include projection and output expansion. Ordinary scan measurements force component contraction to exercise the changed path and include order selection. The small trefoil scan regressed in this sample; other scan differences are modest and include control variation. The default sparse engine does not use this new dispatch, and no general recognition speedup is claimed. Reproduction

scripts are `fast/benchmark_homogeneous.py` and `fast/benchmark_adaptive_graded.py`; their complete samples are in `fast/results/homogeneous_integration_20261008.json` and `fast/results/adaptive_graded_20261008.json`.

The full suite passes 229 tests. New tests exhaust every pair of four-variable homogeneous polynomials, test overlap rejection and byte padding, compare mixed inputs with the ranked fallback, verify projection behavior, and combine forced component algebra with adaptive cancellation. The isolated incoming archive passed its 74 tests, the independent eleven-test algebra suite, 2,100 associativity checks, 8,768 transfer comparisons, and replay of ten finite-quotient certificates. Local logs are retained under `synthesis/data/`.

## 16.5 A limit of fixed finite-quotient portfolios

The archive's optional  $A_5$  search has not been merged into the current pipeline. Its witnesses are useful nontriviality certificates; exhaustion must remain inconclusive. The report also supplies a structural limitation worth retaining in the research plan.

For prime  $p > 3$ , the torus-knot group of  $T(3, p)$  has presentation  $\langle x, y \mid x^3 = y^p \rangle$ , with  $x^3 = y^p$  central. In any finite image in which the order of the image of  $y$  is coprime to  $p$ , that image is a power of its central  $p$ th power. It is therefore central, so the image generated by  $x, y$  is abelian. In particular this holds for  $A_5$  when  $p \geq 7$ .

For a coloring by a finite quandle  $Q$  with  $|Q| < p$ , its inner action lies in the symmetric group on fewer than  $p$  points, so the same argument makes the induced group image abelian. All meridian images then agree. The common right translation fixes each color by idempotence, making every crossing preserve its under-arc color; following the one-component knot forces the coloring to be constant. Thus this family needs palettes of at least  $p$  for nonconstant colorings, although its usual braid diagram has only  $2p$  crossings. These knots are already decided by our three-braid or homogeneous diagram stages. The obstruction rules out a universal fixed-palette strategy; it does not rule out larger adaptive targets, bounded-seed searches, or a quasi-polynomial algorithm by another method.

## 17 Exact homological windows and adaptive widening

Report 13 supplies an exact way to compute a prescribed interval of unreduced Khovanov homology over  $\mathbb{F}_2$  without constructing every chain group. We integrate its window scanner and explicit mirror bound into the current implementation, then add a bounded adaptive recognition stage. This stage first queries degree zero, widens only while its budget permits, and falls back to the complete selected backend if the answer is inconclusive. The earlier adaptive cancellation strategy can also run inside each window. Both mechanisms are optional.

### 17.1 Why deleting chain groups can be exact

Raw cube degree counts 1-resolutions. Each remaining crossing contributes homological support  $\{0, 1\}$ , and delooping preserves homological degree. Suppose the requested final raw interval is  $[a, b]$  and  $r$  crossings remain. Retain the current complex only in

$$[a - r - 1, b + 1]. \quad (4)$$

This is brutal truncation: groups outside the interval and maps incident to them are deleted. The two extra degrees are guards.

**Lemma 17.1.** *Truncation to (4) preserves the final homology in  $[a, b]$  after the actual remaining crossing extensions and closure.*

*Proof.* A current object of degree  $p$  can contribute only to final degrees  $p + q$  with  $0 \leq q \leq r$ . For a final degree  $j \in [a - 1, b + 1]$ , every contributing  $p = j - q$  belongs to  $[a - r - 1, b + 1]$ . Thus all chain groups and maps between them in this three-degree neighborhood survive unchanged. Homology in degree  $j$  uses precisely its group and the incoming and outgoing differential. Additive planar extension, closure, and the Khovanov functor preserve these equalities.  $\square$

Neither guard can be dropped in general. The complex  $\mathbb{F}_2 \xrightarrow{1} \mathbb{F}_2$  in degrees  $a - 1, a$  has zero  $H^a$ ; deleting the first group invents a class. The same example in degrees  $b, b + 1$  proves the need for the other guard. Homology in a guard degree can itself be artificial and is never returned as part of the answer.

After processing  $i$  of  $n$  crossings, the lower bound is  $a - (n - i) - 1$  and the upper bound remains  $b + 1$ . At each step the code allocates only the delooped children in that band, then constructs crossing saddles and transferred old maps only when both endpoints survive. This is exactly the truncated extension, so the lemma applies before allocation as well as after it. An object ceiling therefore limits retained allocation, rather than an unnecessarily constructed full extension. The frontier still advances when the retained complex is empty.

Unit cancellations may be interleaved arbitrarily: they are chain homotopy equivalences, and the remaining additive extension and closure operations preserve their homotopy equations. Applying the lemma after each extension and this invariance after each cancellation proves correctness inductively. The proof compares each intermediate state after its own actual suffix; it does not require the truncated and full scanners to choose the same pivots. An isolated object in degree  $h$  may additionally be removed when  $[h, h + r] \cap [a, b] = \emptyset$ , since its independent continued summand has no homology in the target. Incident nonzero maps forbid this shortcut.

## 17.2 Mirror choice and a parameterized complexity bound

Let  $s_0, s_1$  count circles in the all-zero and all-one complete smoothings. Intersect the target with  $[0, n]$  first; an empty intersection has zero homology. Define

$$Q(n, k) = \sum_{j=0}^k 2^j \binom{n}{j}, \quad B_0 = 2^{s_0} Q(n, \min(n, b + 1)).$$

A prefix state with  $j$  one-resolutions has at most  $s_0 + j$  closed circles: complete it with zero-resolutions, then compare the resulting state with the all-zero smoothing using  $j$  surgeries, each changing the circle count by at most one. Thus its delooping gives at most  $2^{s_0+j}$  objects. Summing over at most  $\binom{n}{j}$  states bounds every retained allocation by  $B_0$ . Cancellation and truncation remove physical objects and cannot increase this bound; surviving labels extend injectively to cube labels.

If  $w$  is the largest frontier size in the selected order, the ordinary explicit coefficient representation has cost

$$2^{O(w+1)} (n + 1)^{O(1)} (B_0 + 1)^{O(1)}.$$

There are polynomially many object pairs and pivot updates per stage, and at most  $2^{w/2}$  dotted basis elements in a morphism space. This argument counts objects actually present, so it requires no common boundary disk or Catalan matching bound.

Mirroring reverses raw degree: the rank in original degree  $h$  equals the mirror rank in degree  $n - h$ . It preserves the crossing graph and the width of the same crossing order, while interchanging  $s_0$  and  $s_1$ . The corresponding bound is

$$B_1 = 2^{s_1} Q(n, \min(n, n - a + 1)).$$

The implementation compares these exact integers and selects the smaller bound, mapping returned degrees back to the original diagram. This is a proved size-bound selection, not a prediction of

observed running time. The integers have  $O(n)$  bits since  $Q(n, k) \leq 3^n$ . The recorded per-stage profile explicitly states whether its internal degrees belong to the original or mirrored diagram.

For normalization let  $\nu$  be the number of negative crossings returned by `Diagram.signs()`; normalized degree is raw degree minus  $\nu$ . The source braid word uses a different sign convention: the closure of `[1]` on two strands has  $\nu = 1$  and raw homology in degree one. Counting negative braid letters would select the wrong window. If  $s$  is the Seifert-circle count and  $\mu = \min(\nu, n - \nu)$ , surgery comparison with the oriented smoothing gives  $s_0 \leq s + \nu$  and  $s_1 \leq s + n - \nu$ . For the normalized band  $[-d, d]$ , the resulting bound is

$$2^{O(w+s+\mu+d)}(n+1)^{O(\mu+d+1)}. \quad (5)$$

It is quasi-polynomial when  $w + s = O(\log^2(n+2))$  and  $\mu + d = O(\log(n+2))$ . For a  $t$ -strand braid scanned in braid order,  $s = t$  and  $w \leq 2t$ . These are bounds for the prescribed query. No theorem here bounds a detecting radius for every nontrivial knot, finds such a low-width presentation for every input, or turns an agreeing partial query into a complete unknot recognizer.

### 17.3 An introspective recognition policy

The new recognition options are `window_radius`, `window_max_objects`, and `window_seconds`; the corresponding CLI flags use hyphens. A missing radius disables the stage. An enabled stage runs after the existing inexpensive certificates and simplification, before the selected complete backend. Each actual factor is queried independently and its PD is included in the evidence.

The requested radius is a maximum. The stage tries normalized bands  $[-d, d]$  at radii  $0, 1, 2, 4, \dots$ , including the requested endpoint once and stopping when every raw degree is covered. Each wider query restarts: discarded chain groups cannot be recovered from a narrow scan. The last successful crossing order is reused, including when the selected mirror changes. All queries for one factor share the same time allowance (default 0.1 seconds) and retained-object ceiling (default 20 000), both bounded by the enclosing recognition limits. Before another query, the policy stops if the remaining local time is less than twice the last query's time. This is an empirical progress rule, not a competitive bound.

An exact queried profile differing from  $\{0 : 2\}$  proves knottedness, since every queried band includes normalized degree zero. Agreement is inconclusive unless the query covers all raw degrees  $0, \dots, n$ . Only in that complete case does rank two certify the unknot using the existing Khovanov detection theorem. For example, a trefoil agrees in normalized degree zero and is detected after widening. A local time, object, or memory limit abandons the probe and continues the full backend; exhausting the enclosing deadline returns `UNKNOWN`.

This differs operationally from adaptive cancellation: that method pauses between complete pivots and resumes the same complex when its residue shortcut fails. Window widening must restart. Both fit the same design principle of measuring progress, switching only at mathematically valid boundaries, and retaining an exact fallback. Their combination does not improve the general exponential worst-case guarantee.

The standalone `window` CLI and the Python functions in `fastunknot/window_scan.py` and `fastunknot/window_bounds.py` expose exact partial ranks independently of recognition. CLI `-normalized` interprets input endpoints as normalized degrees and adds a separate normalized profile; `by_degree` always uses original raw degrees. The API result distinguishes `window_rank` from `complete_rank`. A partial rank of two alone is never a verdict.

### 17.4 Validation and measured tradeoffs

The integrated suite passes 245 tests. New checks compare 100 deterministic random knot braids on two through five strands with full homology, using shuffled crossing orders and arbitrary

intervals. Every interval is checked with ordinary, residue, and adaptive cancellation; the first 20 also force the dense component kernel. Intermediate differentials are checked to square to zero. Other regressions cover both guards, mirror degree mapping, wrong-sign hazards, factor evidence, partial agreement, complete coverage, local versus global limits, and budget-sensitive widening. Separately, report 13 passes its 64 scanner tests and 10 hierarchy tests; report 14 passes 94 tests. These later review runs do not alter the as-delivered archive trees. Logs are retained under `synthesis/data/`.

| Input              | Full/window | Probe 0 | Probe 4 | A/A   |
|--------------------|-------------|---------|---------|-------|
| conway             | 0.973       | 0.988   | 0.959   | 0.950 |
| kinoshita_terasaka | 1.142       | 1.030   | 1.030   | 0.984 |
| hard_unknot_8      | 0.983       | 0.514   | 0.207   | 1.032 |
| stress_braid5_36   | 1.374       | 1.478   | 1.469   | 0.988 |
| torus_4_7          | 8.460       | 0.885   | 1.230   | 1.009 |
| two_strand_201     | 2.733       | 0.741   | 0.946   | 1.019 |

Ratios are median paired baseline/new times; values above one are faster.

The table uses seven shuffled paired rounds on CPython 3.13.14. A/A compares identical full-decision arms. Preparation and imports are excluded; order selection and mirror selection are included. The raw-window ratio compares *different output contracts*: degree zero versus all homology. The two probe columns compare complete decision procedures with fallback, using a two-second shared local window budget and maximum radii zero and four. Earlier certificates are disabled in those timed arms to isolate this stage. With default certificates enabled, Jones decides Conway and Kinoshita–Terasaka, the short-braid engine decides the hard unknot, Alexander decides the stress braid, and Bennequin decides both torus examples. Thus these are fallback measurements, not claimed default-pipeline speedups.

The stress input reduces peak retained allocation from 17 694 to 7 692 objects and its degree-zero rank is 698, already an obstruction. At a 10 000-object cap, the full baseline returns UNKNOWN; the window stage returns KNOTTED. Conversely, the hard unknot gains no allocation reduction and maximum-radius-four probing is about 4.8 times slower than the complete baseline. The large raw-window gain on the four-strand torus example does not directly translate to a decision gain: degree zero agrees with the unknot and requires widening or fallback. These negative results justify keeping the policy optional.

## 17.5 Review boundaries for the adjacent reports

Report 13’s finite nilpotent transfer and terminal patterned-ball routine are separate proposals; their passing tests do not supply the missing complete hierarchy construction. They are not substituted for a knot recognition backend here. Report 14’s interval normalization, decision quotients, and scalar Fitting split were initially held for grading review. Section 18 records their subsequent optional integration, including the additional restriction of scalar basis changes to recovered quantum-shift blocks. These operations remain separate from the exact window scanner; an agreeing partial window may fall back to the new complete decision backend, but its shortened model is never used to answer an exact window query.

## 18 Interval normalization, continuation quotients, and graded splitting

Report 14 contributes a different compression mechanism: change bases to expose direct summands, then shorten certain entire summands when only a final unknot verdict is required. The production

implementation now includes optional **barcode** and **fitting** recognition backends and exact-rank interfaces. We retain the standard backend as default. The scalar splitter is strengthened to respect recovered quantum shifts by default, allowing its representation to retain the homogeneous coefficient invariant from Section 16.

### 18.1 Exact decomposition of a pure component

Let  $m$  be one matching on  $2p$  boundary endpoints. Its endomorphism ring is

$$R_m = \mathbb{F}_2[x_1, \dots, x_p] / (x_1^2, \dots, x_p^2).$$

Every element  $\theta$  with zero constant term satisfies  $\theta^2 = 0$ : squares of nonconstant monomials vanish, and cross terms cancel in characteristic two. Call a connected differential component pure if all its objects are  $m$  and every nonzero entry is one fixed nonzero  $\theta$  of this form. Singletons are also considered pure.

Write a pure component's degreewise maps as  $\theta B_h$ , where  $B_h$  is binary. Crucially  $B_{h+1}B_h$  need not vanish: it is the factor  $\theta^2$  that makes the actual differential square to zero. The  $B_h$  form a representation of a finite linearly oriented quiver, whose interval decomposition is a simultaneous scalar change of basis. Define

$$\rho(a, b) = \text{rank}(B_{b-1} \cdots B_a), \quad \rho(a, a) = \dim V_a.$$

The number of intervals supported on  $[a, b]$  is

$$\rho(a, b) - \rho(a-1, b) - \rho(a, b+1) + \rho(a-1, b+1),$$

with unavailable ranks interpreted as zero. Indeed  $\rho(a, b)$  counts intervals covering  $[a, b]$ , and this finite difference selects exactly the intervals born at  $a$  and ending at  $b$ . Successive image-space bases compute these ranks without materializing all matrix products.

Reinserting  $\theta$  gives a chain isomorphism to a direct sum of intervals  $I_l(m, \theta)$ , each with  $l$  consecutive copies of  $m$  and maps  $\theta$ . The implementation reconstructs intervals in degree order and then shares identical templates using the existing nonnegative multiplicity polynomials. This decomposition preserves full raw homology and every later additive continuation. It is applied to whole differential components, never to a submatrix that still has unaccounted attachments.

In exact mode, lengths and birth shifts are retained. In decision mode, birth shifts may be discarded and multiplicities saturated at three. These are two distinct interfaces. In particular, scalar coefficients remain in  $\mathbb{F}_2$ , whereas multiplicities are counts and must not be reduced modulo two.

### 18.2 Why length two is enough for the decision observation

Put  $A = \mathbb{F}_2[\epsilon]/(\epsilon^2)$  and let  $J_l$  have  $l$  free rank-one terms with differential  $\epsilon$ . For an actual remaining crossing sequence and ordinary closure  $F$ , put  $D = F(m)$  and let  $\epsilon$  act by  $F(\theta)$ . This action commutes with the suffix differential and squares to zero. In the strict raw complex, additive planar gluing gives

$$F(I_l(m, \theta)) \cong J_l \otimes_A D.$$

This module structure, not linearity alone, justifies the replacement.

For a bounded complex  $D$  of finite-dimensional  $A$ -modules, the finite dual-number derived classification gives

$$r_l(D) = \dim H(J_l \otimes_A D) = al + 2 \sum_{j \geq 1} b_j \min(l, j), \quad (6)$$

where  $a, b_j$  are nonnegative and finitely supported. Here  $a$  counts shifted simple modules and  $b_j$  counts shifted free intervals  $J_j$ . Report 14 supplies an elementary proof: take a bounded-above degreewise finite free resolution and cancel its units downward. The remaining matrices are  $\epsilon B_h$ . In an exact far-left tail,  $2r_h - \text{rank } B_{h-1} - \text{rank } B_h = 0$  forces constant rank and invertible  $B_h$ . Choose identity bases on that tail, cut it once, and interval-decompose the finite remainder. Intervals born after the cut stay finite; those born at the cut extend left and represent shifted simple modules. Thus only finitely many summands are needed.

For completeness, the tensor calculation behind (6) is

$$\dim H(J_l \otimes_A J_j) = 2 \min(l, j), \quad \dim H(J_l \otimes_A \mathbb{F}_2) = l.$$

The first differential is  $\epsilon(u + v)$  on  $A \otimes \mathbb{F}_2[u, v]/(u^l, v^j)$ . Its homology dimension is twice the cokernel dimension of multiplication by  $u + v$ , namely  $2 \min(l, j)$ . The second differential vanishes. Since  $J_l$  is bounded and free, tensoring preserves the quasi-isomorphisms used above.

A nonzero radical endomorphism requires a nonempty matching. Every complete smoothing of an actual suffix glued to it contains at least one circle, so its unreduced chain-space dimension is even. Homology has the same parity, implying  $r_1(D) = a + 2 \sum b_j$  is even and therefore  $a$  is even. For  $l \geq 2$ , if  $a > 0$  or any  $b_j > 0$  with  $j \geq 2$ , then already  $r_2(D) \geq 4$ . Otherwise  $r_l(D) = 2b_1$  is constant. Consequently

$$\min(3, r_l(D)) = \min(3, r_{\min(l, 2)}(D)). \quad (7)$$

Length one cannot replace length two uniformly: close a two-arc matching to one circle and take  $\theta = x_1 + x_2$ . The two dots become the same circle operator and cancel, so the interval's closed rank is  $2l$ . For arbitrary algebraic suffixes without the evenness premise, the simple module gives  $r_l = l$ , and cutoff two fails. The optional algebraic cutoff three remains valid in that broader setting.

Shortening preserves an observation, not the homotopy type or the exact homology. Repeated replacements are sound because every later continuation is also a continuation before the replacement. Direct sums and copies preserve capped-rank equality, and saturation commutes with addition and multiplication of nonnegative counts. At final closure, elimination leaves isolated generators; their saturated total is exactly the original rank capped at three. Rank two certifies the unknot, and a capped value three means at least three, hence at least four for a classical knot. The code labels shortened-model object statistics accordingly and never exports their degree counts as exact knot homology.

### 18.3 Certified scalar splitting with recovered grading

Purity need not be visible in the current basis. The optional Fitting search looks for scalar endomorphisms  $Q$  satisfying  $Qd = dQ$ , solving the binary equations coefficient by coefficient. The archive permits variables between objects with equal matching and homological degree. We additionally require equality of recovered quantum shift by default.

For a nonzero homogeneous entry  $u \rightarrow v$ , let  $d_{uv}$  be its dot degree and  $c_{uv}$  the overlay-circle count. The required relation is

$$q_v - q_u = p - c_{uv} + 2d_{uv}.$$

Propagating these differences along the undirected differential graph recovers all  $q_v$  up to one additive constant per component. The code checks homogeneity of every inspected entry and rejects inconsistent cycle sums. Scalar variables exist only within blocks  $(m, h, q)$ . Both solving and basis-change verification use these blocks. Cache keys include the relative shifts, so a cached ungraded search cannot silently stand in for a graded search or a geometrically different shift constraint.

For a candidate  $Q$ , take a power  $Q^N$  with  $N$  a power of two at least the largest block dimension. Then  $\ker Q^N \oplus \text{im } Q^N$  is the stable Fitting decomposition. Because  $Q$  commutes with  $d$ , these spaces form subcomplexes. The code builds bases for both parts, checks invertibility, verifies the commutator independently, transforms every differential coefficient, and requires every entry across the proposed split to vanish. It accepts only nontrivial splits. Optional witnesses record the original and transformed matrices, matching and degree labels, relative quantum shifts, and the change of basis. A separate dense test verifier checks both directions of the basis transformation and rejects corrupted witnesses.

The search is deliberately incomplete: defaults limit each considered component to 48 objects, the scalar system to 1024 variables, accepted splits to 16 per crossing, and candidates to a bounded deterministic set. A local size or search limit leaves the exact component intact. Accepted splits are followed by interval normalization and sharing. Exhausting the global scanner budget gives UNKNOWN. The global time allowance includes order selection, and retained-object ceilings are checked before crossing allocation. The low-level `preserve_grading=False` option retains the archive's exact ungraded search for controlled comparisons; it does not justify the homogeneous bounds below.

For a connected pure component with homogeneous  $\theta$  of dot degree  $d$ ,  $q_v - 2dh_v$  is constant throughout the component. Thus all objects in a given homological layer have the same recovered quantum shift, and its interval basis changes are automatically graded. Sharing identical connected templates preserves the existence of a graded lift up to componentwise shifts. Shortening keeps a graded interval representative, although it does not preserve the original bigraded homology. These facts justify using homogeneous coefficient counts after our restricted scalar splits; no claim to output the erased absolute quantum grading is made.

## 18.4 A sharper checkpoint bound

A pure checkpoint is a stage where, after the implemented splitting and normalization, every component is pure. Such a stage is recognizable; a small maximum gap between such stages is an additional family hypothesis. Let  $w = 2p$  bound the frontier throughout the chosen order, and let  $g$  bound the number of crossings between successive pure checkpoints, including the initial and final stages.

Without grading, there are  $2^{2^p-1} - 1$  possible nonzero radical coefficients on a fixed matching. With our homogeneous invariant there are at most

$$S(p) = \sum_{d=1}^p \left( 2^{\binom{p}{d}} - 1 \right) \leq p 2^{\binom{p}{\lfloor p/2 \rfloor}} \quad (p \geq 1), \quad S(0) = 0.$$

At a checkpoint, a shortened interval type is determined by its matching and coefficient; singletons need no coefficient. With  $M(w) = (w-1)!!$  for  $w > 0$  and  $M(0) = 1$ , there are at most

$$M(w)(1 + S(p)) \text{ templates,} \quad M(w)(1 + 2S(p)) \text{ stored objects.}$$

No common boundary disk is assumed. This bound is independent of the original connected component size, the number of hidden copies, and the original interval lengths. Exact mode retains lengths through  $n+1$  and pays only an additional polynomial factor in  $n$ .

One crossing makes at most eight delooped children per object. Hence an impure stretch costs at most a factor  $8^g$  in stored objects. Gaussian elimination, interval ranks, the bounded scalar search, and coefficient operations have polynomial cost in this physical size and  $2^{O(w)}$ . Multiplicity polynomials have polynomially many degree positions and  $O(n)$ -bit coefficients. The resulting conservative bound is

$$T(n; w, g) \leq n^{O(1)} \exp \left( O \left( g + w \log(w+1) + \binom{w/2}{\lfloor w/4 \rfloor} \right) \right). \quad (8)$$

This sharpens the report’s  $2^{w/2}$  term by incorporating the recovered grading. It gives a complete quasi-polynomial recognizer on families with constructible suitable orders for which the displayed exponent is  $O(\log^2(n+2))$ . It does not prove that arbitrary knot diagrams have such orders or frequent pure checkpoints. A single gap  $g = n$  leaves an exponential bound. A normalization size cap that skips a nontrivial component prevents that stage from counting as a pure checkpoint in the implemented statistics.

## 18.5 Validation and practical scope

All 267 production tests pass. The imported interval tests independently change quiver bases, reconstruct interval multiplicities, and compare dense suffix closures. The scalar tests replay witnesses, check corruption, and compare 80 seeded knot braids with exact full ranks and capped decisions; another 60 seeded braid cases exercise interval normalization. New production regressions cover grading constraints, inconsistent cycles, cache separation, order-selection deadlines, input validation, CLI incompatibilities, and recognition fallback after an agreeing window query. Recovered shifts also agree up to componentwise constants with independently propagated shadow labels on Conway, Kinoshita–Terasaka, and the hard unknot.

Report 14’s independent continuation verifier imports no production code. It enumerates 861 finite complexes of dual-number modules, including nonfree modules and unit differentials, and passes 10 206 threshold comparisons through interval length six. This supports the implementation and finite algebra examples; the all-suffix statement rests on the proof. Its output and the full integrated test log are retained under `synthesis/data/`.

| Input              | Interval exact | Fitting exact | Fitting decision | A/A   |
|--------------------|----------------|---------------|------------------|-------|
| conway             | 0.821          | 0.460         | 0.469            | 1.063 |
| kinoshita_terasaka | 0.841          | 0.487         | 0.481            | 0.992 |
| hard_unknot_8      | 0.819          | 0.777         | 0.795            | 1.028 |
| stress_braid5_36   | 0.747          | 0.776         | 0.766            | 0.988 |
| torus_3_5          | 0.901          | 0.889         | 0.871            | 1.003 |

Ratios are median paired full-scanner/new times; values below one are slower.

These are seven shuffled paired raw-backend rounds on CPython 3.13.14, including order selection and excluding imports and diagram construction. Every exact result is checked degree by degree against the full scanner; every decision result is checked against its rank capped at three. A/A compares identical full-scanner arms. Witness recording is performed separately, outside the timed arms. Earlier certificates are bypassed, so these measurements describe backend costs rather than end-to-end default recognition. Default front-end methods and all inputs are recorded.

The data include overhead and unsuccessful normalization attempts. None of these five actual diagrams produces an eligible nonsingleton pure component, so they demonstrate no real-diagram interval-shortening gain. The synthetic ladder regression does exhibit the intended structural saving: 31 hidden copies of a seven-term interval require 217 objects under literal sharing, seven after exact interval normalization, and two after decision shortening. This is a graded algebraic fixture, not a claim that it appears in the knot corpus. Scalar splitting produces two verified splits on both Conway and Kinoshita–Terasaka, but their existence alone does not imply faster recognition. On this corpus the new modes are slower than the full baseline. The quantum restriction also costs time: the median ungraded/graded Fitting ratio is about 0.85 on those two diagrams and near one on the other three. The measured tradeoffs support keeping both backends optional.

Report 14’s bounded crossing-order optimizer is a separate proposal and is not part of this integration. Its objective is a frontier proxy rather than elapsed scanner time. The general

quasi-polynomial recognition goal still requires a structural theorem or additional machinery beyond these conditional compression bounds.

## 19 Verified rank-two braid preprocessing

The incoming archive `unknot_rank_two_kernels_20261007.zip` is now preserved as report 19. It contributes an exact braid-word compressor. It finds an optimal set of disjoint two-index subwords to replace by the empty word or a single Artin letter. We integrate the search and its independent replay verifier as an optional bounded preprocessing stage, after the inexpensive existing certificates and before the more expensive braid descent and diagram fallbacks. Unlike the closed-three-braid gateway, this procedure proves braid equality inside the surrounding word.

### 19.1 Faithful local states without copying prefixes

For adjacent indices write  $a = \sigma_i$ ,  $b = \sigma_{i+1}$ . The local group is

$$B_3 = \langle a, b \mid aba = bab \rangle = \langle x, y \mid x^2 = y^3 \rangle, \quad x = aba, \quad y = ab.$$

The common element  $z = x^2 = y^3$  is central and has Artin exponent sum six. The quotient by  $z$  is  $C_2 * C_3$ . Thus a local braid is uniquely determined by its reduced quotient word together with its exponent sum: equal quotients differ by  $z^k$ , and equal exponents force  $6k = 0$ .

The quotient normal form must not be cyclically reduced. The conjugate letters  $a$  and  $b$  are not equal inside an arbitrary context. Nor may the central exponent guard be omitted:  $(aba)^2$  has empty quotient word but is nontrivial. For distant indices the local subgroup is  $\mathbb{Z}^2$ , so two exponent counts suffice. A single index uses one count.

In the quotient, the four signed letters have images  $y^2x, xy, xy^2, yx$ . Each appends two tokens to an alternating normal-form stack. The search stores these stacks persistently in a trie with three child slots per node. Push creates or reuses a child, cancellation follows a parent pointer, and combination of order-three tokens uses a child of the parent. Node identity is an exact canonical word identity; no hash fingerprint substitutes for group equality. The prefix tables themselves use deterministic AVL trees in the integrated recognition stage.

### 19.2 An optimal pass over bounded-overlap intervals

Compress maximal runs of a common absolute generator index. Label each edge between consecutive runs by its unordered pair of indices. A maximal string of equal edge labels specifies a maximal two-index interval, called a corridor in the source. Consecutive corridors overlap in exactly one run; nonneighbors are disjoint. Every permitted subinterval lies in a corridor, each letter belongs to at most two, and the sum of their lengths is at most  $2n$ .

Let  $F[j]$  be the maximum number of letters removable from the first  $j$  letters by disjoint permitted replacements. Inside one active corridor, write  $p_j$  for its local prefix product. Maintain an exact table

$$H(q) = \max_{i: p_i = q} (F[i] - i),$$

with an attaining start position. For a target  $v$  of length zero or one,

$$w[i : j] = v \iff p_i = p_j v^{-1}.$$

Therefore the next optimum is

$$F[j] = \max \left( F[j-1], \max_{\text{active corridors}, v} \{j - |v| + H(p_j v^{-1})\} \right).$$

Queries precede insertion of the current endpoint. Backtracking the attaining starts recovers disjoint replacements. The last replacement of any optimal prefix plan is represented in this recurrence, proving optimality by induction. This is optimality for one pass and this explicit rule set, not global minimality among braid presentations or Markov moves.

There are at most five targets per corridor and at most two active corridors. Each adjacent-group update takes two trie-token operations. There are  $O(n)$  exact states, queries, and backpointers. AVL table access costs  $O(\log(n + 2))$  comparisons, giving

$$O(n \log(n + 2)) \text{ word-RAM time,} \quad O(n) \text{ words.}$$

State identifiers, exponent counts, and indices have  $O(\log(n + s + 2))$  bits for  $s$  strands, giving the conservative bit bound  $O(n \log^2(n + s + 2))$ . The low-level package also retains an optional hash backend whose dictionary-time bound is expected rather than deterministic. Recognition uses one AVL pass. Repeated passes remain available through the low-level API, but their conservative total bound is quadratic up to logarithmic factors.

### 19.3 Independent replay and evidence association

The verifier does not import the optimizer or its quotient trie. It uses a central normal form  $z^k f_1 \cdots f_r$  with alternating  $x, y, y^2$  factors. Each signed Artin letter expands into  $z^{-1}$  times the same two positive factors used above; reductions  $xx = z$  and  $y^3 = z$  update the central integer explicitly. A replacement is accepted only if its source and target differ by the identity in this form, or by equal exponent counts in the commuting case.

Stable linked-list node identifiers describe interval endpoints across passes. Inserted target letters get fresh identifiers. Replay checks that endpoints remain live and correctly ordered, that the interval uses at most two indices, that the target is permitted, that equality holds, and that the word strictly shortens. The final output is reconstructed and compared exactly with the certificate. Its input digest binds the transcript to the supplied word; it is not an authenticity signature.

Braid equality preserves exponent parity, so each successful replacement saves at least two letters. Across all passes there are at most  $n/2$  insertions and  $3n/2$  removed nodes. Thus valid-certificate replay takes linear word-RAM work and memory, even when search used multiple passes. The verifier establishes soundness of the replacements; it does not certify the optimizer's separate optimality or saturation claims.

The pipeline requires checked source-braid provenance. It compresses that source, independently replays the transcript, and constructs a new validated PD only if the word is shorter than the current simplified diagram. An accepted replacement restarts recognition on the new source with compression disabled, preserving all other options and the remaining global time. Earlier PD reductions and later reductions lie on distinct branches of the evidence, named **before\_ranktwo** and **after\_ranktwo**; the source word and braid-equality certificate connect the original knot to the restarted input. They must not be concatenated as one PD move trace. The existing factor and twist provenance rules then apply to the restarted recognition without modification.

The stage is enabled by **use\_ranktwo=True** or **-ranktwo**. It has one shared local allowance for search and replay, defaulting to 0.1 seconds, configurable through **ranktwo\_seconds** or **-ranktwo-seconds**. Budgets are cooperative. Local time or allocation failure skips the probe and continues the existing complete route; global exhaustion returns **UNKNOWN**. A failed certificate is an error, never a knot verdict. Already decisive cheap certificates retain priority. Inputs without checked braid provenance bypass this stage.

## 19.4 A constructive quasi-polynomial input class and its barrier

Let  $\kappa_1(w)$  be the smallest output length under the one-pass rules. The optimizer finds this number without receiving a decomposition. A *flat rank-two inflation* of a  $k$ -letter core replaces disjoint letters by equal rank-two words and inserts disjoint rank-two identity blocks. Those blocks themselves define an admissible plan, so  $\kappa_1(w) \leq k$ . Following the verified pass by an uncapped exact exponential homology computation takes

$$O(n \log^2(n + s + 2)) + \text{poly}(\kappa_1(w)) 2^{O(\kappa_1(w))}.$$

For a knot closure the reduced strand count satisfies  $s \leq \kappa_1(w) + 1$ : a word on  $s$  strands whose permutation is one cycle needs at least  $s - 1$  adjacent transpositions. This removes a hidden strand factor. Flat inflations of  $O(\log^2 n)$ -letter cores consequently admit quasi-polynomial recognition without a supplied inflation decomposition. The statement concerns the uncapped route, including `ranktwo_seconds=None` in the integrated API. A local cutoff can miss the compression and invoke an exponential fallback on the larger input even when that fallback eventually completes; a finite global budget can return `UNKNOWN`. The integrated earlier simplifications have polynomial overhead and do not invalidate this conditional bound.

No universal small-core theorem follows. An explicit obstruction is

$$U = abcaba, \quad V = cbacbc, \quad q_h = (UV^{-1})^h abc \quad \text{in } B_4.$$

The positive words  $U, V$  are equal half-twists, so  $q_h$  closes to the unknot and has length  $12h + 3$ . Within a uniform-sign block, exponent sum proves every subword geodesic. Any two-index subword crossing a sign boundary is contained in  $ac^{-1}$  or  $c^{-1}a$ , flanked by index  $b$ ; its commuting exponent vector has norm two, proving geodesicity there as well. Thus no strictly shortening rank-two substitution of any target length can begin on this family. Escaping it requires, for example, length-preserving relations, a larger support, or separately certified closed-knot operations. Increasing the number of identical shortening passes cannot solve that obstruction.

## 19.5 Production validation and end-to-end measurements

The archive’s 33 tests pass locally. Its previously unrun real-upstream smoke test now passes all 16 sleeve cases against the current braid gateway: the gateway alone is inconclusive before compression and certifies the unknot after replay. This gateway result does not by itself establish an end-to-end speedup; the current diagram simplifier may already decide them.

All 272 integrated tests pass. New tests compare 200 random words with an independent exhaustive interval oracle, both dictionaries, both target radii, and the Artin action on a free group. Sixty knot inputs compare full normalized homological profiles before and after shortening, checking  $d^2 = 0$ . Other regressions cover central and conjugacy guards, certificate corruption, the higher-rank barrier, original input counts, evidence scopes, local and global budgets, absent provenance, preservation of cheap verdicts, and restart into the standard, shared, twist, barcode, and Fitting backends. Logs and the actual gateway smoke-test output are retained in `synthesis/data/`.

| Input              | Disabled/enabled | A/A   |
|--------------------|------------------|-------|
| sleeve_1           | 3.397            | 1.019 |
| sleeve_16          | 12.471           | 1.007 |
| sleeve_64          | 33.777           | 1.005 |
| barrier_8          | 0.994            | 1.011 |
| cheap_braid        | 0.979            | 1.014 |
| stress_braid5_36   | 0.559            | 0.969 |
| conway             | 1.012            | 1.014 |
| kinoshita_terasaka | 1.021            | 1.027 |

Ratios are median paired end-to-end times; values above one are faster.

The benchmark uses seven shuffled paired rounds with an identical A/A control. It includes PD construction, all default certificates and simplifications, compression, replay, and the complete fallback; imports are excluded. It uses a five-second global budget and the default 0.1-second compression budget. The source inputs, verdicts, methods, transcripts, and timings are recorded. Separate traced-allocation runs record both their peak and the path actually taken, since instrumentation can affect a time-limited branch. These are measurements of this machine, not asymptotic guarantees or a broadly sampled knot distribution.

The sleeve cases with parameters 1, 16, 64 improve by approximately 3.4, 12.5, and 33.8 times respectively. Their baseline already certifies the unknot through diagram reduction; compression reaches the short source braid and its genus-zero certificate sooner. The stress five-braid instead runs about 1.8 times slower: its probe cannot supply a shorter input than the current simplified diagram, and Alexander still provides the verdict. The barrier fixture is settled by the descending diagram certificate before compression, and the positive braid by Bennequin’s obstruction; neither timing measures a compressor success. The two PD-only named knots bypass the stage. These contrasting outcomes support keeping the new stage optional.

In the separate traced-allocation run, the largest sleeve baseline hit the five-second global deadline. Its recorded peak is censored and must not be compared as a peak of a completed recognition. All ordinary timed arms completed with agreeing verdicts.

## 20 Minimal extremal windows with certified binomial bounds

The incoming archive `ProveIt_Unknot_Extremal_Windows_2026-10-07.zip` is now preserved as report 20. It adds a stronger size analysis for an alternative window schedule. The schedule is now available in recognition through the `-window-strategy` option with value `minimal`, and through `window -minimal` for exact partial homology. The existing allocation-pruned strategy remains the default. Both compute exact requested ranks, but they construct different guard complexes and have different size guarantees.

### 20.1 Cancellation and truncation do not commute

Let  $K = k + 1$  when the desired raw degrees are  $0, \dots, k$ . The new strategy extends the whole retained complex by one crossing, deloops, exhaustively cancels units, and only then discards degrees above  $K$ . In particular it constructs temporary degree  $K + 1 = k + 2$ . The contractible complex  $X \xrightarrow{1} X$  in degrees  $K, K + 1$  explains the order: cancellation removes it, while early truncation leaves a nonzero stalk in degree  $K$ . The latter is artificial guard homology.

This does not contradict the exactness of the earlier support-window algorithm. That algorithm returns degrees below the guard and has a separate bounded-suffix support proof. But that proof does not establish the binomial size bound derived here. The two implementations are kept explicitly separate; the new strategy never suppresses temporary objects before cancellation. Its object ceiling therefore includes their allocation.

### 20.2 Minimality controls all exhaustive pivot choices

In the finite dotted matching category over  $\mathbb{F}_2$ , let  $J$  consist of all morphisms between distinct matchings and all equal-matching morphisms with zero constant coefficient. This is the nilpotent radical; the semisimple quotient has one scalar factor per matching. The positive weight  $m - c(a, b) + 2|S|$  from the radical analysis gives a finite nilpotence bound. An entry outside  $J$  is precisely a unit between equal matchings. Exhaustive scalar-unit cancellation therefore leaves a

minimal complex, meaning one whose differential lies in  $J$ .

Every bounded finite complex splits, by the invertible changes of basis from Gaussian cancellation, into a minimal complex and contractible disks. If minimal complexes  $M, N$  are homotopy equivalent, their homotopy inverse equations reduce modulo  $J$  to degree-wise inverse maps. Lift these inverses using the finite geometric series in the nilpotent matrix ideal. The chain maps are thus degree-wise invertible and give a chain isomorphism. Consequently the minimal matching multiplicity in every homological degree is unique, although matrix entries and computation times can depend on the chosen pivots.

Write  $\tau_K$  for brutal upper truncation. Consider additive extension functors  $F_t$  that preserve chain homotopies and have degree amplitude  $[0, 1]$ . Starting from a common initial complex, compare

$$A_t = \tau_K \text{Min}(F_t A_{t-1}), \quad B_t = \tau_K R_t(F_t B_{t-1}),$$

where  $R_t$  performs any collection of valid unit cancellations. Then

$$B_t \simeq A_t \oplus E_t[K] \tag{9}$$

for a stalk object  $E_t$  in degree  $K$ . In particular  $A_t$  has no larger matching multiplicities than  $B_t$  in any degree.

To prove this, extend the inductive decomposition. Its extra stalk extends only into degrees  $K, K + 1$ . Uniqueness of minimal complexes identifies the minimal part of the extension with the sum of the desired minimal extension and a minimal complex supported in those two degrees. The partially reduced comparison complex is *chain-isomorphic* to that sum plus contractible disks. Truncate this chain isomorphism: disks below  $K$  stay contractible, disks above  $K$  disappear, and disks crossing the cutoff become stalks in  $K$ . The extra minimal two-degree piece also becomes a stalk. This proves (9). It never assumes that brutal truncation preserves an arbitrary homotopy equivalence.

### 20.3 The geometric certificate and inherited object count

The external ingredient is Kelomäki–Schütz, Lemma 5.5, checked directly against the primary paper during this review. For their comparison scan on a nice disc order, its degree- $j$  object count after  $t$  crossings is bounded by  $\binom{t}{j}$  before final closure and by  $2\binom{n}{j}$  at closure. Reducing its coefficients modulo two preserves the allowed cancellations. Applying (9) transfers these object bounds to our exhaustive cancellation policy.

The production certificate verifies an actual crossing permutation, absence of projection loop edges, connected processed and remaining vertex sets, consecutive attaching slots at every intermediate crossing, and the unique final four-slot closure. For a validated plane projection, a cut with connected sides is a bond; its dual cycle supplies a separating curve and a disc presentation. Consecutive attachments are checked explicitly. The certificate records the measured maximum boundary size  $W$ , not an estimate of the best possible girth.

When no order is supplied, the implementation first tries the ordinary fast greedy order. If its certificate fails, a polynomial search attempts a connected-prefix, connected-suffix order on the loopless two-connected projection. It repeatedly chooses a vertex attached to the prefix whose removal leaves the suffix connected, breaking ties by the next cut size. The block decomposition of the connected suffix guarantees a permissible extension under the stated two-connectedness hypothesis. Direct candidate connectivity checks give a conservative  $O(n^3)$  construction cost. The resulting order is reverified. If construction fails, the original order remains valid for exact computation and only the stronger complexity certificate is withheld. Supplied orders are verified without replacement. All planning and verification is charged to the query deadline.

At each certified stage the implementation checks the actual retained degree counts against the binomial bound. This is a diagnostic consequence of the proof, not a substitute for the geometric

hypothesis. Tests also compare matching-by-degree profiles across standard, residue, and adaptive exhaustive cancellation. Optional dense coefficient operations preserve the same exact category and can be used as well.

## 20.4 A uniform bound without the all-state-circle factor

Put

$$B(n, K) = \sum_{j=0}^{\min(n, K)} \binom{n}{j}.$$

Retained population is at most  $2B(n, K)$ . One crossing creates at most eight delooped children per object, so even the fully allocated temporary extension has population  $O(B(n, K))$ . There are quadratically many possible entries and linearly many pivots; a pivot changes quadratically many entries. Each coefficient computation costs  $\text{poly}(n, W)2^{O(W)}$  bit operations. Thus the new strategy has the bounds

$$T \leq \text{poly}(n, W)2^{O(W)}B(n, k+1)^3, \quad S \leq \text{poly}(n, W)2^{O(W)}B(n, k+1)^2.$$

The implementation disables the cross-stage shape cache for this strategy and caps its per-stage coefficient-composition memo at 4096 entries. This avoids silently charging cubic memo storage as quadratic matrix storage. Disc matching and plan metadata has  $2^{O(W)}$  possible types per stage; accumulated stage labels add polynomial factors.

For  $k = O(\log(n+2))$  and  $W = O(\log^2(n+2))$ , these bounds are quasi-polynomial. Unlike the previous elementary support-window estimate, this bound has no all-zero smoothing circle factor. It requires a certified nice order and this particular cancellation schedule. It is not asserted for early-allocation pruning or arbitrary uncertified frontiers.

The API `khovanov_minimal_window_auto` accepts an arbitrary raw interval  $[a, b]$ . On one side it computes through upper degree  $b$  and returns only the requested interval. Mirroring instead computes through  $n - a$  and maps degrees back by  $h \mapsto n - h$ . The default mirror choice minimizes this raw upper depth; it does not estimate wall time. The complexity parameter is the selected upper depth, not the width  $b - a$ . The `binomial_bound_certified` field records whether the nice-order premise was established. A failed certificate does not weaken the exactness of the returned queried ranks.

Recognition can use this strategy inside the existing bounded widening portfolio. Its same normalized-band comparison, local allowance, global limits, restart rule, and complete fallback remain in force. Agreement with the unknot on a partial band is still inconclusive. Widening restarts; it does not attempt to reconstruct discarded chain groups. No new default policy or general quasi-polynomial verdict guarantee is introduced.

## 20.5 Raw endpoint depth has an exact padding obstruction

Insert  $s$  cancelling positive-negative crossing pairs into a diagram. Its knot type and normalized Khovanov homology do not change, while its crossing count rises by  $2s$  and its negative PD crossing count by  $s$ . Therefore the raw profile obeys

$$\beta_{D_s}(r) = \beta_D(r - s).$$

Every discrepant degree relative to the unknot profile shifts by  $s$ , and its distance from either raw endpoint increases by  $s$ . Such padding can be inserted with constant additional frontier width. Consequently bounded width alone does not force a logarithmic detecting endpoint depth. Ordinary diagram simplification removes this particular padding, so this is an obstruction to an unqualified raw-window theorem, not a lower bound for the whole preprocessing pipeline. A

theorem covering all reduced inputs would need additional information. Even shallow agreement on an unknot needs a positive certificate or eventual full coverage.

## 20.6 Validation and comparison with the current window engine

The integrated suite passes 277 tests. New checks compare 100 random knot braids and arbitrary intervals with complete production homology, using three exhaustive reducers and forced dense composition on the first 15 inputs. They check intermediate  $d^2 = 0$ , certified binomial inequalities, mirror labels, invalid orders, padding shifts, time and allocation limits, and the guard-disk counterexample. The CLI and recognition portfolio are tested separately, including an agreeing partial query followed by exact fallback. The actual stage profiles are also compared across reducers.

The archive’s 14 tests pass locally. Its separately rerun independent cube audit compares 393 windows on 100 diagrams, checks 1772 certified stages and 15 padding cases, and reports no failures. Its archived adapter is not applied: the production implementation here targets the current array-based **FastScan**, with its existing coefficient engines and adaptive cancellation. Source archive files remain unchanged; review runs use a scratch extraction. Local logs are retained under `synthesis/data/`.

| Input              | Full/support | Full/minimal | Support/minimal | A/A   |
|--------------------|--------------|--------------|-----------------|-------|
| weaving_14         | 4.52         | 4.09         | 0.895           | 1.002 |
| torus_3_4          | 1.67         | 1.29         | 0.777           | 0.996 |
| padded_trefoil     | 1.74         | 1.20         | 0.708           | 1.009 |
| conway             | 1.63         | 1.35         | 0.834           | 0.994 |
| kinoshita_terasaka | 1.72         | 1.43         | 0.831           | 1.017 |
| stress_braid5_36   | 425.47       | 323.49       | 0.792           | 0.999 |

Ratios are medians of paired times; larger than one favors the denominator.

These seven-round paired measurements use common supplied crossing orders and depth-two raw queries. Full homology and a window have different output contracts; both window strategies answer exactly the same interval. A/A compares identical full-homology calls. Order construction and imports are excluded, while the minimal strategy’s certificate verification is included. These are backend query measurements, not default recognition speedups. In particular, a supplied nice order can differ from the ordinary recognizer’s preferred order.

The new strategy is slower than allocation pruning on all six cases. It sometimes saves retained guard objects nevertheless: on the 14-crossing weaving case, peak pre-cancellation allocation is 492 rather than 516, versus 5046 for full homology in that order. On Conway it instead allocates 84 objects versus 42 for the support strategy, because it constructs the temporary degree needed for cancellation. Its practical tradeoff is an additional proved size guarantee under explicit hypotheses, not a measured replacement for the faster default on this corpus.

## 21 Exact Potts filters and independent component tensors

Report 17 contributes an optional scalar obstruction before the complete homology fallback. The integrated selectors are `potts5`, `potts-exact`, and `potts-exact-factorized`; the existing `matching` Jones backend remains the default. These selectors are independent of the Khovanov backend and of adaptive cancellation. Potts representations of Jones evaluations and width-sensitive contraction are established techniques, including the tensor implementation of Meichanetzidis–Kourtis [26] and Maria’s parameterized analysis of quantum invariants [27]. The contribution here

is the audited implementation, exact coefficient representation, component factoring, and explicit resource and fallback contracts.

### 21.1 Normalization without approximate algebraic numbers

Let  $D$  be a nonempty validated plane knot diagram,  $G$  either checkerboard Tait multigraph,  $v$  its number of vertices, and  $r$  the diagram writhe. Loops and parallel edges are retained. At crossing  $e$ , let  $\epsilon_e \in \{1, -1\}$  be the bracket exponent of the smoothing that omits its Tait edge; put  $s = \sum_e \epsilon_e$ . A subset  $S$  of included edges produces

$$c(S) = 2k(S) - v + |S|$$

smoothing circles. Indeed a regular neighborhood of the spanning plane subgraph has Euler characteristic  $v - |S| = 2k(S) - c(S)$ , including its isolated vertices. With  $\delta = -A^2 - A^{-2}$  and the full bracket of one circle equal to  $\delta$ , expansion of the smoothing sum gives

$$B_D(A) = \delta^{-v} A^s \sum_{S \subseteq E} (\delta^2)^{k(S)} \prod_{e \in S} \delta A^{-2\epsilon_e}.$$

At the integer specialization  $q = \delta^2$ , expanding a spin product shows that the sum is a partition function: each selected edge requires equal endpoint colors, so the assignments constant on  $(V, S)$  number  $q^{k(S)}$ . Writing  $x = A^4$  gives

$$q = x + 2 + x^{-1}, \quad Z_G = \sum_{\sigma: V \rightarrow [q]} \prod_{e=uv} \begin{cases} -x^{-\epsilon_e}, & \sigma(u) = \sigma(v), \\ 1, & \sigma(u) \neq \sigma(v). \end{cases}$$

The normalized Jones polynomial is  $V_D(A^{-4}) = B_D(A)/(\delta(-A^3)^r)$ . Substitution of  $\delta = -(x + 1)A^{-2}$  yields

$$V_D(x^{-1}) = (-1)^{r+v+1} x^h \frac{Z_G}{(x+1)^{v+1}}, \quad h = \frac{s - 3r + 2(v+1)}{4}. \quad (10)$$

For a knot,  $h$  is an integer. The residue modulo four of  $s_\sigma - 3r + 2(c_\sigma - 1)$  is unchanged by a smoothing flip. In the Seifert state  $s_\sigma = r$  and  $c_\sigma - 1 \equiv r \pmod{2}$ ; in the all-omitted state  $c_\sigma = v$ , giving the assertion. Thus the exact obstruction compares  $Z_G$  with

$$Z_U = (-1)^{r+v+1} x^{-h} (x+1)^{v+1}.$$

No fourth root or division by  $x+1$  is computed. The crossing-free knot is handled separately with  $Z_G = Z_U = q$ .

For every integer  $q \geq 5$ , work in  $R_q = \mathbb{Z}[x]/(x^2 - (q-2)x + 1)$ . The discriminant lies strictly between  $(q-3)^2$  and  $(q-2)^2$ , so this polynomial is irreducible and each element has a unique integer pair  $(a, b)$  representing  $a + bx$ . Here  $x^{-1} = q - 2 - x$  and

$$(a, b)(c, d) = (ac - bd, ad + bc + (q-2)bd).$$

Multiplication by an equal-spin edge uses just additions and multiplication by  $q-2$ :

$$-x^{-1}(a, b) = (-(q-2)a - b, a), \quad -x(a, b) = (b, -a - (q-2)b).$$

Partition inequality therefore certifies a nontrivial Jones value exactly. Partition equality is inconclusive. The exact implementations default to  $q = 6$ , whose roots are  $2 \pm \sqrt{3}$ ; no numerical embedding is chosen. Witness pairs are serialized as signed hexadecimal strings, avoiding Python's limit on conversion of very large integers to decimal strings.

## 21.2 A half-frontier bound for every supplied crossing order

At a cut in the crossing order, let  $w_i$  count shadow edges joining a processed crossing to an unprocessed one. A Tait vertex is active when it is incident to both processed and unprocessed edges; write  $f_i$  for their number. Traverse the cyclic boundary walk of an active selected-color face. The processed/unprocessed marks change at least twice. Each change traverses a cut shadow edge. A cut edge has just one side of the selected color and can be charged to just that face. Hence

$$2f_i \leq w_i. \quad (11)$$

This argument allows repeated vertices and edges in face walks, and does not require a common-disk ordering or a Catalan matching hypothesis.

The single-table algorithm introduces a spin at its first incident edge, applies the edge weight, and forgets spins at their last incident edge. Its key records the equality pattern of active colors, canonically naming classes in first-occurrence order. Coefficients aggregate *all* labeled assignments in an orbit. An introduced spin can join any existing class with multiplicity one, or create a new class with multiplicity  $q - k$  when  $k$  classes already occur. Restriction and recanonization implement forgetting. There are at most  $q^{f_i}$  keys; an edge introduces at most two extra vertices. At fixed  $q$ , the work is

$$\text{poly}(n, w) q^{w/2}, \quad w = \max_i w_i,$$

including polynomial bit arithmetic. To justify the latter, each coefficient after  $i$  edges and  $I$  introduced vertices is a sum of at most  $q^I$  assignments with pair norm bounded by  $(q - 1)^i$ . Therefore its bit length is  $O(n \log q)$ , since  $v \leq n + 1$ . The normalization pair also has that bit bound. This is a scalar-query bound, not a bound on full homology.

## 21.3 Independent processed components and exact orbit division

The factored implementation keeps one tensor for each connected component of processed Tait edges. A vertex forgotten from the frontier does not erase interactions through it. Consequently this decomposition follows processed connectivity, not connectivity of just the active vertices. Relative color equalities between two components become relevant only when an edge joins them. A closed component contributes a scalar factor.

For a pattern using  $k$  colors, write  $(q)_k = q(q - 1) \cdots (q - k + 1)$ . Its aggregate coefficient  $C$  is  $(q)_k$  times its per-assignment value, because simultaneous color permutation preserves the partial partition sum. When joining patterns with  $k$  and  $\ell$  classes, enumerate partial bijections between their classes. An alignment with  $K \leq q$  total classes has pre-edge aggregate coefficient

$$\frac{C_L}{(q)_k} \frac{C_R}{(q)_\ell} (q)_K. \quad (12)$$

The integer coordinates of each division must have zero remainder; the code checks this before multiplying pairs. The operation is exact integer division, not division in a finite field or a numerical approximation. Each combined equality pattern determines its two restrictions and alignment uniquely, proving that the merge neither loses nor duplicates assignments.

There are at most  $q!$  relative alignments per pair of keys. Let  $g$  be the largest number of active vertices in any one processed component after an edge. Only the current edge's endpoints can be forgotten at that step, so a merged component has at most  $g + 2$  vertices before forgetting. Thus the Cartesian product of its two input key sets is bounded by  $q^{g+2}$ , not merely by the weaker  $q^{2g}$ . The resulting bound is

$$\text{poly}(n, g) q! q^{g+2}$$

coefficient and key work, with  $O(nq^g)$  logical keys and polynomial coefficient lengths. Here  $g \leq \max_i f_i \leq w/2$ . The extra alignment work can dominate when factoring saves few states.

## 21.4 A specialization collision that changing primes cannot repair

Report 17 analyzes  $W_m = (\widehat{\sigma_1 \sigma_2^{-1}})^m$ . It is a knot exactly when  $3 \nmid m$ . Its three-braid Jones trace identity gives, at  $t + t^{-1} = q - 2$ ,

$$V_{W_m}(t) = q - 2 + T_m, \quad T_0 = 2, \quad T_1 = 3 - q, \quad T_{m+1} = (3 - q)T_m - T_{m-1}.$$

At  $q = 5$  this is  $3 + 2(-1)^m$ , equal to one for every odd  $m$ . In particular the nontrivial members with odd  $m \geq 5$  are missed even in characteristic zero. Changing primes or using the conjugate root preserves this equality. For  $q \geq 6$ , set  $a = q - 3 \geq 3$  and  $U_m = (-1)^m T_m$ . Then  $U_0 = 2$ ,  $U_1 = a$ ,  $U_2 = a^2 - 2 > a$ , and the recurrence proves strict increase thereafter. Thus  $a + 1 + (-1)^m U_m \neq 1$  for every  $m \geq 2$ . For example,  $V_{W_5} = -119$  at  $q = 6$ . This proves detection for this family only. The existing short-braid stage already decides these inputs when their source braid is available.

## 21.5 Integration, budgets, and validation

The optional filter runs in the same place as the existing matching Jones filter and reuses its crossing order. A mismatch returns **KNOTTED**; equality or local state/transition exhaustion continues the existing Alexander, simplification, window, and complete homology stages. The global cooperative deadline still returns **UNKNOWN**. `-jones-backend` selects the filter in recognition, while `jones-backend` selects a standalone query. The standalone query now exits 3 with an inconclusive result when its resource limit is reached: a nonempty error dictionary is never interpreted as an obstruction. Invalid options exit 2.

State limits count represented keys. Inputs and a replacement table can coexist during a merge, and normalized lists and alignment caches consume additional memory, so this is not a resident-memory limit. Transition limits count actual orbit updates, including relative alignments. At fixed  $q \leq 6$  the alignment cache has finitely many bounded entries; larger color counts use lazy alignment generation. The factored wrapper additionally checks the global deadline after final normalization. The archived source remains unchanged.

The archived suite passed 172 tests; the first integrated checkpoint passed 315 tests. The retained independent full-smoothing oracle constructs Laurent Jones polynomials without Tait graphs or production ring multiplication. It checked 248 diagrams and 30,116 cube states, followed by 4,464 comparisons per exact backend over three color counts, two shadings, and three orders. Another 45 weaving checks reached 98 crossings. These are correctness observations, not complexity proofs. The report's certificate replayer, which recomputes the same backend, is not presented as an independent oracle. The ring-valued Fitting proposals in report 17 are not integrated by this change.

## 21.6 Measurements and the quasi-polynomial objective

The reproducible driver `fast/benchmark_potts.py` measures seven shuffled paired rounds with an exact- $q = 6$  A/A control. Kernel timings exclude construction of their common supplied order; the fragmented-order case is deliberately adverse. A separate normal-pipeline comparison includes diagram construction, ordering, cheap certificates, and fallbacks. Matching and modular  $q = 5$  arms evaluate different specializations, so their timing ratios alone do not establish equivalent detection power. Detailed measurements are recorded in `fast/results/potts_20261008.json`.

| Input                          | Exact keys | Factored keys | Exact/factored | Pipeline gain |
|--------------------------------|------------|---------------|----------------|---------------|
| Weaving, 10 crossings          | 5          | 6             | 0.799          | bypassed      |
| Weaving, 98 crossings          | 5          | 6             | 0.934          | bypassed      |
| Fragmented order, 16 crossings | 4111       | 205           | 23.612         | bypassed      |
| Conway                         | 5          | 6             | 0.803          | 1.143         |
| Kinoshita–Terasaka             | 5          | 6             | 0.734          | 1.152         |
| Stress braid, 36 crossings     | 15         | 16            | 0.787          | bypassed      |
| Hard unknot, 8 crossings       | 2          | 3             | 0.749          | bypassed      |

Here “Exact/factored” is the median paired kernel-time ratio; a value above one favors factoring. “Pipeline gain” compares default matching recognition with the single-table exact- $q = 6$  option, including construction and all earlier stages. On Conway and Kinoshita–Terasaka, every compared pipeline reached its Jones filter and returned **KNOTTED**. The factored option’s corresponding pipeline ratios were only 1.069 and 1.053. The other five cases were decided earlier, so their pipeline timings measure noise and common stages rather than a benefit from Potts evaluation.

Factoring reduced transitions from 30,197 to 1,558 on the fixed fragmented order but was slower in all six other kernel cases. The exact A/A ratios ranged from 0.973 to 1.159, making small timing changes unsuitable for a broad speedup claim. The robust large saving on the adverse order supports keeping the factored implementation available; these data do not support making it the default. Nor does the modest normal-pipeline improvement on two fixtures establish superiority of the six-color specialization on unseen diagrams.

At fixed  $q$ , a supplied order with  $w = O(\log^2 n)$  makes this filter quasi-polynomial. The stronger component parameter  $g$  can make a poor global frontier affordable. Neither guarantees a suitable order for every diagram, nor that the scalar differs from the unknot value. The fallback therefore retains its exponential worst case. This integration adds options for a future adaptive scalar-query policy; it does not change the existing progress-based adaptive Khovanov cancellation policy or promote an unmeasured automatic Potts switch.

## 22 Disk-certified transfer after an adaptive pause

Report 21 supplies the geometric certificate needed to apply a finite radical transfer to actual scan prefixes. The integrated option **-reduction disk-adaptive** extends the existing adaptive policy: ordinary sparse cancellation first, the cheap binary survivor test second, and a bounded full-transfer attempt only if adjacent surviving degrees prevent that shortcut. Successful transfer retains the radical differential between those degrees. Failure of the geometric or local-work gate resumes the same partially reduced complex. The default remains **standard**. This is an implementation of another exact cancellation strategy within the local-complex framework of Bar-Natan [22], not a new recognition criterion or a general quasi-polynomial algorithm.

### 22.1 Certifying the boundary from the processed rotation system

For a processed prefix  $P$ , pair the darts of every internal edge and fix each frontier dart. Call this permutation  $\alpha$ ; let  $\sigma$  rotate the four darts at each crossing. The cycles of  $\sigma\alpha$  are boundary walks of the ribbon neighborhood of  $P$ . The implementation checks that this prefix is connected, that

$$|V(P)| - |E_{\text{internal}}(P)| + \#\text{boundary walks} = 2,$$

and that all frontier darts occur on one boundary walk. These conditions certify a genus-zero neighborhood with one marked boundary component. Filling its unmarked holes gives a disk. In a validated connected full diagram, no unprocessed crossing component can hide in such a hole: a connection to the rest would produce a frontier dart on that boundary.

Reading the marked boundary gives the common cyclic endpoint order. Every smoothing is embedded in this disk, so its matching is noncrossing in that order. Delooping closed circles leaves endpoint pairings unchanged. The code also checks all distinct live matching types against the order, using the usual stack test for nested arcs. Numeric edge labels do not themselves specify an order. In particular the three pairings of four points cannot all be noncrossing in one cyclic order; applying a disk-algebra theorem to them without this check would be invalid.

The certificate records the boundary walks, vertex and internal-edge counts, cyclic order, and a digest of the actual prefix. The digest binds data but does not authenticate a source. Replay recomputes all the fields. Certificate construction is linear in indexed prefix darts; checking every prefix independently costs  $O(n^2)$  such graph operations. The adapter records the actual supplied scan order and verifies its current frontier against the certificate. It does not replace a good greedy order automatically.

The report also supplies a polynomial ear-insertion construction of connected prefix-and-suffix orders for biconnected crossing graphs. Such a cut admits the disk certificate, but the construction need not preserve a small width. The helper is retained for experiments and independently checked on smoothing states; no claim of a width approximation or a universally good order follows.

## 22.2 Retaining the maps that the binary profile cannot determine

For a frontier of  $2b$  points, let  $J$  be the positive-weight radical in its disk matching category. A dotted basis map from matching  $a$  to matching  $c$  has weight  $b - k(a, c) + 2|S|$ , where  $k(a, c)$  is the number of overlay circles. Composition adds weights; the only weight-zero basis maps are identity maps of equal matchings. The earlier radical analysis gives  $J^{2b} = 0$  for  $b \geq 1$ . Briefly, the weight bound first gives  $J^{2b+1} = 0$ ; a potentially nonzero product of  $2b$  positive factors would consist entirely of undotted weight-one surfaces and produce a fully dotted endomorphism. Over  $\mathbb{F}_2$ , an undotted positive-genus component vanishes and every surviving genus-zero component leaves an output circle undotted. The fully dotted term is therefore impossible. If all positive maps preserve their matching, the square-free endomorphism ring gives the stronger exponent  $b + 1$ .

Split the actual differential as  $d = d_0 + \delta$ , retaining in  $d_0$  only the constant coefficient of maps between equal matchings. Undotted maps between distinct matchings belong to  $\delta$ ; bit zero alone is not an identity test. Binary elimination constructs a contraction onto the residue homology with inclusion  $i$ , projection  $p$ , and degree-minus-one homotopy  $h$ :

$$pi = 1, \quad d_0h + hd_0 = 1 + ip, \quad hi = ph = h^2 = 0, \quad d_0i = pd_0 = 0.$$

All three maps have scalar identity coefficients. Hence nilpotence makes the following inverse series finite:

$$\begin{aligned} I &= (1 + h\delta)^{-1}i, & D &= p\delta(1 + h\delta)^{-1}i, \\ H' &= (1 + h\delta)^{-1}h, & P &= p(1 + \delta h)^{-1}. \end{aligned}$$

These are the characteristic-two perturbation formulas; the standard homological perturbation lemma supplies the deformation retract [29]. In particular

$$D = \sum_{j=0}^{2b-2} p\delta(h\delta)^j i$$

for  $b \geq 1$ , with at most  $b$  positive factors needed in the matching-preserving case. At a closed frontier  $D = 0$  and binary homology already suffices. The output has exactly the previously predicted minimal matching multiplicities, with its nonzero adjacent-degree maps intact.

The kernel streams one residual column at a time: apply  $\delta$ , accumulate its projection by  $p$ , and replace the current vector by its image under  $h\delta$ . It computes an additional final zero check rather

than silently cutting off a nonzero term. Unexpected failure of nilpotence raises an algebraic error; it does not become a local resource decline or a knot verdict. Optional full certificates include  $I, P, H'$  and are verified by separate typed matrix multiplication. The checks include both square-zero equations, both chain-map equations,  $PI = 1$ ,  $dH' + H'd = 1 + IP$ , and the three side conditions  $H'I = PH' = (H')^2 = 0$ .

### 22.3 The adaptive implementation and its transactional boundary

As before, a stage permits  $\max(256, 4N)$  ordinary Schur-update pairs before pausing between complete pivots. A successful cheap residue/degree-gap test finishes the stage immediately. Otherwise the disk option takes a compact snapshot and attempts the geometric certificate and finite transfer. The new differential and reverse incidences are built separately, and all five object arrays are installed together only after the last budget check. The old coefficient-result cache is cleared at installation; typed algebra plans remain valid. A declined attempt leaves the old differential and its completed sparse cancellations intact.

The default experimental allowance is  $\max(4096, 64N)$  cooperative polls for one attempt. Geometry traversal, contraction columns, series steps, coefficient products, and scalar incidences poll this guard. The global deadline is checked first at each poll; global exhaustion still raises `ScanLimit` and becomes `UNKNOWN` in recognition. Local exhaustion only declines the attempt. This poll allowance is neither a bit-operation budget nor a memory ceiling: binary vector work and a coefficient composition can cost more than one elementary operation. There is no competitive-runtime theorem for this scheduler. Direct kernel calls can perform the complete finite transfer without that experimental allowance.

The new policy is available to raw homology, recognition, allocation-pruned windows, and minimal windows. The window adapter records each processed crossing before either its ordinary or pruned extension calls cancellation, so the geometric certificate remains bound to the correct prefix. Existing component coefficient engines are compatible; component-owner arrays, saturated weights, races, and alternative scanner representations are not silently reinterpreted. The same standard-scanner restrictions as the earlier adaptive policy apply. Evidence includes attempts, completed transfers, geometry declines, local-budget fallbacks, transfer compositions, and maximum radical depth. Full replay certificates are an explicit kernel diagnostic, not an extra default allocation during recognition.

### 22.4 Cost separation and the missing global parameter bound

Let  $N$  be the pre-reduction object count,  $r$  the residue survivor count,  $E_\delta$  the number of nonzero radical entries, and  $S_h, S_p$  the numbers of scalar incidences in the homotopy and projection. Write  $A_b, M_b$  for coefficient addition and correctly typed composition costs. The column algorithm has the conservative mixed-cost bound

$$O(N^3) + O(N + E_\delta + \ell r(E_\delta M_b + (S_h + S_p)A_b)), \quad \ell \leq 2b.$$

The first term is binary contraction. Dense coefficient work is thus  $O(brN^2M_b)$ , instead of a generic  $O(N^3M_b)$  scalar-pivot bound. It can help when  $r \ll N$  and sparse elimination would create expensive fill. It need not help sparse matrices. The product cache is bounded at 4096 entries; optional full certificates require additional storage and work.

For a fixed disk-certified full scan, let  $B$  be its maximum boundary half-width and  $R$  its largest minimal explicit object count. Attaching one crossing creates at most eight objects per old object, so stage input size is at most  $8R$ . Coefficients occupy at most  $2^B$  bits and even elementary composition has cost  $\text{poly}(B)2^{O(B)}$ . Noncrossing matching types number at most  $4^B$ . Charging

algebra caches and geometry as well as matrices yields

$$\text{poly}(n, 1 + R)2^{O(B)}.$$

This conditional explicit-scan bound is compatible with ordinary exhaustive pivots too; the new transfer improves how expensive coefficient work is organized. It does not prove smaller intrinsic minimal multiplicities. The condition  $B + \log(1 + R) = O(\log^2 n)$  is sufficient for quasi-polynomial time, but no universal extraction theorem for those parameters is supplied.

## 22.5 Why changing crossing order alone cannot suffice

Report 21 constructs an unknot diagram on an  $m \times m$  crossing grid, with  $n = m^2$ . Pair exterior endpoints adjacent in their remaining cyclic order, always joining distinct open paths until only one closed strand remains. This adds no crossings and preserves the internal grid as a spanning subgraph. Make the first visit to each crossing an overpass; the result is descending and hence an unknot.

Every ordering has a prefix containing between  $m^2/3$  and  $2m^2/3$  vertices. If at least  $m/3$  rows are mixed across the prefix cut, each contributes a horizontal cut edge. Otherwise there is both a full and an empty row: mixed rows alone cannot contain either side's required size. Each column then contributes a vertical cut edge between those rows. Thus every ordering has at least  $m/3$  raw frontier edges, or  $B \geq m/6 = \Omega(\sqrt{n})$ . This refutes a universal polylogarithmic *raw-order-width* strategy. It is not a recognition lower bound: the descending certificate and ordinary Reidemeister simplification bypass these diagrams. In the production audit, all generated inputs through 64 crossings were recognized before homology; independent cube ranks were checked only through nine crossings.

## 22.6 Validation and measured scope

The report's 52 tests pass unchanged. Its source-derived fixture was audited against the pinned Git blobs for `scan_fast.py`, `planar.py`, and the scoped geometry definitions; executable ASTs agree after stripping documentation. The production suite now passes 344 tests, including 100 seeded small diagram comparisons with an independent cube, intermediate minimal-profile comparisons, adjacent-map synthetic checks, failure atomicity, both window implementations, coefficient composition, and global limits. An eager test control deliberately exercises transfer on actual prefixes; this is distinct from observing a production adaptive switch.

The separately rerun geometric audit accepted 71 of 80 order candidates, checked 14,658 complete smoothing states and 71 independent cube ranks, and generated and verified seven typed full chain-homotopy certificates. Data and the source audit are retained under `synthesis/data/`; the reproduction driver is `fast/check_disk_geometry.py`.

Seven shuffled paired rounds in `fast/benchmark_disk_transfer.py` compare fresh states under ordinary, A/A, residue-adaptive, and disk-adaptive policies. Dense synthetic two-term blocks with nonzero survivor maps gained 1.126, 2.948, 4.078, and 3.307 times for the four retained cases; every disk run made one successful full-transfer call. Independent binary regular- representation homology agrees before and after reduction. These synthetic blocks, including their coefficient-degree mixtures, are not asserted to be actual knot-prefix complexes.

The sparse diagonal control stayed on ordinary pivots: no transfer attempt was made, and its ratio was 0.959 rather than the eager prototype's large regression. The four full diagram scans (Conway, Kinoshita–Terasaka, the 36-crossing stress braid, and the eight-crossing hard unknot) made zero full transfer calls. Their ratios ranged from 0.960 to 1.010, while A/A controls ranged from 0.992 to 1.012. Normal recognition decided all four in earlier certificate or invariant stages. These measurements support the overhead guard on these controls; they establish no recognition

speedup from full transfer. Defaults remain unchanged and the goal of general quasi-polynomial recognition remains open.

## 23 Exact long-run recurrences and streamed macro homology

Report 18 extends the existing twist macro complex in two directions: an exact compressed homological profile for one sufficiently long run, and a separate backend that streams degree layers to reduce retained storage. The module `fastunknot.twist.continuation` exposes both on signed run-encoded braid input, with structural recognition before homology in its recognition mode. Its homology mode always computes the requested invariant. The existing production defaults and `backend="twist"` adapter retain their original macro implementation. Ordinary recognition additionally offers the opt-in `-braid-profile` structural evaluation on checked source words.

### 23.1 The repeated differential, not just repeated dimensions

Select a run  $\sigma_i^{\epsilon m}$  and freeze the remaining context. Let its macro homological support be  $[a, b]$ , where  $a$  is the sum of negative context exponents and  $b$  the sum of positive ones. Write  $W = b - a$  and  $m_0 = W + 2$ . When the selected run is in its turnback sector  $E$ , the direct sum of all context groups is a fixed vector space  $C$  of dimension  $M$ . Let  $D$  be the context differential and  $L$  the sum of the dots on the cap and cup. The repeated interior differential is the *same* square-zero operator

$$F = D + L, \quad \eta = M - 2 \operatorname{rank} F.$$

Indeed  $D^2 = 0$ , each dot squares to zero, and distinct dot actions commute. The Frobenius identities make exterior dots commute with every context saddle, giving  $DL = LD$  and  $F^2 = 0$  over  $\mathbb{F}_2$ . The argument restricts to the reduced marked-circle subcomplex. This is an ungraded endomorphism statement about  $C$ ;  $D$  and  $L$  need not have the same original bidegree.

For a positive run, complete copies of  $C$  occur in total macro degrees  $b + 1 \leq h \leq a + m$ . At  $m = m_0$  there are exactly two adjacent complete copies, in degrees  $b + 1, b + 2$ , with boundary map  $F$  between them. For a negative run the complete copies lie in  $b - m \leq h \leq a - 1$ , and the reference map is the boundary in degree  $a - 2$ . Geometry depends on whether the selected sector is  $I$  or  $E$ , not on its positive internal index, which identifies these interior maps exactly.

Let  $P_{m_0}(z)$  be the reference reduced homological polynomial and  $\Delta = m - m_0 \geq 0$ . Split its positive-run profile into  $A_+$  in degrees at most  $b + 1$  and  $B_+$  in degrees at least  $b + 2$ . For the negative run split into  $A_-$  at most  $a - 2$  and  $B_-$  at least  $a - 1$ . Then

$$\begin{aligned} P_m(z) &= A_+(z) + z^\Delta B_+(z) + \eta \sum_{h=b+2}^{a+m-1} z^h, \\ P_{-m}(z) &= z^{-\Delta} A_-(z) + B_-(z) + \eta \sum_{h=b-m+1}^{a-2} z^h. \end{aligned}$$

The lower and upper end pieces retain both incident boundary maps, with the displayed shift where needed. Every inserted degree has  $F$  as both incoming and outgoing differential and therefore has dimension  $\eta$ . There are exactly  $\Delta$  inserted degrees, so

$$R(m) = R(m_0) + \Delta\eta.$$

The implementation extracts  $\eta$  from the reference *boundary rank*, not from either end-piece Betti number. Those Betti numbers need not equal the interior dimension. These formulas are proved identities of the macro complex; they are not extrapolations fitted to sampled ranks.

### 23.2 Why the slope is positive

Let  $J_E$  be the link obtained by replacing the selected run with its cap and cup. Coherent braid orientation forces the two outside paths at that box to join top ends to bottom ends. Adding the cap and cup therefore puts both marked arcs on the same actual component, regardless of the other components of the closure.

Jaeger’s weighted-dot sliding theorem states that the characteristic-two twisted complex depends on the sum of weights on each actual component [30]. Two unit weights on that component cancel. In the report’s explicit specialization, sliding across one crossing uses a reverse saddle  $H$  with  $H^2 = 0$  and  $DH + HD = X_p + X_{p'}$ ; the change of basis  $1 + H$  conjugates the corresponding dot-deformed differentials. Thus the ordinary reduced cube with the two-dot deformation has the homology of  $J_E$ . Relative local reductions in the other run boxes commute with the exterior dots, so this identification passes to the macro context as well. Consequently

$$\eta = \dim_{\mathbb{F}_2} \widetilde{Kh}(J_E; \mathbb{F}_2) > 0.$$

Positivity follows from the nonzero reduced Jones Euler polynomial of a nonempty link: its value at one has absolute value  $2^{\ell-1}$ . A free integral homology summand therefore survives over  $\mathbb{F}_2$ . This argument does not use the generally false assertion that adding an arbitrary null-homotopic operator preserves homology.

The dot argument identifies a total dimension after collapsing grading. The original run’s homological profile follows instead from the separate identification of the complete groups and both incident maps above. Quantum grading is not returned. The raw continuation API retains macro degrees. Converting to the original production cube convention requires  $h_{\text{cube}} = n_+^{\text{original}} - h_{\text{macro}}$  and, for a knot, doubling reduced dimensions. The shortened reference’s positive crossing count is not the original shift. Also, shortening can change component parity: recognition checks the original run list, and the reference may be a link.

### 23.3 Compressed output and a fixed-context bound

The tail algorithm keeps two finite sets of exceptional degree–dimension pairs and one constant interval, with inclusive endpoints. Exceptions and the interval are disjoint. Queries and total rank use integer arithmetic; explicit expansion is optional and capped. The tests include exponents with 100,001 bits while the two-strand reference still has three macro states and four reduced basis elements. This is an in-memory API test; the JSON frontend retains Python’s decimal-integer conversion limit.

The reference has  $2W + 2$  expanded crossings. At  $s$  strands a conservative basis bound is  $2^{s-1}3^{2W+2}$ , so even elementary assembly and elimination give  $2^{O(s+W)}$  reference work, with polynomial dependence on the binary length of  $m$  for reconstruction. For fixed context and strand count this removes enumeration of the long exponent. For growing context the reference can still be exponential. On a knot closure the context must touch all other strand separators, so  $s \leq W + 2$ ; a context of  $W = O(\log^2 n)$  yields a quasi-polynomial bound in the expanded crossing count  $n$ . This remains a restricted homology statement.

Since  $\eta > 0$ , an explicit list of the inserted nonzero degrees has at least  $\Delta$  entries. No algorithm returning that expanded list can be polynomial in  $\log m$  for fixed context. The compact output contract is essential, rather than an optional presentation improvement. The largest run is the default candidate. Two runs cannot both strictly dominate their complement, so this one-run recurrence cannot simply be iterated to compress every large exponent independently.

### 23.4 Direct structural evaluation on run-encoded braids

For each generator index let  $p_i, n_i$  count its positive and negative occurrences, and set  $P = \sum p_i$ ,  $N = \sum n_i$ ,  $e = P - N$ ,  $u_+ = |\{i : p_i > 0\}|$ ,  $u_- = |\{i : n_i > 0\}|$ . The braid's signed Seifert graph is a path multigraph on  $s$  vertices. On a one-component closure, the existing structural formulas become

$$e + s - 2u_+ - 1 \leq s(K) \leq e - s + 2u_- + 1, \quad 2g = P + N - s + 1.$$

Its homogeneity defect is the number of indices occurring in both signs. Genus zero certifies the unknot; positive homogeneous genus or an interval excluding zero certifies knottedness. A zero-containing interval alone is inconclusive. Sparse permutation tracking uses each exponent's parity to check the original component count without expanding the word or allocating an array for untouched strands. Missing a strand separator rejects a knot candidate immediately; otherwise  $s - 1$  is bounded by the number of runs. These checks are polynomial in the encoded input length.

The production PD convention reverses the Artin crossing sign. Public Seifert fields therefore use writhe  $-e$  and interval  $[-U, -L]$  when the Artin interval is  $[L, U]$ . Separate Artin fields make that conversion explicit. Tests compare the entire dictionary with the independent PD Seifert graph, rather than comparing only verdicts.

Every one-component dominant-run input is already structurally knotted. For a positive run of magnitude  $m \geq W + 2$ , the lower bound equals  $2(P - u_+) - d$ , with  $d = m + W - s + 1$ , and

$$2(P - u_+) - d \geq 2(m - 1) - (m + W - s + 1) = m - W + s - 3 \geq s - 1 > 0.$$

The negative case is symmetric. Thus tail timings do not represent a newly solved recognition family. The report's signature-packing diagnostic also adds no cases: its selected definite rank is at most  $P - u_+$  or  $N - u_-$ , so its successful inertia bound is dominated by the same interval. Its diagnostic and verifier are retained for reproducibility, but recognition uses the cheaper dominating calculation.

### 23.5 Streaming changes storage, with an explicit time tradeoff

The separate streamed backend enumerates bounded compositions of degree totals. Each macro state is visited once, without repeatedly scanning the whole Cartesian product for each degree. A depth-first enumerator prunes prefixes using suffix capacities and avoids recursion limits. Its total enumeration work is  $O(t \prod_j (|e_j| + 1))$  for  $t$  runs.

Only two adjacent state indexes are live. Local Frobenius maps retain circle indexes instead of exponentially large label tables; generated columns feed directly into binary elimination. Geometry and map caches have bounded entry and weighted-slot capacities. If square-zero checking is requested, two adjacent matrices are additionally retained. The limits distinguish total enumerated states and basis from live states, layer basis, matrix bit payload, column references, geometry vertices, and map slots. They are mathematical storage envelopes, not allocator or RSS reservations. The returned degree, chain-dimension, and boundary-rank dictionaries still grow with the number of degrees; two live state layers do not imply constant total memory.

After computing ranks of  $d_{h-1}$  and  $d_h$ , the value  $\beta_h = \dim C^h - \text{rank } d_{h-1} - \text{rank } d_h$  is final. On a knot, a cumulative finalized reduced rank greater than one certifies **KNOTTED** and permits early exit. Such a result explicitly has **reduced\_rank=null**, a lower bound, and **homology\_complete=false**. Agreement with rank one before completion cannot certify the unknot. Full homology mode never uses this early exit.

The combined RLE frontend checks structural certificates first, then uses the selected **tail**, **streaming**, or **macro** backend. It is an explicit interface, not an unmeasured automatic storage switch. Resource exhaustion returns **UNKNOWN**; CLI exit codes are 0 for completion, 2 for invalid input or configuration, and 3 for exhaustion. Tail's finite reference and reconstruction share one

cooperative deadline in the production integration, and mutable streaming budgets are copied and revalidated. Tail’s square-zero flag refers to the finite reference, not to an unbuilt enormous matrix. The ordinary source-word profile stage is opt-in and never assigns the whole source braid to an individual connected-sum factor.

## 23.6 Validation and measurements

The archive passed 254 tests unchanged. The production suite now passes 391, including 400 tail/macro profile comparisons, 80 independent cube comparisons, both signs and boundary thresholds, original component parity, huge binary exponents, streamed enumeration/maps/storage limits, partial-rank semantics, and deadline/budget regressions. Separate reruns produced 832 independently formulated recurrence comparisons, 220 cube comparisons, and 500 checks identifying the slope with a separately extracted ordinary turnback subcomplex. Their records are in `synthesis/data/twist-*-audit.json`.

Seven paired A/B/A rounds compare complete homology contracts, with tail’s compact profile expanded only for validation outside timing. At exponent 1025, the two-strand, figure-eight-context, and four-strand-context cases gained 38.70, 58.59, and 46.84 times respectively. Their reduced reference bases were 4, 93, and 126, versus 1027, 15393, and 18486 for full macro assembly. The no-compression control ratio was 0.974. Raw samples and the separate A/A control are retained in `fast/results/twist_tail_20261008.json`.

The separate eight-case streaming benchmark includes fresh A/A controls, seven shuffled rounds, and independent traced-allocation runs. Streaming was slower in every full-homology case: macro/stream ratios ranged from 0.229 to 0.694. Peak traced allocation ratios ranged from 1.020 to 3.064 in streaming’s favor. These are Python allocation peaks, not process RSS; early-decision arms have a different output contract and are recorded separately. Data are in `fast/results/twist_streaming_20261008.json`. This supports streaming as a storage option, with no default change. The structural benchmark separately measures encoded input versus expansion and recognition on fresh prebuilt diagrams; these scopes must not be combined into a general recognition-speedup claim. For the four-strand family with runs  $(1, m)$ ,  $(2, -(2m - 1))$ ,  $(3, m)$  and odd  $m = 17, 129, 1025, 4097$ , the source-profile option gained 2.296, 2.824, 2.905, and 3.334 times on fresh prebuilt diagrams. Its A/A ratios ranged from 0.997 to 1.022. These inputs have homogeneous positive genus despite an Artin Rasmussen interval  $[0, 0]$ , so the homogeneous criterion, not interval equality, supplies the verdict. The separate encoded-input comparison includes expansion, PD construction, and Seifert evaluation in its baseline; its ratios range from 94.4 to 29687.0. These large numbers primarily measure the benefit of preserving compressed input. Morton’s 11-crossing case remains a structural-only inconclusive control, with no full-pipeline timing in this experiment. All raw data are in `fast/results/braid_profile_20261008.json`.

## 24 Arithmetic continuations and checked Montesinos sources

Report 22 supplies a useful change of representation: some prefixes whose explicit homotopy complexes are necessarily large have short arithmetic descriptions sufficient for the actual closure decision. Production now implements its supplied-Montesinos classifier and a separate optional local obstruction. This is a complete polynomial decision on a specified source grammar, not an algorithm discovering that grammar in every PD. The incoming archive remains unchanged; this section describes the maintained integration.

## 24.1 What the source describes

An input consists of an integer  $e$  and separate finite rational tangles with ordinary continued fractions. Its knot is

$$N([e] + T(p_1/q_1) + \cdots + T(p_s/q_s)), \quad \gcd(p_i, q_i) = 1, \quad q_i > 0.$$

The summands remain separate throughout. Reducing their numerical sum would lose topological information: two  $1/2$  summands give uncanceled numerator 4, whereas the numerical sum is 1. A quarter rotation sends a slope to  $-1/x$ ; a rotation followed by crossing reversal gives  $1/x$ . These conventions agree with the rational-tangle operations in [31]. The compiler performs those operations on actual four-terminal crossing graphs, joins adjacent terminal pairs, and adds the numerator caps. A lazy crossing-switch bit avoids repeatedly mirroring earlier crossings during continued-fraction construction.

For each continued fraction, read coefficients backwards from  $(p, q) = (1, 0)$ :

$$(p, q) \mapsto (ap + q, p).$$

Every update matrix is unimodular. Hence no gcd computation is required, intermediate infinity is harmless, and the final pair is primitive. A final infinite slope is rejected. Write  $p_i = a_i q_i + \beta_i$  with  $0 \leq \beta_i < q_i$ , add all  $a_i$  to  $e$ , and discard integral summands. The remaining  $r$  pairs  $(\beta_i, \alpha_i)$  have  $0 < \beta_i < \alpha_i$  and  $\alpha_i \geq 2$ . Integral-twist transfer through rational summands preserves the numerator closure. Set

$$Q = \prod_{i=1}^r \alpha_i, \quad D = eQ + \sum_{i=1}^r \beta_i Q / \alpha_i. \quad (13)$$

The implemented accumulator starts at  $(D, Q) = (e, 1)$  and updates  $(D, Q) \mapsto (D\alpha + \beta Q, Q\alpha)$ , without cancellation.

**Lemma 24.1** (Component gate). *The numerator closure in this grammar is a knot exactly when  $D$  is odd.*

*Proof.* A primitive rational column modulo two specifies one of the horizontal, vertical, and crossed endpoint pairings, respectively  $(0, 1)$ ,  $(1, 0)$ , and  $(1, 1)$ . Twisting and reciprocal operations preserve this correspondence. If  $k > 0$  denominators are even, the open horizontal sum has  $k - 1$  internal closed strings and its numerator closure has  $k$  components. If  $k = 0$ , the number of components is one or two according as  $e + \sum_i \beta_i$  is odd or even. Formula (13) has odd parity exactly for  $k = 1$ , or for  $k = 0$  with the stated odd sum. For  $k \geq 2$  every term is even. This also covers the empty exceptional list.  $\square$

**Proposition 24.2** (Decision on supplied Montesinos knots). *After the component gate, the answer is UNKNOT precisely when  $r \leq 2$  and  $|D| = 1$ ; otherwise it is KNOTTED.*

*Proof.* Use the classical double-cover construction: the cover is a Seifert manifold whose group, in this convention, is

$$\langle h, x_1, \dots, x_r \mid [h, x_i] = 1, x_i^{\alpha_i} h^{\beta_i} = 1, x_1 \cdots x_r = h^e \rangle.$$

Rational tangle balls lift to solid tori; their meridians have slopes  $\alpha_i x_i + \beta_i h$ . Abelianizing this presentation gives homology order  $|D|$  when  $D \neq 0$ . If  $r \geq 3$ , kill  $h$  and all but three of the  $x_i$ . The resulting triangle group with orders  $a, b, c \geq 2$  is nontrivial. Here is an explicit verification avoiding an algorithm for triangle groups. Put  $\lambda = e^{\pi i/a}$ ,  $t = 2 \cos(\pi/b)$ ,  $s = 2 \cos(\pi/c)$ ,

$$u = \frac{s - \lambda^{-1}t}{\lambda - \lambda^{-1}}, \quad v = \frac{\lambda t - s}{\lambda - \lambda^{-1}}, \quad X = \begin{pmatrix} \lambda & 0 \\ 0 & \lambda^{-1} \end{pmatrix}, \quad Y = \begin{pmatrix} u & 1 \\ uv - 1 & v \end{pmatrix}.$$

Both determinants are one and the traces of  $Y, XY$  are  $t, s$ . Their distinct eigenvalues give  $X^a = Y^b = (XY)^c = -I$ . Thus  $X, Y, (XY)^{-1}$  define a nontrivial projective representation;  $X$  has projective order  $a$ . The cover cannot be  $S^3$  and the knot cannot be the unknot. This is a symbolic proof, not a numerical subroutine.

For at most two exceptional summands the closure is two-bridge, even though the open sum need not be rational. The rational-link classification gives the unknot exactly when its numerator parameter  $|D|$  equals one. In the two-summand case this is also the rational gluing criterion of [32, Theorem 5]. The Seifert-cover and local-closure foundations are discussed in [33, Theorem 4.9].  $\square$

This detects determinant-one nontrivial knots. For example, for every odd  $a \geq 3$  take slopes  $-1/a, 1/(a+2), 1/r$  with  $r = (a^2 + 2a - 1)/2$ . The signed pretzel expression is  $-a(a+2) + 2r = -1$ , so the normalized positive-denominator determinant has absolute value one. All three denominators exceed one, proving nontriviality. The usual genus-one Seifert matrix for odd pretzel parameters  $(p, q, r)$  is

$$\frac{1}{2} \begin{pmatrix} p+q & q+1 \\ q-1 & q+r \end{pmatrix}.$$

Its determinant is  $(pq + pr + qr + 1)/4 = 0$  here; its Alexander polynomial is a unit and its symmetrization has signature zero. Thus these easy arithmetic decisions include a family on which those two obstructions are inconclusive.

**Bit cost and output.** Let  $B$  include all binary coefficients, signs, and list delimiters. Each continued-fraction column has bit length bounded by the sum of  $1 + \lceil \log_2(1 + |a|) \rceil$  along its block. All final columns together have  $O(B)$  bits. Multiplying an  $s$ -bit partial entry by a  $b$ -bit coefficient costs  $O(sb + s)$  with schoolbook arithmetic; summing over the input costs  $O(B^2)$ . The divisions producing  $(a_i, \beta_i)$  and the uncanceled accumulator have the same bound, by charging products to both operand lengths. The classifier therefore uses  $O(B^2)$  bit operations and  $O(B)$  bits of integer/source data. This statement does not count Python object overhead as a single bit. The certificate has  $O(B)$  encoded bits; integers exceeding 1024 bits use exact hexadecimal records. The independent replay uses forward matrix products and full denominator-product sums, with a conservative  $O(B^3)$  bound because it also replays prefix size statistics. The reported maximum entry size concerns tracked states, not every temporary or peak memory.

## 24.2 Provenance, budgets, and integration

The compressed API is `montesinos_certificate`, taking  $e$  and the tangle list. The Diagram constructor `from_rational` (also `from_montesinos`) builds all crossings and validates the resulting spherical, one-component PD. Its default expansion ceiling is 100000 crossings, checked before allocating crossings. Input parsing and construction precede the ordinary recognition timer. The ordinary JSON grammar uses a `montesinos` object with `e` and `tangles`; its CLI expands that input, rather than silently switching to the compressed interface.

Only the checked constructor attaches a detached immutable source. Bare PD input has no such assertion. Mirroring negates all coefficients; serialized sources are reconstructed and must match the stored PD up to half-turns of individual rows. Multiple serialized source types are rejected. An initialized Diagram cannot be reinitialized with a different PD while retaining old provenance. Equality and hashing still depend solely on PD. These are supported API invariants, not protection against arbitrary mutation of interpreter internals.

The recognizer checks this source before other certificates; `use_rational=False` or `-no-rational` disables that stage. Arithmetic polls the same cooperative deadline and the recognizer checks again after certificate construction. The source is used only for the original whole diagram, before

factorization. The rank-two restart option continues to carry all recognition settings. PDs without this new source retain the previous pipeline. The general quasi-polynomial flag stays false.

### 24.3 Literal local obstructions with a checked disk

For a genuine two-string tangle with  $r \geq 2$  finite nonintegral summands, Nogueira–Salgueiro’s criterion says it embeds in some unknot exactly when  $r = 2$  and

$$p_1 q_2 + p_2 q_1 \equiv \pm 1 \pmod{q_1 q_2}.$$

We use only its negative direction. A verified occurrence failing this criterion proves the entire host knot nontrivial, regardless of the exterior. An allowed complement gives no positive verdict for the actual host. The implemented templates require initial  $e = 0$ , connected crossing shadow, four distinct ports, and no closed strand component.

The source compiler recomputes fractions and builds the literal crossing pattern. Its rotation system must be spherical with every port on one facial boundary in the declared order. Opposite-dart tracing must produce exactly two open strings. An occurrence maps crossings injectively into a freshly validated host PD. Offsets zero and two preserve the cyclic order and under-pair. Every internal edge must match and every other dart must lead outside the selected set, yielding four distinct cut edges.

These checks prove the needed disk condition. Take a regular neighborhood of the connected selected graph, ending halfway along its outgoing edges. Every remaining graph component attaches at a port, since the entire knot shadow is connected. All ports lie in one face, so the remaining graph lies there too. Fill the other neighborhood boundary components with their empty complementary disks. The result is a disk meeting precisely the four cut edges; thickening it gives a tangle ball. An abstract four-edge cut alone would not justify this argument.

Fixing one root image and its half-turn forces the image and rotation of every neighbor through internal edges. Connectedness leaves only  $2n$  root trials, each costing  $O(k)$  for a  $k$ -crossing template. The matching core thus costs  $O(nk)$  word operations. Public host validation also sorts labels, adding  $O(n \log(n+2))$ ; source reading and compilation must be charged separately. A huge zero-coefficient description is not free just because it expands to few crossings. Python dictionaries use the usual table-cost model in this count; indexed arrays give the corresponding deterministic algorithmic bound. The six default patterns have slopes  $(1/3, 1/3)$ ,  $(1/2, 1/5)$ ,  $(1/3, 1/4)$  and their mirrors.

`recognize_with_subtangles` is a separate opt-in wrapper. No match continues ordinary recognition with the remaining time; exhaustion yields UNKNOWN. The production integration also checks time after successful final verification, closing a gap in the delivered wrapper. Time values must be finite and nonnegative. Template compilation and PD validation are cooperative operations, not interruptible real-time allocations.

An abstract extension enumerating at most  $h$  coefficients and all separators, with coefficients in  $[-n, n]$  and total expanded size at most  $n$ , has at most  $\sum_{\ell \leq h} (2n+1)^\ell 2^{\ell-1} = (n+2)^{O(h)}$  candidates. Compiling and checking them gives quasi-polynomial discovery when  $h = O(\log n)$ . This is a restricted literal-witness theorem from report 22, not the implemented fixed catalogue and not a bound for every knot diagram.

### 24.4 Why arithmetic helps when a small linear summary cannot

Let

$$A = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}, \quad v_w = G_w(1, 1)^t, \quad x_w = G_w(3, 2)^t$$

for words  $w \in \{A, B\}^m$  and their literal matrix products  $G_w$ . Two twists realize each letter on rational tangles. The associated prefix and mirrored continuation have at most  $2m + 1$  and  $2m + 3$  crossings. All cross closures are knots: the two columns have parity  $(1, 1)$  and  $(1, 0)$ . The rational gluing test is  $|\det(v_u, x_w)| = 1$ .

For  $u = w$  the determinant is  $-1$ . For distinct words, their positive-cone images are disjoint:  $A$  sends a positive column into  $x > 2y$ , while  $B$  sends it into  $y > 2x$ ; remove the common prefix to prove the assertion inductively. Consequently  $G_w^{-1}v_u = (a, b)^t$  has one nonpositive and one positive integer coordinate. The determinant  $2a - 3b$  has absolute value at least two, and odd parity raises this to at least three. Thus the Boolean continuation matrix is the  $2^m$ -dimensional identity over every field. Distinct exact continuation states require at least  $m$  bits; a linear representation of all Boolean answers needs dimension at least  $2^m$ . Neither statement is a general running-time lower bound. The signed determinant matrix has rank at most two, and its nonlinear absolute-value-one readout decides the same family using  $O(m)$  bits and  $O(m^2)$  bit operations.

There is also an explicit full-complex obstruction inside unknots. Put  $G_k = (AB)^k$ ,  $v_k = (p_k, q_k)^t = G_k(1, 1)^t$ , and  $x_k = G_k(3, 2)^t$ . The whole closure is an unknot with  $N_k = 8k + 4$  displayed crossings. Its designated rational prefix has  $4k + 1$  crossings and

$$p_0 = 1, \quad p_1 = 7, \quad p_{k+2} = 6p_{k+1} - p_k, \quad p_k \geq (3 + 2\sqrt{2})^k.$$

Any full, explicitly delooped four-ended complex homotopy equivalent to that prefix needs at least  $p_k/2$  summands. Indeed, closing each matching and reducing on a fixed exterior marked arc yields at most two scalar generators per summand. Closing and applying the marked TQFT is additive and preserves chain-homotopy equations. Homology dimension is therefore at most twice the summand count, whereas its graded Euler characteristic at quantum parameter  $i$  gives  $\dim \widetilde{\text{Kh}} \geq \det = p_k$ .

If the quantum grading is preserved, the stronger bound is  $p_k + q_k$ . At  $i$ , a reduced closure with more than one circle contributes zero because  $i + i^{-1} = 0$ ; a one-circle closure contributes absolute value one. Numerator closure bounds one matching's multiplicity below by  $p_k$  and denominator closure bounds the other by  $q_k$ . Both are knots since  $p_k, q_k$  are odd. Orientations may be chosen separately: the orientation-free bracket complex changes only by global grading shifts, which do not change these absolute-value estimates. This obstruction concerns that specified prefix, full homotopy type, and expanded summands. It does not cover every order, binary multiplicity encodings, or the actual arithmetic decision procedure.

For adaptive recognition the consequence is concrete: bounding geometric frontier alone does not bound full surviving object count. When checked arithmetic structure is available, its decision stage should precede a full complex. For general PDs, discovering and composing such structures without expansion remains a research obligation; switching between existing complex representations alone does not prove the missing bound.

## 24.5 Production validation and measured scope

All 152 delivered tests pass unchanged. The maintained suite has 411 tests, including two additional resource/option regressions. The patch hash and all 81 delivered patched-file hashes agree with the patch manifest; all 82 source Git blobs agree with the pinned commit. The package-wide checksum launcher cannot run because the delivered tree lacks its `SHA256SUMS` file; the narrower checks are recorded explicitly in `rational-source-audit.json`. Independent reruns against production check 1200 sources: 718 knot-PD Alexander comparisons, 482 link component comparisons, and 1436 mirrored pickle checks. The local audit rejects 12 forged certificates and a crossing switch, checks 40 reorder/half-turn/relabel/mirror instances, and finds no false rejection on 31 known unknots. The identity diagnostic checks 1398101 matrix entries through  $m = 10$ . The checkpoint audit constructs 21 whole diagrams and adds 14 Alexander and six independent cube comparisons. These are finite implementation checks; the proofs above supply asymptotic conclusions.

`fast/rational_research/benchmark.py` retains seven paired rounds with fresh prebuilt Diagrams, one-second recognition budgets, and a 20000-object cap. Each round measures baseline, five source-enabled samples, then baseline. The after/before ratio is a drift control, not a separate randomized A/A trial. Construction plus new recognition is recorded separately; recognition-only speedups must not be read as construction-inclusive speedups. Local patterns use the same before/new/after protocol, with the template cache warm. Raw data and exact source hashes are in `fast/results/rational_20261008.json`.

| Input                | Crossings | Old ms  | New ms | Old/new or censoring   |
|----------------------|-----------|---------|--------|------------------------|
| Diagonal $m = 2$     | 12        | 2.953   | 0.0300 | 96.8×                  |
| Diagonal $m = 4$     | 20        | 6.157   | 0.0302 | 203.2×                 |
| Diagonal $m = 6$     | 28        | 15.494  | 0.0333 | 454.3×                 |
| Diagonal $m = 8$     | 36        | 121.454 | 0.0658 | 1896.4×                |
| Diagonal $m = 10$    | 44        | 133.061 | 0.0673 | object limit, no ratio |
| Diagonal $m = 12$    | 52        | 94.483  | 0.0394 | object limit, no ratio |
| Off-diagonal $m = 8$ | 36        | 0.102   | 0.0282 | 3.64×                  |
| Pretzel $a = 3$      | 15        | 1.154   | 0.0286 | 41.4×                  |
| Pretzel $a = 7$      | 47        | 3.446   | 0.0317 | 109.0×                 |
| Pretzel $a = 31$     | 575       | 52.195  | 0.0527 | 992.6×                 |

Ratios are medians of paired ratios, not ratios of displayed medians. The 44- and 52-crossing baselines stop at the object cap; they are not completed solves. Source-enabled construction plus recognition ranges from 0.175 to 0.700 ms on these diagonal cases, and is 5.37 ms for the 575-crossing pretzel. The largest compressed pretzel input uses a 65537-bit parameter and takes 1.46 ms median for arithmetic only; no PD is built. These compressed times describe a different input contract.

Local search loses on all five measured cases: old/new ratios are 0.682 (Conway), 0.634 (eight-crossing unknot), 0.256 (trefoil), 0.876 (planted two-summand pattern), and 0.160 (planted three-summand pattern). In the planted cases the old modular Alexander or Rasmussen stage is already cheaper. This is why local search remains explicit. Its proven rejection family permits arbitrarily complicated exteriors, but these measurements do not establish an end-to-end gain on that family.

## 25 Full transfer ordered by quantum grading

The graded kernel from report 23 supplies another exact response to slow sparse cancellation. It computes all maps between scalar survivors, including adjacent-degree maps, without the common-disk certificate required by Section 22. Production exposes `graded` and `graded-adaptive` as optional reduction policies. This integration covers the graded kernel; Section 26 reviews the report’s twist-threshold, succinct interface and geometric-dependency proposals.

### 25.1 Retaining the grading rather than reconstructing it afterwards

At a frontier of  $2k$  points let  $c(a, b)$  be the number of circles in the overlay of matchings  $a, b$ . A basis morphism dotted on a set  $S$  of those circles has intrinsic weight

$$\omega(a, b, S) = k - c(a, b) + 2|S|.$$

The existing coefficient engine and grading audit use the convention  $q_b - q_a = \omega$  for every nonzero monomial in a homogeneous differential entry. The new scanner keeps these object shifts during allocation:

$$q' = q + i + c - 2 \text{popcount}(\ell),$$

where  $i \in \{0, 1\}$  is the smoothing,  $c$  is the number of delooped circles, and  $\ell$  is their binary label. The array follows the exact object order used by `FastScan.add_crossing`. Scalar cancellation preserves the shifts of all remaining objects. This is the ordinary dotted grading and cancellation calculus of [22], used here as an algorithmic order.

Weights are nonnegative because  $c(a, b) \leq k$ . Equality requires equal matchings and no dots: a circle in the overlay then consists of the two copies of the same arc. Composition adds weights on every nonzero term. In the engine's dot-count convention, for a composition through  $b$  with input dot counts  $u, v$  and output count  $r$ ,

$$2r = 2u + 2v + k - c(a, b) - c(b, c) + c(a, c),$$

which proves additivity by substitution. Thus writing  $d = d_0 + \delta$ , the zero-weight part  $d_0$  is a direct sum of binary complexes at fixed matching and  $q$ , and  $\delta$  strictly raises  $q$ . An undotted morphism between distinct matchings is not a scalar identity. Taking weight zero in  $d^2 = 0$  proves  $d_0^2 = 0$ .

## 25.2 A contraction with representatives and homotopies

For each fixed matching and quantum shift, binary elimination splits each chain group as

$$C^h = B^h \oplus R^h \oplus L^h,$$

where  $B^h = \text{im } d_0^{h-1}$ ,  $R^h$  represents its quotient in  $\ker d_0^h$ , and  $d_0 : L^h \rightarrow B^{h+1}$  is an isomorphism. Let  $I, P$  include/project  $R$  and let  $H$  invert the last isomorphism, vanishing on the other summands. Then

$$PI = 1, \quad d_0 I = P d_0 = 0, \quad d_0 H + H d_0 = 1 + IP, \quad H^2 = HI = PH = 0.$$

The implementation constructs bases, lifts, and inverse coordinates; a list of ranks would not supply the maps needed for this step. Each scalar map preserves matching and  $q$ , and  $H$  lowers homological degree by one.

There are finitely many object shifts, so  $H\delta$  and  $\delta H$  are nilpotent. In characteristic two form the finite inverses

$$U = (1 + H\delta)^{-1}, \quad V = (1 + \delta H)^{-1}.$$

The finite perturbation identity [29] gives

$$D = P\delta UI, \quad I' = UI, \quad P' = PV, \quad H' = UH = HV. \quad (14)$$

These are a special chain contraction from  $(C, d)$  to  $(R, D)$ :

$$dI' = I'D, \quad P'd = DP', \quad P'I' = 1, \quad dH' + H'd = 1 + I'P', \quad H'^2 = H'I' = P'H' = 0, \quad D^2 = 0.$$

For an explicit algebraic check,  $Z = UI$  solves  $Z = I + H\delta Z$ . Using  $d_0\delta + \delta d_0 + \delta^2 = 0$  and the scalar contraction gives  $dZ = ID + H\delta dZ$ , hence  $dZ = ZD$ . The right-hand calculation gives the projection identity. Expanding the finite inverses with  $H^2 = HI = PH = 0$  proves the side identities. Multiplying  $dH' + H'd + 1 + I'P'$  on the left by  $1 + H\delta$  and on the right by  $1 + \delta H$  reduces it to zero using  $d^2 = 0$  and the scalar homotopy equation. The multipliers are invertible. Finally  $D^2 = P'd^2I' = 0$ . Every term of  $D$  contains  $\delta$  and has positive weight, so the returned complex has no scalar unit left to cancel.

This termination argument uses the actual finite grading and typed composition law. It does not invoke a uniform nilpotence claim for arbitrary ungraded formal matching tables. Both the disk-certified and graded algorithms remain applicable only to valid complexes in their respective contracts.

### 25.3 Computing a path sum without enumerating its paths

For each scalar survivor column  $e$ , evaluate

$$Z = Ie + HW, \quad W = \delta Z, \quad De = PW$$

one quantum block at a time in increasing order. All contributions to a block arrive from lower blocks before it is processed; binary  $P, H$  then mix within that block. Each radical edge is propagated at most once for this source, combining equal destinations by XOR. This computes the finite sum in (14) without constructing matrix powers or enumerating exponentially many individual paths.

There is also a source-dependent band:

$$0 \leq \omega = k - c(a, b) + 2|S| \leq k + c(a, b) \leq 2k.$$

A source of shift  $q_0$  cannot reach a nonzero coefficient beyond  $q_0 + 2k$ . The kernel drops such destinations before composition. It still retains higher-order corrections within the band. For example, two radical steps with a scalar contracting arrow between them can yield  $x_1x_2$  even when  $P\delta I = 0$ . On a single arc the analogous  $x^2$  term is zero. Both cases are explicit regression tests, as is the nonzero minimal map  $a \xrightarrow{x} a$ . Replacing the latter by two isolated survivors would change later gluing.

Let  $N$  count allocated object slots including holes left by completed sparse pivots,  $S$  count scalar survivors, and  $E_\delta$  count positive-weight entries. There are at most  $SE_\delta$  coefficient-composition attempts. If  $A_H, A_P$  count nonzero scalar incidences, the remaining transfer work is bounded by  $O(S(N + E_\delta + A_H + A_P))$  coefficient additions and control operations, after polynomial binary preprocessing. Packed indices and integer costs must also be charged. The implementation's global bit columns and arrays can depend on  $N$ , even if few slots remain live. Thus a live-only bound must first account for compaction or the size of the allocated arrays. The component coefficient engine costs  $2^{O(k)} \text{poly}(k)$  per operation, giving an explicit bound polynomial in  $N, S$  and exponential in  $k$ . None of these statements bounds necessary object multiplicity.

Optional diagnostic mode additionally computes all of  $I', P', H'$  and checks the eight displayed identities in the full cobordism algebra. It is excluded from timed production reductions. If the whole differential is scalar and diagnostics are disabled, the kernel obtains quantum-resolved homology multiplicities directly and avoids constructing unused contraction maps.

### 25.4 Adaptive scheduling and installation

The eager policy transfers each stage. The adaptive policy first allows  $\max(256, 4M)$  candidate Schur update pairs, where  $M$  is the live count at stage entry. The existing sparse eliminator pauses only between complete pivots and clears its candidate queues. Full transfer then consumes that partially reduced complex, retaining completed cancellations. Counters distinguish stages, actual switches, coefficient attempts, identity shortcuts, and scalar-only reductions.

The maintained adapter prepares the new incoming-edge index before replacing any differential arrays, checks the shared deadline again, and installs the matching, degree, quantum, outgoing and incoming arrays together. A failure while preparing the replacement preserves the current differential; composition caches may have acquired reusable entries. Tests force both a final-boundary deadline expiration and an incoming-index allocation failure, then resume the same object successfully. The kernel also polls during long composition sequences. A global resource failure propagates from the raw API and becomes UNKNOWN in recognition; no rate of progress is a topological verdict.

Direct constructors have an optional transfer-object ceiling; exceeding it resumes ordinary cancellation. The public reduction modes retain the existing object and time ceilings. Their policy is

a heuristic, not a competitive running-time guarantee or a byte-memory budget. In particular, a dense binary basis change can be more expensive than the sparse pivots it replaces and can change future fill. The default remains **standard**.

Both modes require minimum-fill pivots, bit coefficients, self-inverse cancellation, and one scan order. Standard and component coefficient composition and tail finishing are supported. Homological windows are rejected before early recognition certificates: their pruned allocation needs a separate quantum-label adapter. Shared, saturated, twist and other alternative scanners are also rejected. Existing disk-adaptive window modes remain available.

## 25.5 Validation and measurement scope

The delivered report 23 snapshot passes all 281 tests unchanged and its 114 source Git blobs match the declared baseline. Those full archive tests also cover additions outside this integration; passing them does not constitute integration of those additions. Production adds 13 graded integration tests and two transactional-resource tests to the previous 411-test suite. The transfer audit uses 86 diagrams in 258 order presentations and checks 1812 full stage contractions, with eight identities each. Four scanner variants agree on raw ranks and three quantum-tracking variants agree on quantum-resolved ranks. Synthetic controls retain nonzero minimal maps, test higher corrections, exercise capacity fallback and resume after sparse progress. Independent set-coefficient scans provide an additional oracle on 18 small random diagrams.

The maintained drivers are in **fast/graded\_research**; raw audit and source-provenance evidence is in **synthesis/data/graded-\***. Measurements use seven shuffled paired rounds, an identical standard A/A arm, fresh states and batched calls. Whole-diagram timings cover complete raw scans including reduction setup, but exclude construction and order selection; they bypass recognition filters. Synthetic timings cover the entire reduction stage and exclude cloning. Synthetic controls are valid graded algebraic complexes, not proved realizations of classical knot prefixes.

| Case               | Old/eager | Old/adaptive | A/A   | Adaptive switches |
|--------------------|-----------|--------------|-------|-------------------|
| hard_unknot_8      | 0.689     | 0.957        | 0.998 | 0                 |
| conway             | 0.557     | 0.920        | 0.929 | 0                 |
| kinoshita_terasaka | 0.616     | 0.980        | 1.019 | 0                 |
| unknot_braid40     | 0.633     | 0.886        | 0.984 | 0                 |
| stress_braid5_36   | 0.260     | 0.942        | 1.021 | 0                 |
| synthetic 16       | 0.888     | 0.741        | 0.999 | 1                 |
| synthetic 32       | 1.909     | 1.582        | 0.992 | 1                 |
| synthetic 64       | 4.968     | 4.174        | 1.008 | 1                 |
| synthetic 128      | 13.180    | 11.266       | 1.015 | 1                 |

Ratios above one favor the new policy. Each is a median of paired ratios. The five actual diagrams invoke zero adaptive transfers. Eager transfer slows all five raw scans; tracking the grading also adds overhead to adaptive scans. The largest synthetic case gains 13.18× eagerly and 11.27× adaptively, retaining a nonzero dotted minimal differential. The smallest synthetic case regresses. A/A ratios range from 0.929 to 1.021 across all cases, so small changes should not be treated as reliable improvements. No end-to-end recognition gain on the actual-diagram controls is established. The raw measurements are in **fast/results/graded\_transfer\_20261008.json**.

The conditional complexity target remains explicit. If all allocated intermediate state counts satisfy  $N = n^{O(\log n)}$ , all frontiers have  $k = O(\log^2 n)$ , and there are polynomially many stages, these explicit operations fit a quasi-polynomial bound. The integration proves neither universal hypothesis. Section 24 shows why even a four-ended frontier can have large necessary expanded multiplicity. The next algebraic refinement in report 24 prunes paths irrelevant to survivors and chooses propagation direction; its performance and topology obligations remain separate from this kernel’s correctness.

## 26 Succinct transport, the earlier rank law, and repair dependencies

The remaining parts of report 23 overlap substantially with the integrated one-run recurrence of Section 23. We retain that backend and its established profile schema. The useful additions are exact transport for enormous integers, an independent structural replay, a sharper statement of the total-rank threshold, and a precisely conditional repair bound. None requires a second competing implementation of the same tail.

### 26.1 An earlier total-rank threshold does not give a smaller cap for free

Fix a selected signed run of magnitude  $m$ , and let the remaining expanded context have degree range  $[a, b]$  and length  $L = b - a$ . For a positive run, the interior starts at degree  $b + 1$ . At  $m = L + 1$  there is exactly one full chain slice  $V$ . At  $m = L + 2$  there are two, with the canonically identified square-zero matrix  $A : V \rightarrow V$  between them. Write  $v = \dim V$ ,  $\rho = \text{rank } A$ , and  $\nu = v - 2\rho$ .

All lower boundary matrices are fixed; all upper boundary matrices agree after translating degrees. Increasing  $m$  from  $L + 1$  inserts one more full slice and one matrix  $A$ , without changing any other matrix rank. The field identity

$$\dim H(C) = \sum_h \dim C^h - 2 \sum_h \text{rank } d^h$$

therefore proves

$$R(m) = R(L + 1) + (m - L - 1)\nu \quad (m \geq L + 1).$$

For a negative run use the reversed coordinate  $q = -h$ ; the original cochain matrix still goes from  $h$  to  $h + 1$ , so its central source is  $a - 2$ , as in the existing implementation. The formula is about total rank, not a decomposition of the single full slice's two boundary contributions. Exact degree profiles still use the  $L + 2$  cap.

The integrated recurrence now reports  $L + 1$  and  $R(L + 1) = R(L + 2) - \nu$  in its compressed-result certificate. It still builds the  $L + 2$  reference: the earlier equality by itself does not construct the missing square matrix or compute its rank from a single slice. No reference cost reduction is claimed. An independent referee run checked 392 explicit complexes: 96 signed contexts at four magnitudes plus eight sharp boundary examples. It checks canonical interior matrix equality,  $d^2 = 0$ , Euler characteristics, the rank law and full profiles against the ordinary macro builder. The boundary examples  $\sigma_1^{L+1}\sigma_1^{-L}$  are unknots, so a dominance rejection threshold cannot simply be lowered to  $L + 1$ .

The cap also has a sharper elementary size estimate. A support with  $r$  active turnbacks has at most  $s + r$  closure circles on  $s$  strands, since adding one turnback changes the count by at most one. Summing reduced dimensions over active-run choices gives, at selected magnitude  $L + 2$ ,

$$K \leq 2^{s-1}(2L + 5) \prod_{i \neq j} (1 + 2|e_i|) \leq 2^{s-1}(2L + 5)3^L.$$

For an original knot, every strand separator occurs. Apart from the selected generator at least  $s - 2$  distinct generators occur in the context, so  $s \leq L + 2$ . Finite-reference geometry, rank computations and compressed output consequently cost  $2^{O(L)} \text{poly}(N)$  for succinct input bit length  $N$ . This becomes quasi-polynomial when  $L = O(\log^2(N + 2))$ . Reading an explicit word has its additional linear input cost. Discovering a small context for arbitrary diagrams is still unproved, and dominant-run knot decisions were already covered by the existing structural criteria.

## 26.2 JSON is part of the succinct representation contract

The previous Python API could compute exponents with tens of thousands of bits, but ordinary JSON decimal conversion could reject their input or output. This included degree *keys* of the finite exceptional profile, not just rank values. The maintained run frontend now accepts literal integers or explicitly signed hexadecimal strings. Above 4096 bits it emits hexadecimal strings for both integer values and integer dictionary keys; smaller values retain the ordinary JSON representation. No global decimal conversion limit is relaxed. Booleans, floats and ambiguous numeric strings are rejected. Distinct dictionary keys must retain distinct encodings.

The frontend accepts exactly one of **runs** and **word**; the latter is already an explicit list of signed generators. Neither path constructs a PD, and run input is never expanded. Recognition, homology and structural profile modes retain their existing meanings and resource outcomes. A final deadline check now covers the arithmetic after source counting, including successful structural answers.

`decode_degree_profile` reconstructs integer points and interval fields without enumerating the plateau. It rejects duplicate degrees after decoding. This function validates a transport schema, not a homology proof; the normal profile-query functions operate on its decoded Python result. The structural certificate verifier decodes only numeric fields prescribed by its exact schema, rejecting additional fields and boolean/float impostors. Its replay uses a separately written dense permutation and generator-support calculation, independent of the sparse producer. Before allocating the permutation it checks  $s \leq \#runs + 1$ , which every knot source must satisfy. It recomputes the PD/Artin sign conversion and both Rasmussen intervals. This checks arithmetic premises, not the imported topology.

New regressions send  $2^{20000} + 1$  through the actual CLI with both signs, reconstruct the degree profile, and verify the exact rank while constructing only four reference basis vectors. A five-strand example with four balanced huge signed runs has no dominant exponent; its homogeneous source certificate round-trips and verifies while the producer is disabled by the test. Tampered fields, duplicate degree encodings, original component parity, and unchanged global decimal limits are checked. These changes enable an already-proved succinct computation to cross the public interface; they do not provide a new homology runtime improvement.

## 26.3 What an acyclic repair certificate would need to establish

The report's geometric proposal is an accounting theorem for a supplied fixed DAG, not a constructed hierarchy of a knot exterior. Suppose module  $i$  has an integer counter  $0 \leq x_i \leq g_i$ . Every charged event strictly decreases its distinguished counter and can increase only counters at its out-neighbors. Only these charged events may increase counters. Partition time at events of predecessors of  $i$ ; within each interval  $i$  has at most  $g_i$  events. Thus the event counts obey

$$r_i \leq g_i \left( 1 + \sum_{j \rightarrow i} r_j \right).$$

Induction in topological order gives the weighted-path bound

$$\sum_i r_i \leq \sum_{\pi \neq \emptyset} \prod_{v \in \pi} g_v \leq \ell g \sum_{j=0}^B (dg)^j,$$

where  $\ell$  is the number of vertices,  $d$  bounds outdegree,  $g$  bounds the numerical counter values, and  $B$  bounds path length in edges. The exact path sum is computable by dynamic programming without enumerating paths. Polynomial  $\ell, d, g$  and logarithmic  $B$  give a quasi-polynomial number of charged events. A short binary encoding of an enormous  $g$  is insufficient; the event count depends on  $g$ , not merely its bit length.

The maintained research diagnostic agrees with independent path enumeration on 380 fixtures and rejects cycles and invalid bounds. A complete DAG has bound  $(g + 1)^\ell - 1$ , while a chain has  $\sum_{j=1}^\ell (\ell - j + 1)g^j$ ; sparse edges alone do not control depth. For a knot algorithm one must still construct and validate the modules, fixed dependencies, counter bounds, and legal update rules. Disjoint local supports do not by themselves imply independence of global nonseparateness, homology predicates or attachment certificates. Total work must also charge uncharged steps, event cost, and representation size. We retain this as a research target rather than installing an unsupported geometric scheduler.

## 27 Survivor corridors and adaptive transfer direction

Report 24 refines the full transfer of Section 25. It removes algebraic paths that cannot contribute to a surviving map and chooses which end of each remaining component to propagate from. The maintained implementation exposes this as `corridor` and `corridor-adaptive`; neither changes the default sparse reducer. The latter first permits  $\max(256, 4N_{\text{live}})$  ordinary Schur-update pairs in a stage. Completed pivots are retained when that allowance is exhausted, and the remaining complex is transferred exactly. The policy therefore responds to observed elimination work in the spirit of introsort. Unlike introsort's fallback theorem, it supplies neither a competitive runtime guarantee nor a bound on the number of objects generated by a scan.

### 27.1 From perturbation transfer to a directed graph

Write  $d = d_0 + \delta$ , with  $d_0$  the weight-zero scalar differential, and let  $(I, P, H)$  be its typed binary special contraction. Here  $H$  preserves matching type and quantum shift and lowers homological degree by one;  $\delta$  raises homological degree by one and strictly raises quantum shift. For the degree- $h$  part of

$$D = P\delta(1 + H\delta)^{-1}I = \sum_{r \geq 0} P\delta(H\delta)^r I,$$

make one vertex  $z_a$  for each original degree- $h$  object and one  $w_b$  for each degree- $(h + 1)$  object. Include terminal vertices  $s, t$  for the survivors in those degrees. Its edges are precisely

$$s \xrightarrow{I_{as}} z_a, \quad z_a \xrightarrow{\delta_{ba}} w_b, \quad w_b \xrightarrow{H_{ab}} z_a, \quad w_b \xrightarrow{P_{tb}} t,$$

omitting zero entries. The  $I, H, P$  edges carry typed identities, whereas the radical edges retain their actual cobordism coefficients. Ordering by quantum shift, and at equal shift by phase  $w, s, z, t$ , makes every edge go forward. Thus this graph is acyclic even when the homological degrees of the survivors are consecutive.

**Proposition 27.1** (Exact survivor restriction). *The matrix entry  $D_{ts}$  is the sum of the ordered coefficient products over all directed paths from  $s$  to  $t$ . Deleting every vertex not both reachable from an input and able to reach an output preserves all entries of  $D$ .*

*Proof.* Expansion of  $P\delta(H\delta)^r I$  gives exactly the paths with  $r$  homotopy edges. Quantum acyclicity makes the expansion finite. A path contributing to any matrix entry has both required reachability properties at each vertex, so deleting the other vertices removes none of its terms. Conversely every retained path was already present. An edge between two retained vertices lies on a complete path by concatenation, so retaining all such edges is safe.  $\square$

Reachability is computed by Boolean union, not by coefficient XOR. It is an overestimate of algebraic support: equal path products can cancel and nonzero factors can have zero composite. In particular, neither a nonempty corridor nor the scalar survivor multiplicities decide a homology rank. The implementation evaluates the entire remaining differential.

The weak components of the retained graph include its terminal vertices. They give disjoint blocks of this adjacent-degree map. Within a block, forward propagation accumulates each source's coefficients in topological order; reverse propagation accumulates each target's suffix coefficients in reverse order. For an edge  $u \xrightarrow{f} v$ , the latter forms  $g \circ f$  when the current suffix is  $g : v \rightarrow t$ . It does not reverse the order of multiplication or assume that differently typed coefficients commute. Distributivity and associativity show that both evaluations give the same finite path sum.

## 27.2 What the direction heuristic bounds

Let  $L(u)$  be the set of sources reaching  $u$  and  $R(v)$  the set of targets reachable from  $v$ . For a retained component  $G_j$ , define

$$F_j = \sum_{(u,v) \in E_j} |L(u)|, \quad B_j = \sum_{(u,v) \in E_j} |R(v)|.$$

The **reach** policy constructs these endpoint sets as packed integers and chooses forward when  $F_j \leq B_j$ , reverse otherwise.

**Proposition 27.2** (Propagation bound). *The number of coefficient edge propagations under this policy is at most  $\sum_j \min(F_j, B_j)$ . If component  $j$  has  $a_j$  inputs and  $b_j$  outputs, the alternative **ports** policy uses only Boolean reachability flags and bounds the propagation count by  $\sum_j |E_j| \min(a_j, b_j)$ .*

*Proof.* Forward propagation from one source processes an edge at most once, and only if that source reaches its initial vertex. Summing gives  $F_j$ . The suffix argument gives  $B_j$ . The implemented choice takes their minimum. Replacing the two reach-set sizes by the total numbers of ports gives the second bound, choosing the side with fewer ports.  $\square$

These bounds count attempted edge propagations, not cache misses or seconds. The code bypasses typed identities and caches equal products. It also scans every ordered vertex of a component once per chosen root, including vertices whose current coefficient is zero. If  $r_j$  is the number of roots on the chosen side, that costs  $O(\sum_j |V_j| r_j)$  dictionary visits. The **reach** choice need not minimize this term or choose the side with fewer ports. Nor does its bound include scalar setup and endpoint masks.

## 27.3 Sparse scalar components and complete costs

The scalar graph has only equal-matching, equal-shift identity arrows. The **components** engine finds its weak components and contracts each separately. Isolated vertices are survivor identities; a component consisting of one scalar arrow has only its reverse homotopy entry. Other components use the existing binary contraction on local indices. The result stores sorted tuples of nonzero indices for  $I, P, H$ , including empty tuples at dead slots, instead of integers indexed by all old objects.

This changes representation without changing the established scalar basis. Within a matching/shift/degree group, elimination sees source and target indices in their original order. Operations in disjoint scalar components cannot interact. Each selected kernel representative has a distinct largest original source index, its dependent-column anchor. Sorting group keys and restoring anchor order therefore reproduces the original representative order. The same permutation of projection columns, and unchanged local homotopy coefficients, recover all three original maps entrywise.

Let  $N$  count allocated slots, including dead slots; let  $E_{\text{in}}$  count input differential entries, and  $M_{\text{sc}}$  count nonzero scalar-map indices. If scalar components have sizes  $s_i$ , a conservative binary setup

bound is, up to polynomial factors in index and label bit lengths,

$$A_{\text{sc}} = O \left( N + E_{\text{in}} + M_{\text{sc}} + \sum_{i:s_i > 2} s_i^3 \right).$$

The scan of  $E_{\text{in}}$  is necessary to discover scalar entries. Across adjacent degrees, graph sizes satisfy  $V = O(N + S)$  and  $E = O(E_{\delta} + M_{\text{sc}})$ , where  $S \leq N$  is the number of survivors. Graph construction, sorting, Boolean reachability, and union by size with path compression cost near-linear work in this explicit representation, apart from index and label arithmetic.

For the **ports** policy the remaining bound has the form

$$T \leq A_{\text{sc}} + A_{\text{graph}} + \Lambda \sum_j |V_j| r_j + \Lambda(1 + C(k)) \sum_j |E_j| r_j, \quad r_j = \min(a_j, b_j),$$

where  $\Lambda$  charges index/dictionary arithmetic and  $C(k)$  bounds coefficient work at  $2k$  boundary points. A conservative coefficient envelope is  $\text{poly}(k)2^{O(k)}$ . The **reach** version replaces the edge term by  $\sum_j \min(F_j, B_j)$ , but uses its actually chosen  $r_j$  in the vertex term and additionally pays up to  $O((V + E)S)$  bit work for endpoint masks and population counts. Python hash-table timings are empirical; ordered dictionaries give a deterministic accounting with polynomial index overhead.

Sparse indices matter even for a singleton column: an integer  $2^j$  uses  $j + 1$  bits of payload. Many such global columns can have quadratic total bit length while their sparse representation has only linearly many indices. Conversely, a dense connected scalar block can still require large local binary matrices. This optimization does not remove all fill.

## 27.4 A strict separation, and its scope

Fix the three-arc dot algebra  $\mathbb{F}_2[x, y, z]/(x^2, y^2, z^2)$ . Take  $R$  degree-zero survivors  $s_j$  at shift zero and one degree-one survivor  $t$  at shift six. Add identity pairs  $L_0 \rightarrow B_0$  at shift two and  $L_i \rightarrow B_i$  at shift four for  $1 \leq i \leq M$ , with  $L$  in degree zero and  $B$  in degree one. The only radical maps are

$$s_j \rightarrow B_0 : x, \quad L_0 \rightarrow B_i : y, \quad L_i \rightarrow t : z.$$

This is a homogeneous two-term complex, hence  $d^2 = 0$ . For odd  $M$  its transferred entry is  $D_{ts_j} = Mxyz = xyz \neq 0$ . Forward propagation visits  $R(1 + 2M)$  radical edges, while reverse propagation visits  $R + 2M$ . The latter accumulates the common suffix once before distributing it to the  $R$  sources. For even  $M$  the output vanishes by parity despite unchanged structural reachability, as a regression test checks.

There are  $R + 2M + 3$  original objects,  $R + 1$  isolated scalar survivors, and  $M + 1$  identity pairs. Sparse scalar setup has no nontrivial binary blocks. Using **components** and **ports**, the single-target graph is evaluated once in reverse; all coefficients lie in a fixed eight-dimensional algebra. Including input reading, scalar setup, sorting and output writing, the stage takes  $O((R + M) \log(R + M + 2))$  index-word operations and  $O(R + M)$  words. The previous forward evaluation alone takes  $\Omega(RM)$  propagations. At  $R = M = 255$ , the radical counts are 130305 and 765; the contraction uses 768 sparse indices versus 164735 packed payload bits. These are representation counts, not heap measurements.

This is a separation between complete transfer implementations on an explicit algebraic family. Its realization by knot prefixes is unproved, and minimum-fill sparse cancellation is already efficient on the family. The default public corridor policy still uses **reach** and the global scalar engine; the near-linear theorem refers specifically to the direct **ports/components** configuration.

## 27.5 Integration, validation, and measured limits

The new reducers support the standard scanner, minimum-fill pivots, bit coefficients, the existing component coefficient engines, and tail contraction. Races, other scanner backends, and window

truncation are rejected before early certificates. Window integration needs its own quantum-shift and truncation argument. Global resource exhaustion raises the ordinary scan limit in the raw API and yields UNKNOWN in recognition. Full replacement arrays, reverse indices, and an empty cache are allocated before a final deadline check and installation. Tests inject both allocation failure and expiry after transfer, then resume successfully on the preserved complex. Completed earlier sparse pivots remain valid.

The maintained independent audit compares seven configurations entrywise against the frozen forward reference on 555 prefixes from 80 diagrams: 3885 comparisons, including partially cancelled inputs. All eight special contraction identities are checked at each prefix. Separate tests compare graded homology, continued scans and independent set coefficients. The full maintained suite passes 458 tests. The stronger graded support-only certificate uses the condition

$$\Delta > 0, \quad k - c(a, b) \leq \Delta \leq k + c(a, b), \quad \Delta \equiv k - c(a, b) \pmod{2}$$

for a possible radical map. Its audit accepts 196 of 639 stages in 72 diagrams, exactly the same stages as the older degree-gap shortcut. The experimental support reducer is retained for comparison, not substituted for an established production policy.

The delivered package lacks its advertised full SHA-256 manifest. The maintained reproduction wrapper verifies the separate 20-file integration manifest and runs the archive in a temporary copy: 268 scanner tests, the transfer and support audits, and the independent surface-cover suite all pass. This is a narrower provenance claim than verifying every file. The surface-cover theorem, reviewed and extended in Section 29, has a separate presentation contract and is not used to make a knot verdict in this integration.

| Raw scan        | Std/corridor | Std/adapt. | Std/sparse | A/A   | Switches |
|-----------------|--------------|------------|------------|-------|----------|
| two_strand_201  | 0.105        | 0.808      | 0.084      | 1.019 | 0        |
| torus_3_10      | 0.327        | 0.932      | 0.305      | 0.987 | 0        |
| alternating_3_4 | 0.348        | 0.938      | 0.302      | 1.017 | 0        |
| morton          | 0.422        | 0.927      | 0.411      | 1.013 | 0        |
| conway          | 0.341        | 0.923      | 0.309      | 0.991 | 0        |
| hard_unknot_8   | 0.500        | 0.918      | 0.490      | 0.986 | 0        |

Sparse denotes the component-scalar/Boolean-port configuration. Ratios above one favor the denominator.

| Constructed stage | Fwd/corridor | Fwd/sparse | Std/sparse | A/A   |
|-------------------|--------------|------------|------------|-------|
| source_heavy      | 2.390        | 2.214      | 0.340      | 0.988 |
| sink_heavy        | 0.461        | 0.426      | 0.336      | 1.016 |
| balanced          | 0.623        | 0.592      | 0.299      | 1.014 |
| dead_branches     | 6.234        | 5.457      | 1.188      | 0.993 |
| shared_suffix_15  | 1.303        | 1.558      | 0.204      | 1.017 |
| shared_suffix_63  | 4.194        | 4.859      | 0.172      | 0.981 |
| shared_suffix_255 | 15.661       | 18.141     | 0.154      | 1.041 |
| dead_leaves_255   | 0.677        | 0.860      | 0.187      | 0.993 |

Fwd denotes the maintained quantum-ordered forward transfer. These timings include scalar and graph setup.

All six actual scans favor ordinary sparse cancellation: eager corridor transfer is approximately 2.0–9.5 times slower. The adaptive policy switches zero times, with standard/adaptive ratios 0.808–0.938; its quantum bookkeeping therefore buys no transfer benefit in these cases. The

sparse-scalar/port refinement also loses on every actual scan. Actual standard/control ratios span 0.986–1.019. On the constructed size-255 shared-suffix case, the sparse refinement is 18.14 times faster than the maintained forward transfer, but ordinary sparse cancellation is still approximately 6.49 times faster than that refinement. The constructed dense dead-branch case gives a different result: eager corridor and adaptive corridor beat ordinary cancellation by factors 1.406 and 1.380, respectively. Other constructed cases include regressions. These observations support retaining optional policies and publishing their negative results, rather than changing the default or claiming a uniform timing improvement.

The measurements use fresh states, seven shuffled paired warm rounds, and identical standard/control arms. Actual raw scans include construction of the scanner, quantum tracking and all reduction work; diagram construction and shared crossing-order choice are outside timing. Synthetic timings include the complete reduction stage, with fixture construction outside timing. Early recognition filters are bypassed, so these are not end-to-end recognition speedups. Raw samples, repetition counts, exact checks and source hashes are retained with the result file.

Finally, a scan with every allocated  $N_i \leq \exp(O(\log^2(n+2)))$  and every  $k_i = O(\log^2(n+2))$ , suitably bounded labels and an affordable order, would meet the target by the displayed bounds and ordinary extension costs. No such uniform bounds have been proved for arbitrary diagrams. Corridor size also depends on the scalar basis; it is not a knot invariant. Faster evaluation of an explicit stage leaves the central multiplicity and representation problems unresolved.

The weaker general sub-exponential target is also unproved for this implementation. Here sub-exponential means  $2^{o(n)}$  in the crossing count, not merely a speedup over the previous program. Under the same polynomial label-size assumptions, the stage accounting would suffice if both  $\max_i k_i = o(n)$  and  $\max_i \log(N_i + 2) = o(n)$ , with an order found in sub-exponential time. No such simultaneous guarantee has been established. Adaptive switching among the current representations does not supply it. This implementation status must also be distinguished from Lackenby’s announced quasi-polynomial result: the July 2026 hierarchy paper explicitly leaves some step runtimes unspecified in its Section 9 [17].

## 28 Certified cyclic Garside preprocessing

Report 25 extends the earlier rank-two interval optimizer to arbitrary supports in an Artin braid group. The maintained implementation shares exact prefix arithmetic across cyclic cuts and target words. It supplies an optional source-braid probe, not a new knot invariant. A successful proposal must shorten the current simplified diagram, must pass independent replay, and restarts recognition under the remaining global budget. The probe follows the existing braid, Seifert, descending, and modular Alexander obstructions; its default setting is disabled.

### 28.1 Exact states and a separately checked equality witness

Let  $b$  be the strand count and  $\Delta$  the positive half-twist of length  $B = \binom{b}{2}$ . Classical Garside theory gives a unique left-greedy form

$$g = \Delta^z s_1 \cdots s_r,$$

where  $z \in \mathbb{Z}$  and the  $s_i$  are proper nonidentity simple braids. A simple braid crosses each pair at most once and is determined by its permutation. These are classical facts, not new compression theorems; see Dehornoy [28]. The implementation stores complete permutation arrays and the integer power. Sorting and binary search compare these exact states; neither a permutation quotient nor a hash fingerprint is used to decide braid equality.

Our convention is  $p[i] =$  bottom position of top strand  $i$ . For two adjacent simple factors  $A, C$ , a

positive atom  $\sigma_{i+1}$  can move from the start of  $C$  to the end of  $A$  exactly when

$$C[i] > C[i+1], \quad A^{-1}[i] < A^{-1}[i+1].$$

The move swaps values  $i, i+1$  in  $A$  and positions  $i, i+1$  in  $C$ . The first condition factors  $C = \sigma_{i+1}C'$  and reduces simple length by one; the second makes  $A\sigma_{i+1}$  simple and increases its length by one. Thus the local product is unchanged. An identity right factor is deleted. Positive input letters append an atom; negative letters use

$$\sigma_i^{-1} = \Delta^{-1}(\Delta\sigma_i^{-1}), \quad S\Delta^{-1} = \Delta^{-1}\tau(S), \quad \tau(p)[i] = b-1-p[b-1-i].$$

After transfers make all pairs left weighted, leading half-twists are extracted into  $z$ . Any half-twist factor must then be initial, since a left-weighted factor immediately before  $\Delta$  must itself be  $\Delta$ .

**Proposition 28.1** (Elementary normalization bound). *Normalizing an  $m$ -letter signed word uses at most  $Bm(m-1)/2$  atom transfers, at most  $m$  simultaneous simple factors, and  $O(m^2b^3)$  index-word operations. Appending one atom to a normal state with  $r$  factors costs  $O((r+1)^2b^3)$  conservatively.*

*Proof.* For a temporary factor list indexed from zero, use  $\Phi = \sum_j j \ell(s_j)$ . Each transfer decreases it by one before any deletion. Deletion and leading half-twist extraction cannot increase it, and  $\tau$  preserves simple length. The  $k$ th append raises it by at most  $B(k-1)$  and creates at most one factor. Summing proves the transfer bound. Each pair test or transfer scans  $O(b)$  permutation positions; failed tests advance the scan index, and a successful transfer backs it up by at most one. List copying, twisting and deletion fit the stated quadratic bound. Starting with  $r$  factors instead gives initial potential  $O(B(r+1)^2)$  and the single-append bound.  $\square$

The certificate records local transfer positions, generator indices, and half-twist extraction counts. A separate verifier imports neither the normalizer nor the optimizer. It applies the elementary permutation operations only after checking their inequalities and indices. Therefore it proves equality of the represented braid throughout the trace, even without trusting normal-form uniqueness or the optimizer's choices. Version one checks equality of a source interval to the identity or a signed generator. Version two checks that source times inverse target replays to the identity. It also checks source binding, rotation, disjoint intervals, strict shortening, target radius, and reconstructed output. The digest is an input-binding aid; replay on the actual source supplies the equality proof. It certifies neither kernel optimality nor a knot verdict. A cyclic cut preserves oriented closure by conjugacy, while uncut substitutions preserve the braid element itself.

## 28.2 Sharing the algebra across cuts and targets

Write  $w$  for the input of length  $n$ , and  $P_j$  for the exact prefix products of the doubled word  $ww$ . A source interval  $[i, j)$  equals a target word  $v$  precisely when

$$P_i^{-1}P_j = v \iff P_i = P_jv^{-1}.$$

This ordering matters in a noncommutative group. The doubled table lets all  $n$  cyclic cuts query the same prefix states. The radius- $\rho$  dictionary contains every literal signed word of length at most  $\rho$ , including the empty word; its size is

$$M = \sum_{d=0}^{\rho} (2b-2)^d.$$

Different words may represent the same braid, which causes no correctness problem. A trie of reversed inverse target words shares the appends needed to evaluate  $P_jv^{-1}$  for one endpoint. Canonical query states are looked up in the sorted prefix table.

For a fixed cut  $s$ , let  $F[j]$  be the maximum saving on the first  $j$  letters. Immediately before processing  $j$ , maintain

$$H(q) = \max_{0 \leq i < j: P_{s+i}=q} (F[i] - i).$$

The recurrence is

$$F[j] = \max \left( F[j-1], \max_{v \in D} \{j - |v| + H(P_{s+j}v^{-1})\} \right).$$

Only afterwards insert  $F[j] - j$  under  $P_{s+j}$ , excluding empty source intervals. Backpointers are updated only on a strict improvement.

**Proposition 28.2** (Optimal disjoint one-pass kernel). *The recurrence returns the shortest word obtainable by disjoint interval substitutions to the chosen dictionary at that cut. Minimizing across all cyclic cuts gives the corresponding cyclic optimum.*

*Proof.* An optimal prefix plan either retains its last letter or ends with a replacement  $[i, j) \mapsto v$ . The latter saves  $F[i] + j - i - |v|$  and is permitted exactly by the prefix-state equality. Conversely every term extends a valid optimal prefix plan. Induction proves the recurrence. A nonshortening substitution has saving at most  $F[i] \leq F[j-1]$  and cannot win strictly, so recorded intervals strictly shorten. Backtracking to their starts makes them disjoint. Any circular disjoint plan admits a cut at a block boundary or an unused boundary, including a block covering the entire circle, and is therefore among the tested cuts.  $\square$

This is not optimization over arbitrary sequences of substitutions, conjugations or Markov moves. It extends the earlier interval dynamic program by supplying exact unrestricted group queries. The lower bound

$$k \geq \max(|\exp(w)|, b - \#\text{cycles of the strand permutation})$$

allows an exact early exit: each letter changes exponent by one and changes permutation cycle count by at most one. Both quantities are unchanged by conjugacy. In particular, a knot closure needs at least  $b - 1$  letters.

### 28.3 Height-sensitive costs and the small-core theorem

Let  $R = \max_{j \leq 2n} \ell_c(P_j)$ , where  $\ell_c$  counts proper simple factors. Inversion preserves this length, and multiplication is subadditive: move half-twist powers left, concatenate factor lists, and normalize without increasing their number. Thus interval prefixes have height at most  $2R$ , and target queries at most  $R + \rho$ ; always  $R \leq 2n$ . The following conservative word bound includes a winning proof and replay:

$$\begin{aligned} T = & O(nM(R + \rho + 1)^2 b^3 + nM(R + \rho + 1)b \log(n + 2) \\ & + n^2 M + M\rho \log(M + 2) + n(R + \rho + 1)b^3). \end{aligned}$$

The first term charges the  $O(nM)$  exact append queries, the second their full-state comparisons, and the third all per-cut scalar dynamic programs. The next term builds the target dictionary and trie. In the last term, disjoint winning source intervals total at most  $n$  letters and their strictly shorter targets total at most  $n$ ; the height bound and transfer potential therefore charge proof generation and replay to  $O(n(R + \rho + 1)b^3)$ . Proof size is  $O(n(R + \rho + 1)b^2)$  elementary records. Search workspace, excluding that output, is

$$O(n(R + 1)b + nM + M\rho + \rho(R + \rho + 1)b)$$

words. Each index word occupies  $O(\log(n + b + \rho + M + 2))$  bits, giving the corresponding bit-storage bound. Elementary index arithmetic adds at most a polynomial in this logarithm to the word-operation cost in bit time. With fixed  $\rho$ , the computation is polynomial in  $n, b$ ; leading conservative terms are  $O(n^3 b^4)$  for radius one and  $O(n^3 b^5)$  for radius two. An arbitrary binary-encoded radius is not a fixed parameter: its dictionary is exponential in  $\rho \log(b + 1)$ .

A cheaper portfolio stage first expands the whole-word normal form only if its calculated Artin length is smaller than the input. It independently proves this equality. If the candidate attains the universal lower bound, it returns immediately; otherwise it compares that candidate with the cyclic kernel. The returned word is no longer than the kernel optimum, but need not itself be an optimal radius-specific interval plan. Evidence distinguishes `whole-normal-form` from `cyclic-kernel`.

**Proposition 28.3** (Constructive restricted recognition bound). *For fixed radius  $\rho$ , uncapped certified preprocessing followed by an uncapped conventional exact recognizer has cost*

$$\text{poly}_\rho(n, b) + \text{poly}(k)2^{O(k)},$$

where  $k$  is the resulting word length. It is quasi-polynomial on families whose cyclic radius- $\rho$  optimum is  $O(\log^2(n + 2))$ .

*Proof.* The optimizer and verifier have the polynomial bounds just given. Closure is preserved and remains one component, so  $b \leq k + 1$ . A conventional Khovanov cube on the resulting  $k$  crossings has  $2^k$  resolutions with at most  $b + k$  circles each. Hence its enhanced-state dimension is  $2^{O(k)}$ , and exact matrix ranks over  $\mathbb{F}_2$  can be obtained in polynomial work in that dimension. Reduced rank one detects the unknot: the rational unknot-detection theorem and the universal coefficient theorem imply the same rank-one criterion over  $\mathbb{F}_2$ . The maintained exact fallback supplies this decision. Substitution of the small-core bound gives  $2^{O(\log^2(n+2))}$ . Local probe cutoffs are disabled in this theorem.  $\square$

There is a proved family for which the decomposition need not be supplied. Partition a short core into blocks of length at most  $\rho$ , replace each by a no-shorter equal braid word, and insert disjoint identity blocks, forming  $y$ . Surround it by  $u, v$  with  $vu = 1$ , forming  $uyv$ . A cyclic cut exposes  $yvu$ ; deleting the sleeve and identity blocks and shortening the inflated core blocks is an admissible disjoint plan. The optimizer discovers a result no longer than the original core without being told this decomposition. This generalizes rank-two inflation to arbitrary supports and multi-letter targets. It does not establish a small core for every unknot or a general sub-exponential recognition algorithm.

## 28.4 Two limitations of short-word approaches

First, increasing target radius can be essential even for an unknot. In  $B_4$  write  $a = \sigma_1$ ,  $b = \sigma_2$ ,  $c = \sigma_3$  and, for  $m \geq 2$ ,

$$X_m = a^{-m} c^m a^{m+1} c^{-(m-1)}, \quad W_m = X_m b.$$

Since  $ac = ca$ ,  $X_m = ac$  and the closure of  $W_m$  is the unknot. Nevertheless, the cyclic radius-one optimum is  $4m + 1$ , while radius two returns three letters. In  $\langle a, c \rangle \cong \mathbb{Z}^2$ , the prefix path visits successively  $(0, 0), (-m, 0), (-m, m), (1, m), (1, 1)$ . It is simple and has no nonconsecutive lattice-neighbor vertices. Thus no interval in  $X_m$  is an identity or a long signed-letter replacement. An interval containing the sole  $b$  has form  $pbq$  with  $p, q$  in this commuting subgroup. The permutation double cosets force both  $p, q$  to be pure if this equals a signed  $b$ . Write  $p = a^{2r} c^{2s}$  and  $q = a^{2u} c^{2v}$ . Signed crossings between original strand labels occur on pairs 12, 34, 13, 24 with counts  $2r, 2s, 2u, 2v$  respectively, and pair 23 has count  $+1$ . These equality invariants force  $p = q = 1$  and the positive

sign. The simple prefix path then permits only the original single letter, which does not shorten. This covers intervals at every cyclic cut. Radius two replaces  $X_m$  by  $ac$ ; the four-cycle permutation requires at least three output letters. A whole-word normal form also exposes the short simple braid, so the practical portfolio avoids the larger dictionary on this family. The theorem separates interval rules, not all braid algorithms.

Second, preserving the braid element itself cannot universally give a polylogarithmic core, even at three strands. In  $B_3$ , the unknot braid

$$G_m = a^{m+1}ba^{-m} = a^m(ab)a^{-m}$$

has minimum equal-braid Artin length exactly  $2m + 2$ . To see this, quotient  $B_3$  by its center as  $C_2 * C_3$ , with  $x = aba$ ,  $y = ab$ . The image of  $G_m$  is  $(y^2x)^my(xy)^m$ , of reduced syllable length  $4m + 1$ . Each signed Artin letter contributes at most two syllables, so an equal word has length at least  $2m + 1$ . Exponent sum two forces even length, giving  $2m + 2$ , attained by  $G_m$ . A shortening subinterval would contradict this geodesicity. Cyclic rotation, however, exposes the sleeve  $a^{-m}a^m$  and leaves  $ab$  of length two. Existing free cyclic cleanup already handles this example. The obstruction applies to equality-only explicit words, not to compressed representations or closure-preserving transformations in general.

## 28.5 Production contract and empirical results

The `-garside` option retains the original checked source braid through PD simplifications. The proposal compares its verified output length with the current PD's crossing count, not merely with source length. An accepted source branch and the earlier PD move trace are recorded separately; they are not concatenated into a fictitious move sequence. Restart preserves backend, coefficient, reduction, window, and other user options, subtracts elapsed time from the global allowance, and disables this probe to prevent recursive reapplication.

Search, dictionary construction, and independent replay share a cooperative local deadline and operation allowance. The host's global check always runs first. A local limit or allocation failure skips the probe; global exhaustion yields UNKNOWN. Invalid certificates propagate as errors. Even the lower-bound shortcut checks resources. These are cooperative limits, not hard process-memory or interrupt-latency guarantees; sorting and individual array operations can be noninterruptible.

The existing modular Alexander obstruction runs before the optional probe. When it is inconclusive and the probe declines, this result is reused on the unchanged single-factor diagram. Individual factors receive their own checks. The selected Jones and requested exact Alexander stages follow the probe: successful compression can avoid their work on the larger PD. This ordering avoids spending a search allowance on a knot already detected by the inexpensive modular test, while retaining the opportunity to shorten before a larger invariant computation. Source-free PD input bypasses the probe. No production default changes.

The delivered suite passes 47 tests, including 7016 short-word normal forms checked against a faithful Artin action and 1512 independent interval optimization comparisons. All 45 archive hashes and six inspected source Git blobs were checked. Three blobs are tied to the named source commit and path; three were recorded only by exact blob. The previously unexecuted adapter smoke now passes. Production adds normalized-homology comparisons on 60 small knot closures, independent replay without the normalizer, backend/reduction restarts, corrupted certificates, shared resource hooks, and filter-priority/cache-reuse tests. All 517 maintained tests pass. These are executable checks, not formal verification of Python semantics or exhaustive knot classification.

| Recognition input | $n$ | Off/radius 1 | Off/radius 2 | A/A   |
|-------------------|-----|--------------|--------------|-------|
| barrier_h4        | 51  | 0.979        | 0.971        | 1.003 |
| rectangle_m16     | 65  | 0.977        | 1.007        | 1.032 |
| geodesic_m12      | 26  | 0.865        | 0.888        | 0.981 |
| hidden_sleeve_m8  | 24  | 0.841        | 0.823        | 1.008 |
| unknot_h2_u8      | 43  | 1.018        | 1.037        | 0.989 |
| unknot_h2_u16     | 59  | 0.811        | 0.814        | 0.997 |
| figure8_h2_u8     | 45  | 1.099        | 1.073        | 0.998 |
| figure8_h2_u16    | 61  | 1.102        | 1.107        | 1.014 |
| positive_h2_u8    | 49  | 0.775        | 0.763        | 0.961 |
| positive_h2_u16   | 65  | 0.834        | 0.827        | 1.018 |
| unknot_h8_u8      | 115 | 2.000        | 1.974        | 1.000 |
| unknot_h8_u16     | 131 | 2.053        | 2.042        | 0.997 |
| figure8_h8_u8     | 117 | 1.094        | 1.099        | 1.005 |
| figure8_h8_u16    | 133 | 1.096        | 1.092        | 1.006 |
| positive_h8_u8    | 121 | 0.870        | 0.876        | 1.000 |
| positive_h8_u16   | 137 | 0.882        | 0.869        | 1.000 |
| unknot_h16_u8     | 211 | 1.529        | 1.523        | 1.004 |
| unknot_h16_u16    | 227 | 1.340        | 1.327        | 0.988 |
| figure8_h16_u8    | 213 | 1.077        | 1.098        | 1.004 |
| figure8_h16_u16   | 229 | 1.099        | 1.096        | 0.996 |
| positive_h16_u8   | 217 | 0.895        | 0.902        | 0.975 |
| positive_h16_u16  | 233 | 0.911        | 0.911        | 0.997 |
| conway            | 11  | 0.960        | 0.982        | 0.993 |
| hard_unknot_8     | 8   | 0.750        | 0.749        | 0.996 |
| stress_braid5_36  | 36  | 1.182        | 1.196        | 1.046 |

Ratios are medians of paired complete-recognition times; values above one favor the enabled policy.

| Isolated compressor | $n$ | Repeated/shared | A/A   |
|---------------------|-----|-----------------|-------|
| single_crossing     | 1   | 0.891           | 1.004 |
| positive_4_braid    | 9   | 3.901           | 0.983 |
| ranktwo_barrier_h1  | 15  | 7.769           | 1.016 |
| ranktwo_barrier_h4  | 51  | 22.510          | 1.000 |
| geodesic_m12        | 26  | 6.231           | 1.009 |
| hidden_sleeve_m8    | 24  | 6.441           | 1.000 |
| seeded_5_braid      | 22  | 9.832           | 0.979 |

Both compressor arms search all cuts without lower-bound pruning and generate and replay only the winning certificate.

The complete-recognition experiment keeps the normal filters enabled and uses fresh validated diagrams, with their construction outside timing. Probe search, replay, candidate construction, and restarted recognition are included. Seven shuffled paired warm rounds compare disabled, identical control, and radius-one/radius-two enabled policies. The local allowance is 0.1 seconds and 100000 ticks, with 100000 target words; all arms have the same two-second global and 50000-object ceilings. An exhausted local probe is not censored if the remaining exact pipeline completes; a global UNKNOWN would receive no speedup ratio. Raw samples, repeats, verdicts, stages and source hashes are retained. Isolated compressor measurements are reported separately and do not imply the same gain in recognition.

All 25 inputs complete with agreeing verdicts in every measured arm. The four larger inflated-unknot fixtures have radius-one gains of 1.34–2.05 relative to the disabled pipeline. The two smaller ones have ratios 1.018 and 0.811, so successful shortening need not reduce total work. Several inputs are already decided before any probe; enabled-option overhead makes some very short certificates slower, with a ratio as low as 0.750. The identical-control ratios range from 0.961 to 1.046. These observations support an optional policy, not a universal or default performance win. On modular-Alexander-detected examples the enabled path returns before factorization, which also accounts for modest gains unrelated to Garside arithmetic. Source-free PD inputs do not run the probe.

Two retained diagnostic experiments explain the placement choice. Putting compression before modular Alexander made some already-detectable knots roughly 140 times slower. Moving all invariant filters before it removed that regression but lost the useful inflated-unknot gains, because Jones work on the larger PD had already been spent. The final placement lies between modular Alexander and Jones. These diagnostics retain their own source hashes and are not pooled with the final timings.

The isolated shared-prefix compressor improves by 3.90–22.51 on the six multi-crossing fixtures, but loses on the one-letter control. At the 51-letter barrier, sharing uses 714 algebra appends instead of 18207. That mechanism result is distinct from the full recognition table: the ordinary pipeline already decides the displayed barrier without invoking the compressor. Radius two does not deliver a consistent recognition gain on this sample, since the inexpensive whole-normal-form proposal often already attains the lower bound.

## 29 Compressed surface covers and ordered attachment points

Report 24’s second contribution concerns geometric compression rather than chain cancellation. Its dihedral-cover classifier is now maintained in `surface_cover.py`. We also extend it with exact comparison of components carrying an ordered list of points over a fixed basepoint. This is a possible interface for part of the attachment bookkeeping in a future hierarchy algorithm. It currently receives a covering presentation; it does not extract one from a knot exterior and supplies no knot verdict.

The general correspondence between connected covers and transitive fundamental-group actions, with equivariant bijections representing cover isomorphisms, is classical [11, Section 1.3]. The formulas below specialize it to binary affine data. They are not a replacement for general normal-surface orbit algorithms: Agol–Hass–Thurston already give polynomial algorithms for orbit counting and associated normal-surface topology [12]. Our new marked signatures have the narrower global dihedral input promise stated here.

### 29.1 A checked presentation and three unmarked families

Let  $F$  be a connected compact surface with  $b \geq 1$  boundary circles and free fundamental group of rank  $r = 1 - \chi(F)$ . The input specifies its orientability, genus (crosscap number when nonorientable), and  $b$ , together with  $r$  monodromies on a fibre of binary size  $W$ :

$$g_i(x) = \epsilon_i x + a_i \pmod{W}, \quad \epsilon_i \in \{1, -1\}.$$

Each assignment defines a cover because this fundamental group is free. Peripheral words are generated from the canonical surface schema: the last is the inverse of the product of handle commutators, or crosscap squares, followed by the first  $b - 1$  boundary generators. Arbitrary supplied words are not accepted as a peripheral-system certificate. Closed bases are outside this interface. The disk is included.

If all signs are positive, put  $d = \gcd(W, a_1, \dots, a_r)$  and  $m = W/d$ . Otherwise fix  $R(x) = -x + a_0$  and take the gcd of  $W$ , all positive shifts, and all differences  $a_i - a_0$  for negative generators. In either case the translation subgroup is generated by  $T(x) = x + d$ . Products of two reflections give the differences, and conjugating a translation by a reflection negates its shift; Bezout supplies the converse containment.

Without reflections there are  $d$  components of degree  $m$ , all isomorphic over  $F$ . With a reflection, components are the orbits of  $r_0 \mapsto a_0 - r_0$  on residues modulo  $d$ . Let  $f$  be the number of solutions of  $2r_0 = a_0 \pmod{d}$ . There are  $(d - f)/2$  paired-residue components of degree  $2m$  and  $f \leq 2$  fixed-residue components of degree  $m$ . All paired components are equivariantly isomorphic: for two nonfixed residues  $r_0, s_0$ , send

$$r_0 + jd \mapsto s_0 + jd, \quad a_0 - r_0 + jd \mapsto a_0 - s_0 + jd.$$

These maps commute with  $T, R$ , hence with every monodromy generator. Thus at most three records, with binary multiplicities, suffice.

When two fixed residues occur, their covers are isomorphic exactly when  $m$  is odd. An equivariant map between their transitive translation orbits must be a translation by  $A = d/2 + dt$ . Commutation with  $R$  requires  $2A = 0 \pmod{W}$ , or  $2t = -1 \pmod{m}$ . This is solvable exactly for odd  $m$ . The records retain distinct component labels even when their cover types coincide. Type equality never means that external attachments can be discarded. The three-type bound is sharp: for a genus-one base with two boundaries and maps  $x + d, -x, -x$ , even  $d \geq 4, m \geq 2$  give component genera  $m + 1, m/2, m/2 + 1$  and boundary counts  $2m, m + 2, m$ .

## 29.2 Orientation and peripheral information are retained

An affine sign is not the orientation character of a surface loop. Attach the actual character  $w_i \in \mathbb{Z}/2$  to every generator. For the translation increments  $u_j$ , retain bits  $v_j = w_j$  for positive generators and  $v_j = w_j + w_0$  for reflection differences. An extended gcd identity gives  $(d, \tau)$  as a combination of  $(W, 0)$  and  $(u_j, v_j)$ . The signed translation action is consistent precisely when

$$m\tau = 0 \pmod{2}, \quad v_j = (u_j/d)\tau \pmod{2} \quad \text{for every } j.$$

If a relation fails, subtracting multiples of  $(d, \tau)$  produces  $(0, 1)$ , an orientation-reversing loop fixing every sheet. Otherwise the signed translation group is  $(jd, j\tau)$ , with no orientation-reversing translation stabilizer. A reflection stabilizes a fixed-residue component at  $r_0$  with orientation bit  $w_0 + ((2r_0 - a_0)/d)\tau$ . Thus all paired components are orientable in the consistent case, and this last bit decides each fixed component. Pure translation covers use the same consistency test. In particular, an even-degree connected cyclic cover of a Möbius band is an annulus; an odd-degree one remains a Möbius band.

For boundary monodromy  $x + c$ , put  $q = \gcd(m, c/d)$ . There are  $q$  lifts of degree  $m/q$  on a single-residue component, or  $2q$  on a paired one. A boundary reflection exchanges the two residues of a paired component, giving  $m$  lifts of degree two. On a fixed residue  $r_0$ , its fixed points solve  $2y = (c - 2r_0)/d \pmod{m}$ . If this has  $\nu$  solutions, the profile is  $\nu$  degree-one and  $(m - \nu)/2$  degree-two lifts, omitting zero multiplicities. Together with

$$\chi(C) = \deg(C)\chi(F), \quad 2 - \#\partial C - \chi(C) = \begin{cases} 2g(C), & C \text{ orientable,} \\ g(C), & C \text{ nonorientable,} \end{cases}$$

these profiles determine the topology. The code checks the resulting integer identities. They describe the orientation interval bundle with orientable total space; a different bundle character would be extra input.

The component of sheet  $x$  is labeled by  $x \bmod d$ , together with its reflection partner when distinct. A boundary translation cycle has label  $x \bmod \gcd(W, c)$ ; a reflection cycle has label  $\{x, c - x$

$(\text{mod } W)\}$ . These give individual component and boundary-lift queries without enumerating other sheets or lifts.

### 29.3 An exact extension for ordered marked points

Fix the normalized input presentation. A marked component consists of one of its components and an ordered list  $(x_1, \dots, x_q)$  of fibre points in that component. Repeated points are allowed. Equivalence means an isomorphism over the identity of  $F$  taking each  $x_i$  to its corresponding point. The implementation's component-selector sheet identifies the component and is not an additional mark. This definition does not include unparameterized boundary-circle markings or arbitrary attaching maps.

**Proposition 29.1** (Rooted equivariant transport). *An equivariant map between connected components is uniquely determined by the image  $y$  of one point  $x$ . For two translation components any  $x, y$  are admissible and the map is  $z \mapsto z - x + y$ . For two paired components, any  $x, y$  are admissible and the map is*

$$f_{x,y}(z) = \begin{cases} z - x + y, & z \equiv x \pmod{d}, \\ z + x - y, & z \equiv a_0 - x \pmod{d}, \end{cases} \pmod{W}.$$

*For two fixed components it exists precisely when  $2(y - x) = 0 \pmod{W}$ , and is then the translation  $z \mapsto z - x + y$ . A paired and a fixed component cannot be isomorphic over the base.*

*Proof.* Transitivity forces  $f(gx) = gy$  for every monodromy element  $g$ . Translations act regularly on a single residue class. On a paired component, the points have unique expressions  $T^j x$  or  $T^j Rx$ , for  $j \in \mathbb{Z}/m$ ; the two displayed affine formulas follow. They are bijective and commute with  $T, R$ . On a fixed component the translation formula is forced, and commuting with  $R$  is equivalent to  $2(y - x) = 0 \pmod{W}$ . The degree difference excludes the remaining case. These proofs include  $W = 1, 2$  and  $m = 1$  even when formal affine signs describe the same permutation.  $\square$

For  $q > 0$ , take  $x = x_1$ . A complete marked signature is obtained as follows. For translation components record  $j_i = (x_i - x)/d \pmod{m}$ . For paired components record  $(e_i, j_i)$  according as  $x_i = T^{j_i} x$  or  $T^{j_i} Rx$ , with  $e_i = 0$  or  $1$ . For fixed components record all  $j_i = (x_i - x)/d \pmod{m}$  and also

$$k_x = (2x - a_0)/d \pmod{m}.$$

The extra value records the reflection stabilizer of the root. Equality of  $k_x, k_y$  is exactly the rooted transport condition. In all cases include the unmarked cover-type identifier. For  $q = 0$ , that identifier alone suffices. The API also includes the entire normalized base and monodromy in its signature, so comparisons between different presentations cannot silently become a claim of equivalence under a change of base.

**Corollary 29.2** (Exact marked-component comparison). *Within one checked presentation, two signatures are equal if and only if there is an equivariant isomorphism respecting every ordered mark. After preparing the cover, a signature costs  $O(q)$  arithmetic operations on binary integers. The unique map determined by two admissible anchors can be evaluated at any requested sheet in a constant number of such operations.*

*Proof.* An equivariant map preserves the root stabilizer and the relative  $T, R$  coordinates. Conversely, equal signatures satisfy the rooted transport criterion; the resulting map takes all coordinates, hence all marks, to their prescribed targets. The empty-mark case is the previously proved unmarked type classification. No search through sheet bijections occurs.  $\square$

**Proposition 29.3** (Two marks defeat a constant type bound). *There can be  $W$  inequivalent ordered two-point markings of one connected component, even though its unmarked cover has just one type. If the points must be distinct, there can be  $W - 1$  types.*

*Proof.* Take the annulus cover with monodromy  $T(x) = x + 1 \pmod{W}$ . Any equivariant automorphism is a translation, since the image of zero determines it. The pairs  $(0, j)$  are equivalent precisely when their second coordinates agree. Exclude  $j = 0$  for the distinct-point version.  $\square$

Thus polynomial bit complexity for each state and each equivalence query does not bound the number of possible marked states by a polynomial in  $\log W$ . Nor does this example show that an unknot hierarchy must visit all these states. A quasi-polynomial search bound needs an additional restriction on reachable attachment data or a way of skipping states.

## 29.4 Implementation bounds, verification and measurements

Let  $L$  be the total peripheral-word length and let  $B$  bound input integer bit lengths and  $\log(r + L + b + q + 2)$ . The classifier uses  $O((r + L + b)B^3)$  elementary bit operations conservatively; here  $L = O(r + b)$  for the canonical schema. It outputs  $O((r + L + b)B)$  bits. Prepared marking queries use  $O(qB^2)$  bit work and  $O(qB)$  extra storage using elementary division bounds. The normalized model header is shared inside a prepared index; comparing or serializing complete signatures includes its size. No loop depends on  $W$ , its divisors, the component count, or the number of unmarked boundary lifts.

**CoverIndex** builds its own validated classification and returns a defensive copy of its summary. A serialized summary is not accepted as a proof of a covering presentation. Integer fields support the existing literal-integer or signed-hexadecimal transport, including values beyond Python's decimal conversion limit. The standalone command supports a cooperative deadline; cancellation propagates and never becomes a topology answer. Checks occur in schema construction, gcd iteration, boundary processing and marked queries. Individual integer operations and bulk serialization are not hard-interruptible.

The ten maintained tests derived from the archive repeat 13846 comparisons against explicit lifted-spine graph search, orientation propagation, and boundary permutation cycles. The new independent audit exhausts all two-generator affine actions through six sheets. Across 364 presentations it performs 196712 marked and 1061 unmarked equivalence comparisons, 9100 rooted comparisons and 38746 transported-sheet checks against literal equivariant bijections. Additional tests cover binary sheet counts beyond  $2^{24000}$ , invalid marks, ordered repetitions, normalized input binding, defensive copies, cancellation, and hexadecimal CLI transport. The complete maintained suite passes 534 tests.

| Supplied cover    | Sheets | Records | Expanded/compressed | A/A   |
|-------------------|--------|---------|---------------------|-------|
| annulus           | 16     | 1       | 4.06                | 1.034 |
| mobius_reflection | 16     | 3       | 4.15                | 0.991 |
| three_types       | 16     | 3       | 3.52                | 0.990 |
| annulus           | 256    | 1       | 15.65               | 1.016 |
| mobius_reflection | 256    | 3       | 25.41               | 1.156 |
| three_types       | 256    | 3       | 17.58               | 0.988 |
| annulus           | 4096   | 1       | 244.61              | 1.072 |
| mobius_reflection | 4096   | 3       | 396.16              | 0.956 |
| three_types       | 4096   | 3       | 318.54              | 1.046 |
| annulus           | 65536  | 1       | 4621.74             | 1.045 |
| mobius_reflection | 65536  | 3       | 8290.72             | 0.989 |
| three_types       | 65536  | 3       | 7002.16             | 0.970 |

Ratios are median paired times for complete cover topology; sheet expansion is an independent oracle, not a competing compressed algorithm.

| Sheet exponent | Marks | Query (ms) | JSON bytes | A/A   |
|----------------|-------|------------|------------|-------|
| 32             | 2     | 0.010      | 100        | 1.001 |
| 32             | 32    | 0.077      | 456        | 0.999 |
| 32             | 512   | 1.187      | 6135       | 1.008 |
| 1024           | 2     | 0.014      | 698        | 1.013 |
| 1024           | 32    | 0.192      | 5534       | 1.009 |
| 1024           | 512   | 3.059      | 82897      | 1.001 |
| 16384          | 2     | 0.412      | 8286       | 0.998 |
| 16384          | 32    | 12.330     | 70055      | 1.000 |
| 16384          | 512   | 148.544    | 1058344    | 1.004 |

Marked queries reuse a prepared index, with sheet count  $W = 2^{\text{exponent}}$ . Preparation and serialization are outside query timing.

The topology benchmark keeps all input validation, peripheral evaluation, orientation and genus classification inside the compressed timer. Its expanded arm independently constructs the lifted spine and boundary permutations. Seven shuffled paired warm rounds retain all samples and A/A controls, with input generation and result comparisons outside timing. On 65536 sheets the observed expanded/compressed ratios range from 4622 to 8291; these measure avoidance of explicit expansion, not superiority to another compressed topology algorithm. The topology controls range from 0.956 to 1.156, so small constant-factor differences should not be overinterpreted.

The separate prepared-query experiment uses  $W = 2^{32}, 2^{1024}, 2^{16384}$  and 2, 32, 512 ordered points spread across a paired component. It includes mark validation and signature construction, but excludes the shared preparation and serialization. At  $W = 2^{16384}$ , 512 marks take about 149 milliseconds and the JSON signature has 1058344 bytes. The latter illustrates that mark output size is charged explicitly. Query A/A controls range from 0.998 to 1.013. These finite timings support the implementation checks; the polynomial bit-complexity claim rests on the arithmetic proof.

These measurements concern a geometric kernel with a supplied presentation. They are not timings of unknot recognition. To use it in a knot-exterior hierarchy we still need a checked extraction of the base and common fibre coordinate, an exact representation of all relevant attachments, and a bound on the hierarchy's search and repair states. Ordered fibre points are only one part of that attachment interface. The general recognizer does not dispatch to this module or claim a sub-exponential bound from it.

## 30 Causal RIII search: review and unambiguous certificates

The next intake placed three archives in arrival order as reports 26–28: determinant continuations, causal RIII search, and classical closure resets. The second archive calls itself “Report 25” internally; our collection number is 27 to avoid colliding with the earlier Garside archive. Original checksum manifests were verified in scratch before placement, then omitted under the repository intake rules. Delivered source and results remain unchanged apart from the mandated line-ending normalization of report 28’s benchmark CSV. The retained intake audit records all original hashes, normalized hashes and source pins. Local unit reruns pass 35, 7 and 29 tests respectively; these counts do not establish mathematical soundness or integration with the current recognizer.

### 30.1 What the causal argument can give

Report 27 considers bounded-local reversible moves on a fixed set of sites. Each move has a footprint containing everything it reads and changes. Actions on disjoint footprints commute in both directions and preserve each other’s legality. Terminal reductions also have local footprints. A move is a *birth* when its footprint is disjoint from the union of all preceding footprints. That union is accumulated support, not merely the footprint of the last move.

Every birth can commute to the front of the trace: it is disjoint from every earlier action. The births are pairwise disjoint and can be ordered canonically. Remaining actions retain an intersection with the preceding support. A reduction may become available sooner after this rearrangement, in which case the trace is truncated. Thus one can search a normal form with a canonical sequence of births followed by connected extensions. Connectedness of the full dependency graph does not imply that every chronological prefix was connected.

For RIII on a fixed input diagram, let  $D(T)$  be the twelve darts at the three crossings of a triangular face  $T$ , and let  $\alpha$  be the edge pairing. The conservative footprint

$$F(T) = D(T) \cup \alpha(D(T)), \quad |F(T)| \leq 24$$

contains every modified pairing. Inverting the face leaves this footprint unchanged. Globally there are  $O(N)$  triangular faces in a connected  $N$ -crossing spherical diagram. The number whose footprint meets a set of  $O(k)$  darts is  $O(k)$ , since only incident crossings and their partners need inspection. The report consequently obtains a conditional search bound

$$N^{b+O(1)}(Ck)^k$$

for traces of length at most  $k$  and birth number at most  $b$ , with an absolute constant  $C$ . A depth-first implementation without a global memo table uses polynomial workspace in  $N, k$ . The delivered abstract prototype does retain a memo table, so its memory usage should not be identified with the latter claim.

For example, fixed  $b$  and  $k = O(\log N / \log \log N)$  give polynomial time;  $b, k = O(\log N)$  give quasi-polynomial time. These are parameterized statements. No argument bounds the parameters uniformly for arbitrary unknots. Repeated preprocessing also requires the bound at each episode selected by the actual deterministic reduction policy; existence of a different successful sequence of choices is insufficient.

We reran the report’s independent breadth-first comparisons in scratch. The generated audit covers 5,000 six-site systems, with 1,660 witnessed instances, 3,209 already reducible and 131 without a depth-six witness. The exhaustive three-site catalog covers 608,400 instances through depth four, with 171,968 witnessed, 163,800 already reducible and 272,632 without a witness. Both reproduce zero discrepancies in the recorded normal-form checks. These are abstract Boolean systems, not knot-diagram benchmarks; they exercise the prototype, not the proposed production adapter.

### 30.2 Why immediate inverse pruning needs care

The proposed RIII adapter suppresses the inverse of the preceding move. The abstract prototype deliberately does not. For unrestricted shortest paths an adjacent inverse pair is removable, but this need not preserve the birth bound. Here is an explicit four-site example under precisely the abstract locality assumptions. Start with all bits zero:

| Move     | Footprint        | Toggled bits | Guard           |
|----------|------------------|--------------|-----------------|
| <i>A</i> | $\{0, 1, 2, 3\}$ | $\{0, 1\}$   | $x_2 = x_3 = 0$ |
| <i>B</i> | $\{0, 2\}$       | $\{2\}$      | $x_0 = 0$       |
| <i>C</i> | $\{1, 3\}$       | $\{3\}$      | $x_1 = 0$       |

Each guard avoids the toggled bits, so every legal move is an involution. The terminal predicate is  $x_2 = x_3 = 1$ . The trace *BC* succeeds but has two births. The trace *AABC* succeeds with one birth: after *AA* the physical state is again zero, while accumulated support contains all four sites. After the first *A*, both *B* and *C* are disabled, so the inverse *A* is essential for this one-birth trace.

Exhaustion of all words through length four gives exactly *AABC* and *AACB* as first-unlocking traces with at most one birth. Suppressing adjacent inverses loses both. The script `data/causal_inverse_audit.py` compares this independent enumeration with the delivered prototype and retains the source hashes and results. This is an abstract counterexample to the pruning justification; it is not an actual RIII counterexample. It shows that locality alone cannot justify transplanting that extra pruning rule into the claimed complete parameterized search. Physical-state memoization likewise cannot ignore support and search phase. The existing bounded local simplifier remains a heuristic; its failure is not a knottedness certificate.

### 30.3 Integrated prerequisite: identify the actual triangular face

Review also exposed a separate defect in the existing production trace format. Recording only the three crossing indices does not uniquely identify an RIII move. The two-strand braid  $(1, 1, -1)$  has the PD code

$$((5, 4, 0, 1), (1, 0, 2, 3), (2, 4, 5, 3)).$$

Both  $(11, 4, 0)$  and  $(9, 2, 6)$  are legal triangular faces on crossings  $\{0, 1, 2\}$ . Their inversions give distinct labeled PD codes:

$$\begin{aligned} &((0, 1, 2, 0), (3, 4, 1, 3), (2, 4, 5, 5)), \\ &((0, 1, 1, 2), (3, 4, 4, 0), (5, 5, 3, 2)). \end{aligned}$$

The old replay routine selected a face according to traversal order and therefore could reconstruct the wrong intermediate diagram. Both moves preserve knot type; the defect concerns reproducibility of the recorded sequence, including the legality of its subsequent moves.

New RIII records include a `triangle` field containing the face’s three input dart indices. These indices remain fixed after earlier crossing deletions. Replay checks distinct, present crossing indices, literal integer types, range, face membership and RIII legality. It recovers cyclic order from the current pairing, so reordering the three supplied dart indices cannot accidentally request a different reconnection. Legacy crossing-only RIII records are accepted when they identify a unique face, and rejected with an ambiguity error otherwise. R1/R2 serialization is unchanged. This is a certificate correctness fix, not a new recognition criterion or an asymptotic improvement.

Five new regression tests cover the explicit two-face example with all six dart orders, invalid certificates, every candidate RIII face in deterministically generated knot diagrams, legacy unambiguous replay, and JSON round trips after crossing deletions. The full integrated suite passes 539 tests in 109.018 seconds at that checkpoint; the log is `data/reidemeister-trace-integrated-tests.txt`.

## 30.4 Integrated search and adaptive dispatch

The production module `fastunknot/causal_r3.py` now implements the support-aware normal-form search. It uses an explicit depth-first stack with no global memo table. Each frame owns a journal of at most 24 pairing entries. Failure, local trial exhaustion and exceptions restore those entries and the path; successful first-unlocking traces remain applied. The retained footprint unions, per-frame candidate sets and pairing journals use polynomial workspace in  $N, k$ . Candidate enumeration polls an optional resource callback, as do terminal tests and the main search loop.

An additional pruning rule is justified under a narrower hypothesis than in the delivered adapter. Suppose an action with footprint  $F$  is applied when the earlier accumulated support is  $S$ . Its immediate inverse can be skipped if  $F \subseteq S$ . In that case deleting the pair preserves the physical state, accumulated support, birth count and last birth key. It can only delay the start of the connected phase; all subsequent connected actions remain legal in the normal form. The resulting trace is shorter. If  $F \not\subseteq S$ , the inverse is retained, including inverses of initial births. This distinction preserves the preceding counterexample and the parameterized completeness argument.

The `r3_search` option accepts `last`, `clustered`, and `adaptive`; the last-move neighborhood implementation remains the default. The clustered implementation searches with a caller-specified `r3_births` bound. Adaptive dispatch first gives the last-move search half the remaining trial allowance. If it fails to make progress, the clustered search uses the remaining shared allowance. When the total trial limit is disabled, the first phase is still limited to  $5N$  trials before switching. Each phase uses iterative deepening up to `r3_depth`. At a depth of at least two, the first move is not filtered by the last-step face shortcut. If a completed search at that depth tried zero nondegenerate moves, there is no legal first RIII move at all. Both search implementations stop deepening in that case, and adaptive dispatch also skips its second phase. This is a proved dead end for RIII-only search; it says nothing about crossing-increasing moves or knot type. In particular, failure of the face-filtered depth-one search alone does not justify stopping.

Successful reductions trigger another episode, with the same accumulated trial counter. Only paths ending in a legal RI/RII reduction are committed. The independent starting moves and retained support are local to each search episode, not retained across changes of crossing count.

The API and CLI expose the depth, births and trial allowance. The existing `use_r3=False` switch still disables the whole stage. Global time exhaustion interrupts either implementation and yields `UNKNOWN`; local trial exhaustion is inconclusive and continues the exact recognition pipeline. These options are preserved through certified source-reduction restarts and passed to each summand. Recognition records per-implementation trial counts and switch counts for the new modes.

Eleven new production tests compare bounded searches against unrestricted all-face enumeration on actual diagrams, verify disjoint births commute, check exact trace replay, retain inverse moves that add support, and exercise nested cancellation and injected mutation failures. They also check the no-first-move shortcut and deeper recognition of a stored hard unknot. The full integrated suite passes 550 tests in 98.849 seconds. Its log is `data/causal-r3-integrated-tests.txt`; the separate focused log is `data/causal-r3-production-tests.txt`.

## 30.5 Actual-diagram evidence and limits

The reproducible discovery run in `benchmark_causal_r3.py` uses seed 2703 and tests 12,000 random braid words on four to six strands. Of these, 2,296 close to knots and 1,537 remain above six crossings after the historical search at depth four and ten trials per input crossing. On those 1,537, each new mode ties the resulting crossing count in 1,529 cases and leaves more crossings in eight; neither improves it. This is a negative practical result for the proposed dispatch at those settings, even though the unlimited clustered search has a stronger parameterized completeness claim. Finite budgets and the choice of committed reductions matter.

Another 1,200 generated unknots use the existing conjugation-and-scrambling constructor with 800 rewriting steps. All already reduce below seven crossings with the historical simplifier; none enters the stalled-diagram comparison. These are easy examples for existing RIII preprocessing and supply no evidence of a gain from clustered search. The data and exact source hashes are in `results/causal_r3_discovery_20261008.json`. For all 3,496 validated diagrams, the current default simplifier’s complete trace and output PD code also agree exactly with pinned revision 28c974650; the new no-first-move shortcut changes work, not choices.

The stored hard unknot corpus gives a separate lead: two examples admit further reductions when the historical search depth is raised from four to six, using fewer than forty trials. This benefit comes from greater depth, not from the new support representation. Full-pipeline paired measurements are maintained separately from the discovery counts. The final run covers 21 inputs, seven shuffled paired rounds and three fresh constructions per arm, after one untimed warmup. The baseline loads both the simplifier and recognizer from revision 28c974650, sharing all other modules; an identical control arm measures variation. All recognition results finish and agree, with every default invariant and structural filter enabled and a five-second global limit.

At depth six, the historical search improves full recognition time by  $1.71\times$  on stored survivor 0 (sixteen crossings reduced to twelve) and  $3.61\times$  on survivor 1 (fifteen crossings reduced to zero). The adaptive depth-six arm gives  $1.48\times$  and  $3.65\times$  respectively; the second example finishes in its historical phase without switching. Across all inputs, the depth-six historical arm ranges from  $0.907\times$  to  $3.605\times$ , with median  $0.957\times$ . Consequently the default depth remains four; selective examples do not justify a universal increase.

The final default arm, including stricter option validation, cooperative deadline checks and the no-first-move shortcut, ranges from  $0.945\times$  to  $1.061\times$  in full recognition, with median  $0.989\times$ . The recognition control ratios range from  $0.983\times$  to  $1.054\times$ . For survivor 8 the no-first-move shortcut reduces isolated simplification from about 391 to 325 microseconds (paired ratio  $1.20\times$ ), with little effect on full recognition. On the empty diagram, validation adds about 1.34 microseconds to isolated simplification and 0.56 microseconds to recognition. These costs are retained in the reported measurements.

At depth four, the clustered and adaptive full-recognition median ratios are  $0.942\times$  and  $0.931\times$ ; neither supplies evidence for a default switch. Raw simplifier ratios must not be confused with end-to-end speedups: for example, spending more on simplification can save a larger subsequent homology computation on the depth-six witnesses. Inputs, individual samples, verdicts, methods, per-mode trial counts and exact source hashes are retained in `results/causal_r3_integration_20261008.json`. These are finite measurements, not asymptotic evidence.

A local search budget may stop preprocessing inconclusively; it cannot replace the exact fallback. The conditional bound still requires bounded unlocking parameters at each episode selected by the actual reduction policy. Neither this integration nor adaptive switching proves a general sub-exponential or quasi-polynomial recognition theorem.

## 31 Marked determinant continuations and an adaptive exact fallback

Report 26 supplies a stronger observation of a partially scanned complex: complete each genuine relative direct summand with the actual remaining diagram, reduce at a marked point, and retain its Euler characteristic in each quantum degree modulo four. Its norm can detect knottedness before the whole scan closes. The maintained implementation is `fastunknot/shadow_scan.py`, selected by `backend="shadow"`. It evaluates actual completed diagrams by exact integer determinants. The report’s reusable terminal kernel is verified separately; a new geometric producer for that kernel is still a research prototype. Neither observation gives a general bound on when a useful

summand appears.

### 31.1 Why the marked completion is necessary

At a proper prefix of a crossing order, choose a point on an edge incident to the last unprocessed crossing. The remaining tangle retains this point through every prefix observation. The marked circle carries the action of  $A = \mathbb{F}_2[x]/(x^2)$ . The relative cobordism maps, cancellations, and their explicit chain homotopies commute with this action. Completing the relative complex and applying the marked quotient therefore preserves these chain homotopies: apply the quotient to each identity  $fg - 1 = dh + hd$ . No assertion that the quotient functor is exact or flat is needed.

Suppose the maintained relative complex splits as whole differential components  $C_a$ , with multiplicities  $m_a$ . After the actual suffix is attached and the marked quotient is applied, it follows that

$$\widetilde{\text{Kh}}(K; \mathbb{F}_2) \cong \bigoplus_a H(\widetilde{\text{cl}}(C_a))^{\oplus m_a},$$

up to the recorded grading shifts. This is a statement about components of a genuine relative knot complex. An arbitrary graded matrix satisfying  $d^2 = 0$  is insufficient evidence of this provenance. There is also a crucial endpoint restriction. After the final crossing is processed, individual unmarked scalar generators need not be free marked  $A$ -summands. Dividing the Euler value of each such generator by  $q + q^{-1}$  would be invalid. The observer rejects the final stage, and the complete fallback instead uses the established unreduced rank test.

### 31.2 A four-coordinate lower bound

For a finite bigraded completed complex  $X$ , write

$$\sigma_r(X) = \sum_{h, j \equiv r \pmod{4}} (-1)^h \dim_{\mathbb{F}_2} X^{h,j}, \quad r = 0, 1, 2, 3.$$

This vector is unchanged by a graded chain homotopy equivalence. A uniform homological shift changes its overall sign and a uniform quantum shift cyclically permutes its coordinates. Thus neither changes its  $\ell^1$  norm. Taking Euler characteristics after passing to homology proves

$$\|\sigma(X)\|_1 \leq \dim_{\mathbb{F}_2} H(X), \quad B_4 = \sum_a m_a \|\sigma(\widetilde{\text{cl}}(C_a))\|_1 \leq \dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(K; \mathbb{F}_2).$$

More precisely, if  $E_r, O_r$  are the even and odd homological dimensions in residue  $r$ , the deficit is  $2 \sum_r \min(E_r, O_r)$ . Since reduced Khovanov homology detects the unknot,  $B_4 > 1$  certifies knottedness. A value of zero or one is inconclusive.

**Aggregation order matters.** For each whole differential component the implementation recovers relative quantum shifts using `recover_shifts`. A nonzero coefficient of dot degree  $d$  between matchings with  $m$  arcs and  $c$  glued circles requires  $q_{\text{target}} - q_{\text{source}} = m - c + 2d$ . Consistency is checked on all edges and cycles. The completed object vectors are shifted, multiplied by the object's homological sign, and *added before taking a norm*. Taking separate object norms would discard legitimate differential cancellations and could produce an unsound rejection.

Exact refinement of a direct-sum decomposition cannot decrease  $B_4$ : this is the triangle inequality, together with additivity of the Euler vector. Shared isomorphic components add their multiplicities. The production scanner caps those multiplicities at three; it preserves the threshold  $\min(2, B_4)$ , rather than the full uncapped bound. Tests compare both exact multiplicities and this saturated threshold.

### 31.3 Computing the vector without enumerating the cube

For a nonempty closed diagram use the raw reduced convention

$$J(q) = \sum_{s \in \{0,1\}^n} (-q)^{|s|} (q + q^{-1})^{c(s)-1}.$$

Its quantum degrees have one parity  $\epsilon = (c(0) - 1) \bmod 2$ . Put  $a = J(1)$  and  $d = i^{-\epsilon} J(i)$ , which is an integer. Only residues  $\epsilon$  and  $\epsilon + 2$  can occur, and

$$\sigma_\epsilon = (a + d)/2, \quad \sigma_{\epsilon+2} = (a - d)/2.$$

The engine verifies the parity and the exact divisions. The existing orientation-based closure calculation supplies the unreduced value  $2a$  at  $q = 1$ ; no exponential state sum is involved.

For the other specialization, construct the actual completed dart involution and follow the face permutation  $d \mapsto \text{rot}(\alpha(d))$ . Check that its Euler characteristic is that of a disjoint union of spheres, then checkerboard color its faces. A disconnected projection has  $J(i) = 0$ . For a connected projection choose either color as black, with  $v$  black regions. At crossing  $e$ , let  $b_e$  be the smoothing bit which does *not* join its black sectors, put  $w_e = (-1)^{b_e}$ , and set  $B = \sum_e b_e$ . With  $L_G^{(0)}$  a grounded cofactor of the signed Tait Laplacian,

$$J(i) = (-i)^{B+v-1} \det L_G^{(0)}.$$

One-circle smoothings correspond to spanning trees of the Tait graph; expanding their signed weights gives this formula by the matrix-tree identity. The identity is polynomial in the edge weights, so negative weights do not require a positivity assumption. Tait loops contribute to  $B$  even though they make no Laplacian contribution. Dropping this phase, or replacing the determinant by its absolute value, would corrupt cancellation within a component.

Either checkerboard color is valid if its phase is computed consistently. The maintained engine chooses the smaller color class to reduce the dense cofactor dimension. Fraction-free Bareiss elimination, with row pivoting and exact integer division, computes the determinant. Its integer intermediate sizes are polynomially bounded for these integer Laplacians; the determinant calculation for one completion has polynomial bit complexity in its crossing count.

For a single parity vector the norm is  $\max(|a|, |d|)$ , using  $|a + d| + |a - d| = 2 \max(|a|, |d|)$ . Thus the observation retains both Euler and determinant information. The significant new possibility is their application to separate genuine completed components, whose contributions may cancel in the global polynomial. Abstract examples of stronger component bounds do not by themselves establish that a knot scan realizes those examples.

### 31.4 A budgeted observer on the same exact scan

The driver first tries the existing component Euler obstruction, then the four-coordinate bound. Stage zero is always eligible. Later observations are skipped while there is just one component copy, since its completed Euler vector is the unchanged global vector up to shifts. Observation resumes when a nontrivial direct-sum decomposition appears. The suffix preparation and completed values are cached by stage and boundary matching. Euler and determinant observations share the `euler_max_states` allowance, default 4096 distinct queries.

An additional `shadow_max_work` allowance, default one million, charges geometry visits, dense matrix allocation and arithmetic updates. It is an operation allowance, not a count of bit operations or a hard wall-clock guarantee. The cooperative global deadline is polled during geometry and between determinant elimination rows, including on cache hits. An incomplete determinant is never cached as a completed value.

At this integration checkpoint, exhaustion of either local allowance disabled further observations and continued the *same* capped exact scan. Section 35 subsequently refines this policy by retaining a separately query-capped Euler reserve after marked-work exhaustion. No restart, discarded completed pivot, or speculative verdict is needed. Global resource exhaustion still returns UNKNOWN through the recognition API. An unknot verdict is available only when the complete rank computation has closed. The empty diagram is handled by its known unreduced rank two.

This is an adaptive policy in the sense suggested by introsort: expensive optional work has a budget and a complete fallback. Its guarantee is that inference cannot continue consuming unbounded local work after that allowance. Unlike introsort’s worst-case sorting theorem, it does not improve the fallback’s asymptotic bound. The default recognition backend remains unchanged because the measurements below do not show a broad end-to-end benefit.

### 31.5 A reusable terminal kernel, including singular interiors

Report 26 also proposes factoring a common signed Laplacian into interior vertices  $I$  and boundary terminals. A completion identifies terminals; it must leave interior vertices distinct. A nonsingular interior block could be eliminated by an ordinary Schur complement, but signed weights allow a singular interior even when some completed cofactor is nonzero. The archive uses symmetric one- or two-coordinate pivots to extract a nonsingular principal block  $P$ . The remaining interior block is zero, of dimension  $r$ . With residual coupling  $R$ , terminal matrix  $S$ , and grounded partition matrix  $Q$  for a query with  $p$  terminal blocks, its cofactor is

$$\det P \det \begin{pmatrix} 0 & RQ \\ Q^\top R^\top & Q^\top S Q \end{pmatrix}, \quad Q \text{ has } q = p - 1 \text{ columns.}$$

If  $r > q$ , or  $RQ$  lacks row rank  $r$ , the answer vanishes. In particular,  $r > b - 1$  makes every query vanish when there are  $b$  terminals. At the critical size  $q = r$  the answer becomes

$$(-1)^r \det P \det(RQ)^2,$$

independent of  $S$ . Consequently two nonzero critical answers have the same sign and rational square class; their product is a positive integer square. These are identities for a verified common graph and partition, not a license to identify arbitrary regions of unrelated completions.

**An explicit geometric producer.** The new research program `fast/determinant_research/terminal_geometry_audit.py` retains the original black regions incident to the suffix crossings, and retains exactly those crossings as signed edges. Original black regions touching a frontier edge become terminals. For each completed matching it verifies that completed faces retain the original colors, that each retained original black region maps to one completed face, and that every fiber containing more than one old region consists only of terminals. These conditions establish the graph quotient by checking each retained edge’s endpoints; no dense matrix comparison is required. The phase uses the *completed* black vertex count.

The program constructs one archive terminal kernel per suffix and verifies its algebraic certificate. Disconnected completed projections have zero determinant specialization and are handled separately. Any other failed geometry check declines the query; it cannot certify a knot. Across 300 random braid attempts, 78 validated knots and all encountered prefix matchings, 3186 connected queries passed the checks and agreed with direct completed determinants. A further 1317 queries had disconnected projections. There were no color, region-splitting, or interior-identification failures; the largest terminal set had size ten. The input words, crossing orders, and source hashes accompany these 4503 queries in `data/determinant-terminal-geometry-audit.json`. This is finite evidence for a useful construction, not a proof that every realizable matching passes its checks.

The recognizer currently uses direct completed determinants; kernel reuse needs a maintained implementation, interruptible exact arithmetic, and measurements of setup amortization before production dispatch is justified.

### 31.6 Independent checks and actual performance

The unchanged delivered package passes its 35 tests. Its full validation rerun covers 701 diagrams, 133923 direct cube states, 83 homology calculations, 62904 columns checked for  $d^2 = 0$ , and 4000 quotient queries on 160 signed graphs, including 57 singular interiors. The upstream probe performs 176 scans of 44 diagrams, using two crossing orders and both physical and shared components. All 820 proper-prefix observations obey the exact-rank bound and the appropriate monotonicity condition.

Ten maintained tests independently compare Bareiss elimination with permutation expansion, completed vectors with direct cube polynomials, and component bounds with reduced homology. They check ordinary and capped multiplicities, mirror diagrams, global deadlines, partial determinant interruption, shared cache limits, exact fallback with zero inference budget, invalid inputs, and actual CLI dispatch. They also verify that observing a component does not mutate its differential. The full integrated suite passes all 560 tests.

One concrete strict improvement is the natural crossing order of  $T(3, 5)$ . Its initial four-coordinate norm is one. After nine of ten crossings, the completed components have total norm seven, while their reduced Euler bound is only one. The marked backend therefore rejects at stage nine; the Euler backend reaches the final stage ten. Similar one-stage improvements occur on the measured  $T(3, 7)$  and  $T(3, 11)$  inputs. An earlier certified stage does not automatically imply a faster runtime.

| Input                 | Full/shadow | A/A   | Raw/shadow | Raw/Euler | A/A   |
|-----------------------|-------------|-------|------------|-----------|-------|
| unknot                | 0.985       | 0.997 | 0.079      | 0.969     | 1.020 |
| trefoil               | 1.014       | 1.028 | 3.015      | 0.980     | 0.974 |
| conway                | 0.979       | 1.015 | 0.948      | 0.990     | 0.993 |
| kinoshita_terasaka    | 1.007       | 1.013 | 0.951      | 0.992     | 0.987 |
| hard_unknot_8         | 0.991       | 1.002 | 0.854      | 1.019     | 1.002 |
| grid_scrambled_unknot | 0.986       | 0.982 | 0.803      | 0.980     | 0.979 |
| stress_braid5_36      | 1.000       | 0.993 | 360.190    | 1.032     | 0.987 |
| conway_sum_2          | 1.004       | 1.004 | 1.475      | 1.539     | 1.007 |
| conway_sum_8          | 1.012       | 0.969 | 4.144      | 5.603     | 1.003 |
| torus_3_5             | 1.007       | 1.002 | 1.023      | 1.002     | 1.003 |
| torus_3_7             | 1.002       | 1.017 | 1.000      | 0.983     | 0.986 |
| torus_3_11            | 0.994       | 1.000 | 0.972      | 0.976     | 0.997 |

Ratios are median paired times; values above one favor the denominator. Full uses the previous default recognizer; raw uses the saturated scanner. A/A compares identical implementations in each scope.

The complete recognizer shows no material benefit from the new backend on these inputs: old-default/marked ratios range from 0.979 to 1.014, while the identical-control range is 0.969–1.028. The changed default entry point has old/new ratios 0.962–1.019, with median 0.998. In contrast, the raw marked scanner is about 3.02 times faster on the trefoil and 360 times faster on the 36-crossing stress braid, rejecting both at stage zero. Existing recognition filters already decide those inputs. The Conway sums gain from component Euler information available to the old Euler backend too; that backend is faster than the marked observer on both measured sums. The raw marked scanner is slower on Conway, Kinoshita–Terasaka, both nonempty unknot fixtures, and the empty diagram. The one-stage torus improvements give ratios of 1.023, 1.000, and 0.972, respectively, against saturated scanning. These results support optional use and further work on

selective observation, rather than enabling the backend by default. For scale, the empty raw scan rises from about 2.56 to 34.90 microseconds, while the stress braid falls from about 635 to 1.72 milliseconds; these are medians of the individual arm times.

The reproducer is `fast/benchmark_shadow.py`. It compares twelve inputs over seven shuffled paired rounds, with a duplicate control arm. Recognition measurements keep every default filter enabled and include fresh validated diagram construction. A separate raw-decision scope bypasses those filters and compares the saturated, Euler and marked scanners. Each call has a five-second and 50000-object allowance; any timed repetition returning UNKNOWN suppresses ratios for that case. The historical recognition control loads only `recognize.py` from commit 3a6d80ed6, sharing its unchanged dependencies. Imports and one warmup are excluded. Complete samples and source hashes are retained in `fast/results/marked_shadow_20261008.json`.

### 31.7 What is still missing for a general complexity theorem

Each determinant query is polynomial, but the number and size of physical relative components need not be small. The report’s stopping-prefix bound has the conditional form  $\text{poly}(n, R)2^{O(W)}$ , where  $R$  bounds the actual maintained physical data and  $W$  bounds the frontier through the first decisive observation. Neither parameter has a universal bound sufficient for quasi-polynomial recognition. A planar separator bound on a diagram alone does not bound the surviving algebraic multiplicities or ensure an early decisive decomposition.

The new observer is useful precisely because it may stop before computing all homology. That possibility does not establish that it stops early on every nontrivial knot, nor that the complete fallback is subexponential on all unknots. The earlier assessment of Lackenby’s hierarchy claim remains unchanged: it is a plausible geometric route, while this implementation still lacks a general quasi-polynomial or subexponential running-time proof.

## 32 Classical closures, first jets, and bounded diagram resets

Report 28 studies a different way to avoid carrying a large complex through the entire crossing order. When a whole relative summand has one matching, its *actual classical completion* imposes restrictions absent from an arbitrary algebraic continuation. In certain cases the entire current scan has the same total homology rank as that completed matching alone. We can then splice the matching into the remaining diagram and start a fresh scan with fewer crossings. The maintained decision-only backend `closure`, implemented in `fastunknot/closure_scan.py`, combines these resets with closure-rank obstructions and bounded optional work. The standard backend remains the default.

### 32.1 Source audit and the exact scope of the rerun

The delivered package distinguishes its shortened reference fixture from the production checkout. We ran its AST checker against the current checkout. The scanner agrees after docstrings are removed, but the geometry fixture’s `compose` omits four maintained fast paths: zero factors, the two identity cases, and equal endomorphism factors. The latter uses the square-zero radical identity. Moreover, the geometry blob listed in the provenance file is not the blob at the stated inspected revision 13f6dbfdd: the actual blob there starts 92135876, whereas the supplied pin starts 1078526e. The other three pins match that revision. The earlier intake ledger already flagged this mismatch, but its summary and catalog wording overstated the verification. Those summaries are now corrected. The detailed follow-up is retained in `data/closure-reset-source-audit.json`. The 29 delivered tests pass with imports redirected to the actual production package. We also reran the independent cube audit through `fast/closure_research/audit_upstream.py`. This

adapter changes only the import paths, output destinations, and scope description; the delivered files remain unchanged. All 480 driver comparisons on 160 diagrams, 480 interval-rank checks, and 130 quiver-rank checks pass. Eighty diagrams reset and two have early nontriviality certificates. No nonsingleton eligible block occurs in this corpus. The independent cube does not import scanner code.

### 32.2 Why a first jet controls a knot completion

Let  $m$  be a nonempty geometric matching with  $p$  arcs, and let a whole differential component supported on  $m$  have coefficient matrix

$$Q(\mathbf{x}) = A + \sum_{i=1}^p x_i B_i + \text{higher dot terms}, \quad Q^2 = 0, \quad B = \sum_i B_i,$$

over  $\mathbb{F}_2[x_1, \dots, x_p]/(x_i^2)$ . The matrix raises homological degree by one. Gluing the actual suffix gives a link  $L_m$ , whose Khovanov complex  $C$  carries commuting square-zero dot operators  $X_i$ . The completed block has differential  $D = d_C + Q(X)$  on  $C \otimes V$ . The component must have no incoming or outgoing attachments; a selected submatrix does not meet this premise.

**The chain-level calculation.** Moving a marked point across a crossing has reverse-saddle homotopy  $H$  with  $d_C H + H d_C = X_a + X_b = \delta$ ,  $H^2 = 0$ , and commutation with all fixed dot operators. It also satisfies  $H\delta = 0$ . These are strict chain identities, recorded in Nahm's Lemma 2.4 and its cited predecessors [10]; equality of induced dot maps on homology would not suffice here. For  $P = \partial Q / \partial x_i$ , differentiation of  $Q^2 = 0$  gives  $PQ + QP = 0$ . Thus  $G = 1 + HP(X)$  is self-inverse and

$$GDG = D + \delta P(X).$$

Indeed the linear error is  $\delta P$ , while the quadratic error is  $H d_C H P^2 = H \delta P^2 = 0$ . There is no requirement that  $P^2 = 0$ . This conjugation moves one variable's dot action to the new marked edge.

If  $L_m$  is a knot, all marks can consequently be moved to one edge. All higher monomials vanish under  $x_i \mapsto X$ , leaving  $A + XB$ . The characteristic-two vertical operator  $\nu$ , which changes one circle label  $x$  to 1 and sums over circles, satisfies

$$\nu^2 = 0, \quad \nu d_C = d_C \nu, \quad \nu X + X \nu = 1.$$

Hence  $C = E \oplus XE \cong \mathbb{F}_2[x]/(x^2) \otimes E$ , where  $E = \text{im } \nu$  is a subcomplex. Contracting  $(V, A)$  onto its homology transfers  $A + xB$  to  $xB_*$ : two perturbation factors vanish because  $x^2 = 0$ . If  $t = \dim H(V, A)$  and  $b = \text{rank } B_*$ , the transferred complex has dimension  $2t$ , differential rank  $b$ , and homology dimension  $2(t - b)$ . Therefore

$$\dim H(\text{cl}(Q)) = \kappa(Q) r(L_m), \quad \kappa(Q) = \dim H(V, A) - \text{rank } B_*,$$

where  $r$  denotes total unreduced characteristic-two Khovanov rank. The formula preserves total rank; it does not assert equality of the original open complexes or their bigraded profiles.

After exact scalar-unit cancellation,  $A = 0$ , so

$$\kappa = N - \sum_h \text{rank } B_h \geq 1.$$

The implementation reads only the parity of singleton-monomial coefficients in each entry, then computes adjacent binary ranks. Counting all monomials modulo two would give the wrong matrix. Neither barcode reconstruction nor ranks of products of adjacent matrices are needed. The maintained reader declines blocks containing scalar units; the general formula above explains the theory but is not an additional production scalar eliminator.

### 32.3 Pure blocks and the indispensable link distinction

For a pure block  $Q = \theta T$  with one nonzero radical coefficient, put  $\beta = N - \sum_h \text{rank } T_h$  and let  $\sigma = \sum_i [x_i] \theta$  be its total-linear parity. The scalar quiver has  $\beta$  intervals, although  $T^2$  need not be zero. When  $\sigma = 1$ , the derivation identity  $\nu\theta + \theta\nu = U$  has  $U^2 = 1$  and  $\nu U = U\nu$ . Then  $U\nu$  splits the suffix complex into a free dual-number factor for the action of  $\theta$ . This proves completed rank  $\beta r(L_m)$  for *every* classical link completion.

If  $\sigma = 0$  and the completion is a knot, the first-jet matrix vanishes, giving completed rank  $Nr(L_m) \geq 2N$ . If the completion has at least two components, each quiver interval contributes at least  $r(L_m) \geq 4$ . The first inequality follows from the monotone interval profile already proved in the continuation analysis; the second follows from the link's Euler specialization. Consequently a pure block has lower bound

$$\begin{cases} 2\beta, & \sigma = 1, \\ 2 \min(N, 2\beta), & \sigma = 0. \end{cases}$$

Any even pure block with an edge therefore certifies rank at least four. The implementation uses this threshold directly instead of computing unnecessary exact interval data.

A nonpure block requires the knot-completion premise. For example, on the two-component unlink,  $x_1 + x_2$  acts nontrivially, and a three-term interval joined by this coefficient has rank four. Applying the knot formula would incorrectly give twelve. The maintained tests include this counterexample; no completion with several components is silently treated as a knot.

### 32.4 Recognition control, splicing, and fallback

The backend scans with exact component sharing and multiplicities capped at three. At each proper prefix it inspects whole differential components. Pure components give the closure-uniform lower bounds above. Otherwise the driver constructs the actual completion for each eligible matching, checks its spherical rotation system, and counts link components. A knot completion supplies  $2\kappa$  as a lower bound. Contributions are multiplied by their nonnegative integer multiplicities and added; rank at least four rejects the original knot.

When the entire scan has one component copy, one matching, and  $\kappa = 1$  with a knot completion, that residual knot has the same total rank as the current computation. Each matching arc joins two boundary labels of the unprocessed suffix. The splice merges these labels, canonicalizes the remaining PD code, and retains the selected crossing order. Removed smoothing circles have already been handled by delooping; they are not reintroduced. A fresh scanner starts from empty boundary on the strictly shorter residual diagram. No claim of a knot isotopy between the original and residual diagrams is needed or made.

The local `closure_max_work` allowance, default one million, charges observer traversal, polynomial extraction, binary elimination, and completion geometry. It is an operation allowance, not a bit-cost or strict wall-clock bound. The global cooperative deadline has priority. Local exhaustion disables observation and continues the current exact scan, preserving all completed algebraic work. A partially computed bound or residual is never committed as a reset. Global exhaustion returns `UNKNOWN` through recognition. Only final closed rank two certifies the unknot; a proper-prefix failure to reject is inconclusive.

The decision interface exposes capped ranks and reset events. It is not connected to the exact homology or window APIs, because resets discard grading information. The optional backend retains the same alternate backend restrictions as the existing shared scanners.

### 32.5 Actual reset gaps and what they do not prove

Let  $g$  be the maximum number of crossings processed between resets or the final decision, including the initial and terminal segments. After  $j$  crossings in one segment, there are at most  $4j$  frontier labels and, using a deliberately loose delooping bound, at most  $8^j$  physical objects. The coefficient basis has size  $2^{O(j)}$ . Exact arithmetic, elimination, first-jet inspection, and geometry checks cost  $\text{poly}(n)2^{O(j)}$  in bit operations. Saturated sharing does not increase the literal object bound. The maintained ordering heuristic has polynomial overhead, and preserving the suffix order means each original crossing is processed only once across all segments. Thus this implementation satisfies the parameterized bound

$$T(n, g), S(n, g) \leq \text{poly}(n)2^{O(g)}.$$

The recorded `max_gap` is this execution parameter, including scanning after observation has been disabled. A polylogarithmic bound on *actual* gaps would give quasi-polynomial recognition. We do not know such a bound for arbitrary knots or unknots, or an efficiently constructible ordering that guarantees it. Adaptive fallback does not supply that missing theorem.

### 32.6 Expanded discovery, validation, and paired measurements

The maintained discovery program `fast/closure_research/audit_blocks.py` samples 1200 braid attempts with seed 2801, accepting 312 knots. Each is scanned in a random order and the width heuristic's order, with a quarter-second and 3000-object allowance per scan. All 624 scans complete: 4972 crossing stages produce 785 singleton observations and no nonsingleton pure or first-jet block. This larger negative result supports no claim of actual-diagram gains from the mixed-coefficient rule. It does not prove that such blocks cannot occur. Inputs, orders, limits and source hashes are retained in `data/closure-reset-block-discovery.json`.

The maintained unit checks directly complete forty mixed polynomial matrices against the independent trefoil cube, verify  $d^2 = 0$ , and compare their ranks with  $\kappa r(L_m)$ . They also compare sixty random knots against the independent cube under resets, no resets, and zero observation budget; compare actual splices with the independent implementation; check malformed blocks and external attachments; and test global deadlines, CLI dispatch, and local exhaustion after successful resets. A seven-crossing Coxeter unknot has six resets and maximum gap one. This is an illustrative family already easy for earlier structural recognizers. The complete maintained suite passes all 567 tests. Shape caching is tested both enabled and disabled; automatic mode applies the established repeated-stage heuristic to each residual diagram, so the raw comparison uses the same cache policy as the existing saturated scanner.

| Input                     | Full/reset | Raw/reset | Raw/no reset | Raw/Euler | Raw A/A |
|---------------------------|------------|-----------|--------------|-----------|---------|
| unknot                    | 0.996      | 0.104     | 0.108        | 0.984     | 0.965   |
| trefoil                   | 1.028      | 0.953     | 1.126        | 0.991     | 1.180   |
| conway                    | 0.999      | 0.938     | 0.942        | 0.996     | 1.005   |
| kinoshita_terasaka        | 1.001      | 0.942     | 0.939        | 0.997     | 0.998   |
| hard_unknot_8             | 1.022      | 0.889     | 0.923        | 0.961     | 1.001   |
| grid_scrambled_unknot     | 0.986      | 0.646     | 0.719        | 0.978     | 0.993   |
| stress_braid5_36          | 1.001      | 0.927     | 0.949        | 1.014     | 0.990   |
| conway_sum_2              | 0.994      | 1.420     | 1.427        | 1.538     | 1.003   |
| conway_sum_8              | 1.012      | 5.283     | 5.241        | 5.576     | 1.005   |
| R28 cancel_pairs_26       | 1.003      | 0.465     | 0.589        | 0.973     | 1.000   |
| R28 coxeter_24            | 0.978      | 0.297     | 0.383        | 0.974     | 1.006   |
| R28 figure_eight          | 1.001      | 0.951     | 0.900        | 0.962     | 0.996   |
| R28 positive_2braid_11    | 1.015      | 0.940     | 0.941        | 0.993     | 1.004   |
| R28 trefoil               | 1.009      | 0.959     | 0.962        | 0.994     | 0.996   |
| R28 trefoil_then_curls_12 | 1.007      | 2.034     | 1.967        | 1.996     | 0.980   |
| R28 trefoil_then_curls_24 | 1.004      | 2.401     | 2.371        | 2.307     | 1.016   |
| R28 trefoil_then_curls_6  | 1.003      | 1.627     | 1.617        | 1.633     | 0.990   |
| torus_3_5                 | 0.995      | 0.965     | 0.961        | 0.988     | 1.007   |
| torus_3_7                 | 0.998      | 0.968     | 0.969        | 0.994     | 0.998   |
| torus_3_11                | 1.000      | 0.929     | 0.922        | 0.987     | 1.002   |

Ratios are median paired times; values above one favor the denominator. Full uses the previous default recognizer; raw uses the saturated scanner. The no-reset arm retains closure observations but does not restart.

The full-recognition ratios lie between 0.978 and 1.028, with median 1.001, while the identical full-control ratios span 0.976–1.081. All twenty inputs are decided by earlier enabled filters, so this corpus shows no complete-recognition gain from closure dispatch. The unchanged default path has median old/new ratio 1.008, with variation consistent with the controls. One raw trefoil control reaches 1.180; its small scanner differences should not be interpreted as reliable gains.

The raw observer improves the Conway sums by about 1.42 and 5.28 times, and the trefoil-with-curls examples by 1.63, 2.03, and 2.40 times. These are early obstructions with *zero resets*; the existing Euler backend already provides comparable gains. The actual reset cases are slower: saturated/reset ratios are 0.889 for the hard unknot, 0.646 for the scrambled grid unknot, 0.465 for cancellation pairs, and 0.297 for the Coxeter example. Their reset counts are 1, 4, 13, and 23. The no-reset control is faster on each of these four inputs, showing that splicing and scanner reinitialization add overhead here. There is no observed nonsingleton first-jet compression in this benchmark either. These measurements justify retaining the optional backend for further research and leaving automatic recognition dispatch unchanged.

`fast/benchmark_closure.py` measures full recognition separately from raw saturated, Euler, reset, and no-reset scanners. It includes the eight delivered examples and twelve maintained examples, uses seven shuffled paired rounds, includes validated diagram construction, and retains an identical control arm in each scope. Imports and one warmup are excluded. Calls have five-second and 50000-object limits; any timed repetition returning `UNKNOWN` suppresses that case’s timing ratios. The old recognition arm loads only `recognize.py` from `e6f975521`, sharing unchanged dependencies. All raw samples and source hashes are retained in `fast/results/closure_reset_20261008.json`.

### 33 Adaptive boundary summaries for repeated completion queries

Repeated observations of one suffix should share its topology. The maintained Euler backend now starts with direct closure walks and switches to a pair of boundary path summaries when enough distinct queries reach the same stage. A separate research implementation extends this

idea to report 26's signed Tait kernel. Its common vertices are *cut-face fragments*; this choice makes both the quotient construction and its geometric checks depend on the boundary rather than on another traversal of the entire suffix. Neither improvement bounds the number of relative generators or proves a general subexponential recognition algorithm.

### 33.1 Two path summaries determine the raw Euler value

Fix a suffix with  $n$  crossings and  $b$  boundary darts. Its partial edge involution is  $\alpha$ ; at each crossing the opposite-dart pairing describes link strands, and the zero-smoothing pairing describes the all-zero state. Combining either local pairing with the internal edges gives disjoint closed circles and paths whose endpoints lie on the boundary. Store the number of closed circles and the induced boundary matching for each of these two graphs. Preparation takes  $O(n + b)$  combinatorial work and temporary space, while the retained summaries have size  $O(b)$  apart from the integer counts.

For a proposed boundary matching  $m$ , the circles of the completed graph are exactly the stored closed circles plus the alternating cycles of the stored boundary matching with  $m$ . Thus two boundary walks compute the link component count  $c$  and the all-zero circle count  $c_0$ . Coverage is checked: every boundary label must occur once, with no missing, repeated, or unknown endpoint. No suffix dart need be revisited by these queries.

Here is why these counts suffice even for the sign. Choose any orientation of the classical completion, let  $h$  be the number of one-smoothings in its oriented Seifert state, and let  $s$  be its number of Seifert circles. Changing one smoothing changes circle parity, giving  $s \equiv c_0 + h \pmod{2}$ . The orientable Seifert surface has Euler characteristic  $s - n$  and  $c$  boundary components, hence  $s - n \equiv c \pmod{2}$ . This also holds for a disconnected surface. Therefore

$$h \equiv n + c + c_0 \pmod{2}, \quad \chi_{\text{raw,unred}}(m) = (-1)^{n+c+c_0} 2^c.$$

This is the unnormalized Euler value needed by the existing relative scan. The production caller supplies classical completions from a valid scan; these two path summaries alone are not a test for virtual planarity. At the empty scalar terminal the convention is  $n = c = c_0 = 0$ , with value one; it does not denote an implicitly added crossingless unknot.

Each subsequent query uses  $O(b)$  combinatorial operations. This is not an  $O(b)$  bound on bit complexity independent of  $n$ : the integer  $2^c$  itself has  $c + 1 = O(n + b)$  bits. The implementation retains exact integers and the existing coefficient-size statistics.

### 33.2 When the maintained backend switches

The summaries are implemented in `fast/fastunknot/boundary_connectivity.py`.

`AdaptiveClosureEuler` in `fast/fastunknot/euler_scan.py` uses the independent orientation-based `ClosureEuler` for the first four distinct, uncached queries at a stage. Later queries prepare the summaries if  $b \leq n$ ; a wide boundary on a short suffix keeps direct evaluation. This is an empirical setup policy, not an optimal stopping theorem. A low-level `compression_after=0` option forces eager preparation for diagnostics. The public Euler backend uses the adaptive policy; the standard recognizer backend remains unchanged.

Changing the stage discards the active summaries. Cached exact values do not consume another query allowance. The global deadline is checked even on a cache hit, and the local query cap is checked before preparation. A summary is published only after both path traversals finish, so an interrupted setup cannot leave a usable half-summary. Local inference exhaustion continues the same exact saturated scan, while global exhaustion returns `UNKNOWN`, as before. The direct closure evaluator and the older suffix-smoothing recurrence remain independent references.

Five new tests cover random classical completions and their mirrors, independent recurrence values and explicit circle counts, switching and stage changes, the wide-boundary guard, cache and budget

behavior, malformed coverage, and interruption after the first of the two path summaries. The full maintained suite passes all 572 tests; logs are retained in `data/boundary-connectivity-tests.txt` and `data/boundary-connectivity-integrated-tests.txt`.

### 33.3 A common Tait graph made from cut-face fragments

The research implementation `fast/determinant_research/boundary_tait.py` starts with a checkerboard coloring of the original classical diagram. Write  $\rho$  for cyclic rotation of a crossing's darts. A completed face follows the permutation  $\rho\alpha$ . Retain only its edges arising from internal suffix edges. Each resulting component is either a closed face cycle or an open face path. These *fragments*, rather than entire original regions, are the common graph vertices. In particular, a region that splits when the prefix is removed is represented by separate fragments.

For a boundary label  $a$  with dart  $d_a$ , store the fragment containing  $d_a$  as its outgoing endpoint and the fragment containing  $\rho(d_a)$  as its incoming endpoint. Pairing labels  $a, b$  inserts exactly the two face connections

$$d_a \longrightarrow \rho(d_b), \quad d_b \longrightarrow \rho(d_a).$$

Consequently every completed face is obtained by identifying these open fragments, while every closed fragment remains untouched. This proves the face reconstruction directly from the partial permutation; it does not assume that the original region partition survives a new closure.

There are at most  $2b$  open fragments. A query runs union-find only on them and on the suffix projection components incident to the boundary. Counts of closed fragments and closed projection components are stored once. Every inserted face connection must preserve the inherited checkerboard color. If  $F$  is the resulting face count and  $k$  the resulting projection component count, the query also requires

$$F = n + 2k.$$

The completed rotation system has  $V = n$  and  $E = 2n$ . Its orientable ribbon components have total Euler characteristic  $F - n = 2k - 2 \sum g_i$ ; the displayed check is therefore equivalent to every  $g_i = 0$ . It certifies spherical completion for accepted queries. Color incompatibility is a decline of this particular representation, even when a completion is classical; it is never evidence that a knot is nontrivial.

Choose the smaller shading by fragment count. Each crossing contributes one signed edge between its two black fragments, with the same local weight as in Section 31. The black open fragments are terminals. All identifications of the common graph occur among these terminals; black closed fragments are fixed interior vertices. Thus an accepted connected completion has exactly the quotient Tait graph required by report 26's terminal kernel. Its phase is recomputed from the resulting black vertex count  $v$ :

$$J_{\text{raw}}(i) = (-i)^{B+v-1} \tau(G/m).$$

The crossing sum  $B$  includes loop edges even though loops contribute zero to the Laplacian. If the completed projection is disconnected, the specialization is zero and no kernel is needed. If there is no boundary, one black vertex serves as the artificial grounding terminal.

The common Laplacian is prepared once; the delivered exact rational kernel is built lazily. Canonical terminal partitions cache completed integer cofactors, so distinct matchings inducing the same partition share a value. Phase and topology counts are still computed for each matching. The two boundary path summaries give the raw value at one and the parity  $\epsilon = c_0 - 1 \pmod{2}$ . Together with the value at  $i$ , they reconstruct the four marked residues by the existing  $(a + d)/2, (a - d)/2$  formula.

The geometric query costs  $O(b\alpha(b))$  combinatorial operations. For the determinant, the singular-safe elimination described earlier leaves a border of dimension  $r + q$ , where  $r$  is interior nullity

and  $q$  is one less than the number of terminal blocks. If  $r > q$ , the cofactor vanishes; otherwise  $r + q = O(b)$ . This controls matrix dimension, not coefficient bit length or the number of scan queries. Rational arithmetic can still depend polynomially on the original suffix size. Production integration also requires an interruptible maintained kernel and useful amortization on actual observer traces. The present prototype imports report 26’s kernel and has no production deadline or local-work interface; production shadow observations continue to compute their closures directly.

### 33.4 Independent geometry audit

`fast/determinant_research/audit_boundary_tait.py` repeats all 4,503 actual completion queries on 582 proper-prefix stages of 78 retained input diagrams. Every accepted value agrees with an independently rebuilt completed diagram, including component counts, zero circles, raw Euler values, and the Gaussian value at  $i$ . There are 1,317 disconnected projections, 15 singular stage kernels, maximum terminal count ten, and maximum interior nullity two. Kernel certificates are checked independently.

A further exhaustive pairing audit covers boundary sizes four through eight in the first 20 retained diagrams: among 3,468 pairings, 745 are accepted, 472 classical completions are declined because the inherited coloring does not fit, and 2,251 nonclassical completions are rejected. Every accepted pairing passes the independent classical completion check and both value comparisons. The scope, exact inputs, and source hashes are recorded in `data/boundary-tait-audit.json`. These finite checks support the implementation; the fragment and Euler-characteristic arguments above supply the general accepted-query contract.

### 33.5 Measured reuse and its limits

`fast/benchmark_boundary.py` uses seven shuffled paired rounds with identical control arms and one excluded warmup. Every timed sample includes fresh engine setup. The isolated workload consists of 200 distinct actual matchings from one nine-crossing prefix, all with 18 boundary darts. Appended cancelling braid pairs increase the suffix from 5 to 37, 133, and 517 crossings without changing the knot. Matching-stream discovery is outside the primitive timings. These deliberately easy added crossings isolate reuse; they are not a hard family for complete recognition.

| $n$ | Euler adaptive | Euler eager | A/A   | Tait boundary | A/A   |
|-----|----------------|-------------|-------|---------------|-------|
| 5   | 0.873          | 0.906       | 1.006 | 1.152         | 1.004 |
| 37  | 2.589          | 2.622       | 1.002 | 5.829         | 0.993 |
| 133 | 7.273          | 8.332       | 1.001 | 19.476        | 1.008 |
| 517 | 15.828         | 23.640      | 0.997 | 53.905        | 1.002 |

Ratios are median paired direct/alternative times; values above one favor the alternative. A/A compares identical direct implementations.

The adaptive Euler policy achieves up to 15.8 times speedup, including its first four direct queries and summary setup. Eager setup reaches 23.6 times on the longest suffix. On the shortest suffix, the guard prevents preparation but the adaptive wrapper still costs about 15 percent in this isolated 200-query workload. This overhead is retained in the table.

The research determinant path reaches 53.9 times speedup, including coloring, fragment geometry, kernel construction, and partition-cache setup. However, all four of these common graphs have nine vertices, all nine are terminals, and interior nullity is zero. These gains therefore demonstrate boundary geometry and partition reuse, not amortization of a nontrivial interior elimination or a singular-kernel runtime advantage. Only 61 or 96 distinct cofactors are computed for each 200-matching stream; disconnected queries also avoid determinants. Independent kernel-certificate checking is performed in the separate audit, outside these timings.

| Input                 | Raw adaptive | A/A   | Full adaptive | A/A   |
|-----------------------|--------------|-------|---------------|-------|
| conway                | 0.997        | 1.003 | 1.001         | 0.958 |
| kinoshita_terasaka    | 0.996        | 1.000 | 0.990         | 0.989 |
| conway_sum_2          | 1.005        | 1.001 | 1.010         | 1.015 |
| conway_sum_8          | 1.008        | 1.003 | 0.987         | 1.000 |
| hard_unknot_8         | 0.997        | 0.999 | 0.982         | 1.002 |
| grid_scrambled_unknot | 1.010        | 1.005 | 0.990         | 0.999 |
| stress_braid5_36      | 0.961        | 0.985 | 0.994         | 0.997 |

Ratios are median paired direct/alternative times; values above one favor the alternative. A/A compares identical direct implementations.

For complete raw Euler scans, the baseline loads the earlier `euler_scan.py` from revision `f0179ab50`; the other dependencies are shared. Status, method, and observation stage agree in every paired run. None of these seven raw scans prepares a boundary summary. They show no consistent gain, and the stress case is about four percent slower. For complete recognition with the Euler backend requested, earlier filters decide every example, so no Euler evaluator runs. No whole-recognizer speedup is established. The raw samples, per-arm evidence, inputs, and source hashes are retained in `fast/results/boundary_reuse_20261008.json`. Future scheduling decisions must distinguish frequent cheap boundary queries from early-stopping scans; neither a favorable microbenchmark nor an adaptive switch supplies the missing universal width bound.

## 34 Signed Tait block products for actual observer workloads

The preceding boundary experiment isolates a useful primitive, but actual recognition decides when that primitive runs. We therefore traced the unmodified marked observer on every retained example before choosing its next optimization. Most examples ask for just the initial determinant. The determinant-one torus and grid examples ask for two additional distinct closures at a later stage. No example in this collection reaches the repeated-query threshold of four at any stage. This evidence favors reducing the cost of a single completed Tait graph before enabling terminal-kernel setup in the production observer.

The maintained shadow backend now factors larger signed Tait graphs at their articulation vertices. For example, the initial eightfold Conway connected sum has a  $41 \times 41$  full cofactor, but its block factors require matrices of size at most  $6 \times 6$ . This optimization applies directly to each actual completion, with the original signed phase retained. It makes no assumption that a partial complex splits as a knot connected sum.

### 34.1 Why the signed product identity is exact

Let  $G$  be the black Tait multigraph of a connected classical projection, with integer edge weights  $w_e \in \{1, -1\}$ . Its weighted tree sum is

$$\tau(G) = \sum_{T \text{ spanning tree of } G} \prod_{e \in T} w_e.$$

Loops never occur in a tree and may be discarded for this computation. For parallel edges between  $u$  and  $v$ , a tree can use at most one, so replacing them by the single weight  $w_{uv} = \sum_{e: u \leftrightarrow v} w_e$  preserves the sum. This is a polynomial identity and does not require positive weights. A zero combined weight may be deleted. If these deletions disconnect the graph, the sum is zero even though the original knot projection remains connected.

Suppose the resulting graph is connected, and let  $G_1, \dots, G_s$  be its edge blocks, with each bridge

treated as a two-vertex block. Then

$$\tau(G) = \prod_{j=1}^s \tau(G_j).$$

To prove this, first split at an articulation vertex into two graphs meeting only at that vertex. Restricting a spanning tree to either graph gives a spanning tree there: a disconnected restriction cannot be reconnected by leaving and returning through the same articulation vertex. Conversely, the union of two such trees is connected and has no cycle. The bijection preserves the product of edge weights. Induction on the block-cut tree gives the formula. Thus the identity also holds when factors are negative or vanish by cancellation. A bridge contributes its combined weight; the one-vertex graph has tree sum one.

For a nontrivial block, the integer matrix-tree theorem identifies its tree sum with any grounded Laplacian cofactor. The implementation uses the existing fraction-free Bareiss determinant with exact-division checks and row-level interruption points. It stops once a zero factor is found. No positive-definite elimination or nonsingular-interior premise is used.

The full specialization is still

$$J_{\text{raw}}(i) = (-i)^{B+v-1} \prod_j \tau(G_j),$$

where  $v$  is the black vertex count of the *full completed graph* and  $B$  is the original crossing sum. In particular, loops and cancelled parallel edges continue to contribute to the crossing phase. Multiplying independently normalized block Jones polynomials without these conventions would not justify the implementation.

This factorization is performed after the actual closure has been checked. It cannot automatically be applied to the fixed common graph before arbitrary terminal identifications: those identifications can create cycles crossing old articulation blocks. Terminal-kernel reuse and completed-graph block factorization therefore have different geometric premises.

## 34.2 Implementation, resources, and complexity

`fast/fastunknot/tait_blocks.py` combines parallel weights and uses an iterative low-link depth-first search to produce edge blocks. A stack of DFS frames replaces recursion, so a long chain does not impose Python's recursion-depth limit. When a child's low-link index is at least its parent's discovery index, the edge stack is popped through that tree edge to emit a block. A visit count detects disconnection of the nonzero graph.

The observer chooses the smaller checkerboard shading as before. For a cofactor of size at most sixteen it retains the direct dense path; larger cofactors use block decomposition before any dense matrix is allocated. This cutoff avoids adding graph bookkeeping to the smallest queries. It is an implementation policy, not a claim that sixteen is universally optimal. The observer exposes block, bridge, cancellation, and block-determinant counts. `max_unsplit_cofactor_size` records the full graph's size, while `max_cofactor_size` now records the largest matrix actually allocated. The preexisting `determinants` field continues to count connected closure specializations, including interrupted attempts.

Write  $b_j = |V(G_j)| - 1$ . Apart from integer bit costs, the determinant path uses

$$O\left(|V| + |E| + \sum_j b_j^3\right) \text{ work and } O\left(|V| + |E| + \max_j b_j^2\right) \text{ auxiliary space,}$$

with bridge factors evaluated directly. This replaces the full cofactor's cubic arithmetic and quadratic allocation by the corresponding block costs. The graph decomposition itself is linear.

Integer sizes still grow with the input; the displayed arithmetic count is not a unit-cost claim for arbitrarily large numbers. These polynomial savings do not bound the Khovanov scanner’s exponential state growth or establish a general subexponential recognition algorithm.

All graph traversal, edge aggregation, matrix allocation, and elimination paths charge the existing local work allowance and poll the global deadline. Only a complete four-residue result enters the closure cache. A partially multiplied block product is local state and is discarded on interruption. The implementation measured in this section disables observations on local exhaustion and continues the same exact capped scan. The later scheduler in Section 35 retains a query-capped Euler phase; a global deadline still yields `UNKNOWN`. The bounded-work policy counts combinatorial and arithmetic updates, not integer bit operations. The change remains inside the optional `shadow` backend; default recognition still uses `standard`.

### 34.3 Validation and workload evidence

Five new tests compare 245 random small weighted multigraphs with independent spanning-tree enumeration and 192 larger graphs with independent dense cofactors. They include parallel edges, loops, negative weights, deletion of cancelled edges, and connected graphs whose signed tree sum vanishes. A 5,000-vertex bridge chain verifies the iterative traversal and avoids any dense allocation. Four larger classical diagrams and their mirrors match independently reconstructed values at both one and  $i$ ; this checks that the block product retains the global phase.

An injected interruption after one completed block verifies that no partial result is cached. Resumption recomputes the exact value, global deadlines win even on a cache hit, and a real 2,000-unit optional allowance on the Conway sum exhausts locally while exact scanning still returns knotted. All 577 maintained tests pass after the final cutoff change. The test records are `data/tait-block-tests.txt` and `data/tait-block-integrated-tests.txt`.

`fast/benchmark_tait_blocks.py` keeps three scopes separate: the initial closure query, the complete raw marked scan, and complete recognition with the shadow backend requested. Each sample includes fresh diagram and observer setup. Two additional inputs concatenate 16 and 32 oriented Conway traversals to form larger connected sums; their validated PD codes are retained in the benchmark data. Seven paired rounds shuffle direct, identical control, and block-factor arms; imports and one warmup are excluded. The direct baseline loads `shadow_scan.py` from revision `2c7c8df3f`, sharing other dependencies. Actual observer traces are collected separately outside the timed calls. Inputs, source hashes, work counts, and raw samples accompany the resulting tables.

| Input                | Initial | A/A   | Raw    | A/A   | Full  | A/A   |
|----------------------|---------|-------|--------|-------|-------|-------|
| Conway               | 0.983   | 0.994 | 1.004  | 0.999 | 0.999 | 0.999 |
| Conway double three  | 0.999   | 0.996 | 1.007  | 1.005 | 0.998 | 0.998 |
| Conway pretzel       | 1.043   | 1.061 | 0.972  | 0.995 | 0.967 | 0.992 |
| Conway sum 2         | 0.995   | 1.000 | 1.015  | 0.998 | 0.995 | 0.993 |
| Conway sum 3         | 1.008   | 1.009 | 0.984  | 0.987 | 1.048 | 1.001 |
| Conway sum 8         | 2.152   | 0.999 | 1.186  | 1.009 | 1.030 | 1.027 |
| Figure eight         | 1.001   | 1.026 | 1.009  | 1.013 | 1.003 | 0.994 |
| Grid determinant one | 0.991   | 0.982 | 0.999  | 0.993 | 1.005 | 0.989 |
| Scrambled unknot     | 0.984   | 1.006 | 0.986  | 0.998 | 0.996 | 0.997 |
| Hard unknot 8        | 0.994   | 0.997 | 1.018  | 1.013 | 1.003 | 0.994 |
| Kinoshita–Terasaka   | 0.996   | 1.004 | 0.987  | 0.991 | 1.001 | 1.004 |
| Stress braid 36      | 1.004   | 1.000 | 0.998  | 0.986 | 0.986 | 0.992 |
| $T(3, 5)$            | 0.994   | 1.007 | 0.999  | 0.997 | 0.999 | 0.990 |
| Trefoil              | 0.989   | 0.996 | 0.982  | 0.995 | 0.992 | 1.007 |
| Crossingless unknot  | —       | —     | 0.986  | 1.002 | 0.983 | 1.003 |
| Braid unknot 40      | 1.302   | 0.989 | 1.082  | 1.009 | 1.000 | 1.002 |
| Conway sum 16        | 5.236   | 1.007 | 1.623  | 1.008 | 1.000 | 0.977 |
| Conway sum 32        | 17.418  | 0.995 | 21.547 | 1.024 | 1.001 | 1.000 |

Ratios are median paired direct/block times; values above one favor block evaluation. A/A compares identical direct implementations. Every scope includes fresh setup.

**Choosing the direct-path cutoff from measured overhead.** The first run, preserved in `fast/results/tait_blocks_threshold8_20261008.json`, used cutoff eight and the implementation at `07c02e10c`. It improved the eightfold Conway sum’s initial query by 2.10 times, but made the irreducible size-16 stress cofactor about nine percent slower. The size-11 twofold sum also lost about five percent. Raising the cutoff to sixteen removes those decomposition costs, at the price of foregoing the initial run’s modest size-16 threefold sum gain. The displayed final run uses this revised policy. Both source versions and all measurements are retained; the comparison does not replace the earlier unfavorable samples with the later ones.

**Single queries and complete raw scans.** The final initial-query speedups are 2.15, 5.24, and 17.42 times for 8, 16, and 32 Conway summands. The 40-crossing braid unknot improves by 1.30 times: all twenty effective blocks are bridges, so it allocates no dense determinant matrix. The size-16 irreducible stress case now keeps its direct path and has ratios near one. The corresponding complete raw scan gains are 1.19, 1.62, and 21.55 times for the three larger sums, and 1.08 times for the braid unknot.

The 32-fold sum illustrates why resource behavior matters. Its initial unrestricted direct query needs 1,398,785 work units, compared with 13,252 for block products. With the normal million-unit optional allowance, the old driver stops observing during this query and scans all 352 crossings, eventually certifying knottedness from closed rank. The new driver uses only 14,619 units including its later Euler query and obtains a component-Euler certificate after 11 crossings. Its largest determinant is  $5 \times 5$ , compared with the original  $160 \times 160$  cofactor. The raw benchmark therefore requires agreement of verdicts while recording method and stage differences explicitly. The large gain here includes preserving the opportunity for later inference, not just faster arithmetic. Both runs complete; no timeout is assigned a speedup ratio.

**Complete recognition and the remaining limit.** Earlier recognizer filters decide all 18 measured inputs before the shadow backend runs. Full-recognition ratios range from 0.967 to 1.048, with no demonstrated overall benefit from the new determinant path. For the 32-fold sum the existing factorization and Jones filter already return in about 15 milliseconds, ahead of

either raw scanner. The results support the optimization inside the optional observer; they do not justify changing the default backend or claiming a general asymptotic improvement in unknot recognition. Final inputs, traces, source hashes, and measurements are in `fast/results/tait_blocks_20261008.json`.

## 35 Preserving a cheap observer after marked-work exhaustion

Block products reduce the cost of many determinant queries, but a large irreducible cofactor can still exhaust its optional allowance. The earlier scheduler then stopped *all* observations, including cheap component-Euler checks. We now distinguish those resource limits. This is analogous to introsort at the scheduling level: an expensive optional method has a bounded attempt, then gives way to a cheaper exact method, then to the complete scanner when its own allowance is exhausted.

The marked backend has the one-way progression

marked residues and Euler  $\longrightarrow$  Euler only  $\longrightarrow$  unobserved capped scan.

The first transition occurs on marked-work exhaustion; shared query exhaustion can take either observer state directly to the unobserved scan. The crossing order, simplified relative complex, completed pivots, exact cached values, and global deadline survive every transition. There is no restart and no new query allowance.

### 35.1 Two limits with separate meanings

Let  $W$  be `shadow_max_work` and  $K$  be `euler_max_states`. While marked observations are active,  $W$  pays for their geometry, matrix allocation, arithmetic updates, and accompanying Euler queries, as before. The exception `ShadowWorkBudget`, a subtype of `EulerBudget`, identifies exhaustion of this particular allowance. The driver then stops requesting marked observations permanently.

If query capacity remains, later Euler queries use the direct closure connectivity evaluator with the *same* exact cache and state counter. Its new geometric work is separately measured by `euler_fallback_traversed_darts`; it is not charged to, or hidden by resetting, the already spent marked-work allowance. The corresponding `euler_fallback_evaluations` and `euler_fallback_states` fields distinguish all completed fallback requests from newly computed values. Cached values do not consume another query, but still check the global deadline. Result fields `shadow_exhausted` and `euler_exhausted`, together with `observer_mode`, distinguish which methods remain available.

This is an explicit resource-semantics change: setting  $W = 0$  disables marked work but permits the query-capped Euler reserve. Setting  $K = 0$  disables all new completion queries. Either limit may be explicitly unbounded through the existing API option `None`; an unbounded  $K$  does not give a finite fallback-query guarantee. No additional public switch is required.

If  $k$  distinct Euler values have already been computed, the fallback can compute at most  $K - k$  more. Each direct completion walk uses  $O(n)$  combinatorial work, so additional *new closure geometry* is  $O(n(K - k))$ . This bound does not include every observation cost: aggregating whole relative components still visits the current generators, and cache keys encode boundary matchings. Integer bit costs also depend on the suffix size and coefficient growth. In particular, this is not a polynomial bound on the complete Khovanov scan, whose number of live generators remains uncontrolled in general.

## 35.2 Why partially completed observations are safe to abandon

For a whole relative component, completion gives a genuine chain complex and its signed Euler characteristic  $\chi$ . Therefore  $|\chi| \leq \dim H$ . Summing these inequalities with the already certified nonnegative component multiplicities gives the existing Euler rank lower bound. Capping the final nonnegative bound preserves its decision meaning; the individual signed Euler values themselves remain exact integers. Thus an Euler certificate does not depend on finishing a marked determinant.

The implementation first completes the component-Euler bound for a stage. If marked work fails afterward, the incomplete marked bound is discarded. That stage’s already completed Euler bound remains valid; later stages may request further Euler values. If the marked allowance instead runs out while preparing the Euler part, its adapter switches before completing the outstanding query. Both paths use the same budget-checked evaluator and cache. A generic query-cap exception stops all further observations. Global `ScanLimit` exceptions are never converted into local declines: they propagate to `UNKNOWN` at the recognition boundary.

As before, an early lower bound can certify knottedness only. An unknot verdict requires the completed rank calculation. These invariants hold for every allowed transition regardless of how favorable the timing turns out to be. The policy provides bounded optional work and exact continuation; it supplies no general quasi-polynomial recognition theorem.

## 35.3 Transition tests and benchmark design

Five new tests exercise zero marked allowance, exhaustion during a block determinant, one versus two available queries, preserved cached Euler values, and a global deadline expiring during the transition. Two hundred budgeted scans of twenty small random diagrams and their mirrors agree with independent reduced homology. Zero-cap and low-cap paths are checked for both knotted and unknotted outcomes, and no partial observation returns an unknot verdict. Existing fallback expectations now reflect the retained Euler phase. All 582 maintained tests pass; logs are `data/shadow-fallback-tests.txt` and `data/shadow-fallback-integrated-tests.txt`.

`fast/benchmark_shadow_fallback.py` compares the earlier scheduler from revision `82fdfe15b` with the new one, sharing all other dependencies. It times seven shuffled paired rounds with an identical baseline control, fresh input and observer setup, and one excluded warmup. Every successful arm must agree on the verdict. Method and stage differences are retained as evidence, and any `UNKNOWN` suppresses all ratios for that case. The raw backend and full recognition are separate scopes.

Alongside all sixteen existing examples under default limits, the benchmark uses reduced-work and single-query controls. A larger default-limit example joins the Conway knot to  $T(3, 151)$ , giving a validated 313-crossing PD code. It is tested with the ordinary heuristic order and with a supplied order that processes the Conway summand first. The latter case is explicitly raw-only; its favorable ordering is not attributed to the default heuristic. Complete input codes, actual scan orders, budgets, source hashes, and raw samples are retained in `fast/results/shadow_fallback_20261008.json`.

| Input/order              | $W$    | $K$  | Old stop | New stop | Speedup | A/A   |
|--------------------------|--------|------|----------|----------|---------|-------|
| Conway sum 8             | $10^6$ | 4096 | 11       | 11       | 0.997   | 0.996 |
| Conway sum 8             | 0      | 4096 | 88       | 11       | 5.530   | 1.028 |
| Conway sum 8             | 2800   | 4096 | 88       | 11       | 5.154   | 1.008 |
| Conway sum 8             | 2800   | 1    | 88       | 88       | 1.010   | 1.003 |
| $T(3, 5)$                | 0      | 4096 | 10       | 10       | 0.982   | 0.997 |
| Hard unknot 8            | 0      | 4096 | 8        | 8        | 0.984   | 0.998 |
| 313 crossings, heuristic | $10^6$ | 4096 | 313      | 302      | 1.016   | 1.009 |
| 313 crossings, supplied  | $10^6$ | 4096 | 313      | 11       | 4.786   | 1.007 |

Ratios are median paired baseline/fallback times for complete raw scans. Stop columns give the processed crossing count. A/A compares identical baselines.

**Retained query capacity produces the gain.** On the eightfold Conway sum,  $W = 0$  improves the raw scan by 5.53 times, and  $W = 2800$  by 5.15 times. Both schedulers respect the same marked-work cap. In the latter case each spends all 2,800 units and has computed one Euler value; the new scheduler uses one further query to obtain the component-Euler certificate at crossing 11. The baseline completes all 88 crossings. With  $K = 1$ , neither can compute that further value and both finish the full scan, with a timing ratio of 1.01. This negative control checks that the improvement does not come from silently resetting the query budget. With default marked work, this example never switches and its ratio is 0.997.

The retained reserve can also cost time without strengthening a bound. With  $W = 0$ ,  $T(3, 5)$  uses three fallback values but still requires closed rank, with ratio 0.982. The hard eight-crossing unknot uses one value and still finishes the entire scan, with ratio 0.984. These small regressions are part of the comparison, not excluded as inconclusive examples.

**Ordering determines when the reserve can help.** For the 313-crossing connected sum, both policies stop marked work at 999,932 units under the default million-unit allowance. The heuristic order first processes the torus part. Fallback gives a certificate at crossing 302 rather than 313, but the measured speedup is only 1.016, with an A/A ratio of 1.009. This does not demonstrate a substantial timing gain. With the supplied order, fallback needs only one new query and certifies knottedness at crossing 11. Its measured raw-scan speedup is 4.79 times, with an A/A ratio of 1.007. The respective median times are about 170 and 821 milliseconds. This is a scheduler gain for the supplied order, not evidence that the ordinary order selector discovers it.

All 45 case/scope comparisons complete and agree on verdict. None of the sixteen default-limit examples switches modes; their raw ratios range from 0.971 to 1.031. Earlier recognition filters decide every full-recognition case before this observer runs, giving ratios from 0.982 to 1.019 and no demonstrated full-recognizer gain. In particular, the larger connected sum is already detected by the Rasmussen interval filter. The new policy therefore improves robustness of an optional backend under its resource limits while leaving the universal complexity question unresolved.

## 36 Certified square-root frontiers and a subexponential scalar filter

The previous integration reduced the cost of observing a fixed scan. We now address its order. The maintained Python implementation constructs a crossing order of width  $O(\sqrt{n})$  for every closed classical projection with  $n$  crossings. It supplies a separately checkable hierarchy and compares the result with the existing greedy order, retaining the smaller measured frontier. Combined with the exact fixed-color Potts evaluator, this gives a general  $\text{poly}(n)2^{O(\sqrt{n})}$  bit bound for *one scalar Jones specialization*, with optional resource caps disabled. It does not give that bound for unknot recognition: equality with the unknot’s scalar is inconclusive.

This is an implementation of classical planar-separator ideas [8], not a new planar-separator theorem. The contribution here is a constructive order, a graph-only certificate verifier, its integration with the existing exact scalar backend, and explicit accounting of which recognition bottleneck remains. Nothing in this section requires connected prefixes, a common boundary disk, or a bound on surviving Khovanov objects.

### 36.1 A bounded-radius middle band

Represent the projection by a plane multigraph  $G$ : crossings are vertices, diagram arcs are edges, and the PD rows specify the cyclic orders of half-edges. The degree is four, with loops counted twice. The constructor checks that each edge label occurs twice and that the rotation system is spherical on every connected component. Loops may then be deleted for separator construction: they neither connect distinct components nor cross a cut of crossing vertices. Parallel edges remain in the embedded map. Disconnected projections are handled component by component.

Consider one connected recursive subproblem with  $s$  vertices. Take breadth first search levels  $V_0, \dots, V_d$  from a root, and put  $r = \lceil \sqrt{s} \rceil$ . Let  $a$  be the first level for which  $\sum_{i \leq a} |V_i| \geq s/2$ . If  $a \geq r$ , choose a smallest level in  $[a - r, a - 1]$  as the lower boundary  $\ell$ ; otherwise use the empty virtual level  $\ell = -1$ . If  $a + r \leq d$ , choose a smallest level in  $[a + 1, a + r]$  as the upper boundary  $u$ ; otherwise put  $u = d + 1$ . Each real chosen level has size at most  $s/r \leq r$ .

The vertices strictly below  $\ell$  and strictly above  $u$  number at most  $s/2$  on either side. An edge changes its BFS distance by at most one, so removing the two boundary levels separates these outside vertices from the middle band. Retain levels below  $u$ , and, when  $\ell \geq 0$ , contract the connected inner BFS tree on levels at most  $\ell$  to the root. Delete the resulting loops. The resulting plane multigraph  $H$  has radius at most  $2r$ . Give weight one to each original middle-band vertex and weight zero to the contracted root. The total weight is at most  $s$ .

The ribbon-map implementation contracts a tree by following its boundary through the removed half-edges. It does not repeatedly rebuild the graph after individual edge contractions. This matters for long bands and large inner balls. An independent check after construction verifies the actual separator size and every remaining component's size in the original graph.

### 36.2 Three short paths from a dual centroid

For each face of  $H$ , add a center and a spoke to every corner occurrence. Keep the old edges. A bridge or a repeated vertex on a face walk therefore causes repeated spokes, not an assumption that the face boundary is a simple cycle. The resulting map is triangulated; every new center has distance at most  $2r + 1$  from the root. The one-vertex case is handled directly.

Choose a BFS spanning tree  $T$  of this triangulation. The dual edges of primal edges outside  $T$  form a dual spanning tree  $T^*$ . Assign each weighted original vertex to one incident triangular face. Choose a weighted centroid face  $f$  of  $T^*$ : every component of  $T^* - f$  has at most half the total assigned weight. Remove the union  $P$  of the three primal tree paths from the root to the vertices of  $f$ .

To see balance, consider a dual edge incident to  $f$ . Its primal edge and the tree path between its endpoints bound a fundamental cycle. The cycle's vertices lie in  $P$ , since both endpoints belong to  $f$ . It separates the corresponding branch of the dual cotree from the other branches. Consequently each surviving primal component has its assigned faces in one branch of  $T^* - f$ . Any weight assigned to  $f$  itself belongs to a vertex of  $f$ , hence to  $P$ . Every remaining component therefore has at most half the middle-band weight. This argument permits arbitrary nonnegative face weights; it does not require each individual assigned weight to be small.

Return only the *original middle-band vertices* of  $P$ , together with the two boundary levels. Discarding artificial centers from this separator is safe: paths in the original graph use old vertices and old edges only. The old edges were retained during triangulation, so any original path avoiding the returned vertices is also a path in the triangulation avoiding  $P$ . The contracted root represents vertices already separated from the middle by the lower level. Thus the original graph has a separator  $S$  with

$$|S| \leq 2r + 3(2r + 2) = 8\lceil \sqrt{s} \rceil + 6, \quad |C| \leq s/2 \quad (C \text{ a component of } G - S).$$

The constants are deliberately loose. Small separators on practical inputs matter more than optimizing this universal numerical bound.

### 36.3 From the hierarchy to a certified order

Recursively order each component of  $G - S$ , then put  $S$  last. Stop at a fixed leaf size, eight by default; the API permits integer cutoffs from one through 64. Process distinct initial connected components consecutively. At a prefix cut, any crossing arc is incident to a separator on the current recursion path, or to a vertex of the current leaf. Completed sibling components have no edges to unvisited siblings. Since the original degree is four, the frontier is at most

$$4 \left( \sum_{S \text{ on the path}} |S| + |\text{leaf}| \right).$$

Sizes halve along a path. The square-root terms form a convergent geometric sum, while the additive constants contribute  $O(\log n)$ , giving width  $O(\sqrt{n})$ . This holds for every prefix, including prefixes inside a separator's own vertex list.

The JSON certificate records the input digest, leaf cutoff, separator and child lists, flattened order, actual maximum and total frontier, and the recursively computed bound

$$b(\text{leaf}) = 4|\text{leaf}|, \quad b(\text{node}) = 4|S| + \max_C b(C).$$

Each crossing occurs once in the hierarchy. With normalized crossing indices the hierarchy uses  $O(n \log n)$  bits; arbitrary input-label bit length also counts toward input processing. Construction and verification take polynomial time. The verifier rebuilds adjacency directly from edge owners and checks coverage, disjointness, the exact remaining components, half-balance, separator sizes, order, actual profile and the bound. It calls no planar-map or separator constructor. Thus even a mistake in the embedded construction cannot silently publish an incorrect certified frontier bound.

The practical wrapper also computes or accepts a candidate order. It chooses the lexicographically smaller pair (maximum frontier, total frontier) between that order and the separator order. If greedy wins, the separator witness is retained: explicit profile comparison proves that the selected order has the same asymptotic upper bound. This is a bounded-choice ordering policy, not a progress-based restart of a scan. It complements the earlier adaptive elimination and observer fallbacks.

### 36.4 Exactly which subexponential computation follows

For a fixed checkerboard shading, let  $f_i$  count Tait vertices incident to both processed and unprocessed crossings, and let  $w_i$  be the crossing frontier. The corner argument in Section 21 gives  $2f_i \leq w_i$ , including repeated face walks. A fixed  $q \geq 5$  Potts query uses at most  $q^{f_i+2}$  temporary assignments and exact pairs in

$$R_q = \mathbb{Z}[x]/(x^2 - (q-2)x + 1).$$

Coefficient bit lengths are  $O(n \log q)$ ; arithmetic and boundary-key work add polynomial factors. Substituting the certified width yields

$$T_{\text{scalar}}(n) = \text{poly}(n) 2^{O(\sqrt{n})} \quad \text{for fixed } q.$$

This is a bit-complexity statement, not a count of unit-cost operations on unbounded integers. For the existing modular matching filter, allowing all boundary pairings gives the weaker but still subexponential bound  $\text{poly}(n) 2^{O(\sqrt{n} \log(n+1))}$  on the same order. These are universal upper bounds for scalar evaluation, not claims that our implementation or constants are optimal.

The optional `potts-separator` Jones backend constructs the witness and then calls the existing exact evaluator. With the Python API, setting both `max_states=None` and `max_transitions=None` disables the local caps for the theoretical complete query. Production defaults retain the 4096-state and 200 000-transition caps. Exhaustion is inconclusive, and the recognition deadline remains global. A zero local budget declines before constructing a hierarchy. Once a complete certificate has been published, scalar exhaustion retains its order for the homology fallback. If subsequent Reidemeister reductions change crossing indices, the witness is rebuilt on the reduced diagram before supplying an order to a window query or the scanner. A global interruption during preparation publishes no partial certificate.

Three obstacles prevent promoting this to a general recognition theorem. First, agreement at a scalar specialization is not an unknot certificate. Second, a frontier bound alone does not bound the number of surviving relative Khovanov objects or intermediate matrix size. Third, this hierarchy does not certify common-disk prefixes, so the stronger conditional bounds of the disk-transfer and nice-window backends cannot simply be substituted. The descending grid family already described in this report has raw width  $\Omega(\sqrt{n})$  for every order. That obstructs a universal polylogarithmic-width strategy on a fixed diagram, but it is no lower bound for recognition: a descending certificate immediately handles the family.

### 36.5 Validation and measurements

All 589 integrated tests pass. The new tests compare acceptance of 500 random ribbon maps with an independent spherical-map implementation, verify certificate corruption is rejected, and exercise loops, disconnected projections, long bands and grids. Sixteen small knot diagrams are checked against independent cube homology using actual separator orders; both Potts shadings agree with the previous exact evaluator. Resource tests distinguish partial preparation from a completed certificate followed by scalar exhaustion, and check that a changed diagram receives a fresh certificate.

The separate reproducible driver `fast/audit_separator_orders.py` audits 114 inputs, including 100 randomly generated, crossing-permuted and edge-relabelled braid closures. Its independent hierarchy checker uses union-find over edge-owner pairs rather than the production component traversal. It independently counts cut edges and checks balance, coverage, components and the stored bound. The result archive retains all inputs, certificates, source hashes, five shuffled ordering rounds and three shuffled kernel rounds, with warm-ups excluded. Table 1 separates measured frontiers from preparation costs.

| Family                | $n$  | Greedy | Separator | Selected | Greedy ms | Bounded ms |
|-----------------------|------|--------|-----------|----------|-----------|------------|
| Grid $8 \times 8$     | 64   | 10     | 46        | 10       | 0.626     | 4.524      |
| Grid $32 \times 32$   | 1024 | 34     | 214       | 34       | 6.082     | 142.782    |
| Tree medial, height 5 | 62   | 16     | 12        | 12       | 1.118     | 4.492      |
| Tree medial, height 7 | 254  | 64     | 14        | 14       | 5.218     | 26.624     |
| Tree medial, height 9 | 1022 | 256    | 18        | 18       | 23.653    | 140.075    |

Table 1: Widths and median order-preparation times. Bounded preparation includes the separator witness, its verification and the greedy comparison; these are additional costs, not speedups.

The tree-medial family consists of medial diagrams of embedded complete binary trees, with one crossing per tree edge. These are simple nugatory unknot diagrams used to stress the greedy order; they are not hard recognition instances. In isolated modular matching queries, height five falls from a median 47.665 ms to 1.941 ms (24.6 times), excluding order preparation. At heights six and seven, all greedy runs exhaust the state cap while the selected orders finish in 4.126 and 8.788 ms. Those censored runs have no completion-time speedup ratio. All completed scalar answers are inconclusive for recognition.

The exact Potts queries on these same trees differ little between orders: the advantageous checkerboard shading already has trivial frontier. At height seven the medians are 2.572 and 2.585 ms, far below the extra order preparation cost. The grids also retain their better greedy order. These results support keeping the new option explicit: the certified asymptotic bound and a reduction in matching-filter work do not establish a general practical speedup, a full-recognition speedup, or a smaller homology complex.

## 37 Deferring separator work until the scalar query needs it

The separator backend supplies a useful universal bound, but eagerly building its certificate slows many easy scalar queries. The new optional `potts-adaptive` backend starts with the ordinary order and watches *actual completed transitions*. It postpones separator construction until that work becomes large, while retaining the preceding section’s subexponential fixed-color scalar bound. This is the introsort principle applied to one exact invariant query: limit speculative work, then use a method with a proved bound. It is not yet an analogous worst-case bound for the complete unknot recognizer.

### 37.1 The policy and its persistent state

For an  $n$ -crossing input, let the initial work threshold be  $B = 64n$  and the short-tail allowance be  $r_0 = 2\lceil \log_2(n+1) \rceil$ . The evaluator begins on a supplied order, or the existing greedy order when none is supplied. At each completed crossing before the last, it observes the cumulative transition count. The actions are as follows.

1. Below  $B$ , continue normally. A completed scalar answer needs no separator certificate.
2. On first reaching  $B$ , if at most  $r_0$  crossings remain, finish the current table. This enters `polynomial-tail` mode; the argument below supplies a bound without requiring a width certificate.
3. Otherwise construct and independently verify a separator hierarchy. Compare its order with the better of the current order and a freshly computed greedy order, using (maximum frontier, total frontier).
4. If the selected order is unchanged, retain the *same* Potts table and continue. No processed crossing or transition is repeated. This is `certified-continue` mode.
5. A changed order enters `certified-restart` mode. Discard the unfinished scalar table and restart once. The old exception handler is exited before allocating the replacement table, releasing the abandoned evaluator’s traceback and local references.

A local state-cap failure can also trigger the preparation when it has not already happened and transition allowance remains. If the selected order is unchanged, repeating the query cannot repair that failure: publish the completed order certificate and decline immediately. A spent transition allowance never triggers preparation. There is at most one separator preparation and at most one restart. A fixed state cap may still interrupt the polynomial tail, in which case the same rules apply.

The `ordering_policy` evidence records the mode, threshold, short-tail allowance, preparation/restart counts, abandoned crossings and transitions, and total spent transitions. `PottsLimit` carries work counters only; it never exposes a partial partition function as a completed value. The public exact evaluator’s existing complete-result behavior is unchanged.

### 37.2 A single transition budget, including discarded work

Suppose the user supplies transition allowance  $T$ , and the first attempt spends  $a$  transitions. A restarted evaluator receives exactly  $T - a$ . If it spends another  $b$ , the returned count and any

exhaustion exception report  $a + b \leq T$ . There is no fresh allowance for the second implementation. State caps keep their existing meaning as limits on represented table keys; the reported peak is the larger attempt peak, not a byte-memory estimate. Other scalar geometry and coefficient statistics describe the final attempt.

Order preparation is charged to the shared global deadline, not to the scalar-transition counter. The deadline callback propagates through the policy and verifier. A preparation interrupted globally publishes no partial certificate. After a complete witness is available, a later local scalar failure retains it for the recognition fallback. Subsequent crossing-changing Reidemeister simplification rebuilds it on the reduced diagram, as in the preceding section. When the scalar query finishes in ordinary or polynomial-tail mode, no separator witness has been computed; those modes make no width promise for the later homology scan.

### 37.3 Why the deferral preserves the asymptotic scalar bound

Fix the number of colors  $q \geq 5$ . One crossing introduces at most two Tait vertices. From one stored spin-equality state there are at most  $q^2$  extensions, because every canonical extension represents at least one labeled choice for those two vertices. Aggregation and cancellation only decrease the number of states. Moreover, after at least one crossing the number of stored states is at most the number of transitions spent so far.

Immediately before the first completed stage reaching  $B$ , at most  $B + 1$  states are available. The crossing that first passes the threshold therefore takes at most  $(B + 1)q^2$  further transitions. Stopping only at a crossing boundary causes polynomial, rather than unbounded, overshoot. The same observation bounds a final crossing that completes the entire query before the policy is invoked. Canonical-key work and exact coefficient arithmetic have polynomial bit cost at fixed  $q$ .

If the policy enters a short tail of  $r \leq r_0$  crossings, the number of additional transitions is bounded by

$$O\left((B + 1)q^2 \sum_{j=1}^r q^{2j}\right) = O\left(nq^{2r+O(1)}\right) = n^{O(1)} \quad \text{for fixed } q.$$

The degree of this polynomial can be large. This is an asymptotic safeguard, not a promise that every permitted tail is practically cheap. Existing state, transition and deadline limits remain active. If a state-cap failure interrupts the tail and requests a new order, the abandoned tail work is still bounded by this polynomial.

In every other uncapped case the policy eventually obtains a certified  $O(\sqrt{n})$ -frontier order. Continuing an unchanged order or restarting on the selected order costs  $\text{poly}(n)2^{O(\sqrt{n})}$  bit operations. A continuing run does not repeat its earlier work; a restarted run adds only the polynomially bounded first attempt. Construction of the hierarchy is polynomial. Hence the complete adaptive *scalar* query has the same general subexponential upper bound as eager separator evaluation.

This theorem does not assert competitiveness with the fastest order on each input. Under a finite allowance, discarded transitions may prevent completion where some single-order query would have completed. Frontier improvement also does not force fewer spin states after exact cancellations. These are reasons to expose the option and its policy evidence while retaining the existing default, not reasons to reset the budget silently.

### 37.4 Independent checks and measured scope

The six new test cases cover actual measured-work restarts, state-cap restarts, unchanged-order continuation, a completed short tail, exact shared budget boundaries, globally interrupted preparation, zero budgets, invalid options, CLI routing and the complete recognition fallback.

Twenty small diagrams, three color counts and both shadings give 120 comparisons with an independent whole-cube Laurent-Jones oracle. That oracle uses neither Tait graphs nor production ring arithmetic. Its cases exercise policy switches as well as ordinary completion. The five-color blind weaving example still falls through to Khovanov recognition rather than receiving an unknot verdict.

The benchmark uses seven shuffled rounds after an excluded warm-up, fresh diagram construction and all order preparation inside every timer. It separates 15 scalar scopes from 11 full-recognition scopes. Ordinary exact Potts has an identical A/A control; eager separator, final adaptive and the initial adaptive policy are compared on the same scalar queries. Every completed scalar comparison checks cross-multiplied normalized values, because changing shading changes both the partition function and its normalizing factor. Limits remain censored outcomes.

The initial policy lacked the short-tail rule. Its source snapshot matches the hash in the retained initial measurements, and the final driver also reruns it as a paired control. On the  $10 \times 10$  descending grid, the threshold is crossed after crossing 90, at 6555 transitions; the whole query requires 7109. With ten crossings left and  $r_0 = 14$ , the final policy avoids the otherwise unnecessary certificate. Its median time is 21.457 ms versus 30.556 ms for the initial policy, a median paired improvement of 1.415 times. On a shuffled 16-crossing grid, however, the tail rule forgoes a useful restart and is slower than the initial policy (paired ratio 0.913). Both results are retained; the rule is not claimed to optimize every instance.

| Scalar input      | Ordinary | A/A     | Eager   | Initial | Adaptive |
|-------------------|----------|---------|---------|---------|----------|
| Conway            | 0.499    | 0.514   | 1.134   | 0.534   | 0.547    |
| Tree medial 254   | 9.617    | 9.636   | 34.412  | 9.705   | 9.703    |
| Grid 16, shuffled | 5.875    | 5.670   | 1.390   | 5.236   | 5.692    |
| Grid 64, shuffled | limit    | limit   | limit   | 23.877  | 23.893   |
| Grid 100          | 21.241   | 21.129  | 29.452  | 30.556  | 21.457   |
| Grid 144          | 135.286  | 135.237 | 146.759 | 149.998 | 153.190  |

Median milliseconds including all setup. A/A repeats ordinary exact Potts. A limit is a censored query, not a completed scalar value.

On the 254-crossing tree-medial diagram, ordinary and adaptive scalar times are 9.617 and 9.703 ms, versus 34.412 ms for eager separator preparation. On the shuffled 36-, 64- and 100-crossing grids, both adaptive policies finish within the default budgets while ordinary and eager supplied-order queries hit their state caps. The adaptive fallback's fresh greedy comparison is part of this benefit; the eager backend compares only the supplied order with its separator order. These capped runs have no completion-time speedup ratio. On the ordinary 144-crossing grid, the final policy continues the same table after certification and takes 153.190 ms versus 135.286 ms for ordinary exact Potts: the guarantee still has a preparation cost.

Full recognition on the measured inputs gains no consistent advantage over ordinary exact Potts. Earlier certificates bypass the scalar query on many cases. Avoiding eager preparation helps where it would have been used, but this is not a general improvement over the historical matching default. The new backend remains optional.

### 37.5 A relevant geometric result, with a different scope

The research refresh found no new delivered unknot report in the drop zone. Lackenby's revised *Some fast algorithms for curves in surfaces* [7] supplies a useful related primitive: Theorem 1.1 computes geometric intersection numbers of normal 1-manifolds in time polynomial in triangulation size and the logarithms of their weights. Theorem 1.2 gives a corresponding isotopy test. The normalization theorem also depends on the input's simplification number; that extra parameter must not be dropped from a complexity claim. These results strengthen the case that very large

geometric objects can admit efficient compressed operations. They neither bound the depth of an accelerated 3-manifold hierarchy nor the number of surviving chain objects in our implementation. We therefore record them as candidate geometric primitives, without treating them as a missing recognition theorem or adding an unused surface API to the scalar scanner.

## 38 A collision-free Jones query and full polynomial recovery

The fixed-color queries above can certify a nontrivial Jones value, but equality can hide a nonconstant polynomial. The optional `potts-faithful` backend removes this specialization ambiguity. It chooses an input-dependent quadratic integer and either decides whether the *whole* normalized Jones polynomial is one, or recovers that polynomial. With resource caps disabled, both queries have the general bit bound

$$\text{poly}(n) 2^{O(\sqrt{n} \log(n+1))}.$$

This is subexponential in the crossing count. It is a bound for Jones computation, not for complete unknot recognition. The implementation does not assume that the Jones polynomial detects the unknot: a completed identity result still proceeds to an independent recognition certificate.

This section combines elementary coefficient bounds and integer encoding with the preceding maintained separator and Potts algorithms. It makes no priority claim for subexponential Jones computation. In particular, efficient Jones computation under graph-width restrictions predates this implementation; see the literature review in `reports/17/research/literature_landscape.md`.

### 38.1 Uniform coefficient and degree bounds

Write  $V_D(t)$  for the normalized Jones polynomial of an  $n$ -crossing knot, with  $V_\circ = 1$ , and put  $W(x) = V_D(x^{-1})$  for  $x = A^4$ . The bracket state sum has  $2^n$  states. Each state has at most  $n + 1$  circles: changing one smoothing changes the circle count by one, and smoothing a connected  $n$ -crossing projection cannot produce more than  $n + 1$  components. The normalized circle factor is  $(-A^2 - A^{-2})^{c-1}$ , whose coefficient  $\ell^1$  norm is  $2^{c-1}$ . Writhe normalization only changes signs and shifts degrees. Thus

$$\|W\|_1 = \|V_D\|_1 \leq 2^n 2^n = 4^n =: C.$$

The smoothing monomial has  $A$ -exponent of absolute value at most  $n$ ; circle factors contribute at most  $2n$ ; writhe contributes at most  $3n$ . Every resulting exponent is divisible by four for a knot, giving  $|\deg_x| \leq 3n/2$ . We use the simpler integral bound  $D = 2n$ . The crossing-free knot is handled separately with polynomial one.

### 38.2 Choosing a specialization that cannot hide nontriviality

Choose

$$M = 2^{4n+2} = 4 \cdot 16^n, \quad q = M + 2, \quad x^2 - Mx + 1 = 0,$$

and consider the larger real root  $x$ . Since  $M \geq 4$ , this root satisfies  $x > M - 1 > C$ . Suppose that  $W - 1$  is a nonzero Laurent polynomial. Multiplying by a monomial produces an ordinary integer polynomial  $P$  with nonzero leading coefficient of absolute value at least one. Its norm is at most  $C + 1$ , so the sum of absolute values of all its other coefficients is at most  $C$ . At a real argument  $x > C$ , the leading term has absolute value greater than the sum of the lower terms. Thus  $P(x) \neq 0$ , and hence  $W(x) \neq 1$ .

Consequently equality of the exact quadratic-ring pairs for the partition function  $Z$  and the unknot normalization  $U$  is equivalent to  $V_D(t) = 1$  as a polynomial. The normalization is nonzero: its formula is a signed monomial times a power of  $x + 1$ . No floating-point approximation or

probabilistic identity test is used. The same leading-term argument applied to a difference of two bounded polynomials proves injectivity, because  $M - 1 > 2C$ .

The root argument alone would permit a smaller base for identity testing. We deliberately use a single larger base for both identity testing and the coefficient recovery below, keeping their result semantics consistent.

### 38.3 Recovering all coefficients from two integers

The existing evaluator supplies  $Z$  and  $U$  as pairs representing elements of  $\mathbb{Z}[x]/(x^2 - Mx + 1)$ . First recover the normalized pair exactly. If  $U = c + dx$ , its conjugate is  $(c + Md) - dx$  and its norm is

$$N(U) = c^2 + Mcd + d^2.$$

Multiply  $Z$  by this conjugate and divide both coordinates by  $N(U)$ . The divisions have zero remainder because  $Z/U = W(x)$  is a Laurent polynomial with integer coefficients and  $x^{-1} = M - x$  is integral. The code checks the remainders, rather than rounding them.

To understand the coordinate encoding, introduce formal polynomials  $U_{-1}(M) = 0$ ,  $U_0(M) = 1$ ,  $U_1(M) = M$ , and  $U_j(M) = MU_{j-1}(M) - U_{j-2}(M)$ . For  $k \geq 1$ ,

$$x^k = -U_{k-2}(M) + U_{k-1}(M)x, \quad x^{-k} = U_k(M) - U_{k-1}(M)x.$$

The first formula for  $k = 1$  uses  $U_{-1} = 0$ . Each  $U_j$  has coefficient norm at most  $2^j$ , by induction on the recurrence. Therefore reduction of  $W$  yields coordinate polynomials  $a(M), b(M)$  of degree at most  $D$ , each with every coefficient bounded by

$$C2^D \leq 4^n 2^{2n} = 16^n = M/4.$$

The computed integers  $a(M), b(M)$  consequently determine these coordinate polynomials uniquely by balanced base- $M$  expansion. Repeatedly take the residue modulo  $M$ , subtract  $M$  when it is at least  $M/2$ , and divide the remaining difference by  $M$ . This handles negative coordinates as well as positive ones. Actual coefficients lie strictly inside the balanced digit interval; there is no carry ambiguity.

Finally substitute the formal identity  $M = x + x^{-1}$ . A decoded term  $cM^j x^e$ , where  $e \in \{0, 1\}$ , contributes

$$c \sum_{k=0}^j \binom{j}{k} x^{j-2k+e}.$$

Aggregate equal exponents, remove zero coefficients, and negate the exponents to return the standard variable  $t = A^{-4}$ . The implementation independently checks the final degree and coefficient bounds, and consistency with the pair-equality identity result. These checks detect inconsistent input to the decoder; they do not replace the state-sum correctness argument. There are  $O(n)$  output coefficients of  $O(n)$  bits each. Balanced decoding, exact normalization division and binomial expansion take polynomial bit time.

### 38.4 Why exponentially many colors do not cause exponential work

Applying the fixed- $q$  bound  $q^f$  directly would lose the desired result. The implementation instead stores *canonical equality partitions* of the  $f$  active Tait vertices. There are at most the Bell number  $B_f$  such partitions, independent of  $q$ . Its extension loop considers each existing color class and at most one new class, weighted by the number of unused colors  $q - k$ . It never enumerates  $q$  colors.

The certified crossing order has frontier  $w = O(\sqrt{n})$ , hence at most  $f \leq w/2$  active faces of either chosen checkerboard shade. As  $B_f \leq \max(1, f^f)$ , the number of represented states is

$2^{O(\sqrt{n} \log(n+1))}$ . Each crossing introduces at most two vertices; the number of extensions from one state is at most  $(n+3)^2$ . The original scalar coefficient bound is  $O(n \log q)$  bits, now  $O(n^2)$ . Key manipulation, multiplicities, arithmetic and the number of stages therefore contribute only polynomial factors.

The adaptive preliminary scan needs separate accounting because  $q$  is no longer fixed. Before its first threshold crossing it has at most  $64n+1$  states. One completed-stage overshoot costs at most  $(64n+1)(n+3)^2$  transitions. If it takes the short-tail option with at most  $2\lceil \log_2(n+1) \rceil$  crossings remaining, a conservative work bound is

$$\text{poly}(n)(n+3)^{O(\log(n+1))} = 2^{O(\log^2(n+1))}.$$

Thus the mode called **polynomial-tail** in the shared fixed-color policy has only a *quasi-polynomial* bound here. That cost is still dominated by  $2^{O(\sqrt{n} \log(n+1))}$ . All other uncapped runs obtain the separator certificate and continue or restart once. Discarded work and a possible state-cap interruption obey the same bounds. The polynomial construction and verification costs do not change the final bound.

### 38.5 Interface, evidence and resource semantics

`faithful_potts_exact` in `fastunknot/faithful_jones.py` returns the existing exact query fields together with `polynomial_identity`: the algorithm version, crossing count, whether the polynomial is one, and the exponents defining the coefficient and encoding bounds. Its optional `include_polynomial=True` returns sorted Laurent coefficients in `jones_polynomial`, with variable `t` and signed hexadecimal coefficient strings. Raw quadratic pairs and the raw color count remain Python integers.

The command `python -B -m fastunknot jones FILE -backend potts-faithful` requests full recovery. Recognition through `-jones-backend potts-faithful` requests identity testing only. The color count is derived from the input; `-potts-colors` does not override it. The Python API and recognition defaults remain 4096 states and 200000 transitions; the standalone `jones` command has no local caps unless supplied. An exhausted allowance remains inconclusive. The uncapped complexity statement requires `max_states=None` and `max_transitions=None` and no external deadline. A restarted evaluation shares the original transition allowance. Global checks run during decoding too. No incomplete identity or polynomial is published if the query fails before completion.

Exact witnesses encode  $q$  in `q_hex` above 256 bits and retain the existing integer `q` field for smaller values. Large ring descriptions refer symbolically to  $q$ . This avoids Python's decimal-conversion limit without changing process-global settings. The factorized exact backend uses the same ring description and witness conversion.

Identity equality has two deliberately separate consequences: it settles the algebraic question  $V_D = 1$ , but yields **INCONCLUSIVE** for the Jones recognition filter. The complete recognizer retains this stronger evidence and continues to its existing exact fallback. Neither this backend nor the adaptive scheduler establishes a general subexponential recognition bound.

### 38.6 Validation and measurement

The initial integration passed all 602 tests in 111.769 seconds. That run is preserved in `data/faithful-jones-integrated-tests.txt`.

Seven new maintained tests compare 156 full polynomials against the independent whole-cube Laurent oracle: both checkerboard shades and both mirrors of 35 random diagrams, three named fixtures, and the crossing-free knot. Another 187 synthetic bounded Laurent polynomials check recovery at extreme signed degrees and coefficient sizes, including arbitrary exact normalization.

Tests exercise a real certified restart, the precise shared transition limit, the known  $q = 5$  weaving collision, recognition fallback after polynomial identity, cancellation during decoding, zero budgets, CLI output, and 20001-bit color counts through both exact witness backends.

`benchmark_faithful_jones.py` measures fixed  $q = 6$  twice as an A/A control, faithful identity, and full recovery in seven shuffled rounds after an excluded warmup. Every timing includes construction of a fresh diagram and all order preparation. Small inputs have independent cube checks; completed larger recovered polynomials are checked by specialization back to the fixed- $q$  result. These arms deliver different guarantees, so their timings describe the price of stronger information rather than speedups for an identical task. Capped results are retained as such. The backend remains optional, and these invariant timings do not claim faster full recognition.

| Input             | Fixed $q = 6$ | A/A    | Identity | Full polynomial |
|-------------------|---------------|--------|----------|-----------------|
| Conway            | 0.563         | 0.568  | 0.608    | 0.718           |
| Hard unknot 8     | 0.588         | 0.588  | 0.647    | 0.673           |
| Weaving 10        | 0.553         | 0.557  | 0.598    | 0.648           |
| Tree medial 254   | 17.981        | 18.105 | 125.268  | 239.190         |
| Grid 64, shuffled | 21.704        | 21.792 | 25.720   | 25.703          |
| Grid 100          | 21.873        | 21.874 | 38.744   | 38.943          |

Median milliseconds including setup; distinct output guarantees. A limit is a censored query.

In the initial implementation, all 13 measured inputs completed in every arm and round. The Conway knot costs 0.563 ms at fixed  $q = 6$ , 0.608 ms for collision-free identity, and 0.718 ms for full recovery. The larger integer arithmetic is more costly on the 254-crossing tree medial: 17.981 ms, 125.268 ms, and 239.190 ms, respectively. Grid 100 rises from 21.873 ms to 38.744 ms for identity and 38.943 ms for recovery. This cost is a reason to retain the inexpensive fixed specializations and the current default filter; stronger asymptotic information is not a measured per-input runtime improvement.

### 38.7 Using proved identity to avoid redundant reconstruction

The first decoder always formed the norm of the unknot normalization, divided both coordinates and performed balanced decoding, even when  $Z = U$  had already proved  $V_D = 1$ . These potentially large integer operations are unnecessary in that case. The revised decoder validates the input-sized color count and nonzero normalization, checks the two exact pairs for equality, and returns the complete polynomial  $\{0 \mapsto 1\}$  immediately when they agree. For every other input it follows the original recovery path.

This shortcut uses the collision-free theorem above; applying it to an arbitrary fixed specialization would be incorrect. The regression check therefore rejects a fixed- $q$  result, rejects zero normalization even when both pairs are zero, and exercises cancellation immediately before returning the identity. Global checks still run before validation and before this early publication. The complete maintained suite now passes 603 tests in 110.500 seconds, recorded in `data/jones-identity-shortcut-integrated-tests.txt`.

The added benchmark preserves the exact earlier decoder from commit 58006240a, records its source hash, and compares complete polynomial queries in seven shuffled paired rounds after an excluded warmup. Every arm constructs a fresh diagram and includes all ordering and scalar work. A repeated old-decoder arm supplies an A/A control. The full returned results, including all exact pairs, coefficients and work counters, agree in every arm. The retained initial benchmark above and this new paired measurement are separate runs; their absolute times should not be compared across runs.

| Full polynomial query | Before  | A/A     | After  | Paired speedup |
|-----------------------|---------|---------|--------|----------------|
| Trefoil               | 0.161   | 0.165   | 0.167  | 0.966          |
| Conway                | 0.605   | 0.579   | 0.594  | 1.038          |
| Hard unknot 8         | 0.342   | 0.341   | 0.327  | 1.045          |
| Tree medial 126       | 17.718  | 17.727  | 10.135 | 1.742          |
| Tree medial 254       | 142.257 | 141.964 | 71.874 | 1.959          |
| Grid 100              | 38.266  | 38.046  | 38.644 | 0.998          |
| Grid 64, shuffled     | 24.363  | 24.570  | 25.246 | 0.999          |

Median milliseconds including setup. Ratios are medians of paired before/after times for identical complete results.

On the 126-crossing tree medial, the median paired speedup is 1.742; on the 254-crossing example it is 1.959, with median times falling from 142.257 ms to 71.874 ms. Grid and small-knot timings show little change; the trefoil’s paired ratio is 0.966, so no broad improvement is claimed. The large tree inputs benefit because redundant normalization arithmetic is a substantial part of full recovery. The scalar evaluation is still required, and the general asymptotic bound is unchanged. Recognition already requests identity without coefficient recovery, so these are full-Jones-query gains, not gains in the complete recognition pipeline.

### 39 Auditing a claimed route from Jones identity to recognition

A June 2026 preprint by Carmi and Cohen claims that the Jones polynomial detects the unknot [6]. We checked a specific prerequisite: Lemma 5.8 bounds residual Jones–Vassiliev degree by  $\widehat{B} + 2$ , under its machine hypotheses. Section 5.1.1 sets  $\widehat{B} = 3n + \ell - 1 + 11$  from the pre-planting host, and Proposition 5.12 claims the required local construction at a seed crossing, recovering an ordinary odd anti-parallel twist. Together these statements would bound every such residual twist on a three-crossing knot host by  $3 \cdot 3 + 1 - 1 + 11 + 2 = 22$ , independently of its length.

The explicit computation below contradicts this combined consequence. It does not distinguish an error in the conditional algebraic lemma from an error in the claimed geometric realization, and it does not provide a counterexample to Jones unknot detection itself. It is sufficient to reject this proof as a dependency of the maintained recognition algorithm. The stronger Jones identity backend therefore continues to require an independent unknot certificate.

#### 39.1 An explicit host and a twist with fixed exterior

Use the oriented trefoil with PD rows

$$H = ((5, 1, 2, 0), (1, 4, 3, 2), (0, 3, 4, 5)).$$

Keep its first two rows fixed. Replace the last row by a linear twist of odd length  $L$ , preserving its four exterior labels 0, 3, 4, 5. Start with  $(a, b) = (0, 3)$ . For  $i = 0, \dots, L-2$ , insert  $(a, b, 6+2i, 7+2i)$  and replace  $(a, b)$  by  $(7+2i, 6+2i)$ . Finish with  $(a, b, 4, 5)$ . Call the resulting knot diagram  $D_L$ . For  $L = 1$  this is precisely  $H$ , and  $D_L$  has  $L + 2$  crossings. All of its crossings are positive in the orientation given by traversing opposite ports and equal-label arc endpoints.

The two strands in the inserted band are anti-parallel: at every inserted row, exactly one of ports 0, 1 is incoming and exactly one of ports 2, 3 is incoming. The oriented smoothing pairs ports (0, 1) and (2, 3). After smoothing any one band crossing, the others successively become adjacent-port monogons. Deleting these by Reidemeister I moves gives exactly the two exterior rows obtained by smoothing the seed of  $H$ . All moves take place in the inserted band. This shadow calculation is independent of the signs assigned to the remaining crossings.

There is also an explicit host anchor in every odd sign cube: assign the first  $L - 1$  crossings alternating signs  $+, -, \dots$  and the last sign  $+$ . Cancel each consecutive opposite-sign pair by Reidemeister II. The surviving row and its exterior are exactly  $H$ , up to arc relabeling. Thus the audit does not infer applicability merely from a knot-family name or from a numerical resemblance of polynomials; it checks the specified local anti-parallel and anchor behavior.

### 39.2 An independent polynomial calculation

Use  $t$  for the standard Jones variable and  $z = t^{1/2}$  for the variable called  $x$  in the audited preprint. This  $z$  is different from the  $x = A^4 = t^{-1}$  used in our numerical Potts ring. The seed polynomial is

$$V_1(t) = t + t^3 - t^4.$$

Oriented smoothing of a band crossing gives the positive Hopf link, with  $V_0(z) = -z - z^5$ . The four bracket states of the two-crossing smoothed host independently check this value. Switching one band crossing allows a Reidemeister II cancellation and produces  $D_{L-2}$ . The usual oriented Jones skein relation consequently gives

$$t^{-1}V_L(t) - tV_{L-2}(t) = (t^{1/2} - t^{-1/2})(-t^{1/2} - t^{5/2}),$$

or

$$V_L(t) = t^2V_{L-2}(t) + t - t^2 + t^3 - t^4 \quad (L \geq 3 \text{ odd}).$$

In particular, the leading term is  $-t^{L+3}$ . For the smallest positive example exceeding the claimed cap,  $L = 9$ ,

$$V_9(t) = t - t^2 + 2 \sum_{j=3}^9 (-1)^{j+1} t^j - t^{10} + t^{11} - t^{12}.$$

An independent whole-cube bracket computation checks this eleven-crossing diagram using only  $2^{11}$  states. It uses neither a Tait graph nor the production quadratic evaluator. The exact faithful backend independently recovers the same complete polynomial.

Now write  $p = z - z^{-1}$ , so  $z^2 = pz + 1$ . Reducing  $V_L(z^2)$  into  $a(p) + b(p)z$  is a formal operation in a free rank-two  $\mathbb{Z}[p]$ -module. Multiplication by  $z$  sends the pair  $(a, b)$  to  $(b, a + pb)$ . Thus for positive exponent  $k$ , the coefficient of  $z$  in  $z^k$  has degree  $k - 1$  and leading coefficient one. The term  $-z^{2L+6}$  therefore contributes a unique leading term  $-p^{2L+5}$  to  $b(p)$ ; every other Jones monomial has exponent at most  $2L + 4$  and cannot cancel it. Consequently

$$\deg_p V_L = 2L + 5, \quad f_{23}(D_9) = (a_{23}, b_{23}) = (0, -1) \neq (0, 0).$$

This contradicts the degree-22 consequence above. There is no numerical approximation or degree estimation involved. Reflection provides another check: the mirrored family has degree  $2L + 6$ , so the mirror of  $D_9$  has degree 24. For any fixed proposed cap, the same recurrence supplies larger counterexamples to a uniform residual-twist bound.

### 39.3 Reproducibility and the scope of the finding

The maintained research audit is `fast/jones_research/audit_residual_twists.py`, with results in `fast/results/residual_twist_audit_20261008.json`.

It checks lengths 1, 3, 5, 7, 9, 23, 31, both mirrors, and stores each PD, full Jones polynomial, both formal JVP coordinates and their actual degrees. Ten small cases have whole-cube comparisons; all fourteen have faithful recovery and skein-recurrence comparisons. Formal substitution  $p = z - z^{-1}$  reconstructs every original Laurent polynomial, checking the coordinate conversion independently of the numerical quadratic ring. The longer cases include lengths beyond 22, so the discrepancy is not resolved by requiring a twist longer than that threshold.

The geometry record contains 79 individual smoothings and 1576 explicit Reidemeister I deletions with the fixed exterior retained, together with the alternating-sign Reidemeister II anchor traces. Another 42 crossing-sign assignments, exhausting sign cubes of lengths one, three and five, compare whole-cube polynomials with their signed-sum twist reductions. For each assignment, the independent bracket computation sums all smoothing states.

The audit does not construct the four-clasp machine or identify the precise step of its claimed collapse that fails. Its conclusion is the narrower, fully reproducible one: the uniform residual cap and universal geometric realization asserted by the audited statements cannot both be correct. In particular, the preprint does not close our gap between fast Jones computation and fast complete recognition.

## 40 A complete polynomial test on certified treewidth-two projections

### 40.1 The structural restriction that makes determinant one sufficient

The general determinant-one implication is false: the Conway and Kinoshita–Terasaka knots both survive that scalar test. Here we add an explicit graph hypothesis to make it a complete decision rule. Let  $G_D$  be the projection graph of a validated classical one-component diagram  $D$ : vertices are crossings and edges are the arcs between crossings. This is *not* a Tait graph or a knot-complement triangulation graph. Loops and multiple edges do not affect the presence of a  $K_4$  minor.

The structural theorem of Rué, Thilikos and Velona [4] classifies links admitting a  $K_4$ -minor-free projection as disjoint and connected sums of two-strand torus links. For a knot, this reduces to connected sums of  $T(2, q_i)$  with odd  $q_i$ , with trivial factors allowed. Indeed a connected sum of links with  $c_1, c_2$  components has  $c_1 + c_2 - 1$  components, so a two-component torus link cannot occur in a decomposition resulting in one component.

**Proposition 40.1.** *For a classical knot diagram  $D$  with  $\text{tw}(G_D) \leq 2$ ,*

$$D \text{ represents the unknot} \iff \det(D) = 1.$$

*Proof.* Treewidth at most two is equivalent to excluding a  $K_4$  minor. Apply the classification above. For odd  $q$ , the Alexander polynomial of  $T(2, q)$ , up to a Laurent unit and mirroring, is  $(t^{|q|} + 1)/(t + 1)$ , whose value at  $t = -1$  has absolute value  $|q|$ . The Alexander polynomial multiplies under connected sum, hence  $\det(D) = \prod_i |q_i|$ . This product is one exactly when all factors are trivial. Conversely every unknot has determinant one.  $\square$

This uses the given projection, with no supplied braid or Montesinos presentation. A rejected projection does not imply that its knot type has no other diagram in the class. In particular, it is not a knottedness certificate.

Bodlaender, Burton, Fomin and Grigoriev [5] give a stronger, linear-time recognition algorithm on the same class, using generalized diagrams with twist-labelled double edges. Our implementation uses the classification and an exact determinant instead; it does not implement their linear-time algorithm or inherit its time bound. This separates a directly checkable implementation from a future optimization of its arithmetic.

### 40.2 A graph certificate before any positive invariant verdict

The producer in `fastunknot/treewidth_two.py` builds the simple projection adjacency sets. It queues vertices of degree at most two. Deleting a degree-two vertex joins its two remaining

neighbours; existing parallel edges coalesce. It records the deletion order. Every operation preserves the property of having treewidth at most two in both directions. For degree two, suppression is an edge contraction followed by removal of redundant parallel edges, so it cannot increase treewidth. Conversely, a width-two decomposition of the remaining graph has a bag containing the new neighbour edge; attach the deleted vertex in a new bag of size three. The cases of degree zero or one are immediate.

Every nonempty graph of treewidth at most two has a vertex of degree at most two. Consequently elimination either deletes everything and supplies a certificate, or stops at a graph of minimum degree at least three and rejects the input projection. Deletion never increases a remaining vertex's degree: a possible fill edge replaces the incident edge just removed. This justifies the queue implementation. Each deletion examines at most two neighbours and creates at most one fill edge. The graph phase uses  $O(n)$  adjacency operations and  $O(n)$  words; the recorded order uses  $O(n \log(n + 1))$  bits.

A verifier does not trust the producer's queue or adjacency state. It reconstructs projection edges from PD labels, checks that the order is a permutation of all crossings, and replays deletion and fill on an edge set. It rejects any step with more than two remaining neighbours. The simple independent replay is polynomial rather than linear. Crucially, it must succeed *before* an exact determinant of one is allowed to certify an unknot. Certificates also bind the crossing count, schema version, method and exact verdict. The width witness is a sufficient structural certificate, not an implementation of a formal proof of the imported classification.

### 40.3 Exact integer arithmetic and independent verification

The producer builds the signed checkerboard graph using the existing PD corner convention. For either shading, the absolute signed spanning-tree sum is the knot determinant. We choose the shading with fewer vertices, discard loops, sum parallel signed weights and use the articulation-block product from Section 34. The reduced signed Laplacian of every nontrivial block is evaluated by fraction-free Bareiss elimination with exact division and row exchanges when a leading pivot is zero. Bridge blocks contribute just their integer weight. The shared determinant routine was moved to `integer_determinant.py`; the marked-continuation backend retains its existing interface and algorithm.

No modular residue is promoted to an exact determinant. A cancelled parallel sum and a zero leading pivot require exact handling even though a validated classical knot ultimately has an odd, nonzero determinant. The stored determinant is hexadecimal, so large composite knots do not hit Python's decimal-conversion ceiling when their certificates are serialized as JSON.

The independent certificate verifier avoids the checkerboard construction and Bareiss entirely. It constructs Wirtinger arcs by joining only the overpassing arc labels at each crossing. At  $t = -1$ , every Fox row is

$$2x_{\text{over}} - x_{\text{under},1} - x_{\text{under},2},$$

independently of crossing sign and orientation. It removes one row and one column and evaluates the resulting cofactor using exact rational Gaussian elimination. Its absolute value must equal the reported integer, and the verdict must be precisely the determinant-one decision. This provides a separate graph representation, knot-matrix construction and elimination algorithm for replay.

With  $n$  crossings, checkerboard matrices have  $O(n)$  rows and integer entries of size  $O(n)$ . Hadamard bounds give  $O(n \log(n + 1))$  bits for all relevant minors, hence for Bareiss entries; numerator products and exact divisors remain polynomial in bit length. Summing cubic arithmetic cost over blocks gives at most  $O(n^3)$  integer arithmetic operations. Thus the uncapped certificate producer has a polynomial bit bound on this class, with a conservative dense memory bound also polynomial. This is weaker than the cited linear algorithm, but independent of the size of any Khovanov complex.

## 40.4 Adaptive integration and the exact scope of the guarantee

The CLI option `-treewidth-two`, or Python option `use_treewidth_two=True`, enables this stage after checked source certificates and before the usual Seifert and simplification stages. The local allowance defaults to 0.1 seconds and includes lazy module loading, graph certification and determinant work. Its callback first checks the global deadline, then the local one; graph loops and determinant rows poll cooperatively. A local timeout or allocation failure records a skipped probe and continues the existing recognizer. A projection outside the class also continues. A global timeout retains UNKNOWN. No interrupted order or partially multiplied determinant is published as a verdict. The local interval and discarded work remain part of the complete query’s global elapsed time.

The standalone producer has no local cap. An uncapped local query in the Python recognizer sets `treewidth_two_seconds` to None. The polynomial claim belongs to the uncapped restricted-class procedure. A capped probe that falls back to Khovanov does not give the whole pipeline a polynomial guarantee, even on inputs whose projection lies in the class. The optional stage also does not change the general exponential upper bound or establish a general subexponential recognizer.

## 40.5 Validation and whole-query measurements

Eight added tests cover: an independent branch-set search for  $K_4$  minors on all 64 four-vertex graphs and 25 larger graphs; all 170 signed one-component two-strand braids through seven crossings against full cube-of-resolutions recognition; random braid and named-diagram determinants against exact Alexander polynomials; mirrored and relabelled diagrams; forged certificates; CLI dispatch; early and late cancellation; and local fallback versus global exhaustion. The Conway, Kinoshita–Terasaka and stored determinant-one grid knots explicitly test that the graph restriction cannot be omitted. A connected sum of 10,000 trefoils produces determinant  $3^{10000}$  and a JSON-safe certificate on 30,000 crossings. The full suite result is recorded in `data/treewidth-two-integrated-tests.txt`.

`benchmark_treewidth_two.py` measures validation plus full recognition on fresh, source-free PD inputs. Fourteen inputs each receive one excluded warm-up and seven shuffled paired rounds: default, an identical default control, and the optional stage. The interpreter is warm; imports are not part of the measured rounds. All 294 measured queries finish with matching verdicts. Median paired speedups on mixed-sign two-strand unknot diagrams of 101, 501 and 2001 crossings are respectively 1.085, 1.142 and 1.126. The default/control ratios range from 0.954 to 1.038.

Other inputs show why the stage remains optional. The trefoil has a 0.606 speedup ratio, tree medials range from 0.756 to 0.838, and the two 120-crossing trefoil sums have ratios near 0.67: existing Seifert certificates already decide these cheaply. Outside-class inputs range from 0.862 to 1.018. These measurements support a modest improvement on a specific family and a new independently certified input-class bound, not a default-wide performance improvement. Exact inputs, execution order, individual timings, methods and certificates are retained in `results/treewidth_two_20261008.json`.

# 41 An adaptive normal-surface fallback in an isolated worker

## 41.1 A complete geometric decision with a different bottleneck

The built-in complete backend computes Khovanov homology. We now add an optional alternative based on the topology of the knot exterior. For a classical knot  $K \subset S^3$ , its exterior is a solid torus exactly when  $K$  is the unknot. Thus a complete solid-torus recognition algorithm on an

exact triangulation of the exterior decides both outcomes; a failed attempt to find a compressing disc is not by itself a negative verdict.

Normal-surface theory provides an algorithmic framework for that decision. A normal surface in a triangulation with  $t$  tetrahedra has coordinates for four triangle types and three quadrilateral types per tetrahedron. The coordinates are nonnegative integers satisfying face-matching equations; at most one quadrilateral type is permitted in each tetrahedron. These constraints encode properly embedded surfaces, including candidates for compressing discs. The local quadrilateral alternatives are one source of exponential search even though the matching equations are linear. Successful recognition implementations combine such searches with simplification and crushing operations.

Burton and Ozlen [1] develop a complete branching algorithm using normal-surface methods and report experimental polynomial behaviour, while retaining exponential worst-case complexity. This is motivation for trying a different complete algorithm when Khovanov work is expensive, not evidence of a polynomial or subexponential theorem for our inputs.

We use Regina’s documented `isSolidTorus` decision routine [2]. The wrapper first applies the library’s fast link simplification, constructs an ideal triangulation of the complement, and calls that routine. It never substitutes `knowsSolidTorus`, whose Boolean value says whether the property is already known or cheap to compute, rather than whether the manifold is a solid torus. The implementation trusts Regina’s conversion, simplification and exact recognition. It does not yet export and independently check a sequence of normal surfaces or crushing operations.

## 41.2 Convention checks and the trust boundary

Our PD row  $(a, b, c, d)$  lists ports counterclockwise, with under-strand ports zero and two. Regina’s PD input starts at the incoming underpass. The worker obtains that port  $u \in \{0, 2\}$  from the validated orientation and rotates each row by  $u$ . Rotation by two preserves the crossing; rotation by one would interchange over and under and is forbidden. Normalized arc labels increase by one for the external API. The crossing-free input is explicitly one trivial link component, rather than an empty link.

The worker independently revalidates the supplied PD with `Diagram.from_pd`. It checks that the imported link has one component, is classical and has the same writhe. After simplification and complement construction, it checks that the triangulation is valid, orientable and has exactly one boundary component. These checks guard conversion mistakes but are not a replacement for the imported topology.

Every request and result carries a SHA-256 digest of the normalized PD. The digest binds the response to its input; it is not a mathematical certificate. Evidence includes the Regina engine version and installed distribution version (respectively 7.4 and 7.4.1 in this experiment), initial and simplified crossing counts, tetrahedron count, elapsed time and an explicit external-engine trust label. A result with a wrong digest, invalid schema, unsupported status, nonzero worker exit or unparsable output is inconclusive. Only a completed, well-formed `UNKNOT` or `KNOTTED` response can decide the query. Serialized results involving such a verdict, including inside connected-sum factors or restarted queries, explicitly exclude the external wrapper from the built-in Khovanov running-time statement.

## 41.3 Switching after cheap filters, with bounded discarded work

The optional `-regina` flag enables the probe only for an undecided diagram with at least 32 crossings. It runs after the existing one-sided invariant filters and before RIII search and Khovanov construction. Small inputs avoid interpreter and native-library startup. Visible factors are processed as their own diagrams: an external query never receives the whole source braid as if it presented a proper factor.

The local allowance is two seconds by default and is configurable with `-regina-seconds`. Python also permits `regina_seconds=None` for an uncapped local query. The standalone `regina_decide` routine has the same local allowance semantics and accepts diagrams of any size. Installing `fastunknot[normal]` supplies the optional dependency; the default package remains dependency-free and the default recognizer does not dispatch to Regina.

The parent launches a fresh worker using the current Python interpreter. It never imports the native Regina module itself. Native import, request conversion, process startup, simplification and recognition all consume the local query interval. The parent polls communication in slices of at most 50 milliseconds, checking the global callback before each slice and before accepting a result. Retried communication retains partially sent input and already received output. Every exit path kills a live child, waits for it to finish, and closes all three pipes; this includes local expiry, global cancellation and exceptions. Process creation and cleanup remain operating-system operations, so this is not a hard real-time latency guarantee.

A missing dependency, local expiry or failed worker records an inconclusive probe and leaves the original diagram available to the existing RIII and Khovanov stages. A global deadline propagates to `UNKNOWN` after cleanup. Time spent on unsuccessful native work is not reset or refunded. The `max_objects` parameter still bounds the Khovanov scanner; it is not a native-memory ceiling. This worker uses a time allowance rather than an independent bound on Regina’s allocations.

Regina’s simplifier makes random choices. Each worker resets its random engine to the library default, making these calls reproducible on the same build and platform. This does not promise identical simplification paths across machines or versions, and it does not make correctness probabilistic: the simplification moves preserve the knot and the terminal recognizer is an exact decision routine. Exhaustion remains inconclusive.

This is an adaptive portfolio based on failure of the cheaper stages, with a local cap before the built-in complete fallback. It is not simultaneous racing or a claim of constant-factor optimality relative to the best possible backend. A useful next step is to extract independently checkable geometric witnesses. No general subexponential or quasi-polynomial bound follows from the current integration.

#### 41.4 Measurements with explicit censoring

The new fixture directory stores PD data generated by Regina 7.4.1 for its `ExampleLink.monster`, `gordian` and `gst` examples [3]. Their crossing counts are 10, 141 and 48. The last is the Gompf–Scharlemann–Thompson knot from their slice-ribbon investigation, not an unknot fixture. The Conway diagram is also included in the benchmark from the existing corpus.

`normal_research/benchmark.py` runs five measured shuffled rounds plus an excluded warm-up for each of four inputs. Its three arms are the default pipeline, an identical default control, and the optional portfolio. Each arm receives a fresh validated PD, a four-second global allowance and a 50,000-object Khovanov ceiling. Native attempts always start a new interpreter; the portfolio’s local native allowance is two seconds.

| Input          | Default median (s) | Portfolio median (s) | Completed default / portfolio |
|----------------|--------------------|----------------------|-------------------------------|
| Monster unknot | 0.000965           | 0.000968             | 5/5 / 5/5                     |
| GST knot       | 0.001993           | 0.001983             | 5/5 / 5/5                     |
| Gordian unknot | 4.043 (censored)   | 1.788                | 0/5 / 5/5                     |
| Conway knot    | 0.001041           | 0.001043             | 5/5 / 5/5                     |

All conclusive verdicts agree. Both default arms exhaust their time allowance on every measured Gordian query, whereas the portfolio recognizes it in all five. The number 4.043 includes validation and is an observed resource-limited duration, *not* a completion time or a denominator for a

reported completion-speedup factor. Native evidence records simplification from 141 to 111 crossings followed by recognition of a 119-tetrahedron complement.

The dispatch policy matters. Standalone native queries, with a three-second allowance, take about 0.278 seconds for Monster and 1.402 seconds for Gordian; GST exhausts that allowance. The ordinary Alexander filter rejects GST in about two milliseconds, so the integrated portfolio avoids the slower native call. Monster and Conway remain below the crossing threshold. These are four concrete examples and one successful hard-unknot improvement, not a representative average-case study or an asymptotic conclusion. Raw timing samples and verdict evidence are stored in `results/normal_surface_20261008.json`.

Eight new tests check per-crossing conversion, communication across repeated polls, missing dependencies, malformed responses, input binding, pipeline dispatch, fallback and actual worker cleanup on local and global cancellation. With Regina installed they also check six exact native verdicts, global exhaustion and an actual CLI recognition of the Gordian unknot. The full integration log and the dependency-free focused log are retained alongside the article. The optional tests are skipped explicitly when Regina is not installed; the failure-handling tests still run in that configuration.

## 42 Cyclic knot-group certificates and adaptive Whitehead descent

### 42.1 A positive certificate from the full fundamental group

The optional `-group` stage provides a new sufficient test for unknottedness. It simplifies a Wirtinger presentation by exact changes of presentation, records every nontrivial step, and independently replays that record. A successful trace ends with precisely one generator and no nonempty relators. It therefore proves that the knot group is isomorphic to  $\mathbb{Z}$ . For classical knots in  $S^3$ , this implies that the knot is the unknot, by the classical consequence of the Loop Theorem [18]. This criterion concerns the whole fundamental group: its abelianization alone is  $\mathbb{Z}$  for every knot and cannot supply this conclusion.

Here is the topological implication explicitly. Write  $X$  for the compact knot exterior. If  $\pi_1(X) = \mathbb{Z}$ , the map from the boundary torus group  $\mathbb{Z}^2$  cannot be injective. The Loop Theorem supplies an embedded compressing disc. The exterior is irreducible: an embedded sphere bounds two balls in  $S^3$ , and the ball on the side away from the connected knot lies in the exterior. Compressing its torus boundary along the disc gives a sphere; irreducibility then identifies the exterior with a solid torus. Equivalently, the compression slope is the longitude, which bounds a disc and gives a spanning disc for the knot after adjoining the collar annulus. The input contract is essential: our PD loader validates a classical, single-component knot diagram. No conclusion for virtual diagrams or arbitrary finitely presented groups is asserted.

### 42.2 A deterministic presentation and reversible eliminations

For each PD row  $(a, b, c, d)$ , join the labels  $b, d$  along the overpass. The connected classes of arc labels are the Wirtinger generators. Name these classes by increasing minimum arc label; this makes certificate indices independent of the producer's union-find root choices. Orient the knot by following its component from dart zero. At a crossing let  $u, v$  denote the incoming and outgoing underpass generators and let  $o$  be its overpass generator. Our relator is

$$ouo^{-1}v^{-1} \quad \text{at a positive crossing,} \quad ovo^{-1}u^{-1} \quad \text{at a negative crossing.}$$

These choices give the same signed Fox rows as the established Alexander convention. We retain all crossing relators, including the redundant Wirtinger relator, and freely and cyclically reduce

each one. Free reduction changes no word in the free group; cyclic reduction conjugates the relator and hence preserves its normal closure.

Suppose a relator contains a generator  $g$  exactly once, with sign  $\epsilon \in \{1, -1\}$ . Write it as  $Pg^\epsilon S$  where neither  $P$  nor  $S$  contains  $g^{\pm 1}$ . Solve the defining relation:

$$g = P^{-1}S^{-1} \quad (\epsilon = 1), \quad g = SP \quad (\epsilon = -1).$$

Substitute this word, and its inverse for  $g^{-1}$ , into every other relator. Delete this one defining relation and the generator. This is a Tietze elimination, giving an isomorphic presented group. In particular, it is not the operation of imposing an extra relation. The trace records only the original relation index and generator; replay derives the replacement itself. Empty slots retain their original indices.

The producer prefers an eligible pair minimizing  $(|R| - 2) \text{occ}(g)$ , followed by relator length, index and generator. This is a deterministic estimate of expansion cost, not a completeness argument. Renumbering or mirroring a diagram can change the greedy path. Indeed, two of the twelve tested mirror diagrams stall even though every original succeeds. This is a concrete reason to retain other decision procedures in the portfolio.

### 42.3 Cut-based changes of free basis

When no generator can be eliminated, try to shorten the whole relator list by a Whitehead automorphism. Fix a signed letter  $a$  and a subset  $A$  of the signed alphabet with  $a \in A$  and  $a^{-1} \notin A$ . Fix  $a^{\pm 1}$ , and for every other letter  $x$  put

$$\tau_{A,a}(x) = (a^{-1} \text{ if } x^{-1} \in A) \ x \ (a \text{ if } x \in A),$$

where an absent factor means the empty word. This preserves inversion and defines a free-group automorphism. Its inverse uses multiplier  $a^{-1}$  and subset  $(A \setminus \{a\}) \cup \{a^{-1}\}$ . Applying it to every relator therefore preserves the isomorphism type of the quotient group.

For a cyclically reduced list of words, add a weighted undirected edge  $\{x, y^{-1}\}$  for each cyclic adjacent pair  $xy$ , including the last–first pair of each word. Add weights across all relators. For this convention, the exact change in total cyclic length is

$$|\tau_{A,a}(R)|_{\text{cyc}} - |R|_{\text{cyc}} = \text{cut}(A, A^c) - \deg(a).$$

The cut-capacity approach to Whitehead minimization is developed by Roig–Ventura–Weil [19]; their polynomial algorithm handles varying free rank, rather than enumerating exponentially many subsets. We implement the relevant minimization directly on the sum of the relators’ weighted graphs. For each signed multiplier, an exact integer-capacity Edmonds–Karp flow finds a minimum cut separating  $a$  from  $a^{-1}$ . The best strictly negative length change selects the next move. A nonnegative minimum ends this search attempt.

The producer explicitly substitutes the chosen automorphism and checks that the observed length change equals the cut prediction. The verifier needs neither this formula nor the minimum-cut computation: it checks that the supplied pair is a valid Whitehead automorphism and directly applies it. Soundness does not require the proposed move to shorten the words. Thus an error in search strategy cannot by itself create a false positive through the certificate interface.

### 42.4 A reproducible convention audit of a new preprint

Kapovich’s July 2026 preprint on compressed primitivity [20] motivates keeping group words compressed. It states polynomial compressed Whitehead-minimality tests at fixed rank, with further results for compressed primitivity. It also cautions that fast compressed operations alone

do not bound the number of descent steps: elementary descent on  $ab^{2^m}$  can require exponentially many steps in the compressed input length. Fixed-rank hypotheses also need attention when the rank grows with a knot diagram.

Our audit found a convention mismatch in version 1’s equations (10)–(12), as printed together. Its automorphism (10) is the one above, but its graph (11) uses edges  $\{x^{-1}, y\}$ . Take

$$w = abc, \quad A = \{a, b^{-1}\}, \quad \text{multiplier } a.$$

Literal substitution gives  $aa^{-1}bc$ , hence cyclic length change  $-1$ . The printed graph has edges  $\{a^{-1}, b\}$ ,  $\{b^{-1}, c\}$ ,  $\{c^{-1}, a\}$ ; the stated cut has weight two and  $\deg(a) = 1$ , predicting  $+1$  via (12). With our edges  $\{x, y^{-1}\}$  the cut has weight zero, so the predicted change is correctly  $-1$ .

This local inconsistency is repaired by inverting the graph’s vertex labels, or by transforming the associated characteristic pair consistently. It is not a disproof of the paper’s compressed-primitivity claims: minimization over all pairs can survive such a relabelling. Our regression test records the three-letter example and checks 15,552 short-word/characteristic-pair combinations against an independent four-case substitution implementation. The production code currently uses explicit word lists. It does not implement compressed free-group operations or inherit a compressed-input complexity guarantee.

## 42.5 Independent replay and local resource accounting

The certificate contains its version, method, positive verdict, exact normalized input PD, move list, and surviving generator. The checker reconstructs the presentation by graph traversal rather than union–find, reconstructs orientation from paired darts rather than cached Diagram orientation, and performs word cancellation with a separate deque-based implementation. It checks unique occurrence before an elimination and valid alphabet membership and multiplier conditions before a Whitehead move. After every substitution it reduces every remaining relator. Acceptance requires exactly the declared surviving generator and an entirely empty relator list. All relators are accounted for; neither the stored verdict nor a claim of rank one is accepted on its own.

The search and checker share a default wall allowance of 0.05 seconds per attempt. Each separately has a two-million-unit cooperative work budget and a 200,000-letter expansion ceiling. Work units conservatively charge letter processing in blocks, graph scans and flow exploration; they are not a hardware-independent instruction count. Substitution sizes are checked before allocating expanded words. The Whitehead producer uses a conservative bound of three times the current length. A trace that is mathematically valid can therefore be rejected by a resource ceiling. Only a fully completed and checked trace supplies a verdict.

Local exhaustion or stalled search returns **INCONCLUSIVE**; global cancellation propagates and the surrounding recognizer returns **UNKNOWN**. The optional stage runs after cheap invariant obstructions and before the optional normal-surface worker and late Reidemeister III/Khovanov stages. A local failure consumes only its allowance and resumes the portfolio. Verification is inside the local allowance, including the check immediately before acceptance.

## 42.6 What can and cannot be bounded

Let  $n$  be the crossing number, and suppose every expanded relator list in one completed search has total length at most  $B$ . There are at most  $n - 1$  generator eliminations. Between eliminations each chosen Whitehead step decreases a nonnegative integer length, so there are at most  $B$  such steps in each of at most  $n$  phases. The graph has  $O(n)$  vertices and  $O(n^2)$  edges. Edmonds–Karp takes  $O(n^5)$  graph operations per multiplier and we try  $O(n)$  multipliers. Thus, conservatively, the search cost is polynomial in  $n$  and  $B$ , bounded by

$$O(n(B + 1)(B + n^6) \text{polylog}(nB + 2))$$

with straightforward bit accounting for integer capacities. Replay is polynomial in explicit trace size and expanded word volume. This is our implementation-level conditional bound, not a compressed-word theorem.

Two essential gaps prevent a general fast recognition theorem. First, eliminating a generator can substantially expand the other relators; no polynomial or quasi-polynomial bound on  $B$  in terms of  $n$  is established. Second, even unlimited length-decreasing Whitehead search followed by this greedy elimination rule can stall on an unknot. A larger Tietze search would require its own completeness and quantitative bounds. Compressed words could address representation growth, but would not alone supply those missing search bounds. The bounded optional stage preserves the existing complete fallback and does not improve the general exponential upper bound proved for our built-in decision pipeline.

## 42.7 Whole-query measurements and regression tests

`fast/benchmark_group.py` measures 21 inputs in seven randomized A/A/B rounds after one excluded warmup: 441 timed whole queries. Every query starts with fresh validated PD data and carries no source-braid provenance. The two control arms use the default pipeline. The third enables the group stage, including independent replay. Every arm has a 0.25-second global allowance and a 50,000-object Khovanov ceiling. Raw samples, resulting certificates and environment metadata are recorded in `fast/results/group_20261008.json`.

The group stage certifies all twelve maintained Khovanov-only survivor inputs in every measured round. Ten show lower median query costs. The two largest gains are 10.90 to 4.66 ms and 12.86 to 3.86 ms. Two regress, from 4.62 to 4.89 ms and from 3.67 to 4.69 ms. The sum of the twelve median costs is 60.85 ms for the default control, 60.56 ms for the repeated control, and 39.19 ms with group certificates. This aggregate describes these twelve selected inputs, not a distribution of arbitrary knots or an asymptotic law.

An eight-crossing hard unknot and Monster already have cheap default Reidemeister paths; inserting group search increases their median costs from 0.66 to 1.01 ms and 0.95 to 1.82 ms respectively. Easy structural and invariant obstructions generally bypass the new stage. All arms time out on Gordian under this benchmark’s short global cap; these censored observations are not completion times or speedups. The standalone group attempt also exhausts its local work allowance on Gordian. These results support retaining `-group` as opt-in.

Tests cover exhaustive short-word cut identities, minimum-cut versus exhaustive characteristic-pair optimization, the printed-convention counterexample, all twelve survivor certificates, mirror behaviour, 70 random small knots against an independent full Khovanov cube, known nontrivial determinant-one knots, malformed and input-mismatched traces, independent replay with producer helpers disabled, local exhaustion, global and verification-stage cancellation, and actual CLI routing.

The integrated suite passes all 626 tests with the optional Regina backend installed (126.707 seconds); see `synthesis/data/group-integrated-tests.txt`. The rebuilt article contains 154 pages.

## 43 Assessment and recommendations

**What was achieved.** Seven mutually consistent exact recognizers, the seventh now in an accelerated Python version and a Rust version that agree with each other and with nine independently accelerated variants; a verified and cross-validated body of code for the terminal ball-pattern test and for cocycle-dual normal surfaces; a precise, sourced statement of the conditional  $L(g+1)^L$  accounting and of everything a  $n^{O(\log n)}$  implementation would still need; and one recognizer whose

practical reach is well beyond the  $2^n$ -state cubes.

**Obstacles that remain.** The six items of Section 5. They are research problems in 3-manifold topology and algorithm design, not coding tasks: the specification of the accelerated hierarchy exists only as a sketch on slides, and the one detailed paper on the subject states that its steps are not specified precisely enough for a running-time estimate. Until that changes, any “ $n^{O(\log n)}$  implementation” would be an exponential search with a mislabelled bound, which is exactly what all six archives refused to deliver.

**If work continues.** (1) Track the published record: a full version of the 2021 announcement, or the refined Section 10 algorithm with an explicit cost model, would change the picture. (2) Settle the move-set question of Section 3.4 from Dynnikov’s paper, or add shared-corner exchanges to archive 01 and re-run its exhaustive validation. (3) The scanning recognizer now exists in Rust (Section 9); the Python separator backend now guarantees  $O(\sqrt{n})$  frontier width and a subexponential exact scalar query. The next complexity problem is to control intermediate homology multiplicities or supply a different complete decision fallback; order width alone does not settle it. (4) Use the archives’ exact recognizers as oracles for any future geometric implementation, as several of their reports suggest.

## References

- [1] Benjamin A. Burton and Melih Ozlen, *A fast branching algorithm for unknot recognition with experimental polynomial-time behaviour*, arXiv:1211.1079. <https://arxiv.org/abs/1211.1079>.
- [2] Regina 7.4.1 Calculation Engine, *Triangulation<3>::isSolidTorus API documentation*. [https://regina-normal.github.io/engine-docs/classregina\\_1\\_1Triangulation\\_3\\_013\\_01\\_4.html](https://regina-normal.github.io/engine-docs/classregina_1_1Triangulation_3_013_01_4.html).
- [3] Regina 7.4.1 Calculation Engine, *ExampleLink API documentation*. [https://regina-normal.github.io/engine-docs/classregina\\_1\\_1ExampleLink.html](https://regina-normal.github.io/engine-docs/classregina_1_1ExampleLink.html).
- [4] Juanjo Rué, Dimitrios M. Thilikos and Vasiliki Velona, *Structure and enumeration of  $K_4$ -minor-free links and link-diagrams*, European Journal of Combinatorics 89 (2020), 103147, Theorem 9. <https://arxiv.org/html/1806.07855v3>.
- [5] Hans L. Bodlaender, Benjamin Burton, Fedor V. Fomin and Alexander Grigoriev, *Knot Diagrams of Treewidth Two*, arXiv:1904.03117v2, 2019, Theorems 1–3. <https://arxiv.org/html/1904.03117>.
- [6] Avishy Carmi and Eliahu Cohen, *On trivial Jones–Vassiliev polynomials*, arXiv:2606.22410v1, 21 June 2026. <https://arxiv.org/abs/2606.22410>.
- [7] Marc Lackenby, *Some fast algorithms for curves in surfaces*, arXiv:2401.16056v2, 20 March 2026, Theorems 1.1–1.3. <https://arxiv.org/html/2401.16056v2>.
- [8] Richard J. Lipton and Robert Endre Tarjan, *A separator theorem for planar graphs*, SIAM J. Appl. Math. 36 (1979), 177–189. <https://doi.org/10.1137/0136016>.
- [9] Marc Lackenby, *Unknot recognition is quasi-polynomial time*, YMSC Topology Seminar, 7 November 2024, abstract. <https://qzc.tsinghua.edu.cn/en/info/1122/4171.htm>.
- [10] Gheehyun Nahm, *Basepoints in Khovanov homology and nonorientable surfaces*, arXiv:2605.08457v1, 8 May 2026, Lemma 2.4. <https://arxiv.org/abs/2605.08457>.

- [11] Allen Hatcher, *Algebraic Topology*, Cambridge University Press, 2002, Section 1.3. <https://pi.math.cornell.edu/~hatcher/AT/ATpage.html>.
- [12] Ian Agol, Joel Hass and William P. Thurston, *The computational complexity of knot genus and spanning area*, Trans. Amer. Math. Soc. 358 (2006), 3821–3850. <https://arxiv.org/abs/math/0205057>.
- [13] Tetsuya Abe, *The Rasmussen invariant of a homogeneous knot*, Proc. Amer. Math. Soc. 139 (2011), 2647–2656. <https://arxiv.org/abs/1003.5392>.
- [14] Andrew Lobb, *Computable bounds for Rasmussen’s concordance invariant*, Compositio Math. 147 (2011), 661–668. <https://arxiv.org/abs/0908.2745>.
- [15] Report 07, *Unknot recognition: structural certificates and compressed complexes*, delivered 7 October 2026; [reports/07/paper/unknot\\_structural\\_compression.tex](#).
- [16] Marc Lackenby, *Unknot recognition in quasi-polynomial time*, slides, February 2021, 109 pages, supplied as [docs/quasipolynomial-talk.pdf](#).
- [17] Marc Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026, 54 pages. <https://arxiv.org/abs/2607.23350>.
- [18] C. D. Papakyriakopoulos, *On Dehn’s lemma and the asphericity of knots*, Annals of Mathematics 66 (1957), 1–26. <https://doi.org/10.2307/1970113>.
- [19] Abdó Roig, Enric Ventura and Pascal Weil, *On the complexity of the Whitehead minimization problem*, International Journal of Algebra and Computation 17 (2007), 1611–1634. <https://arxiv.org/abs/math/0608779>.
- [20] Ilya Kapovich, *Compressed primitivity problem in free groups*, arXiv:2607.21499v1, 23 July 2026. <https://arxiv.org/html/2607.21499v1>.
- [21] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, Publ. Math. IHÉS 113 (2011), 97–208.
- [22] Dror Bar-Natan, *Khovanov’s homology for tangles and cobordisms*, Geom. Topol. 9 (2005), 1443–1499; and *Fast Khovanov homology computations*, J. Knot Theory Ramifications 16 (2007), 243–255.
- [23] Ivan Dynnikov, *Arc-presentations of links: monotonic simplification*, Fund. Math. 190 (2006), 29–76, arXiv:math/0208153.
- [24] Harry M. Markowitz, *The elimination form of the inverse and its application to linear programming*, Management Science 3 (1957), 255–269.
- [25] Mikhail Khovanov, *A categorification of the Jones polynomial*, Duke Math. J. 101 (2000), 359–426.
- [26] Konstantinos Meichanetzidis and Stefanos Kourtis, *Evaluating the Jones polynomial with tensor networks*, Phys. Rev. E 100 (2019), 033303. <https://arxiv.org/abs/1807.02119>.
- [27] Clément Maria, *Parameterized Complexity of Quantum Knot Invariants*, SoCG 2021, LIPIcs 189, 53:1–53:17. <https://doi.org/10.4230/LIPIcs.SocG.2021.53>.
- [28] Patrick Dehornoy, *Efficient solutions to the braid isotopy problem*, arXiv:math/0703666, 2007. <https://arxiv.org/abs/math/0703666>.
- [29] Marius Crainic, *On the perturbation lemma, and deformations*, arXiv:math/0403266. <https://arxiv.org/abs/math/0403266>.
- [30] Thomas C. Jaeger, *A Remark on Roberts’ Totally Twisted Khovanov Homology*, arXiv:1109.1805, Theorem 3.1. <https://arxiv.org/abs/1109.1805>.
- [31] Louis H. Kauffman and Sofia Lambropoulou, *On the classification of rational tangles*, arXiv:math/0311499. <https://arxiv.org/abs/math/0311499>.

- [32] Louis H. Kauffman and Sofia Lambropoulou, *Hard Unknots and Collapsing Tangles*, arXiv:math/0601525, Theorem 5. <https://arxiv.org/abs/math/0601525>.
- [33] João M. Nogueira and António Salgueiro, *On embeddings of 2-string tangles into the unknot, the unlink and split links*, arXiv:2110.15645, Theorem 4.9. <https://arxiv.org/abs/2110.15645>.

## A Files produced

- `reports/01 – reports/06`: the extracted archives (flat, one level).
- `synthesis/report.tex`, `synthesis/report.pdf`: this report; `synthesis/make_tables.py` regenerates the tables from `synthesis/data/*.json`; the cross-validation scripts are in `synthesis/data/`.
- `fast/`: the `fastunknot` package (version 0.2 with the later work of Section 10: the default scanner `scan_fast.py` and its geometry `planar.py`), tests and examples; `ablation.py` with `results/ablation.json`; the 0.1 benchmark, kept as `results/benchmark_0.1.json`; and the measuring tools of Section 10, namely `perf.py`, `ab.py` and `crosscheck.py`, with their logs in `results/`.
- `proposals/01 – proposals/09`: the nine acceleration proposals, with `proposals/README.md` comparing their ideas.
- `rust/`: the Rust crate, `cross_check.py`, `profile.py` and `results/profile.json`.
- `synthesis/data/proposals_bench.py` and its JSON outputs: the same-machine comparison of all engines.